HZ BOOKS
华章教育

计 算 机 科 学 丛 书

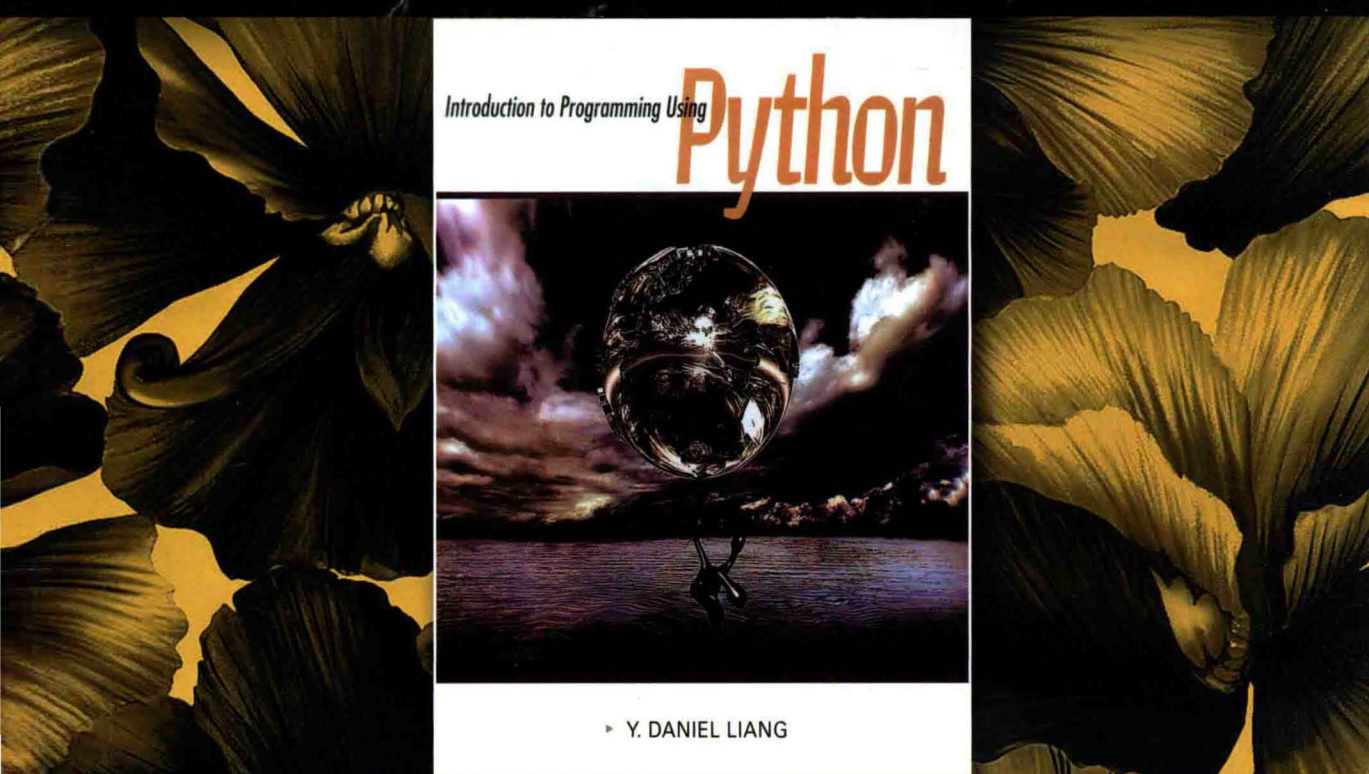
PEARSON

Python语言程序设计

[美] 梁勇（Y. Daniel Liang） 著
阿姆斯特朗亚特兰大州立大学

李娜 译
西安电子科技大学

Introduction to Programming Using Python



机械工业出版社
China Machine Press

Python语言程序设计

Introduction to Programming Using Python

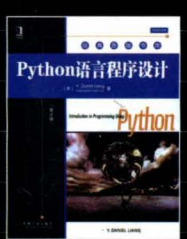
本书保持了梁勇博士系列丛书中一贯的、标志性的教与学的哲学：以实例教，由实践学。书中采用了他所提出的已经经过实践检验的“基础先行”的方法，即在定义类之前，首先使用清晰简明的语言介绍基本程序设计概念，如选择语句、循环和函数；在介绍面向对象程序设计和GUI编程之前，首先介绍基本逻辑和程序设计概念。书中除了给出一些以游戏和数学为主的典型实例外，还在每章的开始使用简单的图形给出一两个例子，以激发学生的学习兴趣。

本书特色

- 以“基础先行”方法介绍基本程序设计概念和方法，帮助学生循序渐进地学习所有必需和重要的基本概念。
- 以“问题驱动”方法讲授程序设计技术，强调问题求解，而非语法。通过广泛的趣味性实例（涉及数学、自然科学、商业、金融、游戏、动画和多媒体领域）来激发学生的学习兴趣，为求解这些问题，适时地引入相关的语法和库。
- 可以灵活介绍GUI相关主题。第1～6章使用内置的Turtle图形模块，其余部分使用Tkinter，这两种工具都是简单、易学的程序设计教学工具。每章的开始都有GUI实例，每章末尾还有专门的GUI练习。

作者简介

梁勇（Y. Daniel Liang） 阿姆斯特朗亚特兰大州立大学计算机科学系Yamacraw教授。之前曾是普度大学计算机科学系副教授，并曾两次获得普度大学杰出研究奖。他所编写的Java教程在美国大学Java课程中采用率最高。



书号：978-7-111-41234-2
定价：79.00元

PEARSON

上架指导：计算机\程序设计
ISBN 978-7-111-48768-5
9 787111 487685 >
定价：79.00元

PEARSON
www.pearson.com

投稿热线：(010) 88379604
客服热线：(010) 88378991 88361066
购书热线：(010) 68326294 88379649 68995259

华章网站：www.hzbook.com
网上购书：www.china-pub.com
数字阅读：www.hzmedia.com.cn

封面设计 · 锡 杉

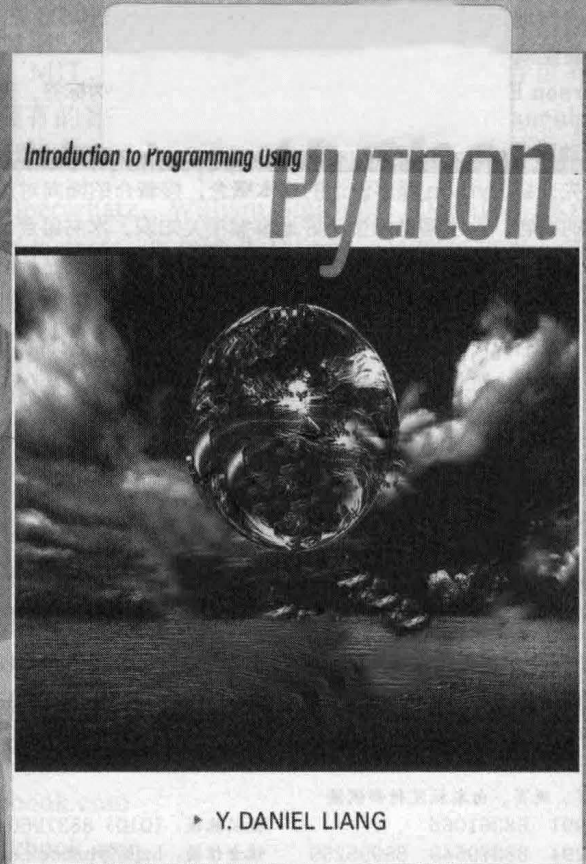
计 算 机 科 学 丛 书

Python语言程序设计

[美] 梁勇（Y. Daniel Liang） 著
阿姆斯特朗亚特兰大州立大学

李娜 译
西安电子科技大学

Introduction to Programming Using Python



机械工业出版社
China Machine Press

图书在版编目（CIP）数据

Python 语言程序设计 / [美] 梁勇（Liang Y. D.）著；李娜译 . —北京：机械工业出版社，2015.2
（计算机科学丛书）
书名原文：Introduction to Programming Using Python

ISBN 978-7-111-48768-5

I. P… II. ① 梁… ② 李… III. 软件工具 - 程序设计 IV. TP311.56

中国版本图书馆 CIP 数据核字（2014）第 287935 号

本书版权登记号：图字：01-2012-5865

Authorized translation from the English language edition, entitled *Introduction to Programming Using Python*, 9780132747189 by Y. Daniel Liang, published by Pearson Education, Inc., Copyright © 2013.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from Pearson Education, Inc.

Chinese simplified language edition published by Pearson Education Asia Ltd., and China Machine Press Copyright © 2015.

本书中文简体字版由 Pearson Education（培生教育出版集团）授权机械工业出版社在中华人民共和国境内（不包括中国台湾地区和中国香港、澳门特别行政区）独家出版发行。未经出版者书面许可，不得以任何方式抄袭、复制或节录本书中的任何部分。

本书封底贴有 Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

本书采用“问题驱动”、“基础先行”和“实例和实践相结合”的方式，讲述如何使用 Python 语言进行程序设计。本书首先介绍 Python 程序设计的基本概念，接着介绍面向对象程序设计方法，最后介绍算法与数据结构方面的内容。为了帮助学生更好地掌握相关知识，本书每章都包括以下模块：学习目标，引言，关键点，检查点，问题，本章总结，测试题，编程题，注意、提示和警告。

本书可以作为高等院校计算机及相关专业 Python 程序设计课程的教材，也可以作为 Python 程序设计的自学参考书。

出版发行：机械工业出版社（北京市西城区百万庄大街 22 号　邮政编码：100037）

责任编辑：李　艺	责任校对：董纪丽
印　刷：北京市荣盛彩色印刷有限公司	版　次：2015 年 4 月第 1 版第 1 次印刷
开　本：185mm × 260mm　1/16	印　张：29.5（含 0.25 印张彩插）
书　号：ISBN 978-7-111-48768-5	定　价：79.00 元

凡购本书，如有缺页、倒页、脱页，由本社发行部调换
客服热线：（010）88378991　88361066　　投稿热线：（010）88379604
购书热线：（010）68326294　88379649　68995259　　读者信箱：hzjsj@hzbook.com

版权所有 · 侵权必究
封底无防伪标均为盗版
本书法律顾问：北京大成律师事务所　韩光 / 邹晓东

文艺复兴以来，源远流长的科学精神和逐步形成的学术规范，使西方国家在自然科学的各个领域取得了垄断性的优势；也正是这样的优势，使美国在信息技术发展的六十多年间名家辈出、独领风骚。在商业化的进程中，美国的产业界与教育界越来越紧密地结合，计算机学科中的许多泰山北斗同时身处科研和教学的最前线，由此而产生的经典科学著作，不仅擘划了研究的范畴，还揭示了学术的源变，既遵循学术规范，又自有学者个性，其价值并不会因年月的流逝而减退。

近年，在全球信息化大潮的推动下，我国的计算机产业发展迅猛，对专业人才的需求日益迫切。这对计算机教育界和出版界都既是机遇，也是挑战；而专业教材的建设在教育战略上显得举足轻重。在我国信息技术发展时间较短的现状下，美国等发达国家在其计算机科学发展的几十年间积淀和发展的经典教材仍有许多值得借鉴之处。因此，引进一批国外优秀计算机教材将对我国计算机教育事业的发展起到积极的推动作用，也是与世界接轨、建设真正的世界一流大学的必由之路。

机械工业出版社华章公司较早意识到“出版要为教育服务”。自 1998 年开始，我们就将工作重点放在了遴选、移译国外优秀教材上。经过多年的不懈努力，我们与 Pearson，McGraw-Hill，Elsevier，MIT，John Wiley & Sons，Cengage 等世界著名出版公司建立了良好的合作关系，从他们现有的数百种教材中甄选出 Andrew S. Tanenbaum，Bjarne Stroustrup，Brain W. Kernighan，Dennis Ritchie，Jim Gray，Afred V. Aho，John E. Hopcroft，Jeffrey D. Ullman，Abraham Silberschatz，William Stallings，Donald E. Knuth，John L. Hennessy，Larry L. Peterson 等大师名家的一批经典作品，以“计算机科学丛书”为总称出版，供读者学习、研究及珍藏。大理石纹理的封面，也正体现了这套丛书的品位和格调。

“计算机科学丛书”的出版工作得到了国内外学者的鼎力相助，国内的专家不仅提供了中肯的选题指导，还不辞劳苦地担任了翻译和审校的工作；而原书的作者也相当关注其作品在中国的传播，有的还专门为其书的中译本作序。迄今，“计算机科学丛书”已经出版了近两百个品种，这些书籍在读者中树立了良好的口碑，并被许多高校采用为正式教材和参考书籍。其影印版“经典原版书库”作为姊妹篇也被越来越多实施双语教学的学校所采用。

权威的作者、经典的教材、一流的译者、严格的审校、精细的编辑，这些因素使我们的图书有了质量的保证。随着计算机科学与技术专业学科建设的不断完善和教材改革的逐渐深化，教育界对国外计算机教材的需求和应用都将步入一个新的阶段，我们的目标是尽善尽美，而反馈的意见正是我们达到这一终极目标的重要帮助。华章公司欢迎老师和读者对我们的工作提出建议或给予指正，我们的联系方法如下：

华章网站：www.hzbook.com
电子邮件：hzjsj@hzbook.com
联系电话：（010）88379604
联系地址：北京市西城区百万庄南街 1 号
邮政编码：100037

华章教育
华章科技图书出版中心

译 者 序

Introduction to Programming Using Python

很荣幸在翻译完 Y. Daniel Liang 的《 Introduction to Java Programming, Comprehensive, Eighth Edition 》[⊖]之后能成为他这本 Python 语言编程书的译者。经过这么长的时间，终于可以交稿了，下面我想结合自己的翻译过程跟大家分享一些感触。

我觉得最值得大家重视的是作者不论在《 Java 语言程序设计》还是在本书中所采用的以问题驱动方式学习程序设计的方法。我们需要明确一点，编写程序是为了解决实际问题，而不是纯粹为了做题。但是，反过来讲，只有做大量的习题才能从做题过程中培养程序设计的能力，从而达到解决问题的目的，因此希望大家在学习过程中明确什么是方式，什么是目标。

当然，本书是讲 Python 的，就不得不说说 Python 的优点。Python 语法简洁、易读、易扩展，具有丰富和强大的类库，一些知名大学已经采用它来教授程序设计课程。例如，麻省理工学院的计算机科学及编程导论课程就用 Python 教授。Python 的设计者在开发时的指导思想就是对于一个特定问题有一种最好的方法来解决就好，所以希望大家在学习过程中能不断体会到这门语言之美。

在整个翻译工作结束时，非常感谢一直负责和我联系的王春华编辑。我一直都不是个按时交稿的译者，教学工作经常会干扰到翻译进度，但她总是耐心等待。翻译接近尾声时，我因为眼睛充血，有些收尾的工作一直在拖延，感谢王老师的理解和支持。我也要感谢做过一些初期协助工作的同学，他们是段超伟、赵欣秋、陈峰、湛孝丰、郝一鸣、吴裕峰、薛二中、王昕伟、陈帅、栾锦泰，特别是段超伟同学，在本书出版时他应该已经在清华大学攻读硕士学位了，祝福他在新的学校取得更大的成绩。当然，也要感谢家人、朋友和同事在翻译过程中对我的生活以及工作的支持，没有他们的支持，我可能没有时间和精力来完成这本书的翻译工作。

由于时间仓促，译者水平有限，译文中难免存在欠妥和纰漏之处，恳请广大读者不吝赐教。

译者

2014 年 10 月

⊖ 本书已由机械工业出版社引进并出版，书名为《Java 语言程序设计（原书第 8 版）》（分基础篇和进阶篇，ISBN：978-7-111-34081-2，978-7-111-34236-6）。——编辑注

本书假设你是一位先前没有任何程序设计经验的程序员新手。那么，什么是程序设计呢？程序设计是指使用程序设计语言编写程序以解决问题。不论你使用的是哪种程序设计语言，解决问题和程序设计的根本都是一致的。你可以使用任何一种像 Python、Java、C++ 或 C# 这样的高级程序设计语言来学习程序设计。一旦知道如何使用其中一门语言编写程序，那么如何使用其他语言编写程序就很容易，因为编写程序的基本技能都是一样的。

那么，使用 Python 学习程序设计的优势在哪里呢？ Python 易于学习，且编程有趣。Python 代码简单、短小，易读、直观，而且功能强大，这样对初学者而言，用它来介绍计算和解决问题是非常有效的。

鼓励初学者通过创建图形学习程序设计。使用 Python 学习程序设计的一个很大原因在于可以从一开始就使用图形来学习程序设计。我们在第 1~6 章使用 Python 内嵌的 Turtle 图形模块，它是一个介绍程序设计基本概念和技术的很好的教学工具。我们在第 9 章介绍 Python 内嵌的 Tkinter，它是开发复杂图形用户界面以及学习面向对象程序设计的一个重要工具。Turtle 和 Tkinter 都相当简单且易于使用。更重要的是，它们都是教授程序设计和面向对象程序设计基础的非常有价值的教学工具。

为了方便教师更灵活地使用本书，我们在第 1~6 章的末尾会讲到 Turtle，所以，可以将它们作为选讲内容跳过去。

本书以问题驱动的方式讲授如何解决问题，这种方式的重点放在问题的解决而不是语法上。我们使用一些涉及范围很广的有趣例子来激发学生学习程序设计的兴趣。鉴于本书的主线是解决问题，这里会介绍解决问题中用到的 Python 语法和库。为了支持问题驱动方式的程序设计教学，本书提供了大量难易程度各异的问题来激发学生的兴趣。为适用于各个专业的学生，这些问题涉及很多应用领域，例如数学、科学、商业、金融管理、游戏、动画和多媒体等。

Python 中的所有数据都是对象。我们从第 3 章开始介绍和使用对象，但是如何定义类将从第 7 章开始。本书首先将重点放在基础上：在编写自定制类之前介绍像选择、循环和函数这样的基本程序设计概念和技术。

教授程序设计的最佳方式是通过实例，而学习程序设计的唯一方法就是通过实践。本书用实例解释基本概念，同时提供了大量不同难度的习题供学生练习。我们的目标是使用大量有趣的例子和习题来教授学生如何解决问题以及如何进行程序设计。

教学特色

本书使用了下面的模块：

- **学习目标**　列出学生应该学会的内容，这样在学完这章之后，学生能够判断自己是否达到这个目标。
- **引言**　提出一个代表性问题，以便学生对该章内容有一个概括了解。
- **关键点**　强调每节中的重要概念。
- **检查点**　提供复习题帮助学生复习相关内容并评估掌握的程度。

- **问题** 通过精心挑选，以一种容易掌握的形式教授问题求解和程序设计的概念。本书使用许多短小的、简单的以及令人兴奋的例子来演示重要的想法。
- **关键术语** 提供对本章重要术语的快速参考。
- **本章总结** 回顾学生应该理解和记住的重要主题，帮助他们加强对该章所学关键概念的理解。
- **测试题** 测试题是在线的，用于学生自我测试对程序设计概念和技术的掌握程度。
- **编程题** 为学生提供应用新技巧的机会。题目的难度等级分为容易（无星号）、适度（*）、困难（**）或具有挑战性（***）。学习程序设计的秘诀就在于练习，练习，再练习。为了达到这个目标，本书提供了大量的练习题。
- **注意、提示和警告** 穿插在整本书中，提供了有价值的建议以及程序开发要点。

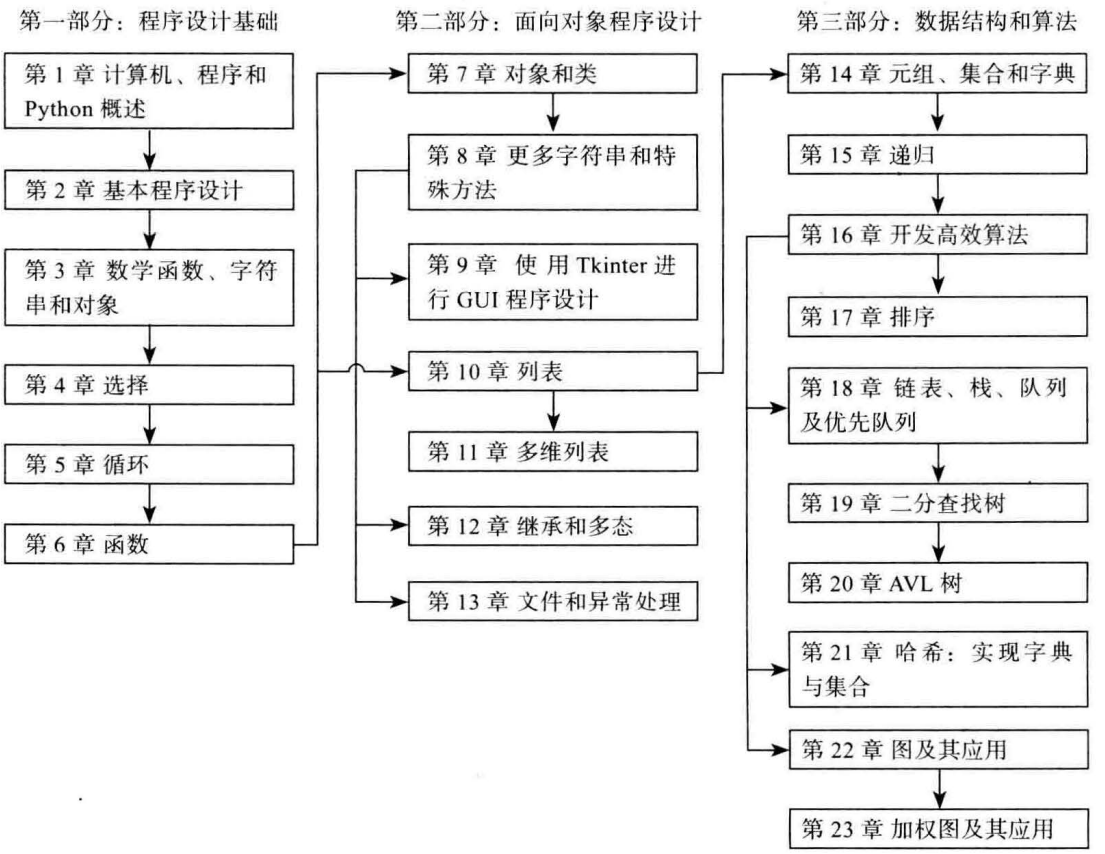
注意：提供关于主题的附加信息并强化重要概念。

提示：教授好的程序设计风格和实践。

警告：帮助学生避免程序设计错误。

灵活的章节顺序

图形是学习程序设计的一个非常有价值的教学工具。本书在第 1 ～ 6 章使用 Turtle 图形，而在书中其他部分使用 Tkinter。但是，教师可以根据需要跳过关于图形的章节或者以后再讨论。下图给出章节之间的相互关系。



注意：第 16 ～ 23 章是配套网站提供的附加章节。

第 10 章可以在第 6 章之后讲解。第 14 章可以在第 10 章之后讲解。

本书的组织结构

全书共分三部分，循序渐进地介绍用 Python 语言进行程序设计的基本知识。前面的章节提供理解程序设计概念的基础知识，并通过简单实例和习题对学生进行指导，随后的章节逐步详细介绍 Python 程序设计，一直到开发复杂的应用程序。

第一部分：程序设计基础（第 1 ～ 6 章）

第一部分是起点，为你学习程序设计做准备。你可以初步了解 Python（第 1 章），并学习基本程序设计技术，包括数据类型、变量、常量、赋值、表达式、运算符、对象以及简单的函数和字符串操作（第 2 ～ 3 章），选择语句（第 4 章），循环（第 5 章），函数（第 6 章）。

第二部分：面向对象程序设计（第 7 ～ 13 章）

这一部分介绍面向对象程序设计。Python 是一种面向对象程序设计语言，它具有抽象、封装、继承和多态等特性，适合编写灵活、模块化和可重用的软件。你将学习面向对象程序设计（第 7 ～ 8 章），使用 Tkinter 进行 GUI 程序设计（第 9 章），列表（第 10 章），多维列表（第 11 章），继承、多态和类设计（第 12 章），以及文件和异常处理（第 13 章）。

第三部分：数据结构和算法（第 14 ～ 15 章和附加章节第 16 ～ 23 章）

本部分介绍典型数据结构课程的主要主题。第 14 章介绍 Python 内嵌的数据结构：元组、集合和字典。第 15 章介绍用递归来编写函数以解决内在递归问题。第 16 ～ 23 章是配套网站的附加章节。第 16 章介绍算法效率以及开发高效算法的常用技术。第 17 章讨论经典的排序算法。第 18 章介绍如何实现链表、队列以及优先队列。第 19 章介绍二分查找树。第 20 章介绍 AVL 树。第 21 章介绍哈希技术。第 22 和 23 章涵盖图算法及其应用。

学生资源网站

学生资源网站（www.cs.armstrong.edu/liang/py）包含下面的资源：

- 复习题的答案。
- 偶数编号编程题的答案。
- 本书例子的源代码。
- 互动的自测题（每章按节组织）。
- 关于 Python IDE、高级主题等补充材料。
- 资源链接。
- 勘误表。

补充材料

本书涵盖了必要的主题，而补充材料则介绍了读者可能感兴趣的主题。本书配套网站中给出下列补充材料：

Part I. General Supplements
- A. Glossary
- B. Installing and Using Python
- C. Python IDLE
- D. Python on Eclipse
- E. Python on Eclipse Debugging
- F. Python Coding Style Guidelines

Part II. Advanced Python Topics
A. Regular Expressions
B. Obtaining Date and Time
C. The `str` Class's `format` Method
D. Pass Arguments from Command Line
E. Database Programming

教师资源网站[⊖]

教师资源网站（www.cs.armstrong.edu/liang/py）包括下面的资源：

- 带交互式按钮的微软 PowerPoint 幻灯片，可以查看全彩、语法项高亮显示的源代码，并且可以在幻灯片状态运行程序。
- 所有复习题和练习题的答案。
- 基于 Web 的测试题产生器。（教师可以从一个超过 800 道题的数据库中选择章节创建测试题。）
- 模拟考试卷。通常，每份模拟考试卷都有四部分：
  - ➢ 多选题或简答题
  - ➢ 纠正编程错误
  - ➢ 跟踪程序
  - ➢ 编写程序
- 项目。通常，每个项目都会给出描述，要求学生分析、设计和实现该项目。

致谢

感谢阿姆斯特朗亚特兰大州立大学给我机会讲授 Python 课程，并支持我将授课内容编写成为教材。教学是我写作这本书的源动力。还要感谢使用本书的教师和学生，他们提出了许多宝贵的意见、建议、错误报告和鼓励。

感谢优秀的评阅人，他们是：

Claude Anderson——罗斯霍曼理工学院

Lee Cornell——明尼苏达州立大学曼凯托分校

John Magee——波士顿大学

Shyamal Mitra——得克萨斯大学奥斯汀分校

Yenumula Reddy——关柏林州立大学

David Sullivan——波士顿大学

Hong Wang——托莱多大学

非常荣幸能和 Pearson 一起工作。感谢组织、生产和推动这个项目的 Tracy Dunkelberger、Marcia Horton、Michael Hirsch、Matt Goldstein、Carole Snyder、Tim Huddleston、Yez Alayan、Jeff Holcomb、Gillian Hall、Rebecca Greenberg 以及他们的同事。

一如既往，特别感谢来自我的妻子 Samantha 的爱、支持和鼓励。

⊖ 关于本书教辅资源，用书教师可与培生教育出版集团北京代表处申请，电话：010-57355169/57355171，电子邮件：service.cn@person.com。——编辑注

出版者的话
译者序
前言

第一部分　程序设计基础

第1章　计算机、程序和Python概述 …… 2
1.1　引言 …… 2
1.2　什么是计算机 …… 3
1.2.1　中央处理器 …… 4
1.2.2　比特和字节 …… 4
1.2.3　内存 …… 5
1.2.4　存储设备 …… 5
1.2.5　输入和输出设备 …… 6
1.2.6　通信设备 …… 7
1.3　程序设计语言 …… 8
1.3.1　机器语言 …… 8
1.3.2　汇编语言 …… 8
1.3.3　高级语言 …… 8
1.4　操作系统 …… 10
1.4.1　控制和管理系统行为 …… 10
1.4.2　调度和分配系统资源 …… 10
1.4.3　调度操作 …… 10
1.5　Python的历史 …… 11
1.6　开始学习Python …… 11
1.6.1　启动Python …… 12
1.6.2　创建Python源代码文件 …… 13
1.6.3　使用Python完成算术运算 …… 14
1.7　程序设计风格和文档 …… 15
1.7.1　恰当的注释和注释风格 …… 15
1.7.2　恰当的空格 …… 15
1.8　程序设计错误 …… 16
1.8.1　语法错误 …… 16
1.8.2　运行时错误 …… 16
1.8.3　逻辑错误 …… 16
1.9　开始学习图形化程序设计 …… 17
1.9.1　绘制图形并给图形添加颜色 …… 17
1.9.2　将笔移到任何位置 …… 19
1.9.3　绘制奥林匹克环标志 …… 20
关键术语 …… 21
本章总结 …… 21
测试题 …… 22
编程题 …… 22
第2章　基本程序设计 …… 25
2.1　引言 …… 25
2.2　编写一个简单的程序 …… 25
2.3　从控制台读取输入 …… 27
2.4　标识符 …… 29
2.5　变量、赋值语句和赋值表达式 …… 29
2.6　同时赋值 …… 31
2.7　定名常量 …… 32
2.8　数值数据类型和运算符 …… 32
2.8.1　运算符/、//和** …… 33
2.8.2　运算符% …… 33
2.8.3　科学记数法 …… 34
2.9　计算表达式和运算符优先级 …… 35
2.10　增强型赋值运算符 …… 35
2.11　类型转换和四舍五入 …… 36
2.12　实例研究：显示当前时间 …… 38
2.13　软件开发流程 …… 39
2.14　实例研究：计算距离 …… 42
关键术语 …… 44
本章总结 …… 44
测试题 …… 45

编程题 …… 45
第3章 数学函数、字符串和对象 …… 50
3.1 引言 …… 50
3.2 常见的Python函数 …… 51
3.3 字符串和字符 …… 54
3.3.1 ASCII码 …… 54
3.3.2 统一码 …… 54
3.3.3 函数ord和chr …… 54
3.3.4 转义序列 …… 55
3.3.5 不换行打印 …… 56
3.3.6 函数str …… 56
3.3.7 字符串连接操作 …… 56
3.3.8 从控制台读取字符串 …… 57
3.4 实例研究：最小数量的硬币 …… 58
3.5 对象和方法简介 …… 59
3.6 格式化数字和字符串 …… 61
3.6.1 格式化浮点数 …… 62
3.6.2 用科学记数法格式化 …… 63
3.6.3 格式化成百分数 …… 63
3.6.4 调整格式 …… 63
3.6.5 格式化整数 …… 63
3.6.6 格式化字符串 …… 64
3.7 绘制各种图形 …… 65
3.8 绘制带颜色和字体的图形 …… 67
关键术语 …… 69
本章总结 …… 69
测试题 …… 69
编程题 …… 69
第4章 选择 …… 73
4.1 引言 …… 73
4.2 布尔类型、数值和表达式 …… 73
4.3 产生随机数字 …… 75
4.4 if语句 …… 76
4.5 实例研究：猜生日 …… 77
4.6 双向if-else语句 …… 80
4.7 嵌套if和多向if-elif-else语句 …… 82
4.8 选择语句中的常见错误 …… 85
4.9 实例研究：计算身体质量指数 …… 87
4.10 实例研究：计算税款 …… 88
4.11 逻辑运算符 …… 90
4.12 实例研究：判定闰年 …… 92
4.13 实例研究：彩票 …… 93
4.14 条件表达式 …… 94
4.15 运算符的优先级和结合方向 …… 95
4.16 检测一个对象的位置 …… 96
关键术语 …… 97
本章总结 …… 97
测试题 …… 98
编程题 …… 98
第5章 循环 …… 107
5.1 引言 …… 107
5.2 while循环 …… 108
5.2.1 实例研究：猜数字 …… 110
5.2.2 循环设计策略 …… 112
5.2.3 实例研究：多道减法题测验 …… 112
5.2.4 根据用户确认控制循环 …… 113
5.2.5 使用哨兵值控制循环 …… 113
5.2.6 输入输出重定向 …… 115
5.3 for循环 …… 116
5.4 嵌套循环 …… 118
5.5 最小化数值错误 …… 119
5.6 实例研究 …… 120
5.6.1 问题：找出最大公约数 …… 120
5.6.2 问题：预测未来学费 …… 121
5.6.3 问题：蒙特卡罗模拟 …… 122
5.7 关键字break和continue …… 123
5.8 实例研究：显示素数 …… 125
5.9 实例研究：随意行走 …… 127
关键术语 …… 129
本章总结 …… 129
测试题 …… 129
编程题 …… 129
第6章 函数 …… 138
6.1 引言 …… 138
6.2 定义一个函数 …… 139

6.3 调用一个函数 …… 140
6.4 带返回值或不带返回值的函数 … 142
6.5 位置参数和关键字参数 …… 145
6.6 通过传引用来传递参数 …… 146
6.7 模块化代码 …… 148
6.8 实例研究：将十进制数转换为十六进制数 …… 150
6.9 变量的作用域 …… 152
6.10 默认参数 …… 154
6.11 返回多个值 …… 155
6.12 实例研究：生成随机ASCII码字符 …… 155
6.13 函数抽象和逐步求精 …… 157
6.13.1 自顶向下设计 …… 158
6.13.2 自顶向下和自底向上的实现 …… 159
6.13.3 实现细节 …… 160
6.13.4 逐步求精的优势 …… 163
6.14 实例研究：可重用图形函数 … 163
关键术语 …… 165
本章总结 …… 165
测试题 …… 165
编程题 …… 166

第二部分 面向对象程序设计

第7章 对象和类 …… 176
7.1 引言 …… 176
7.2 为对象定义类 …… 176
7.2.1 定义类 …… 177
7.2.2 构造对象 …… 178
7.2.3 访问对象成员 …… 179
7.2.4 self参数 …… 179
7.2.5 举例：使用类 …… 180
7.3 UML类图 …… 181
7.4 不变对象和可变对象 …… 184
7.5 隐藏数据域 …… 186
7.6 类的抽象与封装 …… 188
7.7 面向对象的思考 …… 191
关键术语 …… 193
本章总结 …… 194
测试题 …… 194
编程题 …… 194
第8章 更多字符串和特殊方法 …… 198
8.1 引言 …… 198
8.2 str类 …… 198
8.2.1 创建字符串 …… 199
8.2.2 处理字符串的函数 …… 199
8.2.3 下标运算符[] …… 200
8.2.4 截取运算符[start：end] …… 200
8.2.5 连接运算符+和复制运算符* …… 201
8.2.6 in和not in运算符 …… 201
8.2.7 比较字符串 …… 202
8.2.8 迭代字符串 …… 203
8.2.9 测试字符串 …… 203
8.2.10 搜索子串 …… 204
8.2.11 转换字符串 …… 205
8.2.12 删除字符串中的空格 …… 206
8.2.13 格式化字符串 …… 207
8.3 实例研究：校验回文串 …… 208
8.4 实例研究：将十六进制数转换为十进制数 …… 209
8.5 运算符重载和特殊方法 …… 211
8.6 实例研究：Rational类 …… 212
关键术语 …… 216
本章总结 …… 217
测试题 …… 217
编程题 …… 217
第9章 使用Tkinter进行GUI程序设计 … 223
9.1 引言 …… 223
9.2 开始使用Tkinter …… 223
9.3 处理事件 …… 224
9.4 小构件类 …… 226
9.5 画布 …… 231
9.6 几何管理器 …… 233
9.6.1 网格管理器 …… 234

9.6.2 包管理器 …… 234
9.6.3 位置管理器 …… 235
9.7 实例研究：贷款计算器 …… 236
9.8 显示图像 …… 238
9.9 菜单 …… 239
9.10 弹出菜单 …… 241
9.11 鼠标、按键事件和绑定 …… 243
9.12 动画 …… 245
9.13 滚动条 …… 248
9.14 标准对话框 …… 249
关键术语 …… 250
本章总结 …… 251
测试题 …… 251
编程题 …… 251
第10章 列表 …… 258
10.1 引言 …… 258
10.2 列表基础 …… 259
10.2.1 创建列表 …… 259
10.2.2 列表是一种序列类型 …… 260
10.2.3 列表使用的函数 …… 260
10.2.4 下标运算符[] …… 260
10.2.5 列表截取 [start:end] …… 261
10.2.6 +、*和in/not in运算符 …… 262
10.2.7 使用for循环遍历元素 …… 262
10.2.8 比较列表 …… 263
10.2.9 列表解析 …… 263
10.2.10 列表方法 …… 264
10.2.11 将字符串分成列表 …… 265
10.2.12 输入列表 …… 265
10.2.13 对列表移位 …… 266
10.2.14 简化代码 …… 266
10.3 实例研究：乐透数 …… 268
10.4 实例研究：一副扑克牌 …… 270
10.5 扑克牌图形用户界面 …… 272
10.6 复制列表 …… 273
10.7 将列表传递给函数 …… 274
10.8 从函数返回一个列表 …… 276
10.9 实例研究：统计每个字母的出现次数 …… 277
10.10 查找列表 …… 279
10.10.1 线性查找法 …… 280
10.10.2 二分查找法 …… 280
10.11 排序列表 …… 282
10.11.1 选择排序 …… 282
10.11.2 插入排序 …… 284
10.12 实例学习：弹球 …… 286
关键术语 …… 288
本章总结 …… 288
测试题 …… 289
编程题 …… 289
第11章 多维列表 …… 298
11.1 引言 …… 298
11.2 处理二维列表 …… 299
11.2.1 使用输入值初始化列表 …… 299
11.2.2 使用随机数初始化列表 …… 299
11.2.3 打印列表 …… 299
11.2.4 对所有元素求和 …… 300
11.2.5 按列求和 …… 300
11.2.6 找出和最大的行 …… 300
11.2.7 随意打乱 …… 301
11.2.8 排序 …… 301
11.3 将二维列表传递给函数 …… 302
11.4 问题：给多选题评分 …… 303
11.5 问题：找出距离最近的点对 …… 304
11.6 图形用户界面：找出距离最近的点对 …… 306
11.7 问题：数独 …… 307
11.8 实例研究：数独图形用户界面 …… 310
11.9 多维列表 …… 312
11.9.1 问题：每日温度和湿度 …… 313
11.9.2 问题：猜生日 …… 314
关键术语 …… 315
本章总结 …… 315
测试题 …… 316
编程题 …… 316
第12章 继承和多态 …… 330
12.1 引言 …… 330
12.2 父类和子类 …… 330
12.3 覆盖方法 …… 335

12.4 object类 ····· 336
12.5 多态和动态绑定 ····· 339
12.6 isinstance函数 ····· 340
12.7 实例研究：可重用时钟 ····· 342
12.8 类之间的关系 ····· 347
12.8.1 关联 ····· 347
12.8.2 聚合和组合 ····· 348
12.9 实例研究：设计Course类 ····· 349
12.10 为栈设计类 ····· 350
12.11 实例研究：FigureCanvas类 ··· 352
关键术语 ····· 355
本章总结 ····· 355
测试题 ····· 355
编程题 ····· 355
第13章 文件和异常处理 ····· 364
13.1 引言 ····· 364
13.2 文本输入和输出 ····· 364
13.2.1 打开一个文件 ····· 365
13.2.2 写入数据 ····· 366
13.2.3 测试文件的存在性 ····· 367
13.2.4 读数据 ····· 367
13.2.5 从文件读取所有数据 ····· 370
13.2.6 追加数据 ····· 371
13.2.7 读写数值数据 ····· 371
13.3 文件对话框 ····· 373
13.4 实例研究：统计文件中的字符个数 ····· 375
13.5 从网站上获取数据 ····· 376
13.6 异常处理 ····· 378
13.7 抛出异常 ····· 381
13.8 使用对象处理异常 ····· 383
13.9 定义自定制异常类 ····· 384
13.10 使用Pickling进行二进制IO ···· 388
13.10.1 卸载和加载对象 ····· 388
13.10.2 检测文件末尾 ····· 389
13.11 实例研究：地址簿 ····· 390
关键术语 ····· 393
本章总结 ····· 393
测试题 ····· 394
编程题 ····· 394

第三部分 数据结构和算法

第14章 元组、集合和字典 ····· 398
14.1 引言 ····· 398
14.2 元组 ····· 399
14.3 集合 ····· 401
14.3.1 创建集合 ····· 401
14.3.2 操作和访问集合 ····· 402
14.3.3 子集和超集 ····· 402
14.3.4 相等性测试 ····· 403
14.3.5 集合运算 ····· 403
14.4 比较集合和列表的性能 ····· 407
14.5 实例研究：统计关键字 ····· 408
14.6 字典 ····· 409
14.6.1 创建一个字典 ····· 410
14.6.2 添加、修改和获取值 ····· 410
14.6.3 删除条目 ····· 410
14.6.4 循环条目 ····· 411
14.6.5 len函数 ····· 411
14.6.6 检测一个关键字是否在字典中 ····· 411
14.6.7 相等性检测 ····· 411
14.6.8 字典方法 ····· 412
14.7 实例研究：单词的出现次数 ···· 413
关键术语 ····· 415
本章总结 ····· 415
测试题 ····· 416
编程题 ····· 416
第15章 递归 ····· 419
15.1 引言 ····· 419
15.2 实例研究：计算阶乘 ····· 420
15.3 实例研究：计算斐波那契数 ···· 422
15.4 使用递归解决问题 ····· 424
15.5 递归辅助函数 ····· 426
15.5.1 选择排序 ····· 427

15.5.2 二分查找 ····· 427
15.6 实例研究：求出目录的大小 ···· 428
15.7 实例研究：汉诺塔 ····· 430
15.8 实例研究：分形 ····· 432
15.9 实例研究：八皇后 ····· 435
15.10 递归和迭代 ····· 437
15.11 尾递归 ····· 438
关键术语 ····· 439
本章总结 ····· 439
测试题 ····· 439
编程题 ····· 439

附录A Python关键字 ····· 446
附录B ASCII字符集 ····· 447
附录C 数制系统 ····· 448
*第16章 开发高效算法
*第17章 排序
*第18章 链表、栈、队列及优先队列
*第19章 二分查找树
*第20章 AVL树
*第21章 哈希：实现字典与集合
*第22章 图及其应用
*第23章 加权图及其应用

* 第 16 ～ 23 章是附加章节，有需要的读者从 www.personhighered.com/liang 上付费购买。——编辑注

第一部分

Introduction to Programming Using Python

程序设计基础

第 1 章
Introduction to Programming Using Python

计算机、程序和 Python 概述

学习目标

- 演示对计算机硬件、程序和操作系统的基本理解（第 1.2 ～ 1.4 节）。
- 描述 Python 的历史（第 1.5 节）。
- 解释 Python 程序的基本语法（第 1.6 节）。
- 编写和运行一个简单的 Python 程序（第 1.6 节）。
- 解释恰当的程序设计风格和文档的重要性，并提供相应的实例（第 1.7 节）。
- 解释语法错误、运行时错误和逻辑错误之间的区别（第 1.8 节）。
- 使用 Turtle 创建一个基本的图形程序（第 1.9 节）。

1.1 引言

关键点：本书的中心主题就是学习如何编写程序来解决问题。

本书是关于程序设计的。那么，什么是程序设计呢？程序设计是指创建（或开发）软件，这里的软件又称为程序。使用更基本的术语来讲，软件包含的就是一些指令，这些指令告诉计算机或者计算设备应该做什么。

软件就在你的周围，甚至在一些你可能认为不会需要它的设备中。当然，你期望看到的是在个人计算机里找到软件并且使用它，但其实软件在运行的飞机、汽车、手机甚至烤箱上也发挥着作用。在个人计算机中，你可以使用字处理器来编写文档，使用网页浏览器来探索互联网，也可以使用电子邮件程序来发送消息。这些程序都是软件的实例。软件开发者借助程序设计语言这一强大工具来创建软件。

本书介绍如何使用 Python 程序设计语言创建程序。程序设计语言有很多种，其中一部分已经有几十年的历史。每种语言都是为了实现特定目标而发明的——例如：增强前一种语言，或者提供给程序开发者一个全新的或独特的工具集。了解有这么多可用的程序设计语言，你很自然地就会想知道哪个是最好的。但是，实际情况是，没有“最好的”语言。每个语言都有它自己的长处和短处。有经验的程序设计者知道某种语言可能适用于某些情况，而另一种语言可能更适合其他的情况。因此，老练的程序员会试图尽最大努力掌握尽可能多种类的程序设计语言，以便有能力驾驭一个大型的软件开发工具“军火库”。

如果使用一种语言学习编写程序，那么你应该会发现其实学习其他语言也很容易。关键是学习如何使用程序设计方法解决问题，这是本书的主要主题。

你将开始一段令人兴奋的旅程：学习如何编写程序。开始学习之前，我们回顾一下计算机基础、程序以及操作系统等知识是很有帮助的。如果你已经对 CPU、内存、磁盘、操作系统以及程序设计语言等术语非常熟悉，可以跳过第 1.2 到 1.4 节之间的内容。

1.2　什么是计算机

关键点：计算机是存储和处理数据的电子设备。

计算机包括软件和硬件。通常，硬件包括计算机上能看到的物理元素，而软件提供控制硬件并让硬件执行特定任务的不可见的指令。学习一种程序设计语言并不一定需要知道计算机的硬件知识，但是它可以帮助你更好地理解程序的指令在计算机和它的组件上所起的效果。本节介绍计算机硬件组件以及它们的功能。

一台计算机包括下面的主要硬件组件（如图 1-1 所示）。

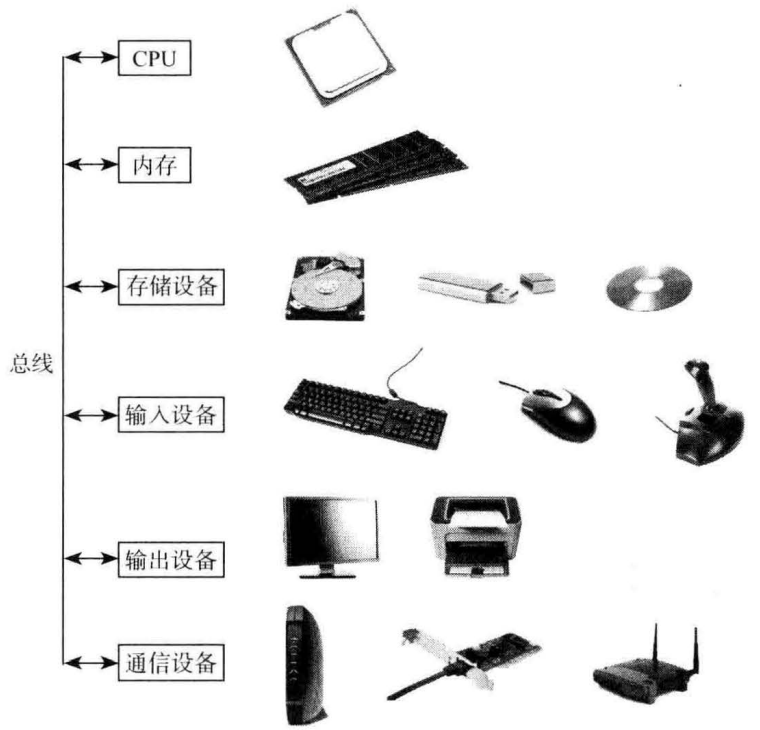


图 1-1　一台计算机包括 CPU、内存、存储设备、输入设备、输出设备和通信设备

- 中央处理器（CPU）
- 内存（主存储器）
- 存储设备（例如：磁盘和光盘）
- 输入设备（例如：鼠标和键盘）
- 输出设备（例如：显示器和打印机）
- 通信设备（例如：调制解调器和网络接口卡）

计算机的组件是通过一个被称作总线的子系统互联的。你可以认为总线是一套运行在计算机组件之间的公路系统，数据和电信号沿着总线从计算机中的一个部分传送到另一个部分。在个人计算机中，总线被内嵌在计算机主板上，主板是将计算机的所有部件连接在一起的电路板，如图 1-2 所示。

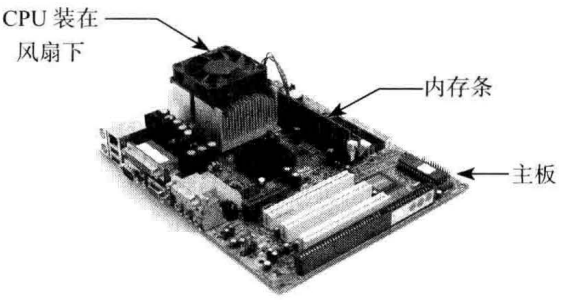


图 1-2　主板连接计算机的各个部件

1.2.1　中央处理器

中央处理器（Central Processing Unit，CPU）是计算机的大脑。它从内存中获取指令然后执行这些指令。CPU 通常由两个组件组成：控制单元（control unit）和算术逻辑单元（arithmetic/logic unit）。控制单元用来控制和协调除 CPU 之外其他组件的动作。算术逻辑单元用来完成数值运算（加法、减法、乘法、除法）以及逻辑运算（比较）。

现在的 CPU 都是内嵌在一块小小的硅半导体芯片上，这块芯片上有数百万个被称作晶体管的小电子开关来处理信息。

每台计算机都有一个内部时钟，该时钟会以一个稳定的速度发射电子脉冲。这些脉冲用于控制和同步各种操作的步调。时钟速度越快，给定时间段内执行的指令就越多。时钟速度的计量单位是赫兹（hertz，Hz），1 赫兹相当于每秒 1 个脉冲。20 世纪 90 年代计算机的时钟速度是以兆赫（MHz）来表示的（1 兆赫兹就是 100 万赫兹），但是随着 CPU 速度的不断提高，现在计算机的时钟速度通常是以千兆赫（gigaherts，GHz）来表示的。Intel 公司最新的处理器运行速度是 3 千兆赫（GHz）左右。

CPU 最初被开发出来时只有一个核。核（core）是处理器中完成读取指令和执行指令的部分。为了提高 CPU 的处理能力，芯片制造商现在生产出来的 CPU 都有多个核。多核 CPU 是一个单独的组件，它具有两个或多个独立的处理器。现在消费者的计算机通常都有两个、三个甚至四个独立的核。相信不久后，市场上就会提供有几十个甚至几百个核的 CPU。

1.2.2　比特和字节

在讨论内存之前，让我们先看看在计算机中是如何存储信息（数据和程序）的。

实际上，一台计算机除了一系列开关以外什么都没有。每个开关都以两种状态存在：开或关。在计算机中存储信息其实就是简单地将一系列开关设置为开或关。如果这个开关是打开状态，那它的值就是 1。如果这个开关是关闭状态，那它的值就是 0。这些 0 和 1 都被解释为二进制数系统中的数字，并称为比特（二进制数）。

计算机中最小的存储单元是字节。一个字节包含 8 个比特。一个像 3 这样的小数字可以被存储为一个单一的字节。为了存储在单个字节中放不下的某个字节，计算机会使用多个字节存储。

各种各样的数据，例如：数字和字符，都被编码成一个字节序列。作为一个程序员，你无需担心数据的编码和解码过程，它们都是由计算机系统基于编码表来自动完成的。编码表是一套规则，这些规则用于控制计算机如何将字符、数字和符号翻译成计算机真正能够使用的数据。大多数规则会将每个字符翻译成一个预定义的数值字符串。例如：在流行的 ASCII 码中，字符 C 被表示为一个字节 01000011。

计算机的存储容量是以字节为单位的，如下所示：

- 千字节（kilobyte，KB）大约是 1000 字节。
- 兆字节（megabyte，MB）大约是 100 万字节。
- 千兆字节或吉字节（gigabyte，GB）大约是 10 亿字节。
- 太字节（百万兆字节）(terabyte，TB）大约是万亿字节。

一页 Word 文档通常会占 20KB，所以 1MB 可以存储 50 页的文档而 1GB 可以存储 50 000 页文档。一部两小时的高分辨率电影通常会占 8GB，所以存储 20 部电影需要 160GB。

1.2.3　内存

计算机的内存由多个有序的字节序列构成，这些字节序列用来存储程序以及这个程序要处理的数据。你可以将内存看作是计算机执行程序的工作区。程序和数据必须在被 CPU 执行之前放在计算机的内存中。

内存中的每个字节都有一个唯一的地址，如图 1-3 所示。地址用来定位存储和获取数据的字节。因为可以以任意顺序访问内存中的字节，所以内存又被称为*随机访问内存*（RAM）。

现在的个人计算机通常都有至少 1GB 的 RAM，但是安装时它们通常多达 2 到 4GB。一般来讲，一台计算机拥有的 RAM 越多，它的运行速度越快，但是对这个简单的经验法则是有限制的。

内存字节永远非空，但是它的原始内容可能对程序毫无意义。一旦有新的内容放入内存，那么内存当前的内容就会丢失。

像 CPU 一样，内存是内置在硅半导体芯片上的，这些芯片的表面上嵌有数百万个静态管。和 CPU 芯片比较，内存芯片没那么复杂，更慢也没那么昂贵。

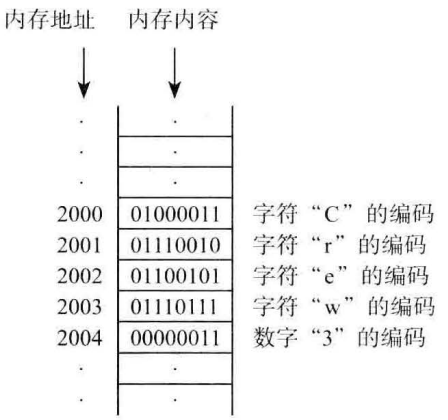


图 1-3　内存在某个独特的内存位置存储数据和程序指令，每个内存位置可以存储一个字节的数据

1.2.4　存储设备

计算机的内存存储数据并不稳定：一旦断开系统电源，所有存储（也可以称为保存）在内存中的信息都会丢失。程序和数据被永久地保存在*存储设备*上，当计算机真的要用到它们的时候再被移到内存中，内存的执行速度还是比永久存储设备快得多。

存储设备主要有三种类型：

- 磁盘驱动器
- 光盘驱动器（CD 和 DVD）
- USB 闪存

*驱动器*是操作像磁盘和 CD 这些介质的设备。存储介质就是存储数据或程序指令的地方。驱动器从这些介质读取数据并且向这些介质写入数据。

1. 磁盘

一台计算机通常至少会有一个硬盘驱动器（如图 1-4 所示）。*硬盘驱动器*用来永久地存储数据和程序。比较新的计算机会有能存储 200GB 到 800GB 数据的硬盘。硬盘驱动器通常安装在计算机内部，当然也可以使用移动硬盘。

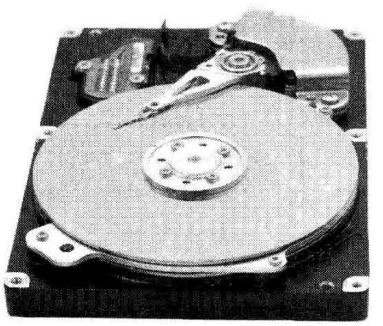


图 1-4　硬盘是一种能够永久保存程序和数据的设备

2. CD 和 DVD

CD 的全称是致密的盘片。光盘驱动器的类型有两种：只读光盘（CD-R）和可擦写光盘（CD-RW）。只读光盘只能用于存储那些永久只读的信息：内容一旦被记录到光盘上，用户是不能修改它们的。可擦写光盘可以像硬盘一样使用，也就是说，可以向这类光盘写入数据，还可以用新数据覆盖这些数据。一张光盘的

容量可以达到700MB。大多数新型的个人电脑都安装了可擦写光驱，它既支持只读光盘也支持可擦写光盘。

DVD的全称是数字化多功能碟片或者数字化视频磁盘。DVD和CD看起来很像，可以使用它们来存储数据。一张DVD上可以保存的信息要比一张CD保存的信息多，一张标准DVD的存储容量是4.7GB。像CD一样，DVD也有两种类型：DVD-R（只读）和DVD-RW（可重写）。

3. USB闪存驱动器

通用串行总线（USB）连接器允许用户将多种外部设备连接到计算机。可以使用USB来将打印机、数字照相机、外接硬盘驱动器，以及其他设备连接到计算机上。

USB闪存驱动器（flash drive）是用于存储和传输数据的设备。闪存驱动器很小——大约就是一包口香糖的大小，如图1-5所示。它就像移动硬盘一样，可以插入计算机的USB端口。USB闪存驱动器目前可用的最大存储容量能够达到256GB。

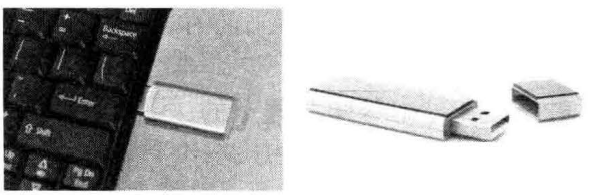


图1-5　USB闪存驱动器是很受欢迎的存储数据的便携设备

1.2.5　输入和输出设备

用户是通过输入和输出设备与计算机进行通信的。最常见的输入设备是键盘（keyboard）和鼠标（mouse）。最常见的输出设备是显示器（monitor）和打印机（printer）。

1. 键盘

计算机键盘是一个用于输入的设备，典型的键盘如图1-6所示。精简的键盘没有数字小键盘。

图1-6　计算机键盘由按键构成，这些按键负责将输入信息发送给计算机

功能键（function key）位于键盘的最上边一排，而且都是以F为前缀顺序排列的数字。它们的功能取决于当前使用的软件。

修饰符键（modifier key）是特殊键（例如：Shift、Alt和Ctrl键），当它和另一个键组合在一起同时按下时，就会改变另一个键的常用功能。

数字小键盘（numeric keypad）位于大多数键盘的右边，是为了快速输入数字的一套独立按键集合，形式上很像一个计算器。

方向键（arrow key）位于主键盘和数字小键盘之间，在很多程序中用于上下左右地移动光标。

插入键（Insert）、删除键（Delete）、向上翻页键（Page Up）和向下翻页键（Page Down）都用在字处理和其他程序中，用来在字处理过程中实现插入文本和对象、删除文本和对象以及向上翻页和向下翻页的功能。

2. 鼠标

鼠标（mouse）是定点设备，用来在屏幕上移动被称作光标的图形指针（通常是一个箭头的形状），或者用于单击屏幕上的对象（例如：按钮）来触发它以完成这个动作。

3. 显示器

显示器（monitor）显示信息（文本和图形）。屏幕分辨率和点距决定显示器的质量。

屏幕分辨率（screen resolution）指定显示器设备水平尺寸和垂直尺寸上像素的个数。像素（“图像元素”的简称）就是在屏幕上构成图像的小点。对于一个 17 英寸的屏幕，分辨率一般为 1024 像素宽 768 像素高。分辨率可以手工设置。分辨率越高，图像就越锐化和清晰。

点距（dot pitch）是指像素之间以毫米为单位的距离。点距越小，显示越清晰。

1.2.6　通信设备

计算机可以通过像拨号调制解调器（调制器 / 解调器）、DSL 或光缆调制解调器、有线网络接口卡或无线适配器等这样的通信设备来连接网络。

- 拨号调制解调器使用电话线并且以高达 56 000bps（每秒比特）的速度传送数据。
- 数字用户线（DSL）也是使用标准电话线来进行连接，但是它可以以比标准拨号调制解调器快 20 倍的速度传送数据。
- 光缆调制解调器使用由光缆公司维护的有线电视线，而且它通常比 DSL 快。
- 网络接口卡（NIC）是一个将计算机连接到局域网（LAN）的设备，如图 1-7 所示。LAN 通常用在大学、企业和政府部门。一个高速的 NIC 被称作 1000BaseT，它可以以每秒 10 亿比特的速率传送数据。
- 无线网络现在在家庭、企业和学校异常流行。现在出售的每一台笔记本电脑都安装有无线适配器，它可以将计算机连接到局域网络或互联网。

注意：检查点问题的答案在配套网站上。

检查点

1.1　什么是硬件？什么是软件？

1.2　罗列出计算机的五个主要硬件组件。

1.3　缩写“CPU”表示什么？

1.4　CPU 的速度使用什么单位表示？

1.5　比特是什么？字节是什么？

1.6　内存是干什么的？RAM 表示什么？为什么内存被称为 RAM？

1.7　用于表示内存大小的单位是什么？

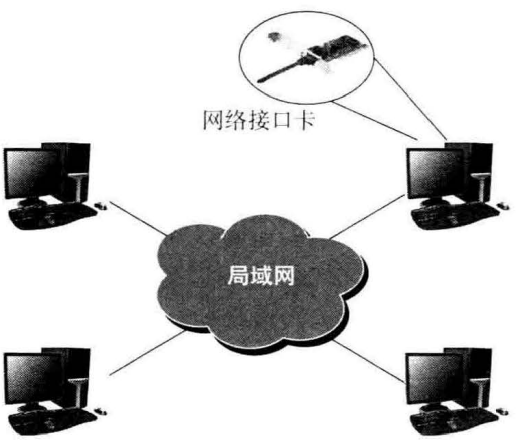


图 1-7　局域网连接相互间距离较近的计算机

1.8　用于表示磁盘大小的单位是什么？

1.9　内存和存储设备最主要的区别是什么？

1.3　程序设计语言

关键点：计算机程序，又称为软件，是告诉计算机要做什么的指令集。

计算机并不理解人类的语言，所以程序必须用计算机使用的语言来书写。现在有几百种程序设计语言，开发它们对人们来说可以让程序设计过程更加简单。但是，所有的程序必须被转换成计算机能够理解的语言。

1.3.1　机器语言

计算机自己的语言（会因计算机的种类不同而有所不同）是它的机器语言——一套内嵌在计算机内的原始指令集。这些指令以二进制代码的形式存在，所以如果给计算机一条用它自己的语言编写的程序，必须输入二进制码的指令。例如：要对两个数字做加法，就必须编写一条二进制码的指令，如下所示：

```
1101101010011010
```

1.3.2　汇编语言

用机器语言进行程序设计是一个繁琐的过程。而且，用机器语言编写的程序非常难以读懂，也很难修改。因此，在计算机发展的早期人们就发明了汇编语言作为机器语言的一个替代品。汇编语言使用一种简短的描述性单词（称为助记符）来表示每个机器语言指令。例如：助记符 add 表示数字的加法而 sub 表示数字的减法。要将数字 2 和 3 进行相加并得到结果，可能要编写一条如下所示的汇编代码的指令：

```
add 2, 3, result
```

开发汇编语言是为了让程序设计更加容易。但是，因为计算机不能理解汇编语言，所以要使用另一种程序——称为汇编器——将汇编语言程序翻译成机器代码，如图 1-8 所示。

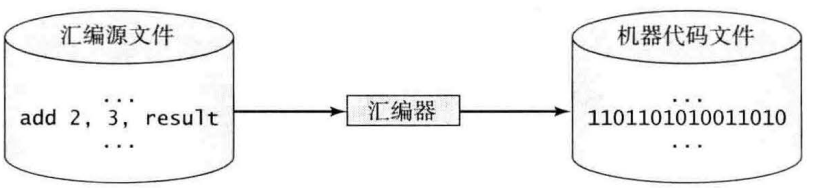


图 1-8　汇编器将汇编语言指令翻译成机器码

用汇编语言编写代码比用机器语言编写代码更加容易。但是，用汇编语言编写代码仍旧是很繁琐的。用汇编语言编写的每条指令本质上讲都对应到机器代码编写的一条指令。用汇编语言编写代码需要知道 CPU 是如何工作的。汇编语言被称为低级语言，因为汇编语言在本质上是接近机器语言的，而且它是不独立于机器的。

1.3.3　高级语言

20 世纪 50 年代，出现了被称为高级语言的新一代的程序设计语言。它们是独立于平台的，也就是说，可以用高级语言编写程序并让它在不同类型的机器上运行。高级语言很像英

语，并且易于学习和使用。高级程序设计语言编写的指令称为语句。例如：这里是一条用于计算半径为 5 的圆的面积的高级语言语句：

```
area = 5 * 5 * 3.14159
```

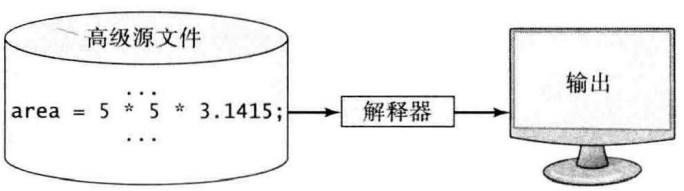
现在有很多种高级程序设计语言，而且每一种语言都是为了特定目的而设计的。表 1-1 罗列出一些流行的高级语言。

表 1-1　流行的高级程序设计语言

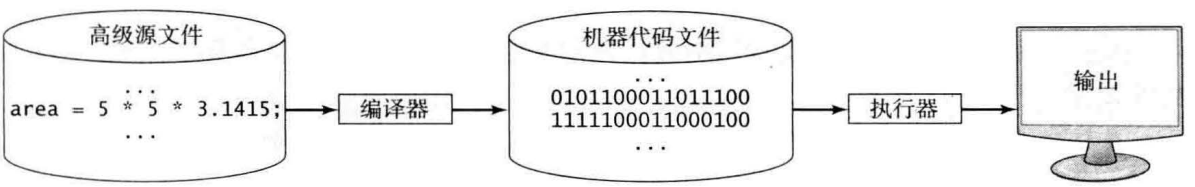
语言	描述
Ada	以 Ada Lovelace 命名，她是在机械的通用计算机上工作的。Ada 语言是为国防部开发的，因此它也主要用于国防项目
Basic	初学者的通用符号指令代码。它是为了初学者易学易用而设计的
C	Bell 实验室开发。C 结合了汇编语言的强大功能以及高级语言的易用性和可移植性
C++	C++ 是基于 C 的面向对象语言
COBOL	面向商业的通用语言，用在商业应用上
FORTRAN	公式翻译。在科学和数学应用中很流行
Java	Sun 公司开发，现在是 Oracle 的一部分，广泛应用于开发平台无关的因特网程序
Pascal	以 Blaise Pascal 命名，他是 17 世纪计算机的先驱。它是一种主要用于讲授程序设计的简单、结构化的通用语言
Python	一种编写短小程序的通用脚本语言
Visual Basic	由微软开发，可以用于快速地开发基于窗口的应用程序

使用高级语言编写的程序称为源程序或源代码。因为计算机不能理解源程序，所以源程序必须被翻译成可执行的机器代码。使用另一个称为解释器或编译器的程序设计工具来完成这个翻译过程。

- 解释器从源代码中读取一条语句，将它翻译成为机器代码或者虚拟机代码，然后立即执行它，如图 1-9a 所示。注意：源代码中的一条语句可以被翻译成几条机器指令。
- 编译器将整个源代码翻译成一个机器代码文件，然后执行这个机器代码文件，如图 1-9b 所示。



a）解释器翻译和执行程序时，一次一句



b）编译器将整个源程序翻译成机器语言文件来执行

图　1-9

使用解释器运行 Python 代码。大多数其他程序设计语言使用编译器进行处理。

检查点

1.10　CPU 能够理解的是什么语言？

1.11　什么是汇编语言？

1.12　什么是汇编器？

1.13　什么是高级程序设计语言？

1.14　什么是源程序？

1.15　什么是解释器？

1.16　什么是编译器？

1.17　解释语言和编译语言之间的区别是什么？

1.4　操作系统

关键点：操作系统（OS）是计算机上运行的最重要的程序。操作系统管理和控制计算机的动作。

一般功能的计算机上流行的操作系统有微软 Windows、Mac OS 以及 Linux。如果不在计算机上安装和运行操作系统，那么像网页浏览器或者字处理器这样的应用程序就不能运行。图 1-10 显示了硬件、操作系统、应用软件和用户之间的相互关系。

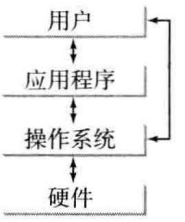


图 1-10　用户和应用程序通过操作系统访问计算机硬件

操作系统的主要任务是：

- 控制和管理系统行为
- 调配和分配系统资源
- 调度操作

1.4.1　控制和管理系统行为

操作系统执行基本的任务，例如：识别来自键盘的输入，将输出结果发送给监视器，管理存储设备上的文件和文件夹，控制像磁盘驱动器和打印机这样的外部设备。操作系统还必须确保同时工作的不同程序和不同用户之间不会相互干扰。除此之外，操作系统还要负责安全问题，确保未经授权的用户和程序不能访问这个系统。

1.4.2　调度和分配系统资源

操作系统负责决定一个程序需要哪些计算机资源（例如：CPU 时间、内存空间、磁盘、输入和输出设备）以及调度和分配这些资源来运行这个程序。

1.4.3　调度操作

操作系统负责调度程序的各种行为以充分利用系统资源。现在的很多操作系统都支持多程序设计、多线程以及多进程以提高系统性能。

多程序设计允许多个程序共享同一个 CPU 同步运行。CPU 比计算机的其他组件更快些。这样，导致大多数时间 CPU 都是空闲的——例如：当等待从磁盘传送数据或者等待其他系统资源响应时。多程序设计操作系统利用这种情况，允许多个程序使用这个 CPU 的闲置时间。例如：多程序设计允许你使用子处理器来编辑文件的同时，你的网页浏览器也可以下载

文件。

多线程允许单个程序同时执行多个任务。例如：字处理程序允许用户编辑文本的同时将它存储到磁盘上。在这个例子中，编辑和存储是同一个应用程序中的两个任务，这两个任务可能是同时运行的。

多进程，或者叫并行处理，使用两个或更多处理器一起完成同时发生的多个子任务，然后将这些子任务的解决方案组合在一起，获取整个任务的解决方案。这就像一个外科手术，几个医生协同工作医治同一个病人。

检查点

1.18　什么是操作系统？罗列出一些流行的操作系统。

1.19　操作系统的主要任务是什么？

1.20　什么是多程序设计、多线程和多进程？

1.5　Python 的历史

关键点：Python 是一种用途广泛、解释性、面向对象的程序设计语言。

Python 是新西兰的 Guido van Rossum 在 1990 年创建的，它以英国流行喜剧“Monty Python 的飞行马戏团”命名。van Rossum 将 Python 开发作为一个嗜好，Python 因其简单、简洁以及直观的语法和扩展库等优势成为工业界和学术界广泛使用的一个流行的程序设计语言。

Python 是一门用途广泛的程序设计语言。这意味着可以使用 Python 为任何程序设计任务编写代码。Python 现在被用在 Google 搜索引擎、NASA 的任务关键项目以及纽约股票交易所的交易处理中。

Python 是解释性的，这表示 Python 代码是被解释器翻译和执行的，每次一句，就像本章早前描述的那样。

Python 是一门面向对象程序设计语言（OOP）。Python 中的数据都是由类所创建的对象。本质上讲类就是一种类型或者某个种类，它能够定义同种类型的对象，这些对象都具有相同的属性以及相同的操作这些对象的方法。面向对象程序设计是开发可重用软件的强大工具。使用 Python 进行面向对象程序设计将从第 7 章开始详细讲解。

现在，Python 是由一个大型的志愿者团队来开发和维护的，你可以从 Python 软件基金会免费获取。Python 的两个版本现在是共存的：Python 2 和 Python 3。使用 Python 3 编写的程序不能在 Python 2 中执行。Python 3 是比较新的版本，但是它不向后兼容 Python 2。这意味着如果你使用 Python 2 的语法编写了一个程序，那它可能无法在 Python 3 解释器中正常工作。Python 提供了一个工具，它可以将 Python 2 所写的代码自动地转换成 Python 3 可以使用的语法。Python 2 最终还是会被 Python 3 所代替。本书教授如何使用 Python 3 来进行程序设计。

检查点

1.21　Python 是解释性的。这是什么意思？

1.22　使用 Python 2 编写的程序可以在 Python 3 中运行吗？

1.23　使用 Python 3 编写的程序可以在 Python 2 中运行吗？

1.6　开始学习 Python

关键点：Python 程序是用 Python 解释器执行的。

我们从编写一个简单的 Python 程序开始，这个程序在控制台上显示消息“Welcome to Python”和“Python is fun”。控制台是一个旧的计算机术语，它是指计算机的文本输入域和显示设备。控制台输入是指从键盘获取输入，而控制台输出是指将输出显示到显示器。

注意：可以在 Windows、UNIX 和 Mac 操作系统上运行 Python。为了获取安装 Python 的信息，可参见配套网站上的补充材料 I.B。

1.6.1　启动 Python

假设已经将 Python 安装在 Windows 操作系统上，在命令行窗口的命令提示符下输入 Python，就可以启动 Python（如图 1-11 所示），或者使用 IDLE（如图 1-12 所示）。IDLE（交互式开发环境）是 Python 的一个集成开发环境（IDE）。可以在 IDLE 中创建、打开、保存、编辑以及运行 Python 程序。你的机器安装了 Python 之后，命令行 Python 解释器和 IDLE 都是可用的。注意：Python（命令行）和 IDLE 都可以通过在 Windows 7 或 Vista 上搜索 Python（Command Line）或 IDLE（Python GUI）直接利用 Windows 开始按钮访问，如图 1-13 所示。

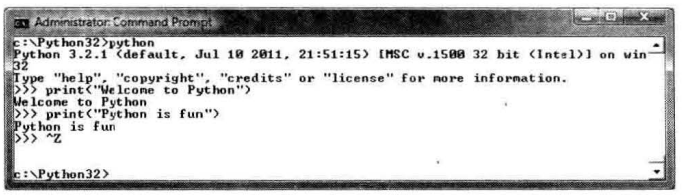


图 1-11　从命令行窗口启动 Python

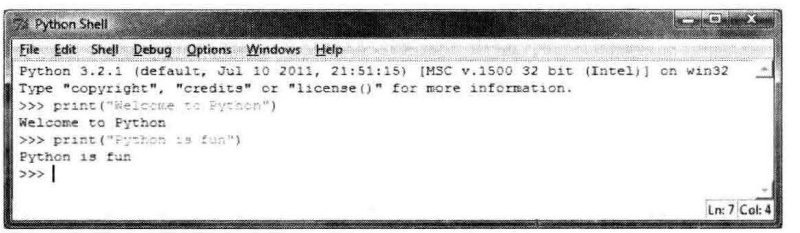


图 1-12　在 IDLE 中使用 Python

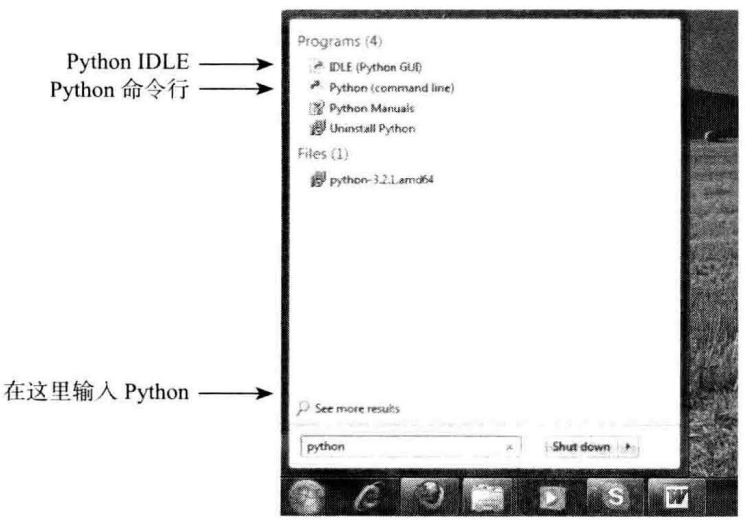


图 1-13　利用开始按钮启动 Python IDLE 和 Python 命令行

启动 Python 之后，你将会看到符号“>>>”。这是 Python 语句提示符，也是你输入 Python 语句的地方。

注意：按照本书中所写的准确输入命令。格式以及其他规则将在本章后面讨论。

现在，输入“print("Welcome to Python")”然后按回车键。控制台上会出现字符串“Welcome to Python”，如图 1-11 所示。字符串是一个程序设计术语，它表示一个字符序列。

注意：Python 字符串两边需要使用双引号或单引号，将它们和其他代码区分开来。就像你在输出中看到的那样，Python 不显示这些引号。

print 语句是 Python 的固有函数之一，可以用它在控制台上显示字符串。函数用来完成一系列动作。print 函数的动作就是在控制台上显示一条消息。

注意：在程序设计专业词汇中，当你使用一个函数时，可以说“调用一个函数”。

接下来，输入“print("Python is fun")”然后按回车键。控制台上会出现字符串“Python is fun”，如图 1-11 所示。可以在语句提示符“>>>”处输入附加语句。

注意：要退出 Python，按 Ctrl+Z 组合键然后再按回车键。

1.6.2　创建 Python 源代码文件

在语句提示符“>>>”处输入 Python 语句是很方便的，但是语句并未被保存。为了保存语句以便今后使用，可以创建一个文本文件来存储语句，然后使用下面的命令执行文件中的语句：

```
python filename.py
```

可以使用像记事本这样的文本编辑器来创建文本文件。这里的文本文件 *filename* 称为 Python 源文件或脚本文件。习惯上，Python 文件的扩展名为 .py。

从脚本文件来运行 Python 程序称为以脚本模式运行 Python。在语句提示符“>>>”后键入一条语句，然后执行它，称为以交互模式运行 Python。

注意：除了在命令行窗口开发和运行 Python 程序之外，也可以在 IDLE 中创建、保存、修改和运行 Python 脚本。有关使用 IDLE 的消息，参见配套网站上的补充材料 I.C。教师可能会要求你使用 Eclipse。Eclipse 是一个流行的交互式开发环境，用来快速开发程序，编辑、运行、调试和在线帮助都集成在一个图形用户界面中。如果你想使用 Eclipse 开发 Python 程序，参见配套网站上的补充材料 I.D。

程序清单 1-1 给出一个 Python 程序，该程序显示消息“Welcome to Python”和“Python is fun”。

程序清单 1-1　Welcome.py

```
1  # Display two messages
2  print("Welcome to Python")
3  print("Python is fun")
```

在本教材中，显示行号是用于参考的，它们不是程序的一部分。所以，在你的程序中不要输入行号。

假设语句存储在一个名为 Welcome.py 的文件中。为了运行这个程序，在命令提示符后输入 python Welcome.py，如图 1-14 所示。

图 1-14　从命令行窗口运行 Python 脚本文件

在程序清单 1-1 中，第 1 行是一条注释，标注这个

程序是什么以及这个程序是如何构建的。注释有助于程序员理解程序。它们不是程序设计语句，所以可以被解释器忽略。在 Python 中，每行注释前都会加一个井号（#），称为行注释，也可以通过在一行或多行上使用三个连续的单引号（'''）括起来达到段注释的目的。当 Python 解释器看到 # 时，就会忽略 # 之后和它在同一行的所有文本。当 Python 解释器看到 ''' 时，就会扫描找到下一个 '''，然后忽略这三个引号之间的任何文本。下面是注释的例子：

```
# This program displays Welcome to Python

''' This program displays Welcome to Python and
    Python is fun
'''
```

下面介绍 Python 中的缩进问题。注意：输入语句是从新行的第一列开始。如果输入的程序如下所示，那么 Python 解释器将会报告错误：

```
# Display two messages
  print("Welcome to Python")
print("Python is fun")
```

不要在语句末尾放置任何标点符号。例如：如果输入下面的代码，那么 Python 解释器将会报错：

```
# Display two messages
print("Welcome to Python").
print("Python is fun"),
```

Python 程序是区分大小写的。例如：在程序中用 Print 替换 print 就会出错。

你已经在程序中看到好几个特殊字符（#、''、()），几乎所有的程序都会用到它们。表 1-2 总结了它们的用途。

表 1-2　特殊字符

字符	名称	描述
()	左括号和右括号	和函数一起使用
#	# 号	表示注释行
" "	双引号	将字符串（即字符序列）括起来
''' '''	段注释	将一段注释括起来

程序清单 1-1 中的程序显示两条消息。一旦你理解了这个程序，就可以很容易地将它扩展为显示更多的消息。例如：可以改写这个程序显示三条信息，如程序清单 1-2 所示。

程序清单 1-2　WelcomeWithThreeMessages.py

```
1  # Display three messages
2  print("Welcome to Python")
3  print("Python is fun")
4  print("Problem Driven")
```

```
Welcome to Python
Python is fun
Problem Driven
```

1.6.3　使用 Python 完成算术运算

Python 程序可以完成各种类型的算术运算，并且显示结果。为了显示两个数 x 和 y 的加法、减法、乘法和除法，使用下面的代码：

```
print(x + y)
print(x - y)
print(x * y)
print(x / y)
```

程序清单 1-3 显示一个程序实例，它计算 $\frac{10.5+2\times3}{45-3.5}$ 然后打印它的结果。

程序清单 1-3 ComputeExpression.py

```
1  # Compute expression
2  print((10.5 + 2 * 3) / (45 - 3.5))
```

```
0.397590361446
```

就像你所看到的，将算术表达式翻译成 Python 表达式是一个简单的过程。我们将在第 2 章进一步讨论 Python 表达式。

检查点

1.24　可以用两种模式运行 Python。解释这两种模式。

1.25　Python 区分大小写吗？

1.26　按照惯例，Python 源文件的扩展名是什么？

1.27　运行 Python 源文件的命令是什么？

1.28　什么是注释？如何表示注释行和注释段？

1.29　在控制台上显示消息“Hello world”的语句是什么？

1.30　找出下面代码中的错误：

```
1  # Display two messages
2    print("Welcome to Python")
3  print("Python is fun").
```

1.31　给出下面代码的输出结果：

```
print("3.5 * 4 / 2 - 2.5 is")
print(3.5 * 4 / 2 - 2.5)
```

1.7　程序设计风格和文档

关键点：好的程序设计风格和正确的文档可以让程序易读并防止出错。

程序设计风格指的是程序的整个样子。当用专业的程序设计风格创建程序时，它们不但会正确执行，而且也会易于阅读、便于理解。这对访问或修改你的程序的其他程序员来说是非常重要的。

文档是属于一个程序的解释性备注和注释的主体。这些备注和注释对程序的不同部分进行解释，帮助其他人更好地理解它的结构和功能。对本章前面的内容，备注和注释都是嵌在程序内部里，当执行程序时 Python 的解释器会直接忽略它们。

程序设计风格和文档与编码一样重要。下面是几个建议规范。

1.7.1　恰当的注释和注释风格

在程序开始的地方要有一个总结性的注释，它解释这个程序是干什么的、其重要特征以及所使用的独特技术。在大程序中，应该有注释介绍每个主要步骤以及任何难以读懂的内容。注释简洁明了是非常重要的，因此，不要让它们密密麻麻，也不要让它们难以阅读。

1.7.2　恰当的空格

一致的空格风格可以让程序更加清晰且易于阅读、调试（找到且解决错误）以及维护。

一个运算符的两边都应该添加一个空格，如下面的语句所示：

```
print(3+4*4)          ← 不好的风格
print(3 + 4 * 4)      ← 良好的风格
```

更多的建议规范可以在配套网站的补充材料 I.F 中找到。

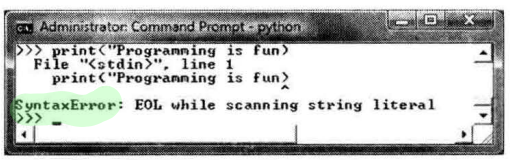
1.8　程序设计错误

关键点：程序设计错误可以分为三类：语法错误、运行时错误和逻辑错误。

1.8.1　语法错误

你会遇到的大多数常见错误都是语法错误。就像任何一种程序设计语言一样，Python 也有自己的语法，你需要遵从语法规则编写代码。如果你的程序违反了这些规则——例如：忘写一个引号或者拼错一个单词——Python 将会报告语法错误。

语法错误来自代码构建过程中的错误，例如：敲错了一条语句，不正确的缩进，忽略某些必需的标点符号，或者使用了左括号而忘了右括号。这些错误通常很容易被检测到，因为 Python 会告诉你这些错误在哪里以及是什么原因造成了这些错误。例如：下面的 print 语句有一个语法错误：



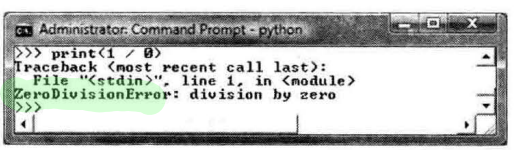
字符串“Programming is fun”少了右引号。

提示：如果你不知道如何更正语法错误，将你的程序和课本中的相同例子一个字符一个字符地进行比较。在学习这门课程的前几周里，你可能要花很多时间找出语法错误。过一段时间之后，你将会熟悉 Python 语法并且能够快速地找出语法错误。

1.8.2　运行时错误

运行时错误是导致程序意外终止的错误。在程序运行过程中，如果 Python 解释器检测到一个不可能执行的操作，就会出现运行时错误。输入错误是典型的运行时错误。当用户输入一个程序无法处理的值时，就会出现输入错误。例如：如果程序希望读取一个数字，而用户输入了一个文本字符串，这就导致程序中出现数据类型错误。

另一个常见的运行时错误是被 0 除。当整数除法的除数为零时就会发生运行时错误。例如：下面语句中的表达式 1/0 就会导致一个运行时错误。



1.8.3　逻辑错误

当程序不能实现它原来打算要完成的任务时就会导致逻辑错误。发生这种类型的错误的

原因有很多种。例如：假设你编写程序清单 1-4 中的程序，这个程序将华氏温度（35 度）转换成摄氏温度。

程序清单 1-4 ShowLogicErrors.py

```
1  # Convert Fahrenheit to Celsius
2  print("Fahrenheit 35 is Celsius degree ")
3  print(5 / 9 * 35 - 32)
```

```
Fahrenheit 35 is Celsius degree
-12.555555555555554
```

你可以得到摄氏 −12.55 度，但这是错的，它应该是 1.66。为了获取正确的结果，需要在表达式中使用 5 / 9 * (35 − 32) 而不是 5 / 9 * 35 − 32。也就是说，需要添加圆括号括住 (35 − 32)，这样，Python 会在做除法之前首先计算这个表达式。

在 Python 中，语法错误事实上是被当作运行时错误来处理，因为程序执行时它们会被解释器检测出来。通常，语法错误和运行时错误都很容易找出并且易于更正，因为 Python 给出提示信息以便找出错误来自哪里以及为什么它们是错的，而查找逻辑错误则非常具有挑战性。

检查点

1.32　三种程序错误是什么？

1.33　如果忘记在字符串后面加右引号，将会产生什么错误？

1.34　如果程序需要从文件中读取数据，但是这个文件并不存在，那么当你运行这个程序时就会导致错误。这个错误是哪类错误？

1.35　假设你编写一个程序计算一个矩形的周长，而你写错了程序导致它计算成矩形的面积。这个错误是哪类错误？

1.9　开始学习图形化程序设计

关键点：Turtle 是 Python 内嵌的绘制线、圆以及其他形状（包括文本）的图形模块。它很容易学习并且使用简单。

初学者通常很喜欢通过图形学习程序设计。因此，我们在本书第一部分的很多章的最后都会用一节讲解图形化程序设计。但是，这些素材不是强制性的，可以跳过它们或者以后再涉及这些内容。

在 Python 中有多种编写图形程序的方法。一个简单的启动图形化程序设计的方法是使用 Python 内嵌的 Turtle 模块。本书后面将会介绍 Tkinter 来开发复杂的图形用户界面应用程序。

1.9.1　绘制图形并给图形添加颜色

下面的程序将演示如何使用 Turtle 模块。后续章节会介绍更多的特性。

1）在 Windows“开始”菜单中选择 Python(命令行）或者在命令提示符下输入“python”来启动 Python。

2）在 Python 语句提示符“>>>”下输入下面的命令来导入 Turtle 模块。这个命令导入 Turtle 模块中定义的所有函数，这样就可以使用所有函数。

```
>>> import turtle # Import turtle module
```

3）输入下面的命令来显示 Turtle 的当前位置和方向，如图 1-15a 所示。

```
>>> turtle.showturtle()
```

使用 Python Turtle 模块进行图形化程序设计很像使用笔进行绘画。箭头表明笔的当前位置和方向。Turtle 的起始位置在窗口的中心。此处，Turtle 是指绘制图像的对象（对象将在第 3 章介绍）。

4）输入下面的命令绘制一个文本字符串：

```
>>> turtle.write("Welcome to Python")
```

你的窗口应该看起来如图 1-15b 所示。

5）输入下面的命令将箭头向前移动 100 像素，向箭头所指的方向绘制一条直线：

```
>>> turtle.forward(100)
```

你的窗口应该看起来如图 1-15c 所示。

为了绘制图 1-15 中的其他部分，继续这些步骤。

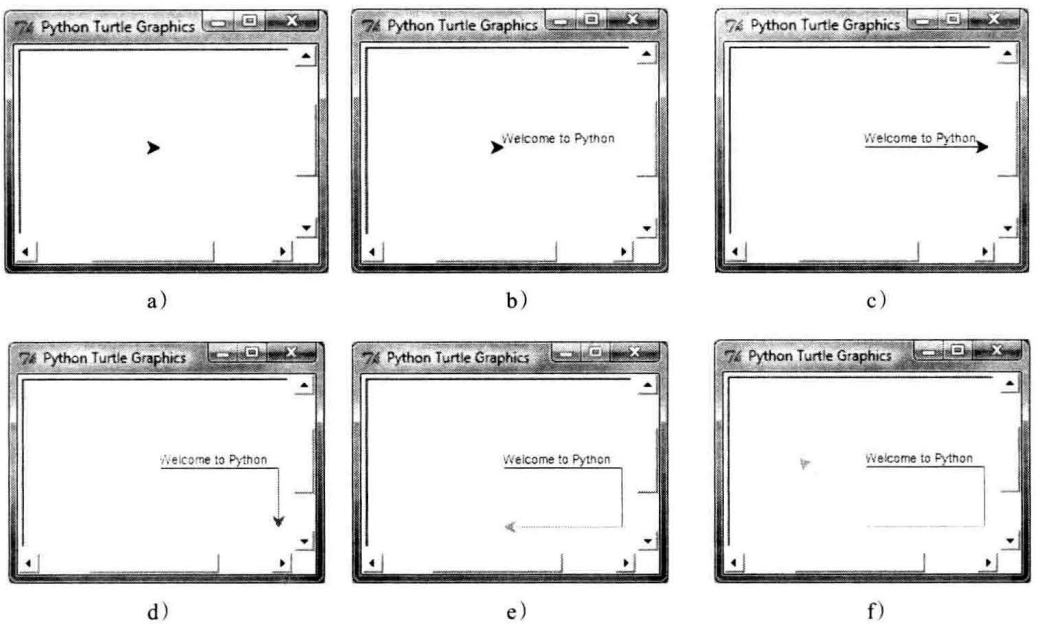


图 1-15　随着每个语句的执行动态地显示图形

6）输入下面的命令将箭头向右转 90 度，将 Turtle 的颜色改为红色，然后将箭头向前移动 50 像素，如图 1-15d 所示。

```
>>> turtle.right(90)
>>> turtle.color("red")
>>> turtle.forward(50)
```

7）现在，输入下面的命令将箭头向右转 90 度，将颜色设置为绿色，然后将箭头向前移动 100 像素来绘制一条直线，如图 1-15e 所示。

```
>>> turtle.right(90)
>>> turtle.color("green")
>>> turtle.forward(100)
```

8）最后，输入下面的命令将箭头向右转 45 度，并将箭头向前移动 80 像素来绘制一条

直线，如图 1-15f 所示。

```
>>> turtle.right(45)
>>> turtle.forward(80)
```

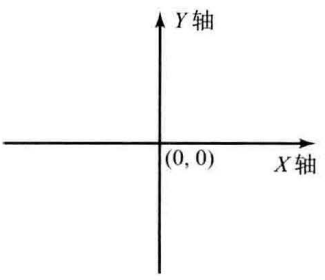
9）现在可以关闭 Turtle 图形窗口并退出 Python。

1.9.2　将笔移到任何位置

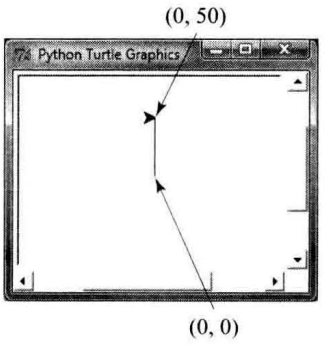
当 Turtle 程序启动时，箭头在 Python Turtle 图形窗口的中心位置，它的坐标是 (0，0)，如图 1-16a 所示。你也可以使用 goto(x，y) 命令将 turtle 移动到任何一个特定的点 (x，y)。

重启 Python 并敲入下面的命令将笔从 (0，0) 移动到 (0，50)，如图 1-16b 所示。

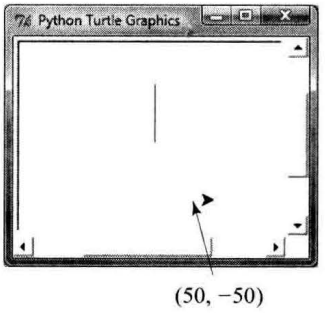
```
>>> import turtle
>>> turtle.goto(0, 50)
```



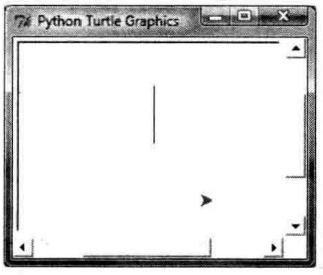
a）Turtle 图形化窗口中心的坐标是 (0, 0)



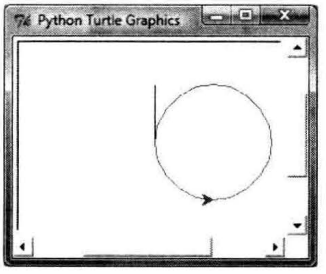
b）移动到 (0, 50)



c）将笔移动到 (50, −50)



d）将颜色设置为红色



e）使用 circle 命令绘制一个圆

图　1-16

也可以使用 penup() 和 pendown() 命令设置抬起或放下笔以控制移动笔时是否绘制一条线。例如：下面的命令将笔移到 (50，−50)，如图 1-16c 所示。

```
>>> turtle.penup()
>>> turtle.goto(50, -50)
>>> turtle.pendown()
```

可以使用 circle 命令绘制一个圆。例如：下面的命令设置颜色为红色（图 1-16d）并且绘制半径为 50 的圆（图 1-16e）。

```
>>> turtle.color("red")
>>> turtle.circle(50) # Draw a circle with radius 50
```

1.9.3　绘制奥林匹克环标志

程序清单 1-5 给出绘制奥林匹克环标志的程序，如图 1-17 所示。

程序清单 1-5　OlympicSymbol.py

```
1  import turtle
2
3  turtle.color("blue")
4  turtle.penup()
5  turtle.goto(-110, -25)
6  turtle.pendown()
7  turtle.circle(45)
8
9  turtle.color("black")
10 turtle.penup()
11 turtle.goto(0, -25)
12 turtle.pendown()
13 turtle.circle(45)
14
15 turtle.color("red")
16 turtle.penup()
17 turtle.goto(110, -25)
18 turtle.pendown()
19 turtle.circle(45)
20
21 turtle.color("yellow")
22 turtle.penup()
23 turtle.goto(-55, -75)
24 turtle.pendown()
25 turtle.circle(45)
26
27 turtle.color("green")
28 turtle.penup()
29 turtle.goto(55, -75)
30 turtle.pendown()
31 turtle.circle(45)
32
33 turtle.done()
```

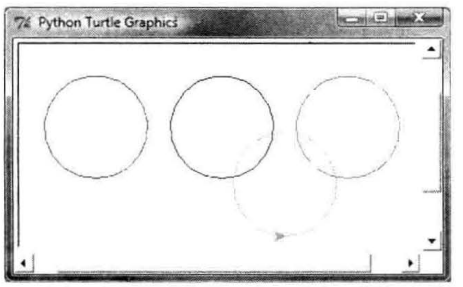


图 1-17　绘制奥林匹克环标志

程序导入 Turtle 模块使用 Turtle 图形化窗口（第 1 行）。它将笔移到（−110，−25）（第 5 行），然后绘制一个半径为 45 的蓝色圆（第 7 行）。类似地，它绘制一个黑色圆（第 9 ～ 13 行）、一个红色圆（第 15 ～ 19 行）、一个黄色圆（第 21 ～ 25 行）以及一个绿色圆（第 27 ～ 31 行）。

第 33 行调用 Turtle 的 done() 命令，它可以导致程序暂停直到用户关闭 Python Turtle 图形化窗口。它的目的是给用户时间来查看图形。没有这一行，图形窗口会在程序完成时立即关闭。

检查点

1.36　如何导入 Turtle 模块？

1.37　如何在 Turtle 中显示文本？

1.38　如何向前移动笔？

1.39　如何设置新颜色？

1.40　不绘制任何东西时如何移动笔？

1.41　如何绘制一个圆？

1.42　程序清单 1-5 中第 33 行的 turtle.done() 的目的是什么？

关键术语

.py file（.py 文件）
assembler（汇编器）
assembly language（汇编语言）
bit（比特）
bus（总线）
byte（字节）
cable modem（光缆调制解调器）
calling a function（调用函数）
central processing unit (CPU)（中央处理器（CPU））
comment（注释）
compiler（编译器）
console（控制台）
dot pitch（点距）
DSL (digital subscriber line)（DSL（数字用户线））
encoding scheme（编码表）
function（函数）
hardware（硬件）
high-level language（高级语言）
IDLE (Interactive DeveLopment Environment)（IDLE（交互式开发环境））
indentation（缩进）
interactive mode（交互式模式）
interpreter（解释器）
invoking a function（调用函数）
line comment（行注释）
logic error（逻辑错误）
low-level language（低级语言）
machine language（机器语言）
memory（内存）
modem（调制解调器）
module（模块）
motherboard（主板）
network interface card (NIC)（网络接口卡（NIC））
operating system (OS)（操作系统（OS））
pixel（像素）
program（程序）
runtime errors（运行时错误）
screen resolution（屏幕分辨率）
script file（脚本文件）
script mode（脚本模式）
software（软件）
source code（源代码）
source file（源文件）
source program（源程序）
statement（语句）
storage device（存储设备）
syntax errors（语法错误）
syntax rules（语法规则）

注意：上面的术语都是在当前章节中定义的。补充材料 I.A 按章罗列出本书所有的关键术语以及对它们的描述。

本章总结

1. 计算机是一个存储和处理数据的电子设备。
2. 计算机包括硬件和软件。
3. 硬件是计算机中可以碰触的物理部分。
4. 计算机程序，也称为软件，是控制硬件并让硬件完成任务的不可见的指令集。
5. 计算机程序设计是指编写让计算机来完成的指令（即代码）。
6. 中央处理器（CPU）是计算机的大脑。它从内存获取指令然后执行它们。
7. 计算机使用 0 和 1 是因为数字设备有两个稳定的电子状态：关和开，习惯上将它们表示成 0 和 1。
8. 比特是二进制数 0 或 1。
9. 字节是 8 比特构成的序列。
10. KB 大约是 1000 字节，MB 大约是 100 万字节，GB 大约是 10 亿字节，而 TB 大约是万亿字节。
11. 内存存储的是 CPU 要执行的数据和程序指令。

12. 内存单元是一个有序字节序列。
13. 内存是不稳定的，因为一旦断电，没有保存的信息就会丢失。
14. 程序和数据被永久地保存在存储设备上，当计算机真要用到它们的时候被移到内存。
15. 机器语言是一套嵌入每台计算机的原始指令集。
16. 汇编语言是一种低级程序设计语言，它使用助记符来表示每一条机器语言指令。
17. 高级语言很像英语，易于学习和编程。
18. 高级语言编写的程序称为源代码。
19. 编译器是一个软件程序，它负责将源程序翻译成机器语言程序。
20. 操作系统（OS）是管理和控制计算机动作的程序。
21. 可以在 Windows、UNIX 和 Mac 上运行 Python。
22. Python 是解释性的，这意味着 Python 解释每条语句，同时处理该语句。
23. 可以在 Python 语句提示符“>>>”下交互地输入 Python 语句，或者在一个文件中存储所有代码，然后使用一条命令执行它。
24. 要从命令行运行 Python 源文件，使用命令 python *filename*.py。
25. Python 中，在一行前加一个 # 号（#）的注释称为行注释，而用三重引号（''' 和 '''）括住一行或几行称为段注释。
26. Python 源代码是区分大小写的。
27. 程序设计错误可以分为三种类型：语法错误、运行时错误和逻辑错误。语法和运行时错误会导致程序意外终止。当程序没有完成它预期的任务时出现逻辑错误。

测试题

本章的在线测试题位于：www.cs.armstrong.edu/liang/py/test.html。

编程题

注意：本书里的偶数编号编程题答案在配套网站上。所有编程题的答案在教师资源网站上。题目的难度等级分为容易（无星号）、适度（*）、困难（**）或具有挑战性（***）。

第 1.6 节

1.1 （显示三个不同的消息）编写程序显示“Welcome to Python”、“Welcome to Computer Science”和“Programming is fun”。

1.2 （显示同样的消息五次）编写程序显示“Welcome to Python”五次。

*1.3 （显示一种模式）编写程序显示下面的模式。

```
FFFFFFF   U      U   NN      NN
FF        U      U   NNN     NN
FFFFFFF   U      U   NN N    NN
FF          U   U    NN  N   NN
FF            UUU    NN    NNN
```

1.4 （打印表格）编写程序显示下面的表格。

```
a       a^2     a^3
1       1       1
2       4       8
3       9       27
4       16      64
```

1.5 （计算表达式）编写程序显示下面表达式的结果。

$$\frac{9.5\times 4.5-2.5\times 3}{45.5-3.5}$$

1.6 （级数求和）编写程序显示 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9 的和。

1.7 （近似 π）可以使用下面的公式计算 $\pi = 4\times\left(1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\frac{1}{9}-\frac{1}{11}+\cdots\right)$。

编写程序显示 $4\times\left(1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\frac{1}{9}-\frac{1}{11}\right)$ 和 $4\times\left(1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\frac{1}{9}-\frac{1}{11}+\frac{1}{13}-\frac{1}{15}\right)$ 的结果。

1.8 （圆的面积和周长）使用下面的公式编写程序，显示半径是 5.5 的圆的面积和周长。

$$area = radius \times radius \times \pi$$

$$perimeter = 2 \times radius \times \pi$$

1.9 （矩形的面积和周长）使用下面的公式编写程序，显示宽度为 4.5 而高为 7.9 的矩形的面积和周长。

$$area = width \times height$$

1.10 （平均速度）假设一个人在 45 分 30 秒内跑了 14 公里，编写程序显示每小时的平均速度是多少英里。（注意：1 英里是 1.6 公里。）

*1.11 （人口预测）美国人口普查局基于下面的假设来预测人口：

每 7 秒 1 人出生；

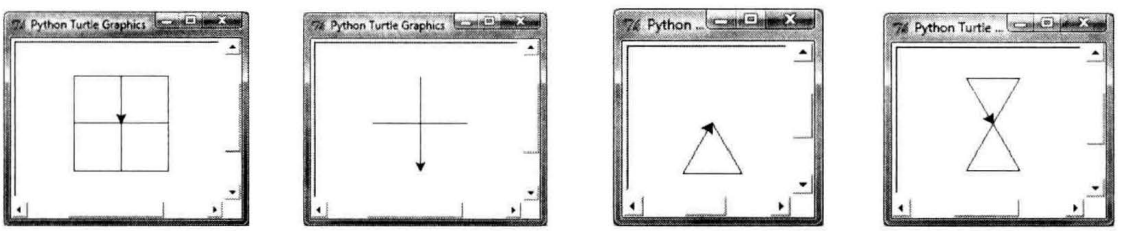
每 13 秒 1 人死亡；

每 45 秒 1 个新移民。

编写程序显示接下来 5 年每一年的人口。假设当前的人口数是 3 120 324 986，每年有 365 天。提示：在 Python 中，可以使用整数除法运算符 // 来完成除法运算。它的结果是一个整数。例如：5//4 是 1（而不是 1.25），10//4 是 2（而不是 2.5）。

第 1.9 节

1.12 （Turtle：绘制正方形）编写程序在屏幕中心绘制正方形，如图 1-18a 所示。



a）绘制正方形　b）绘制十字　c）绘制三角形　d）绘制两个三角形

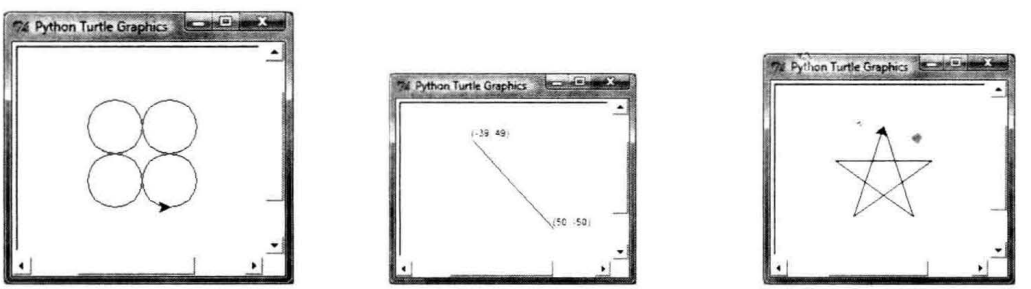
图　1-18

1.13 （Turtle：绘制十字）编写程序绘制如图 1-18b 所示的十字。

1.14 （Turtle：绘制三角形）编写程序绘制如图 1-18c 所示的三角形。

1.15 （Turtle：绘制两个三角形）编写程序绘制如图 1-18d 所示的两个三角形。

1.16 （Turtle：绘制四个圆）编写程序在屏幕中心绘制四个圆，如图 1-19a 所示。



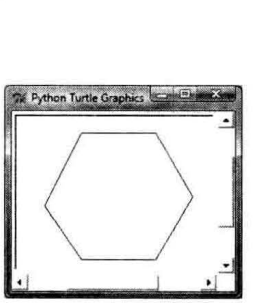
a）绘制四个圆　b）绘制直线　c）绘制五角星

图　1-19

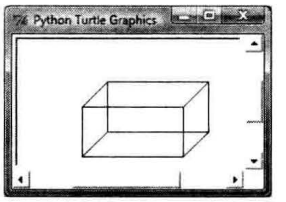
1.17 （Turtle：绘制直线）编写程序绘制一条连接两个点 (−39，48) 和 (50，−50) 的红线，然后显示这两个点的坐标，如图 1-19b 所示。

**1.18 （Turtle：绘制五角星）编写程序绘制一个五角星，如图 1-19c 所示。（提示：五角星每个点的内角度是 36 度。）

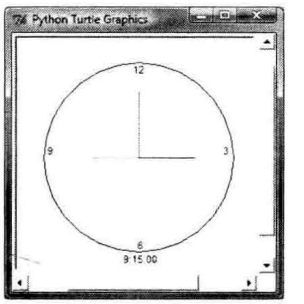
1.19 （Turtle：绘制多边形）编写程序绘制一个依次连接点 (40，−69.28)、(−40，−69.28)、(−80，−9.8)、(−40，69)、(40，69) 和 (80，0) 的多边形，如图 1-20a 所示。



a）绘制多边形



b）显示立方体



c）显示表示时间的时钟

图　1-20

1.20 （Turtle：显示立方体）编写程序显示一个立方体，如图 1-20b 所示。

1.21 （Turtle：显示时钟）编写程序显示一个时钟表示时间 9 : 15 : 00，如图 1-20c 所示。

第 2 章

Introduction to Programming Using Python

基本程序设计

学习目标

- 编写程序完成简单的计算（第 2.2 节）。
- 使用 input 函数从程序的用户处获取输入（第 2.3 节）。
- 使用标识符来命名元素，例如：变量和函数等（第 2.4 节）。
- 将数据赋值给变量（第 2.5 节）。
- 实现同时赋值（第 2.6 节）。
- 定义命名常量（第 2.7 节）。
- 使用运算符 +、-、*、/、//、% 和 **（第 2.8 节）。
- 编写和计算数字表达式（第 2.9 节）。
- 利用简捷运算符简化编码（第 2.10 节）。
- 使用 int 和 round 函数进行数据类型转换和四舍五入（第 2.11 节）。
- 使用 time.time() 获取当前系统时间（第 2.12 节）。
- 描述程序开发过程然后应用它开发一个贷款偿还程序（第 2.13 节）。
- 计算并显示图上两点之间的距离（第 2.14 节）。

2.1 引言

关键点：本章的重点是学习如何使用基本程序设计技巧来解决问题。

在第 1 章我们已经学习了如何创建和运行最基本的 Python 程序。现在你将学习如何通过编写程序来解决问题。通过这些问题，你将会学习到基本的程序设计技巧，例如：如何使用变量、运算符、表达式以及输入和输出。

例如：假设你需要领取学生贷款。假定给出贷款数目、贷款期限以及年利率，你能不能编写一个程序来计算每月的还款金额和总还款金额呢？本章将介绍怎样编写一个类似的程序。沿着这个思路，你会学习到如何通过创建程序深入分析问题、设计解决方案以及实施这个解决方案等基本步骤。

2.2 编写一个简单的程序

关键点：编写一个涉及设计解决问题的策略的程序，然后使用程序设计语言实现这些策略。

首先，让我们来看一个计算圆面积的简单问题。我们该如何编写程序来解决这个问题呢？

编写程序涉及如何设计算法以及如何将算法翻译成程序设计指令或代码。当你编写代码时——即你在编写程序时——你就将一个算法翻译成一段程序。算法描述的是如何通过列出要进行的动作和这些动作的执行顺序来解决一个问题。算法可以帮助程序员在使用程序设计语言编程之前做一个规划。算法可以用自然语言或伪代码（即自然语言与某些程序设计代码的混合应用）描述。这个计算圆面积的程序算法描述如下所示。

1）从用户处获取圆的半径。

2）利用下面的公式计算它的面积：

$$面积 = 半径 * 半径 * \pi$$

3）显示结果。

提示：在开始编写代码前，以算法的方式描述你的程序（或它的相关问题）是一个很好的做法。

在这个问题中，程序需要读取用户从键盘输入的半径。这就产生了两个重要的问题：

- 读取这个半径。
- 将半径存储在程序中。

我们首先来解决第二个问题。半径值被存储在计算机的内存中。为了访问它，程序中需要使用一个变量。变量是一个指向存储在内存中某个值的名字。变量应该尽量选择描述性的名字（descriptive name）而不是用像 x 和 y 这样的名字。例如：在这个例子里，使用名字 radius 表示指向半径值的变量，而使用名字 area 表示指向面积值的变量。

第一步是提示用户指定圆的 radius。你很快将学会如何提示用户输入信息。而现在，为了了解变量如何工作，你可以在编写代码时将一个固定值赋给程序中的 radius。

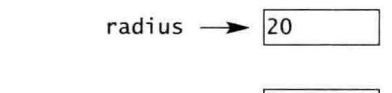
第二步是计算 area，这是通过将表达式 radius*radius*3.141 59 的值赋给 area 来实现的。

在最后一步中，程序将会使用 Python 中的 print 函数在控制台显示 area 的值。

完整的程序如程序清单 2-1 所示。

程序清单 2-1 ComputeArea.py

```
1  # Assign a value to radius
2  radius = 20 # radius is now 20
3
4  # Compute area
5  area = radius * radius * 3.14159
6
7  # Display results
8  print("The area for the circle of radius", radius, "is", area)
```




```
The area for the circle of radius 20 is 1256.636
```

像 radius 和 area 这样的变量指向的值存储在内存中。每个变量都有对应到一个值的一个名字。你可以使用如第 2 行所示那样将一个值赋值给一个变量。

```
radius = 20
```

这条语句将 20 赋值给变量 radius。所以，现在 radius 对应的值是 20。第 5 行的语句

```
area = radius * radius * 3.14159
```

使用 radius 的值来计算表达式并将结果赋给变量 area。下面的表格显示的是随着程序的执行，area 和 radius 的值。该表中的每一行显示的是程序中对应的每行语句被执行之后变量的值。这种显示程序如何工作的方法被称为跟踪程序。跟踪程序有助于理解程序是如何工作的，而且这也是在程序中查错的一个有效工具。

line#	radius	area
2	20	
5		1256.636

如果你已经使用过其他程序设计语言进行过编程，例如：Java，你就会知道必须声明变量的数据类型来明确使用的是什么类型的值，例如：整数或文本字符。但是，在 Python 中你不用这么做，因为 Python 会通过赋值给变量来自动判定数据类型。

第 8 行的语句在控制台上显示四项。你可以使用下面的语法在一条 print 语句中显示任意多项：

```
print(item1, item2, ..., itemk)
```

如果某项是一个数字，那么这数字就会被自动转化为显示一个字符串。

检查点

2.1　显示下面代码的打印输出：

```
width = 5.5
height = 2
print("area is", width * height)
```

2.2　将下面的算法翻译成 Python 代码。

- 第 1 步：使用一个名为 miles 初始值为 100 的变量。
- 第 2 步：将 miles 乘以 1.609 并将它赋值给一个名为 kilometers 的变量。
- 第 3 步：显示 kilometers 的值。

在第三步之后 kilometers 是多少？

2.3　从控制台读取输入

关键点：从控制台读取输入可以让程序从用户处接受输入。

在程序清单 2-1 中，一个半径值被设置在源代码中。为了使用另一个半径值，你不得不修改源代码。可以利用 input 函数输入一个半径值。下面的语句提示用户输入一个值，然后将它赋给变量 variable：

```
variable = input("Enter a value: ")
```

输入的值是一个字符串。你可以使用 eval 函数来求值并转换为一个数值。例如：

“eval("34.5")”返回的是 34.5，“eval("345")”返回的是 345，“eval("3+4")”返回的是 7，而“eval("51+(54*(3+2))")”返回 321。

程序清单 2-2 重写了程序清单 2-1 提示用户输入一个半径值。

程序清单 2-2　ComputeAreaWithConsoleInput.py

```
1  # Prompt the user to enter a radius
2  radius = eval(input("Enter a value for radius: "))
3
4  # Compute area
5  area = radius * radius * 3.14159
6
7  # Display results
8  print("The area for the circle of radius", radius, "is", area)
```

```
Enter a value for radius: 2.5 Enter
The area for the circle of radius 2.5 is 19.6349375
```

```
Enter a value for radius: 23 Enter
The area for the circle of radius 23 is 1661.90111
```

第 2 行提示用户输入一个值（以字符串的形式）然后转化为一个数字，这个过程等价于：

```
s = input("Enter a value for radius: ")  # Read input as a string
radius = eval(s) # Convert the string to a number
```

在用户输入一个数字并按下 Enter 键后，这个数字就被读取并赋给 radius。

程序清单 2-2 显示如何提示用户进行一次输入。但是，你也可以提示进行多次输入。程序清单 2-3 给出了一个从键盘读取多组输入的例子。这个程序读取了三个整数并显示它们的平均数。

程序清单 2-3 ComputeAverage.py

```
1  # Prompt the user to enter three numbers
2  number1 = eval(input("Enter the first number: "))
3  number2 = eval(input("Enter the second number: "))
4  number3 = eval(input("Enter the third number: "))
5
6  # Compute average
7  average = (number1 + number2 + number3) / 3
8
9  # Display result
10 print("The average of", number1, number2, number3,
11     "is", average)
```

```
Enter the first number: 1 Enter
Enter the second number: 2 Enter
Enter the third number: 3 Enter
The average of 1  2  3 is 2.0
```

这程序提示用户输入 3 个整数（第 2 ～ 4 行），计算它们的平均数（第 7 行），然后显示结果（第 10 ～ 11 行）。

如果用户输入的不是数字，这个程序将会以一个运行时错误终止。在第 13 章中，你将学会如何处理这个错误以使程序可以继续运行。

通常，一条语句会在一行的末尾处结束。在前面的程序清单中，print 语句被分成了两行（第 10 ～ 11 行）。这没关系，因为 Python 扫描第 10 行的 print 语句，直到发现第 11 行的后括号才结束。我们说这两句隐式会合了。

注意：在某些情况下，Python 的解释器不能确定在多行中哪里是语句的结尾。你可以在一行的结尾处放置一个继续符号（\）来告诉解释器这条语句继续到下一行。例如，下面的语句：

```
sum = 1 + 2 + 3 + 4 + \
      5 + 6
```

等价于：

```
sum = 1 + 2 + 3 + 4 + 5 + 6
```

注意：本书前几章的大多数程序都会实现三个步骤：输入、处理和输出，它们被称为 IPO。输入是从用户获取输入，处理是使用输入产生结果，输出是显示结果。

检查点

2.3　如何编写一条语句提示用户输入一个数值？

2.4　执行下面代码时，如果用户输入 5a 会发生什么？

```
radius = eval(input("Enter a radius: "))
```

2.5　如何将一个长语句拆为多行？

2.4　标识符

关键点：标识符用于命名程序中标识像变量和函数这样的元素。

如程序清单 2-3 中所示，number1、number2、number3、average、input、eval 和 print 是出现在程序中的事物的名称。在程序设计术语表中，这类名字被称为标识符。所有标识符必须遵从以下规则：

- 标识符是由字母、数字和下划线 (_) 构成的字符序列。
- 标识符必须以字母或下划线 (_) 开头，不能以数字开头。
- 标识符不能是关键字。（参见附录 A，它是一个关键字的列表。）关键字，又被称为保留字，它们在 Python 中有特殊意义。例如：import 是一个关键字，它告诉 Python 解释器将一个模块导入到程序。
- 标识符可以为任意长度。

例如：area、radius 和 number1 都是合法标识符，而 2A 和 d+4 不是，因为它们没有遵从这些规则。当 Python 检测出不合法的标识符时，它就会报告一个语法错误并终止程序。

注意：因为 Python 区分大小写，所以 area、Area 和 AREA 是不同的标识符。

提示：描述性标识符可以使程序更加易于阅读。避免使用简写的标识符。使用完整的单词更具描述性。例如：numberOfStudents 比 numStuds、numOfStuds 或 numOfStudents 更好。我们在书里完整的程序中使用描述性的名字。然而，我们也会偶尔为了简洁起见在代码段中使用像 i、j、k、x 和 y 这样的变量名。这些名字也为代码段提供了一种风格。

提示：变量名使用小写字母，例如：radius 和 area。如果一个名字包含几个单词，将这几个单词连在一起构成一个变量名，第一个单词要小写，而后续的每个单词的第一个字母要大写，例如：numberOfStudents。这种命名方式被称为骆驼拼写法，因为大写的字母好像骆驼的驼峰。

变量命名法

检查点

2.6　下面哪些标识符是有效的？哪些是 Python 关键字（参见附录 A）？

```
miles, Test, a+b, b-a, 4#R, $4, #44, apps
if, elif, x, y, radius
```

2.5　变量、赋值语句和赋值表达式

关键点：变量用于引用在程序中可能会变化的值。

正如在前几节的程序中看到的，变量是引用存储在内存中的值的名字。它们被称为“变量”是因为它们可能引用不同的值。例如：在下面的代码中，radius 的初始值为 1.0（第 2 行），然后它变为 2.0（第 7 行），而 area 被设置为 3.1415926（第 3 行），然后被重置为 12.56636（第 8 行）。

```
1  # Compute the first area
2  radius = 1.0                                    radius ──► 1.0
3  area = radius * radius * 3.14159                  area ──► 3.14159
4  print("The area is", area, "for radius", radius)
5
```

```
6  # Compute the second area
7  radius = 2.0
8  area = radius * radius * 3.14159
9  print("The area is", area, "for radius", radius)
```

radius → 2.0
area → 12.56636

将一个值赋给变量的语句被称为赋值语句。在 Python 中，等号（=）被用作赋值运算符。而赋值语句的语法如下所示：

```
variable = expression
```

一个表达式表示一个涵盖到值、变量和运算符结合到一起并求值的计算。例如：考虑下面的代码：

```
y = 1                    # Assign 1 to variable y
radius = 1.0             # Assign 1.0 to variable radius
x = 5 * (3 / 2) + 3 * 2  # Assign the value of the expression to x
x = y + 1                # Assign the addition of y and 1 to x
area = radius * radius * 3.14159 # Compute area
```

你可以在表达式中使用变量。一个变量可以在赋值运算符“=”的两边同时使用。例如：

```
x = x + 1
```

在这个赋值语句中，x + 1 的结果被赋值给 x。如果在执行这条语句前 x 的值是 1，那执行这句后它就成了 2。

为了将值赋给变量，你必须将变量名放在赋值运算符的左边。这样，下面的语句就是错误的：

```
1 = x    # Wrong
```

注意：在数学中，x=2*x+1 表示一个方程。然而，在 Python 中，x=2*x+1 是对表达式 2*x+1 求值并将结果赋值给 x 的赋值语句。

如果一个值被赋给多个变量，你可以使用类似如下的语法：

```
i = j = k = 1
```

这等价于：

```
k = 1
j = k
i = j
```

每个变量都有它的范围。变量的范围是程序可以引用到变量的部分。定义变量的范围的规则将在本书后面逐步介绍。现在，你所需要知道的是变量在使用前必须被创建。例如，下面的代码是错误的：

```
                    count is not defined yet.
>>> count = count + 1
NameError: count is not defined
>>>
```

count 还没有被定义。

为了改正它，你可以编写如下所示的代码：

```
>>> count = 1  # count is created
>>> count = count + 1  # Now increment count
>>>
```

警告：变量在表达式中使用之前必须被赋值。例如：

```
interestRate = 0.05
interest = interestrate * 45
```

这样的代码是错的。因为 interestRate 被赋值 0.05 而 interestrate 并未被定义。Python 区分大小写，所以 interestRate 和 interestrate 是两个不同的变量。

2.6　同时赋值

Python 也支持如下所示的同时赋值：

```
var1, var2, ..., varn = exp1, exp2, ..., expn
```

它的含义是 Python 计算等号右边的表达式并同时赋值给等号左边相对应的变量。交换变量的值是程序中常见的操作，而同时赋值对完成这一操作十分有用。假设有两个变量 x 和 y，你如何写代码交换它们的值？一个常见的方法是如下引入一个中间变量：

```
>>> x = 1
>>> y = 2
>>> temp = x  # Save x in a temp variable
>>> x = y     # Assign the value in y to x
>>> y = temp  # Assign the value in temp to y
```

但如果你使用下面的语句交换 x 和 y 的值就可以简化这个工作。

```
>>> x, y = y, x # Swap x with y
```

同时赋值也可以用于在一条语句中获取多个输入。程序清单 2-3 给出了一个提示用户输入三个数字然后获取它们平均值的程序。这个程序可以用同时赋值语句来简化，如程序清单 2-4 所示。

程序清单 2-4　ComputeAverageWithSimultaneousAssignment.py

```
1  # Prompt the user to enter three numbers
2  number1, number2, number3 = eval(input(
3      "Enter three numbers separated by commas: "))
4
5  # Compute average
6  average = (number1 + number2 + number3) / 3
7
8  # Display result
9  print("The average of", number1, number2, number3
10     "is", average)
```

```
Enter three numbers separated by commas: 1, 2, 3 Enter
The average of 1  2  3 is 2.0
```

检查点

2.7　变量的命名习惯是什么？

2.8　下面的语句有什么错误？

```
2 = a
```

2.9　在下面语句之后，x、y 和 z 的值是多少？

```
x = y = z = 0
```

2.10　假设 a=1 而 b=2。那么在下面的语句后，a 和 b 的值是多少？

```
a, b = b, a
```

2.7　定名常量

关键点：定名常量（named constant）是一种表示定值的标识符。

变量的值在程序执行的过程中可能会被改变，但是定名常量（或简称为常量）代表永远不会变的固定数据。在我们的 ComputeArea 程序中，π 是一个常量。如果你经常使用它，而不想不停地输入 3.141 59；那么你可以使用一个描述性的名字 PI 代表那个值。Python 没有命名常量的特殊语法。你可以简单地创建一个变量来表示常量。然而，为了区分常量和变量，我们全部使用大写字母来命名常量。例如：你可以重写程序清单 2-1 来使用定名常量 π，例如：

```
# Assign a radius
radius = 20 # radius is now 20

# Compute area
PI = 3.14159
area = radius * radius * PI

# Display results
print("The area for the circle of radius", radius, "is", area)
```

使用常量有下面三个好处：

1）你不必为使用一个值多次而重复性输入。

2）如果你需要修改常量的值（例如：将 PI 从 3.14 改为 3.141 59），你只需要在源代码一处进行修改。

3）描述性名字会提高程序的易读性。

2.8　数值数据类型和运算符

关键点：Python 中有两种数值类型（整数和浮点数）与 +、−、*、/、//、% 和 ** 一起工作。

储存在计算机中的信息通常被称为数据。这里有两种数值数据类型：整数和实数。整数类型 Integer（简写作 int）用于表示整数。实数型用于表示有小数部分的数字。在计算机中，这两种数据类型的存储方式不同。实数型表示为浮点数。我们怎样告知 Python 一个数字是整数还是浮点数呢？一个拥有小数点的数字即使小数部分为零也是浮点数。例如：1.0 是浮点数，而 1 是整数。这两个数字在计算机里的存储方式不同。在程序设计术语表中，像 1.0 和 1 这样的数字被称为字面量。字面量是直接出现在程序中的常量值。

供数值数据类型使用的运算符包括标准的算术符号，如表 2-1 所示。操作数是被运算符操作的值。

+、− 和 * 运算符都很直接明了，但是注意：运算符 + 和 − 既可以用于一元运算也可用于二元运算符。一元运算符只能有一个操作数，而二元运算符有两个操作数。例如：在 −5 中的 − 号是一元的，表示 5 的相反数，而它在 4-5 中是二元的，表示 4 减去 5。

表 2-1　算术运算符

名称	含义	举例	结果
+	Addition	34 + 1	35
-	Subtraction	34.0 - 0.1	33.9
*	Multiplication	300 * 30	9000
/	Float Division	1 / 2	0.5
//	Integer Division	1 // 2	0
**	Exponentiation	4 ** 0.5	2.0
%	Remainder	20 % 3	2

2.8.1　运算符 /、// 和 **

运算符 / 执行浮点除法并产生一个浮点数结果。例如：

```
>>> 4 / 2
2.0
>>> 2 / 4
0.5
>>>
```

运算符 // 执行整数除法并产生一个整数结果，任何小数部分都会被舍掉。例如：

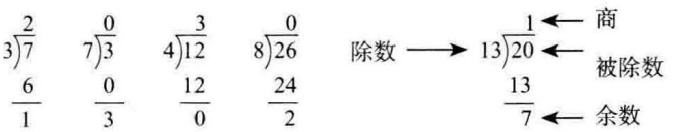
```
>>> 5 // 2
2
>>> 2 // 4
0
>>>
```

为了针对任意数字 a 与 b 计算 a^b（a 的 b 次幂），你可以在 Python 中编写 a**b。例如：

```
>>> 2.3 ** 3.5
18.45216910555504
>>> (-2.5) ** 2
6.25
>>>
```

2.8.2　运算符 %

众所周知，运算符 % 是一个求余或取模运算的运算符，即求出除法后的余数。左侧的操作数是被除数，而右侧的操作数是除数。因此，7%3 结果是 1，3%7 结果是 3，12%4 结果是 0，26%8 结果是 2 而 20%13 结果是 7。



在程序设计中求余运算符非常有用。例如：偶数 %2 总是 0 而奇数 %2 总是 1。这样，你就可以用这个特性判断一个数字是奇数还是偶数。如果今天是星期六，那七天之后又是星期六。假设你和你的朋友十天后要见面。那么十天后是周几？你可以用下面的表达式算出是周二：

程序清单 2-5 给出将以秒计时的一段时间转换为用分和秒计时的程序。例如：500 秒即是 8 分 20 秒。

程序清单 2-5　DisplayTime.py

```
1  # Prompt the user for input
2  seconds = eval(input("Enter an integer for seconds: "))
3
```

```
4  # Get minutes and remaining seconds
5  minutes = seconds // 60      # Find minutes in seconds
6  remainingSeconds = seconds % 60   # Seconds remaining
7  print(seconds, "seconds is", minutes,
8      "minutes and", remainingSeconds, "seconds")
```

```
Enter an integer for seconds: 500 ↵Enter
500 seconds is 8 minutes and 20 seconds
```

line#	seconds	minutes	remainingSeconds
2	500		
5		8	
6			20

第 2 行读取一个整数 seconds。第 5 行使用 seconds//60 获取分钟数。第 6 行（seconds%60）获得除去分钟后剩余的秒数。

2.8.3　科学记数法

浮点数可以用 $a \times 10^b$ 形式的科学记数法来编写。例如：123.456 的科学记数法表示为 $1.234\ 56 \times 10^2$ 而 0.0 123 456 可以表示为 $1.234\ 56 \times 10^{-2}$。Python 使用特殊的语法来书写科学记数法的数字。例如：$1.234\ 56 \times 10^2$ 被写作 1.234 56E2 或 1.23456E+2，而 $1.234\ 56 \times 10^{-2}$ 被写作 1.234 56E-2。字母 E（或 e）代表指数而且可以大写也可以小写。

注意：浮点型用于表示有小数点的数字。为什么它们叫浮点数呢？这些数字在内存中以科学记数法存储。当一个像 50.534 这样的数字被转换为科学记数法是 5.0534E+1，它的小数点移动（浮动）到一个新位置。

警告：当一个变量被赋值一个太大的值而不能存入内存中。这会导致数据溢出。例如：执行下面的语句会导致溢出。

```
>>> 245.0 ** 1000
OverflowError: 'Result too large'
>>>
```

当一个浮点数太小（即太接近 0）会导致下溢，而 Python 会将它近似为 0。因此，你不需要关注下溢。

检查点

2.11　下面表达式的结果是什么？

表达式	结果	表达式	结果
`42 / 5`	———	`2 % 1`	———
`42 // 5`	———	`45 + 4 * 4 - 2`	———
`42 % 5`	———	`45 + 43 % 5 * (23 * 3 % 2)`	———
`40 % 5`	———	`5 ** 2`	———
`1 % 2`	———	`5.1 ** 2`	———

2.12　如果今天是星期二，那 100 天后是星期几？

2.13　25/4 的结果是多少？如果你希望结果是整数应该怎么改写？

2.9　计算表达式和运算符优先级

关键点：Python 表达式计算方式与算术表达式一样。

用 Python 编写一个算术表达式是指使用运算符对算术表达式进行直接的翻译。例如，算数表达式：

$$\frac{3+4x}{5}-\frac{10(y-5)(a+b+c)}{x}+9\left(\frac{4}{x}+\frac{9+x}{y}\right)$$

可以翻译为如下所示的 Python 表达式：

```
(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x +
9 * (4 / x + (9 + x) / y)
```

尽管 Python 有它自己在后台计算表达式的方式，但 Python 表达式和与之相对应的算术表达式的结果是相同的。因此，你可以放心地将算术运算规则应用在计算 Python 表达式上。

首先执行括号内的运算符。括号可以叠加，内层括号里的表达式首先被执行。当一个表达式中使用多个运算符时，使用下面的运算符优先级规则决定计算顺序。

- 首先计算指数运算（**）。
- 接下来计算乘法（*）、浮点除法（/）、整数除法（//）和求余运算。如果一个表达式包含多个乘法、除法和求余运算符，它们会从左向右运算。
- 最后计算加法（+）和减法（-）运算符。如果一个表达式包含多个加法和减法运算符，它们会从左向右运算。

这是一个如何计算表达式的例子：

```
3 + 4 * 4 + 5 * (4 + 3) - 1
                (1) 首先是括号内
3 + 4 * 4 + 5 * 7 - 1
            (2) 乘法
3 + 16 + 5 * 7 - 1
                (3) 乘法
3 + 16 + 35 - 1
        (4) 加法
19 + 35 - 1
            (5) 加法
  54 - 1
        (6) 减法
  53
```

检查点

2.14　如何使用 Python 编写下面的算术表达式？

$$\frac{4}{3(r+34)}-9(a+bc)+\frac{3+d(2+a)}{a+bd}$$

2.15　假设 m 和 r 是整数。请为 mr^2 编写一个 Python 表达式。

2.10　增强型赋值运算符

关键点：运算符 +、-、*、/、//、% 和 ** 可以与赋值运算符（=）组合在一起构成简捷运算符。

经常会出现变量的当前值被使用、修改、然后重新赋值给同一变量的情况。例如，下面的语句就是给变量 count 加 1：

```
count = count + 1
```

Python 允许使用便捷（或合成）运算符将赋值运算符和加法运算符结合在一起。例如，前面的语句可以写作：

```
count += 1
```

运算符 += 被称为加法赋值运算符。所有的简捷运算符都在表 2-2 中给出。

表 2-2　增强型赋值运算符

运算符	名称	举例	等式
+=	Addition assignment	i += 8	i = i + 8
-=	Subtraction assignment	i -= 8	i = i - 8
*=	Multiplication assignment	i *= 8	i = i * 8
/=	Float division assignment	i /= 8	i = i / 8
//=	Integer division assignment	i //= 8	i = i // 8
%=	Remainder assignment	i %= 8	i = i % 8
**=	Exponent assignment	i **= 8	i = i ** 8

警告：在增强型赋值运算符中没有空格。例如：+ = 应该是 +=。

检查点

2.16　假设 a=1，下面的每个表达式都是独立的。那么下面的表达式的结果分别是什么？

```
a += 4
a -= 4
a *= 4
a /= 4
a //= 4
a %= 4
a = 56 * a + 6
```

2.11　类型转换和四舍五入

关键点：如果算术运算符的操作数之一是浮点数，那么结果就是浮点数。

你能否对两个不同类型的数据进行二元运算？答案是肯定的。如果一个整数和一个浮点数同时参与到一个二元运算中，那么 Python 会自动将整数转化为浮点值。这被称为类型转换。所以 3*4.5 和 3.0*4.5 是相同的。

有时候，希望获取小数的整数部分。你可以使用 int(value) 函数来返回一个浮点值的整数部分。例如：

```
>>> value = 5.6
>>> int(value)
5
>>>
```

注意：小数部分被舍掉了而没有进位。

你也可以使用 round 函数对数字进行四舍五入将之转为最近的整数。例如：

```
>>> value = 5.6
>>> round(value)
6
>>>
```

我们将在第 3 章更多地讨论 round 函数。

注意：函数 int 和 round 不会改变要转换的变量。例如：在下面代码中，调用函数后 value 并没有改变。

```
>>> value = 5.6
>>> round(value)
6
>>> value
5.6
>>>
```

注意：函数 int 也可以用于将整数字符串转换为整数。例如：int("34") 返回 34。所以，你可以使用函数 eval 或 int 将字符串转换为整型。哪个会更好些？ int 函数完成一个简单的转换。它不能用于非整型字符串。例如：int("3.4") 将导致错误。函数 eval 可以完成比简单转换更多的功能。它可以用于计算表达式。例如：eval ("3+4") 返回 7。然而，使用函数 eval 有一个微妙的“疑难杂症”。如果数字串前有先导零会使 eval 函数产生错误。相对地，int 函数可以很好地处理这个问题。例如：eval("003") 会导致错误，而 int("003") 会返回 3。

程序清单 2-6 给出一个显示保留小数点后两位的营业税的程序。

程序清单 2-6　SalesTax.py

```
1  # Prompt the user for input
2  purchaseAmount = eval(input("Enter purchase amount: "))
3
4  # Compute sales tax
5  tax = purchaseAmount * 0.06
6
7  # Display tax amount with two digits after decimal point
8  print("Sales tax is", int(tax * 100) / 100.0)
```

```
Enter purchase amount: 197.55 Enter
Sales tax is 11.85
```

行号	销售额	营业税	税款
2	197.55		
5		11.853	
8			11.85

变量 purchaseAmount 的值是 197.55（第 2 行）。营业税是销售额的 6%，所以计算出的 tax 是 11.853（第 5 行）。注意：

```
tax * 100 is 1185.3
int(tax * 100) is 1185
int(tax * 100) / 100.0 is 11.85
```

所以，第 8 行的语句显示保留小数点后两位的税款 11.85。

检查点

2.17　当数据从浮点型转化为整型时，小数点后的部分怎么处理？ int(value) 函数会改变变量 value 吗？

2.18　下面的语句都正确么？如果是，给出它们的结果。

```
value = 4.6
print(int(value))
```

```
print(round(value))
print(eval("4 * 5 + 2"))
print(int("04"))
print(int("4.5"))
print(eval("04"))
```

2.12　实例研究：显示当前时间

关键点：可以使用 time 模块中的 time() 函数来获取当前的系统时间。

这里的问题是编写一个显示当前 GMT 时间的程序，格式为小时：分钟：秒，例如，13:19:18。time 模块中的 time() 函数返回以毫秒为精度的从 1970 年 1 月 1 日 00:00:00 开始到现在的 GMT 时间，如图 2-1 所示。这个时间被称作 UNIX 时间点。这个时间点是时间的开始。1970 年是 UNIX 操作系统正式发布的年份。例如：time.time（）返回 1285543663.205，它表示 1285543663 秒 205 微秒。

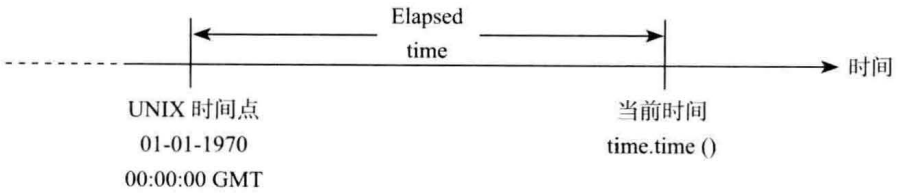


图 2-1　time.time() 函数返回从 UNIX 时间点开始的以毫秒为精度的秒数

你可以利用这个函数获取当前的时间，然后计算出当前秒数、分数和小时数，如下所示：

1）通过调用 time.time() 获取当前时间（自 1970 年 1 月 1 日零时起）(例如，1203183068.328）。

2）使用 int 函数来获取总秒数 totalSeconds(int(1203183068.328)=1203183068)。

3）用 totalSeconds%60 来求现在的秒数（1203183068seconds%60=8，即当前的秒数）。

4）用 totalSeconds 除以 60 求总分钟数 totalMinutes（1203183068seconds//60=20053051 分钟）。

5）用 totalMinutes%60 来求当前分钟数（20053051minutes%60=31，即现在的分钟数）。

6）用 totalMinutes 除以 60 来求总小时数 totalHours（20053051minutes//60=334217 小时）。

7）从总小时数 totalHours%24 来求现在的小时数（334217hours%24=17，即当前小时数）。

程序清单 2-7 给出这个完整的程序。

程序清单 2-7　ShowCurrentTime.py

```
1  import time
2
3  currentTime = time.time() # Get current time
4
5  # Obtain the total seconds since midnight, Jan 1, 1970
6  totalSeconds = int(currentTime)
7
8  # Get the current second
9  currentSecond = totalSeconds % 60
10
11 # Obtain the total minutes
12 totalMinutes = totalSeconds // 60
13
14 # Compute the current minute in the hour
15 currentMinute = totalMinutes % 60
16
```

```
17  # Obtain the total hours
18  totalHours = totalMinutes // 60
19
20  # Compute the current hour
21  currentHour = totalHours % 24
22
23  # Display results
24  print("Current time is", currentHour, ":",
25      currentMinute, ":", currentSecond, "GMT")
```

```
Current time is 17:31:8 GMT
```

variables \ line#	3	6	9	12	15	18	21
currentTime	1203183068.328						
totalSeconds		1203183068					
currentSecond			8				
totalMinutes				20053051			
currentMinute					31		
totalHours						334217	
currentHour							17

第 3 行调用 time.time() 返回以秒为单位的带微秒精度的浮点值表示的当前时间。秒数、分钟数和小时数是通过 // 和 % 运算符从当前时间中计算出的（第 6 ～ 21 行）。

在示例运行中，显示数字 8 为秒数。而希望的输出应该是 08。这可以使用一个在一个数字前加 0 的函数来修正（参见编程题 6.48）。

检查点

2.19　什么是 UNIX 时间点？

2.20　time.time() 返回的是什么？

2.21　如何从 time.time() 的返回值中获取秒数？

2.13　软件开发流程

关键点：程序开发周期是一个包括明确需求、分析、设计、实现、测试、部署和维护的多步骤过程。

开发软件是一个工程过程。软件产品，无论是大还是小，它们都有相同的周期：明确需求、系统分析、系统设计、实现、测试、部署和维护，如图 2-2 所示。

明确需求是寻求理解软件要解决的问题和建立关于软件系统需要完成任务的详细文档的一个正式流程。这个阶段需要用户和开发者之间的进行紧密的交互。本书中大多数例子都很简单，并且它们的需求陈述很明确。然而，在现实世界中，问题并不总是定义明确。开发者需要保持和用户（会使用软件的个人或团体）紧密的联系，仔细研究问题以期明确到底需要软件做什么。

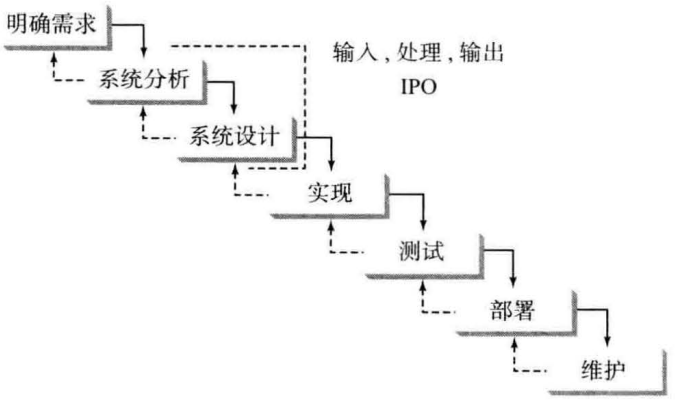


图 2-2　在程序开发周期的任何一步，为了修改错误或解决导致程序不能完成既定功能的问题，退回前一步是有必要的

*系统分析*是指分析数据流和识别系统的输入和输出。当你分析时，它帮你确认哪个输入是最早的，然后帮你搞清楚要产生输出需要哪些输入数据。

*系统设计*是设计从输入获取输出的过程。这一阶段涉及很多层的抽象，将问题分解为可管理的几个组件，然后为每个组件的实现设计策略。你可以将每个组件看作一个完成系统中特定功能的子系统。系统分析和设计的本质是输入、处理和输出（IPO）。

*实现过程*涉及将系统设计翻译成程序。每个组件被编写成各自的程序，然后将它们集成在一起工作。这一阶段需要使用程序设计语言，例如：Python。实现过程设计到编写代码、自测和调试（就是在代码中找被称为小虫子（bug）的错误）。

*测试过程*确保代码满足需求规范并且清除程序 bug。一部分不参与产品设计和实现的工程师组成的独立团队通常进行这类测试。

*部署过程*是使程序可以使用。根据软件类型的不同，有些安装到每个用户的机器上而有些则安装在可以通过互联网访问的服务器上。

*维护过程*涉及产品的更新和升级。一款软件产品必须持续在一个不断变化的环境中完善和升级。这需要定期更新产品来解决最新发现的 bug 并合并这些改变。

为了更直观地看软件开发过程，我们现在创建一个计算贷款支付额的程序。这笔贷款可以是汽车贷款、学生贷款或房屋抵押贷款。作为一个对程序设计教学的介绍，我们专注于需求分析、分析、设计、实现和测试。

第 1 阶段：需求分析

这个程序必须满足以下需求：

- 必须由用户键入利率、贷款数以及贷款的年限。
- 必须计算出每月还贷数和总还款数。

第 2 阶段：系统分析

输出是月供（monthlyPayment）和总还款数（totalPayment），可以通过下面的公式来获取：

$$月供 = \frac{贷款数 \times 月利率}{1 - \frac{1}{(1+月利率)^{年限 \times 12}}}$$

$$总还款数 = 月供 \times 年限 \times 12$$

所以，程序需要输入的是年利率、贷款年限和总贷款数目。

注意：需求分析要求用户必须输入利率、贷款数、贷款年限。但在分析过程中，有可能你会发现输入是不充分的或有些输入对于输出而言是不必要的。如果是这样，你可以返回上一步修改需求分析。

注意：在现实世界里，你会为各行各业的用户工作。你可能会为化学家、物理学家、工程师、经济学家和心理学家开发软件。你不一定会有（或需要）这些行业的完备知识。因此，你不需要知道这些数学公式是怎样推导出来的。所以，在给出利率、贷款数、贷款年限的情况下，你可以利用公式来计算月供。然而，你需要和用户进行交流并理解这个数学模型是如何为系统工作的。

第 3 阶段：系统设计

在系统设计过程中，你需要确定程序中以下几个步骤。

第 1 步：提示用户输入年利率、贷款数、贷款年限和贷款额。

第 2 步：输入的年利率是百分比格式的数字，例如：4.5%。程序需要将它除以 100 转换为小数。因为一年有 12 个月，所以将年利率除以 12 即月利率。所以，为了获取月利率，你需要将百分比格式的年利率除以 1200。例如：如果年利率是 4.5%，那月利率就是 4.5/1200=0.003 75。

第 3 步：使用第 2 步中的公式计算月供。

第 4 步：通过将月供乘以 12 再乘以贷款年限求出还款总额。

第 5 步：显示月供和还款总额。

第 4 阶段：实现过程

实现过程又被称为编码（编写代码）。在公式中，你需要计算 (1 + 月利率)$^{年限 \times 12}$。你可以利用指数运算符将它写作：

```
(1 + monthlyInterestRate) ** (numberOfYears * 12)
```

程序清单 2-8 给出了完整的程序。

程序清单 2-8 ComputeLoan.py

```
1  # Enter annual interest rate as a percentage, e.g., 7.25
2  annualInterestRate = eval(input(
3      "Enter annual interest rate, e.g., 7.25: "))
4  monthlyInterestRate = annualInterestRate / 1200
5
6  # Enter number of years
7  numberOfYears = eval(input(
8      "Enter number of years as an integer, e.g., 5: "))
9
10 # Enter loan amount
11 loanAmount = eval(input("Enter loan amount, e.g., 120000.95: "))
12
13 # Calculate payment
14 monthlyPayment = loanAmount * monthlyInterestRate / (1
15     - 1 / (1 + monthlyInterestRate) ** (numberOfYears * 12))
16 totalPayment = monthlyPayment * numberOfYears * 12
17
18 # Display results
19 print("The monthly payment is", int(monthlyPayment * 100) / 100)
20 print("The total payment is", int(totalPayment * 100) /100)
```

```
Enter annual interest rate, e.g., 7.25: 5.75 ↵Enter
Enter number of years as an integer, e.g., 5: 15 ↵Enter
Enter loan amount, e.g., 120000.95: 250000 ↵Enter
The monthly payment is 2076.02
The total payment is 373684.53
```

variables \ line#	2	4	7	11	14	16
annualInterestRate	5.75					
monthlyInterestRate		0.0047916666666				
numberOfYears			15			
loanAmount				250000		
monthlyPayment					2076.0252175	
totalPayment						373684.539

第 2 行读取年利率，该值在第 4 行被转换为月利率。

计算月供的公式在第 14 到 15 行被翻译成 Python 代码。

变量 monthlyPayment（第 14 行）是 2076.0252175。注意：

```
int(monthlyPayment * 100) is 207602.52175
int(monthlyPayment * 100) / 100.0 is 2076.02
```

所以，第 19 行显示的是保留了小数点后两位的税款 2076.02。

第 5 阶段：测试过程

在实现程序之后，测试过程是利用几组样本输入数据来验证输出是否正确来完成的。如你在后面几章会看到的一样，某些问题会牵扯到许多情况。对于这种类型的问题，你需要设计能涵盖所有情况的测试数据。

提示：这个例子的系统设计阶段确认了几个步骤。一次增加一步来开发和测试这些步骤是一种很好的方法。这个过程可以更容易查明问题也更易于调试。

2.14　实例研究：计算距离

关键点：本节给出计算和显示两点间距离的程序。

假定有两个点，而计算距离的公式是 $\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$。你可以使用 a**0.5 来计算 $\sqrt{a}$。程序清单 2-9 中的程序提示用户键入两个点然后计算它们之间的距离。

程序清单 2-9　ComputeDistance.py

```
1  # Enter the first point with two float values
2  x1, y1 = eval(input("Enter x1 and y1 for Point 1: "))
3
4  # Enter the second point with two float values
5  x2, y2 = eval(input("Enter x2 and y2 for Point 2: "))
6
7  # Compute the distance
```

```
 8  distance = ((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2)) ** 0.5
 9
10  print("The distance between the two points is", distance)
```

```
Enter x1 and y1 for Point 1: 1.5, -3.4 ↵Enter
Enter x2 and y2 for Point 2: 4, 5 ↵Enter
The distance between the two points is 8.764131445842194
```

程序提示用户键入第一个点的坐标（第 2 行）和第二个点的坐标（第 5 行）。然后计算它们之间的距离（第 8 行）并显示这个距离（第 10 行）。

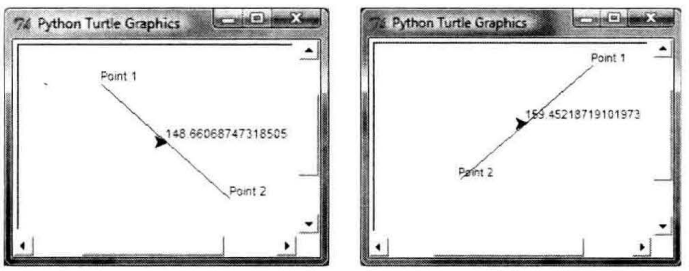


图 2-3　程序显示了一条线和它的长度

图 2-3 解释了程序清单 2-10 中的程序。这个程序：

1）提示用户键入两个点。

2）计算点之间的距离。

3）利用 Turtle 图形显示两点间的连线。

4）在线的中央显示线的长度。

程序清单 2-10 给出这个程序。

程序清单 2-10　ComputeDistanceGraphics.py

```
 1  import turtle
 2
 3  # Prompt the user for inputting two points
 4  x1, y1 = eval(input("Enter x1 and y1 for point 1: "))
 5  x2, y2 = eval(input("Enter x2 and y2 for point 2: "))
 6
 7  # Compute the distance
 8  distance = ((x1 - x2) ** 2 + (y1 - y2) ** 2) ** 0.5
 9
10  # Display two points and the connecting line
11  turtle.penup()
12  turtle.goto(x1, y1) # Move to (x1, y1)
13  turtle.pendown()
14  turtle.write("Point 1")
15  turtle.goto(x2, y2) # Draw a line to (x2, y2)
16  turtle.write("Point 2")
17
18  # Move to the center point of the line
19  turtle.penup()
20  turtle.goto((x1 + x2) / 2, (y1 + y2) / 2)
21  turtle.write(distance)
22
23  turtle.done()
```

```
Enter x1 and y1 for Point 1: -50, 34 ↵Enter
Enter x2 and y2 for Point 2: 49, -85 ↵Enter
```

程序提示用户输入两个点的值（x1，y1）和（x2，y2），然后计算它们的距离（第 4 到 8 行）。接着它移动到（x1，y1）（第 12 行），显示文本 Point 1（第 14 行），绘制从（x1，y1）到（x2，y2）的一条直线（第 15 行），显示文本 Point2（第 16 行）。最后，将它移动到线的中间（第 20 行）并显示距离（第 21 行）。

关键术语

algorithm（算法）
assignment operator(=)（赋值符（=））
augmented assignment（增强型赋值）
camelCase（驼峰拼写法）
compound assignment（复合赋值）
data type（数据类型）
expression（表达式）
floating-point numbers（浮点数）
identifiers（标识符）
incremental development and testing（递增式开发现与测试）
input, process, output (IPO)（输入、处理、输出（IPO））
keyword（关键字）
line continuation symbol（续行符）
literal（字面量）
operands（操作数）
operators（运算符）
pseudocode（伪代码）
reserved word（保留字）
scope of a variable（变量的范围）
simultaneous assignment（同时赋值）
system analysis（系统分析）
system design（系统设计）
type conversion（类型转换）
variable（变量）

本章总结

1. 可以使用 input 函数来获取输入，使用 eval 函数将字符串转化为数值。
2. 标识符是程序中使用的元素的名字。
3. 标识符是由任意长度的英文字母、数字、下划线（_）和星号（*）构成的字符序列。标识符必须以英文字母、下划线（_）开头，不能以数字开头。标识符不能是关键字。
4. 在程序中变量用于存储数据。
5. 等号（=）的作用是赋值运算符。
6. 在使用一个变量前必须对它赋值。
7. Python 中有两种数值数据类型：整数和实数。整数型（简写为 int）适用于整数，而实数型（又称浮点型）适用于有小数点的数字。
8. Python 提供执行数值运算的运算符：+（加法）、-（减法）、*（乘法）、/（除法）、//（整数除法）、%（求余）和 **（指数运算）。
9. Python 表达式中数字运算符的运算法则与算术表达式一样。
10. Python 提供增强型赋值运算符：+=（加法赋值）、-=（减法赋值）、*=（乘法赋值）、/=（浮点数除法赋值）、//=（整数除法赋值）和 %=（求余赋值）。这些运算符由 +、-、*、/、//、% 和 ** 与赋值运算符（=）组合在一起构成增强型运算符。
11. 在计算既有整型又有浮点型值的表达式时，Python 会自动将整型转化为浮点型。
12. 你可以使用 int(value) 将浮点型转换为整型。
13. 系统分析是指分析数据流并且确定系统的输入和输出。
14. 系统设计是一个程序员开发从开始输入到获取输出的流程。

15. 系统设计与分析的实质就是输入、处理、输出。这被称为 IPO。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

教学建议：指导老师可能会要求你写出指定练习题的分析与设计过程，使用自己的语言来分析问题，包括输入、输出以及需要计算什么，并用伪代码描述如何解决这个问题。

调试提示：Python 一般都会给出语法错误的原因。如果你不知道如何改正它，就将程序与书中给出的相似例子一个字符一个字符地仔细比较。

第 2.2 ～ 2.10 节

2.1 （将摄氏温度转化为华氏温度）编写一个从控制台读取摄氏温度并将它转变为华氏温度并予以显示的程序。转换公式如下所示。

```
fahrenheit = (9 / 5) * celsius + 32
```

这里是这个程序的示例运行。

```
Enter a degree in Celsius: 43 Enter
43 Celsius is 109.4 Fahrenheit
```

2.2 （计算圆柱体的体积）编写一个读取圆柱的半径和高并利用下面的公式计算圆柱体底面积和体积的程序：

```
area = radius * radius * π
volume = area * length
```

这里是示例运行。

```
Enter the radius and length of a cylinder: 5.5, 12 Enter
The area is 95.0331
The volume is 1140.4
```

2.3 （将英尺数转换为米数）编写一个程序，它读取英尺数然后将它转换成米数并显示结果。一英尺等于 0.305 米。这里是一个示例运行。

```
Enter a value for feet: 16.5 Enter
16.5 feet is 5.0325 meters
```

2.4 （将磅转换为千克）编写一个将磅转换为千克的程序。这个程序提示用户输入磅数，转换为千克数并显示结果。一磅等于 0.454 千克。这里是示例运行。

```
Enter a value in pounds: 55.5 Enter
55.5 pounds is 25.197 kilograms
```

*2.5 （财务应用程序：计算小费）编写一个读取小计和酬金率然后计算小费以及合计金额的程序。例如：如果用户键入的小计是 10，酬金率是 15%，程序就会显示小费是 1.5，合计金额是 11.5。这里是一个示例运行。

```
Enter the subtotal and a gratuity rate: 15.69, 15 Enter
The gratuity is 2.35 and the total is 18.04
```

**2.6 （对一个整数中的各位数字求和）编写一个程序，读取一个 0 到 1000 之间的整数并计算它各位数字之和。例如：如果一个整数是 932，那么它各位数字之和就是 14。（提示：使用 % 来提取数字，使用 // 运算符来去除掉被提取的数字。例如：932%10=2 而 932//10=93。）这里是一个示例运行。

```
Enter a number between 0 and 1000: 999 Enter
The sum of the digits is 27
```

**2.7 （计算年数和天数）编写一个程序，提示用户输入分钟数（例如：1 000 000），然后将分钟转换为年数和天数并显示的程序。为了简单起见，假定一年有 365 天。这里是一个示例运行。

```
Enter the number of minutes: 1000000000 Enter
1000000000 minutes is approximately 1902 years and 214 days
```

2.8 （科学：计算能量）编写一个程序，计算将水从初始温度加热到最终温度所需的能量。你的程序应该提示用户输入以千克计算的水量以及水的初始温度和最终温度。计算能量的公式是

```
Q = M * (finalTemperature - initialTemperature) * 4184
```

这里的 M 是按千克计的水量，温度为摄氏温度，热量 Q 以焦耳计。这里是一个示例运行。

```
Enter the amount of water in kilograms: 55.5 Enter
Enter the initial temperature: 3.5 Enter
Enter the final temperature: 10.5 Enter
The energy needed is 1625484.0
```

*2.9 （科学：风寒温度）室外有多冷？只有温度值是不足以提供答案的。其他因素，例如：风速、相对湿度和光照都对室外寒冷程度有很大影响。在 2001 年，国家气象局（NWS）实行以新的利用温度和风速来衡量风寒温度。这个公式如下所示。

$$t_{wc} = 35.74 + 0.6215t_a - 35.75v^{0.16} + 0.4275t_a v^{0.16}$$

这里的 *ta* 是华氏温度表示的室外温度，而 *v* 是以里 / 每小时计算的风速。t_{wc} 是风寒温度。该公式不适用于风速在每小时 2 里以下或温度在 −58 华氏度以下及 41 华氏度以上。

编写一个程序，提示用户输入一个 −58 华氏度到 41 华氏度之间的温度和一个大于等于每小时 2 里的风速，然后显示风寒温度。这里是一个示例运行。

```
Enter the temperature in Fahrenheit between -58 and 41: 5.3 Enter
Enter the wind speed in miles per hour: 6 Enter
The wind chill index is -5.56707
```

*2.10 （物理方面：计算跑道长度）假定给出飞机的加速度 a 和起飞速度 v，可以根据以下公式计算出飞机起飞所需要的最短跑道长度。

$$length = \frac{v^2}{2a}$$

编写一个程序，提示用户输入以米 / 秒（m/s）为单位的 v 和以米 / 秒的平方（m/s^2）位单位的 *a*，然后显示最短的跑道长度。这里是一个示例运行。

```
Enter speed and acceleration: 60, 3.5 Enter
The minimum runway length for this airplane is 514.286 meters
```

*2.11 （金融应用程序：投资额）假如你想将一笔钱以固定年利率存入账户。如果你希望三年之后账户中有 5000 美元，你现在需要存多少钱？使用下面的公式可以算出初始存款。

$$最初存款额 = \frac{最终金额值}{(1 + 月利率)^{月数}}$$

编写一个程序，提示用户输入最终金额值、百分比表示的年利率以及年数，然后显示最初存款额。这里是一个示例运行。

```
Enter final account value: 1000 Enter
Enter annual interest rate in percent: 4.25 Enter
Enter number of years: 5 Enter
Initial deposit value is 808.8639197424636
```

2.12 （打印表格）编写一个显示下面表格的程序。

```
a      b      a ** b
1      2      1
2      3      8
3      4      81
4      5      1024
5      6      15625
```

*2.13 （分割数字）编写一个程序，提示用户输入四位整数并以反向顺序显示。这里是一个示例运行。

```
Enter an integer: 5213 Enter
3
1
2
5
```

*2.14 （几何方面：三角形的面积）编写一个程序，提示用户输入三角形的三个顶点（x1，y1）、(x2，y2) 和（x3，y3）然后显示它的面积。计算三角形面积的公式如下所示。

$$s = (side1 + side2 + side3)/2$$

$$area = \sqrt{s(s - side1)(s - side2)(s - side3)}$$

这里是一个示例运行。

```
Enter three points for a triangle: 1.5, -3.4, 4.6, 5,
  9.5, -3.4 Enter
The area of the triangle is 33.6
```

2.15 （几何方面：正六边形的面积）编写一个程序，提示用户输入正六边形的边长并显示它的面积。计算正六边形面积的公式是 $\frac{3\sqrt{3}}{2}s^2$，其中 s 是边长。这里一个示例运行。

```
Enter the side: 5.5 Enter
The area of the hexagon is 78.5895
```

2.16 （物理方面：加速度）平均加速度的定义是速度变化量除以变化所占用的时间，如下公式所示。

$$a = \frac{v_1 - v_0}{t}$$

编写一个程序，提示用户输入以米每秒为单位的初始速度 v_0 和末速度 v_1，以秒为单位速度变化所占用的时间 t，然后显示平均加速度。这里是一个示例运行。

```
Enter v0, v1, and t: 5.5, 50.9, 4.5 Enter
The average acceleration is 10.0889
```

*2.17 （健康应用程序：计算 BMI）身体质量指数（BMI）是以体重衡量健康程度的一种指数。以千克为单位的体重除以以米为单位的身高的平方就可以计算它的值。编写一个程序，提示用户输入

以磅为单位的体重和以英尺为单位的身高，然后显示 BMI 的值。注意：1 磅等于 0.453 592 37 千克而 1 英尺等于 0.0254 米。这里是一个示例运行。

```
Enter weight in pounds: 95.5 Enter
Enter height in inches: 50 Enter
BMI is 26.8573
```

第 2.11 ～ 2.13 节

*2.18 （当前时间）程序清单 2-7 给出的程序显示当前的 GMT 时间。修改程序使之提示用户输入时区，这个时区是用距离 GMT 的小时数表示，然后显示指定时区的时间。这里是一个示例运行。

```
Enter the time zone offset to GMT: -5 Enter
The current time is 4:50:34
```

*2.19 （金融应用程序：计算未来投资额）使用下面的公式编写一个读取投资额、年利率和年数然后显示未来投资额的程序：

$$未来投资额 = 投资额 \times (1 + 月投资率)^{月数}$$

例如：如果你输入金额 1000，而年利率为 4.25%，年数为 1，那么未来投资总额就是 1043.33。这里是一个示例运行。

```
Enter investment amount: 1000 Enter
Enter annual interest rate: 4.25 Enter
Enter number of years: 1 Enter
Accumulated value is 1043.33
```

*2.20 （金融应用程序：计算利息）如果你知道差额和百分比的年利率，你可以使用下面的公式计算下个月月供的利息。

$$利息 = 差额 \times (年利率 / 1200)$$

编写一个读取差额和年利率，然后显示下月要付利息的程序。这里是一个示例运行。

```
Enter balance and interest rate (e.g., 3 for 3%): 1000, 3.5 Enter
The interest is 2.91667
```

**2.21 （金融应用程序：复利值）假设你每月存 100 美元到一个年利率为 5% 的储蓄账户。因此，月利率是 0.05/12=0.004 17。第一个月后，账户里的数目变为：

```
100 * (1 + 0.00417) = 100.417
```

第二个月后，账户里的数目变为：

```
(100 + 100.417) * (1 + 0.00417) = 201.252
```

第三个月后，账户里的数目变为：

```
(100 + 201.252) * (1 + 0.00417) = 302.507
```

依次类推。

编写一个程序，提示用户键入每月存款数然后显示六个月后的账户总额。这里是程序的一个示例运行。

```
Enter the monthly saving amount: 100 Enter
After the sixth month, the account value is 608.81
```

2.22 （人口预测）改写练习题 1.11 来提示用户键入年数，然后显示那么多年后的人口数。这里是程序的一个示例运行。

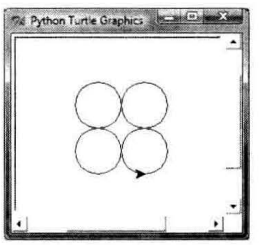
```
Enter the number of years: 5 Enter
The population in 5 years is 325932970
```

第 2.14 节

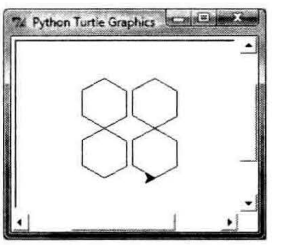
2.23 （Turtle：绘制四个圆）编写一个如图 2-4a 所示的程序，提示用户输入半径并在屏幕中央画四个圆。

2.24 （Turtle：绘制四个正六边形）编写一个如图 2-4b 所示的程序，在屏幕中央画四个正六边形。

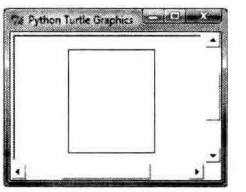
**2.25 （Turtle：绘制一个矩形）编写一个如图 2-4c 所示的程序，提示用户输入矩形中心、长和宽，然后显示这个矩形。



a）绘制四个圆



b）绘制正六边形



c）绘制矩形

图　2-4

**2.26 （Turtle：绘制一个圆）编写一个如图 2-5 所示的程序，提示用户输入圆心和半径并在屏幕中央显示圆和它的面积。

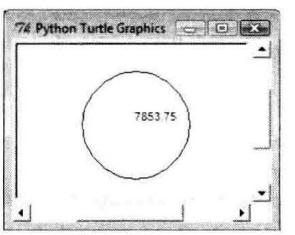


图 2-5　显示圆和它的面积

第 3 章

Introduction to Programming Using Python

数学函数、字符串和对象

学习目标

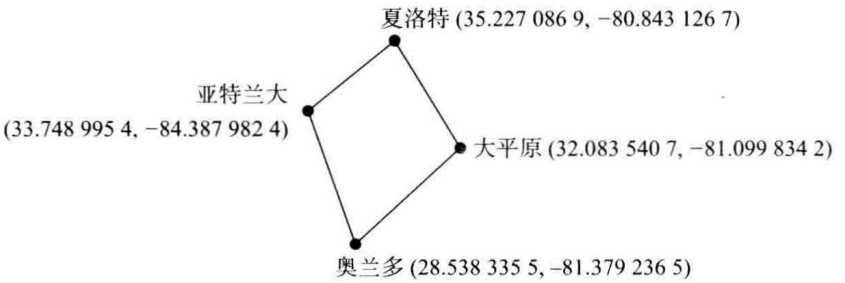
- 使用 math 模块中的函数解决数学问题（第 3.2 节）。
- 表示和处理字符串和字符（第 3.3 ～ 3.4 节）。
- 使用 ASCII 和 Unicode 对字符编码（第 3.3.1 ～ 3.3.2 节）。
- 使用 ord 函数获取一个字符的数值编码以及使用 chr 函数将一个数值编码转换成一个字符（第 3.3.3 节）。
- 使用转义序列表示特殊字符（第 3.3.4 节）。
- 调用带参数 end 的 print 函数（第 3.3.5 节）。
- 使用 str 函数将数字转换成字符串（第 3.3.6 节）。
- 使用运算符 + 来连接字符串（第 3.3.7 节）。
- 从键盘读取字符串（第 3.3.8 节）。
- 介绍对象和方法（第 3.5 节）。
- 使用 format 函数格式化数字和字符串（第 3.6 节）。
- 绘制各种不同的图形（第 3.7 节）。
- 绘制带颜色和字体的图形（第 3.8 节）。

3.1 引言

关键点：本章的重点是介绍函数、字符串和对象以及使用它们来开发程序。

前面的章节介绍了基本的程序设计方法并且教你如何编写简单的程序来解决基本问题。本章介绍 Python 函数来执行常见的数学运算。你将在第 6 章学习如何创建自定义的函数。

假如你需要估计被四个城市所包围的面积，而这四个城市的 GPS 位置（经度和纬度）是已知的，如下图所示。你怎样编写一个程序来解决这个问题？在完成本章的学习之后，你就能够写出这样一个程序。



因为 Python 中所有的数据都是对象，所以有必要早点引进对象，这样就可以开始用它们来开发有用的程序。本章只是简单地介绍了对象和字符串；本书将在第 7 章和第 8 章里进一步介绍对象和字符串。

3.2　常见的 Python 函数

关键点：Python 提供了许多有用的用于解决常见程序设计任务的函数。

函数是完成一个特殊任务的一组语句。Python 语言和其他程序设计语言一样，都提供了一个函数库。你已经使用过 eval、input、print 和 int 函数。这些都是内置函数并且在 Python 解释器里均可用。所以使用这些函数你不用导入任何模块。除此之外，你还可以使用 abs、max、min、pow 和 round 等内置函数，如表 3-1 所示。

表 3-1　简单的 Python 内置函数

函数	描述	举例
abs(x)	返回 x 的绝对值	abs(−2)=2
max(x1,x2,…)	返回 x1,x2,…的最大值	max(1,5,2)=5
min(x1,x2,…)	返回 x1,x2,…的最小值	min(1,5,2)=1
pow(a,b)	返回 a^b 的值，类似 a ** b	pow(2,3)=8
round(x)	返回与 x 最接近的整数，如果 x 与两个整数接近程度相同，则返回偶数值	round(5.4)=5 round(5.5)=6 round(4.5)=4
round(x,n)	保留小数点后 n 位小数的浮点值	round(5.466,2)=5.47 round(5.463,2)=5.46

例如：

```
>>> abs(-3) # Returns the absolute value
3
>>> abs(-3.5) # Returns the absolute value
3.5
>>> max(2, 3, 4, 6) # Returns the maximum number
6
>>> min(2, 3, 4) # Returns the minimum number
2
>>> pow(2, 3) # Same as 2 ** 3
8
>>> pow(2.5, 3.5) # Same as 2.5 ** 3.5
24.705294220065465
>>> round(3.51) # Rounds to its nearest integer
4
>>> round(3.4) # Rounds to its nearest integer
3
>>> round(3.1456, 3) # Rounds to 3 digits after the decimal point
3.146
>>>
```

我们常常为解决数学问题创建一些程序。Python 的 math 模块提供了许多数学函数，如表 3-2 所示。

表 3-2　数学函数

函数	描述	举例
fabs(x)	将 x 看作一个浮点数，返回它的绝对值	fabs(−2)=2.0
ceil(x)	x 向上取最近的整数，然后返回这个整数	ceil(2.1)=3 ceil(−2.1)=−2
floor(x)	x 向下取最近的整数，然后返回这个整数	floor(2.1)=2 floor(−2.1)=−3
exp(x)	返回幂函数 e^x 的值	exp(1)=2.718 28

（续）

函数	描述	举例
log(x)	返回 x 的自然对数值	log(2.718 28)=1
log(x,base)	返回以某个特殊值为底的 x 的对数值	log(100,10)=2.0
sqrt(x)	返回 x 的平方根值	sqrt(4.0)=2
sin(x)	返回 x 的正弦值，x 是角度的弧度值	sin(3.141 59/2)=1
asin(x)	返回 asin 的弧度值	asin(1.0)=1.57
cos(x)	返回 x 的余弦值，x 是角度的弧度值	cos(3.141 59)=−1
acos(x)	返回 acos 的弧度值	acos(1.0)=0
tan(x)	返回 tan（x）的值，x 是角度的弧度值	tan(0.0)=0
degrees(x)	将 x 从弧度转换成角度	degrees(1.57)=90
radians(x)	将 x 从角度转换为弧度	radians(90)=1.57

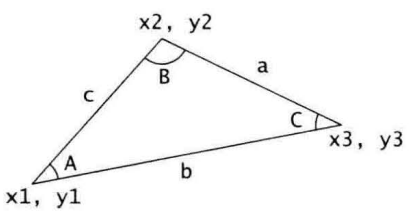
两个数学常量 pi 和 e 也定义在 math 模块中。我们可以通过使用 math.pi 和 math.e 来访问它们。程序清单 3-1 是一段测试一些数学函数的程序。由于这段程序使用了定义在 math 模块中的数学函数，所以 math 模块应该在第一行被导入。

程序清单 3-1　MathFunctions.py

```
1  import math # import math module to use the math functions
2
3  # Test algebraic functions
4  print("exp(1.0) =", math.exp(1))
5  print("log(2.78) =", math.log(math.e))
6  print("log10(10, 10) =", math.log(10, 10))
7  print("sqrt(4.0) =", math.sqrt(4.0))
8
9  # Test trigonometric functions
10 print("sin(PI / 2) =", math.sin(math.pi / 2))
11 print("cos(PI / 2) =", math.cos(math.pi / 2))
12 print("tan(PI / 2) =", math.tan(math.pi / 2))
13 print("degrees(1.57) =", math.degrees(1.57))
14 print("radians(90) =", math.radians(90))
```

```
exp(1.0) = 2.71828182846
log(2.78) = 1.0
log10(10, 10) = 1.0
sqrt(4.0) = 2.0
sin(PI / 2) = 1.0
cos(PI / 2) = 6.12323399574e-17
tan(PI / 2) = 1.63312393532e+16
degrees(1.57) = 89.9543738355
radians(90) = 1.57079632679
```

你可以使用数学函数解决许多计算问题。例如：已知三角形的三条边，你可以使用下面的公式计算出三角形的三个角。




```
A = acos((a * a - b * b - c * c) / (-2 * b * c))
B = acos((b * b - a * a - c * c) / (-2 * a * c))
C = acos((c * c - b * b - a * a) / (-2 * a * b))
```

别被数学公式吓到！就像我们之前在程序清单 2-8 中讨论的那样，为了编写一个计算贷款支付额的程序，我们没必要知道计算公式是如何被推导出来的。上面给出的例子中已知三角形三边的长度，你没必要知道里面的公式是怎么被推导出来的就能写出这样一个计算角度的程序。为了计算三边的长度，我们需要知道三个顶点的坐标并计算两点之间的距离。

程序清单 3-2 是一个示例程序，该程序提示用户输入三角形三个顶点的 x 坐标和 y 坐标，然后显示三个角度。

程序清单 3-2 ComputeAngles.py

```
1  import math
2
3  x1, y1, x2, y2, x3, y3 = eval(input("Enter three points: "))
4
5  a = math.sqrt((x2 - x3) * (x2 - x3) + (y2 - y3) * (y2 - y3))
6  b = math.sqrt((x1 - x3) * (x1 - x3) + (y1 - y3) * (y1 - y3))
7  c = math.sqrt((x1 - x2) * (x1 - x2) + (y1 - y2) * (y1 - y2))
8
9  A = math.degrees(math.acos((a * a - b * b - c * c) / (-2 * b * c)))
10 B = math.degrees(math.acos((b * b - a * a - c * c) / (-2 * a * c)))
11 C = math.degrees(math.acos((c * c - b * b - a * a) / (-2 * a * b)))
12
13 print("The three angles are ", round(A * 100) / 100.0,
14     round(B * 100) / 100.0, round(C * 100) / 100.0)
```

```
Enter three points: 1, 1, 6.5, 1, 6.5, 2.5 ↵Enter
The three angles are 15.26 90.0 74.74
```

本程序提示用户输入三个点（第 3 行）。这条提示信息不是很清楚。所以，应该给用户明确指示如何输入三个点，如下所示。

```
input("Enter six coordinates of three points separated by commas\
like x1, y1, x2, y2, x3, y3: ")
```

这个程序计算两点之间的距离（第 5 ～ 7 行），并且应用公式计算角度（第 9 ～ 11 行）。在第 13 ～ 14 行，以四舍五入保留小数点后两位显示这些角度（第 13 ～ 14 行）。

注意，（x2−x3）*（x2−x3）可以简写成（x2−x3）** 2，round（A*100）/100.0 可以简写成 round（A，2）。

检查点

3.1　计算下面的函数：

(a) `math.sqrt(4)`
(b) `math.sin(2 * math.pi)`
(c) `math.cos(2 * math.pi)`
(d) `min(2, 2, 1)`
(e) `math.log(math.e)`
(f) `math.exp(1)`
(g) `max(2, 3, 4)`
(h) `abs(-2.5)`
(i) `math.ceil(-2.5)`
(j) `math.floor(-2.5)`
(k) `round(3.5)`
(l) `round(-2.5)`
(m) `math.fabs(2.5)`
(n) `math.ceil(2.5)`
(o) `math.floor(2.5)`
(p) `round(-2.5)`
(q) `round(2.6)`
(r) `round(math.fabs(-2.5))`

3.2　三角函数的参数代表一个用弧度表示的角度，对不对？

3.3　编写一条语句，将 47 度角转换成弧度，然后将结果赋值给一个变量。

3.4　编写一条语句，将 π / 7 转换成角度，然后将结果赋值给一个变量。

3.3　字符串和字符

关键点：字符串（第 1 章讲过）是一连串的字符。Python 处理字符和字符串的方式是一样的。

在 Python 里，你除了可以处理数值，还可以处理字符串。一个字符串就是一串包括文本和数字的字符。字符串必须被括在一对单引号 (') 或者双引号 (") 里。Python 没有字符数据类型。一个字符的字符串代表一个字符。例如：

```
letter = 'A' # Same as letter = "A"
numChar = '4' # Same as numChar = "4"
message = "Good morning" # Same as message = 'Good morning'
```

第一条语句将只有字符 A 的字符串赋值给变量 letter。第二条语句将只有数字字符 4 的字符串赋值给变量 number。第三条语句将字符串“good morning”赋值给变量 message。

注意：本书统一使用双引号来括住多个字符构成的字符串，用单引号来括住单个字符的字符串或空字符串。这个习惯与其他程序设计语言是一致的，因此很容易就能让你将一个 Python 程序转换成其他语言程序。

3.3.1　ASCII 码

计算机在内部是使用二进制数的（参见第 1.2.2 节）。在计算机里，一个字符被存储为一连串的 0 和 1。把一个字符映射成它对应的二进制被称为字符编码。对字符编码的方式有很多。编码表定义编码字符的方式。流行的编码标准是 ASCII（美国信息交换标准代码），它是一个比特的编码表，足以表示所有的大小写字母、数字、标点符号以及控制字符。ASCII 码使用 0 到 127 来表示字符。附录 B 中给出 ASCII 码表示的字符。

3.3.2　统一码

Python 也支持统一码。统一码是一种编码表，它能表示国际字符。ASCII 码表是统一码的子集。统一码由统一码协会（Unicode Consortium）建立，支持世界上各种语言所写的文本进行交换、处理和展示。一个统一码以“\u”开始，后面紧跟四个十六进制数字，它们从“\u0000 到 \uFFFF”（有关十六进制数的信息参见附录 C）。例如，“welcome”被翻译成中文后就是两个字符：“欢”和“迎”。这两个字符的统一码表示是“\u6B22\u8FCE”。

程序清单 3-3 中的程序显示两个中文字符和三个希腊字母，如图 3-1 所示。

程序清单 3-3　DisplayUnicode.py

```
1  import turtle
2
3  turtle.write("\u6B22\u8FCE \u03b1 \u03b2 \u03b3")
4
5  turtle.done()
```

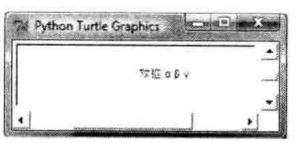


图 3-1　使用统一码在 Python 的 GUI 程序中显示国际字符

如果你的系统里没有安装中文字体，你将看不到相应的中文字符。在这种情况下，为了避免错误，就从你的程序里删除“\u6B22\u8FCE”。希腊字母 α、β、γ 的统一码表示是“\u03b1”、“\u03b2”和“\u03b3”。

3.3.3　函数 ord 和 chr

Python 提供 ord（ch）函数来返回字符 ch 的 ASCII 码，用 chr（code）函数返回 code 所

代表的字符。例如：

```
>>> ch = 'a'
>>> ord(ch)
97
>>> chr(98)
'b'
>>> ord('A')
65
>>>
```

a 的 ASCII 码值是 97，比 A（65）的编码值要大。小写字母的 ASCII 码是从 a 开始，然后是 b、c 依次类推直到 z 的连续整数。大写字母也是一样的。任何小写字母的 ASCII 码与它对应的大写字母的 ASCII 码的差值都一样：32。这是一个很有用的处理字符的特性。例如，任何小写字母的大写字母，如下代码所示：

```
1  >>> ord('a') - ord('A')
2  32
3  >>> ord('d') - ord('D')
4  32
5  >>> offset = ord('a') - ord('A')
6  >>> lowercaseLetter = 'h'
7  >>> uppercaseLetter = chr(ord(lowercaseLetter) - offset)
8  >>> uppercaseLetter
9  'H'
10 >>>
```

第六行将一个小写字母赋值给 lowercaseletter。第七行获取它对应的大写字母。

3.3.4 转义序列

假如你想输出带有引号的字符串。你能编写如下所示的语句吗？

```
print("He said, "John's program is easy to read"")
```

答案是不行！这条语句有一个错误。因为 Python 认为第二个双引号就是这个字符串的结尾，因此，它就不知道该如何处理剩下的字符。

为了解决这个问题，Python 使用一种特殊的符号来表示特殊的字符，如表 3-3 所示。这种由反斜杠“\”和其后紧接着的字母或数字组合构成的特殊符号被称为*转义序列*。

表 3-3　Python 的转义序列

字符转义序列	名称	数值
\b	退格符	8
\t	制表符	9
\n	换行符	10
\f	换页符	12
\r	回车符	13
\\	反斜线	92
\'	单引号	39
\"	双引号	34

字符“\n”也被称为*换行符*或*行结束*（EOL）字符，它们表示一行的结束。字符“\f”让打印机从下一页打印。字符“\r”被用来把光标移动到同一行的第一个位置。字符“\f”和“\r”在本书中很少被用到。

现在，你可以使用下面的语句打印带引号的消息：

```
>>> print("He said, \"John's program is easy to read\"")
He said, "John's program is easy to read"
```

注意：符号 \ 和 " 在一起代表一个字符。

3.3.5 不换行打印

当使用 print 函数时，它会自动打印一个换行符，这会导致输出提前进入下一行。如果你并不想在使用 print 函数后换行，可以使用下面的语法在调用 print 函数时传递一个特殊的参数 end = “anyendingstring”：

```
print(item, end = "anyendingstring")
```

例如，下面的代码：

```
1 print("AAA", end = ' ')
2 print("BBB", end = '')
3 print("CCC", end = '***')
4 print("DDD", end = '***')
```

显示：

```
AAA BBBCCC***DDD***
```

第 1 行打印 AAA 和一个空字符' '，第 2 行打印 BBB，第三行打印 CCC 和 ***，第 4 行打印 DDD 和 ***。注意：第 2 行的''表示一个空字符串。所以，''不会打印任何内容。

你也可以使用下面的语法使用 end 参数打印各项条目：

```
print(item1, item2, ..., end = "anyendingstring")
```

例如：

```
radius = 3
print("The area is", radius * radius * math.pi, end = ' ')
print("and the perimeter is", 2 * radius * math.pi)
```

显示

```
The area is 28.26 and the perimeter is 6
```

3.3.6 函数 str

不同于 char

str 函数可以将一个数字转换成一个字符串。例如：

```
>>> s = str(3.4) # Convert a float to string
>>> s
'3.4'
>>> s = str(3) # Convert an integer to string
>>> s
'3'
>>>
```

3.3.7 字符串连接操作

你可以使用运算符 + 来对两个数字做加法。你也可以使用 + 运算符来连接两个字符串。下面是一些例子：

```
1 >>> message = "Welcome " + "to " + "Python"
2 >>> message
3 'Welcome to Python'
4 >>> chapterNo = 3
5 >>> s = "Chapter " + str(chapterNo)
6 >>> s
7 'Chapter 3'
8 >>>
```

第 1 行把三个字符串连接成一个。在第 5 行，str 函数将变量 chapterNO 中的数值转换成一个字符串。这个字符串与“Chapter”连接在一起得到一个新字符串“Chapter 3”。

增强型赋值运算符 += 也能用来连接字符串。例如：下面的代码就将字符串“ message”与字符串“and Python is fun”连接在一起。

```
>>> message = "Welcome to Python"
>>> message
'Welcome to Python'
>>> message += " and Python is fun"
>>> message
'Welcome to Python and Python is fun'
>>>
```

3.3.8 从控制台读取字符串

为了从控制台读取一个字符串，可以使用 input 函数。例如：下面的代码从键盘读取了三个字符串：

```
s1 = input("Enter a string: ")
s2 = input("Enter a string: ")
s3 = input("Enter a string: ")
print("s1 is " + s1)
print("s2 is " + s2)
print("s3 is " + s3)
```

```
Enter a string: Welcome ↵Enter
Enter a string: to ↵Enter
Enter a string: Python ↵Enter
s1 is Welcome
s2 is to
s3 is Python
```

检查点

3.5　使用 ord 函数找出 1、A、B、a 和 b 的 ASCII 码，使用 chr 函数找出十进制数 40、59、79、85 和 90 所对应的字符。

3.6　如何显示字符 \ 和 " ?

3.7　如何用统一码编写一个字符?

3.8　假如运行下面程序的时候输入 A。那么输出什么?

```
x = input("Enter a character: ")
ch = chr(ord(x) + 3)
print(ch)
```

3.9　假如运行下面的程序的时候输入 A 和 Z。那么输出什么?

```
x = input("Enter a character: ")
y = input("Enter a character: ")
print(ord(y) - ord(x))
```

3.10　下面的代码错在哪里？你能改正吗?

```
title = "Chapter " + 1
```

3.11　显示下面代码的结果。

```
sum = 2 + 3
print(sum)
s = '2' + '3'
print(s)
```

3.4　实例研究：最小数量的硬币

现在，我们来看一个使用本节所讲的特性的示例程序。假如你想开发一个程序将一定数量的钱分类成几个更小货币单元。该程序让用户输入总金额，这是一个用美元和美分表示的浮点值，然后输出一个报告，罗列出等价的货币：美元、两角五分硬币、一角硬币、五分硬币以及美分个数，如示例运行所示。

你的程序应该报告最大数目的美元，然后依次是二角五分硬币、一角硬币、五分硬币以及美分个数，这样就得到最小量的硬币。

下面是编写这个程序的步骤：

1）提示用户输入一个十进制带小数点的数字，例如：11.56。

2）将钱数（11.56）转换成分数（1156）。

3）将分数除以 100 得到美元个数。使用分数 %100 得到余数即是剩余的分数。

4）将剩余的分数除以 25 得到两角五分硬币的个数。使用分数 %25 得到余数即是剩余的分数。

5）将剩余的分数除以 10 得到一角硬币的个数。使用分数 %10 得到余数即是剩余的分数。

6）将剩余的分数除以 5 得到五分硬币的个数。使用分数 %5 得到余数即是剩余的分数。

7）剩余的分数就是一美分硬币数。

8）显示结果。

完整的程序如程序清单 3-4 所示。

程序清单 3-4　ComputeChange.py

```
# Receive the amount
amount = eval(input("Enter an amount, for example, 11.56: "))

# Convert the amount to cents
remainingAmount = int(amount * 100)

# Find the number of one dollars
numberOfOneDollars = remainingAmount // 100
remainingAmount = remainingAmount % 100

# Find the number of quarters in the remaining amount
numberOfQuarters = remainingAmount // 25
remainingAmount = remainingAmount % 25

# Find the number of dimes in the remaining amount
numberOfDimes = remainingAmount // 10
remainingAmount = remainingAmount % 10

# Find the number of nickels in the remaining amount
numberOfNickels = remainingAmount // 5
remainingAmount = remainingAmount % 5

# Find the number of pennies in the remaining amount
numberOfPennies = remainingAmount

# Display the results
print("Your amount", amount, "consists of\n",
    "\t", numberOfOneDollars, "dollars\n",
    "\t", numberOfQuarters, "quarters\n",
    "\t", numberOfDimes,  "dimes\n",
    "\t", numberOfNickels, "nickels\n",
    "\t", numberOfPennies, "pennies")
```

```
Enter an amount, for example, 11.56: 11.56 Enter
Your amount 11.56 consists of
     11 dollars
      2 quarters
      0 dimes
      1 nickels
      1 pennies
```

variables \ line#	2	5	8	9	12	13	16	17	20	21	24
amount	11.56										
remainingAmount		1156		56		6		6		1	
numberOfOneDollars			11								
numberOfQuarters					2						
numberOfDimes							0				
numberOfNickels									1		
numberOfPennies											1

变量 amount 存储的是来自控制台的变量（第 2 行）。这个变量保持不变，因为 amount 必须在程序的结尾显示结果。程序引入一个变量 remainingamount（第 5 行）以存储变化的 remainingAmount。

变量 amount 是一个浮点数，代表的是美元和美分。它被转换为一个表示美分的整型变量 remainingamount。例如：如果 amount 为 11.56，那么 remainingamount 的初始值是 1156。1156//100 是 11（第 8 行）。求余运算符得到除法的余数。因此，1156%100=56（第 9 行）。

这个程序是从 remainingamount 中提取出最大数目的两角五分硬币，然后获得一个新的 remainingamount（第 12 ～ 13 行）。持续相同的过程，程序就可以在剩余数目中得到一角硬币、五分硬币和美分的最大数目。

如示例运行所示，结果中显示：0 个一角硬币、1 个五分硬币和 1 个美分。如果不显示 0 个一角硬币而只是显示 1 个五分硬币和 1 个美分的话就更好了。你将在下一章学习如何使用选择语句修改这个程序（参见编程题 4.7）。

警告：这个例子涉及的一个严重问题是在将一个浮点数转换成整型 remainingamount 的时候可能会损失精度。这就可能导致一个不准确的结果。如果你试图输入 10.03，10.03*100 可能是 1003.999 999 999 999 9。你就会发现程序最终结果为 10 美元和 2 美分。为了解决这个问题，输入用美分表示的整型数值（参见编程题 3.8）。

3.5　对象和方法简介

关键点：在 Python 中，所有的数据（包括数字和字符串）实际都是对象。

在 Python 中，一个数字是一个对象，一个字符串是一个对象，每个数据都是对象。同一类型的对象都有相同的类型。你可以使用 id 函数和 type 函数来获取关于对象的一些信息。例如：

```
1  >>> n = 3  # n is an integer
2  >>> id(n)
```

```
 3  505408904
 4  >>> type(n)
 5  <class 'int'>
 6  >>> f = 3.0  # f is a float
 7  >>> id(f)
 8  26647120
 9  >>> type(f)
10  <class 'float'>
11  >>> s = "Welcome" # s is a string
12  >>> id(s)
13  36201472
14  >>> type(s)
15  <class 'str'>
16  >>>
```

当执行程序的时候，Python 会自动为对象的 id 赋一个独特的整数。在程序的执行过程中，对象的 id 不会改变。然而，每当执行程序时，Python 都可能会赋一个不同的 id。Python 按照对象的值决定对象的类型。第 2 行显示数字对象 n 的 id，第 3 行显示的是 Python 已经被赋值给对象的 id，而第 4 行显示它的类型。

在 Python 中，一个对象的类型由类决定。例如：字符串的类是 str（第 15 行），整数的类是 int（第 5 行），浮点数的类是 float（第 10 行）。术语“class”来自面向对象程序设计，这些都会在第 7 章中讨论。在 Python 中，类（class）和类型（type）是一样的意思。

注意：id 和 type 函数在程序设计里很少用到，但是它们是学习更多有关对象的好工具。

Python 中的变量实际上是一个对象的引用。图 3-2 显示前面的代码中变量和对象之间的关系。

图 3-2　在 Python 中，每个变量实际就是一个指向对象的引用

第 1 行的语句“n=3”是将 3 赋值给了 n，实际上是将 3 赋值给了一个 int 对象，这个对象是由变量 n 引用的。

注意：对于 n=3，我们可以说 n 是一个整型变量，其值为 3。严格说来，n 是一个引用了 int 对象的变量，而这个 int 对象的值为 3。简单讲，说 n 是一个值为 3 的整型变量也可以。

你可以在一个对象上执行操作。操作是用函数定义的。Python 中对象所用的函数被称为方法。方法只能从一个特定的对象里调用。例如：字符串类型里有像 lower() 和 upper() 这样的方法，它们返回大写字母或小写字母写成的新字符串。下面是一些如何调用这些方法的例子。

```
1  >>> s = "Welcome"
2  >>> s1 = s.lower() # Invoke the lower method
3  >>> s1
4  'welcome'
5  >>> s2 = s.upper() # Invoke the upper method
6  >>> s2
7  'WELCOME'
8  >>>
```

第 2 行调用对象 s 上的 s.lower() 方法，返回一个小写字母表示的新字符串，然后将它赋值给 s1。第 5 行调用对象 s 上的 s.upper() 函数，返回一个大写字母表示的新字符串，然后将它赋值给 s2。

正如你在之前的例子中所看到的那样，一个对象调用方法的语法就是 object.method()。

另外一个有用的字符串方法是 strip()，它能被用来移除一个字符串两端的空格符。字符 ' '、\t、\f、\r 和 \n 都是空格符。

例如：

```
>>> s = "\t Welcome \n"
>>> s1 = s.strip() # Invoke the strip method
>>> s1
'Welcome'
>>>
```

注意：如果你在 Eclipse 上使用 Python，Eclipse 会自动在 input 函数输入的字符串后追加 \r。因此，你应该用 strip() 方法移除字符 \r：

```
s = input("Enter a string").strip()
```

有关处理字符串和面向对象程序设计的更多细节将在第 7 章中讨论。

检查点

3.12　什么是对象？什么是方法？

3.13　如何找到一个对象的 id？如何找到一个对象的类型？

3.14　下面哪种陈述是语句 "n=3" 最准确的含义？

（a）n 是一个拥有整型值 3 的变量。

（b）n 是一个对象的引用，该对象的值为整数 3。

3.15　假如 s 是 "\tGeorgia\n"，那么 s.lower() 和 s.upper() 是什么？

3.16　假如 s 是 "\tGood\tMorning\n"，那么 s.strip() 是什么？

3.6　格式化数字和字符串

关键点：你可以使用 format 函数返回格式化的字符串。

我们常常希望显示某种格式的数字。例如：已知数额和年利率，下面是计算利息的代码。

```
>>> amount = 12618.98
>>> interestRate = 0.0013
>>> interest = amount * interestRate
>>> print("Interest is", interest)
Interest is 16.404674
>>>
```

因为利息数是货币，因此我们希望只是显示小数点后两位数。为了达到这点要求，我们编写如下代码。

```
>>> amount = 12618.98
>>> interestRate = 0.0013
>>> interest = amount * interestRate
>>> print("Interest is", round(interest, 2))
Interest is 16.4
>>>
```

然而，格式依旧不正确。小数点后应该有两位小数，就像 16.40 而不是 16.4。你可以使用 format 函数来修改它，如下所示。

```
>>> amount = 12618.98
>>> interestRate = 0.0013
>>> interest = amount * interestRate
>>> print("Interest is", format(interest, ".2f"))
Interest is 16.40
>>>
```

调用这个函数的语法是：

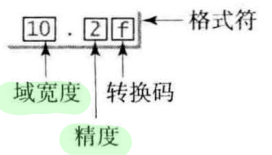
```
format(item, format-specifier)
```

上面的 item 是数字或者字符串，而格式说明符（format-specifier）指定条目 item 的格式。此函数返回一个字符串。

3.6.1　格式化浮点数

如果条目 item 是一个浮点值，你可以用标识符以“*width.precisionf*”的形式给出格式的宽度和精确度。这里的宽度 width 指定得到的字符串的宽度，精确度 precision 指定小数点后数字的个数，而 f 被称为转换码，它为浮点数设定格式。例如：

```
print(format(57.467657, "10.2f"))
print(format(12345678.923, "10.2f"))
print(format(57.4, "10.2f"))
print(format(57, "10.2f"))
```



显示

```
|←— 10 —→|
□□□□ 57.47
 12345678.92
□□□□ 57.40
□□□□ 57.00
```

这里的方箱子（□）表示一个空格。注意：小数点占一个空格。

函数 format（“10.2f”）将数字格式化成宽度为 10，包括小数点以及小数点后两位小数的字符串。这个数字被四舍五入到两个小数位。这样，在小数点前分配 7 个数字。如果在小数点前的数字小于 7 个，则在数字前插入空格。如果小数点前的数字个数大于 7，则数字的宽度将会自动增加。例如：format(12345678.923，“10.2”) 返回的是 12345678.92，它的宽度为 11。

你也可以省略宽度符。如果这样的话，它就被默认为是 0。这样，宽度就会根据格式化这个数所需的宽度自动设置。例如：

```
print(format(57.467657, "10.2f"))
print(format(57.467657, ".2f"))
```

显示：

```
|←— 10 —→|
□□□□ 57.47
57.47
```

3.6.2　用科学记数法格式化

如果你将转换码 f 变成 e，数字将被格式化为科学记数法。例如：

```
print(format(57.467657, "10.2e"))
print(format(0.0033923, "10.2e"))
print(format(57.4, "10.2e"))
print(format(57, "10.2e"))
```

显示：

```
|←——10——→|
□□5.75e+01
□□3.39e-03
□□5.74e+01
□□5.70e+01
```

符号“+”和“−”被算在宽度里。

3.6.3　格式化成百分数

可以使用转换码“%”将一个数字格式化成百分数。例如：

```
print(format(0.53457, "10.2%"))
print(format(0.0033923, "10.2%"))
print(format(7.4, "10.2%"))
print(format(57, "10.2%"))
```

显示：

```
|←——10——→|
□□□□53.46%
□□□□□0.34%
□□□740.00%
□□5700.00%
```

格式“10.2%”将数乘以 100 后加上符号“%”。符号 % 也被算在宽度里面。

3.6.4　调整格式

在默认情况下，一个数的格式是向右对齐的。可以将符号“<”放在格式说明符里指定得到的字符串是以指定的宽度向左对齐的。例如：

```
print(format(57.467657, "10.2f"))
print(format(57.467657, "<10.2f"))
```

显示：

```
|←——10——→|
□□□□□57.47
57.47
```

3.6.5　格式化整数

“d”、“x”、“o”和“b”转换码分别用来格式化十进制整数、十六进制整数、八进制整数和二进制整数。可以指定转换的宽度。例如：

```
print(format(59832, "10d"))
print(format(59832, "<10d"))
print(format(59832, "10x"))
print(format(59832, "<10x"))
```

显示：

```
|←— 10 —→|
□□□□ 59832
59832
□□□□ e9b8
e9b8
```

格式说明符“10d”指定将一个整数格式化为一个宽度为 10 的十进制数。格式说明符“10x”指定将一个整数格式化为一个宽度为 10 的十六进制数。

3.6.6 格式化字符串

可以用转换码 s 将一个字符串格式化为一个指定宽度的字符串。例如：

```
print(format("Welcome to Python", "20s"))
print(format("Welcome to Python", "<20s"))
print(format("Welcome to Python", ">20s"))
print(format("Welcome to Python and Java", ">20s"))
```

显示：

```
|←———— 20 ————→|
Welcome to Python
Welcome to Python
□□□ Welcome to Python
Welcome to Python and Java
```

格式说明符“20s”指定字符串被格式化为宽度在 20 以内的字符串。在默认情况下，字符串是向左对齐的。为了向右对齐，在格式符里加入“>”符号。如果字符串比指定的宽度长，宽度将自动增加到字符串的宽度。

表 3-4 总结了这一小节介绍的格式说明符。

表 3-4　常用的说明符

说明符	格式
"10.2f"	格式化浮点数，宽度为 10 精度为 2
"10.2e"	格式化浮点数（以科学记数法表示），宽度为 10 精度为 2
"5d"	将整数格式化为宽度为 5 的十进制数
"5x"	将整数格式化为宽度为 5 的十六进制数
"5o"	将整数格式化为宽度为 5 的八进制数
"5b"	将整数格式化为宽度为 5 的二进制数
"10.2%"	将数格式化为十进制数
"50s"	将字符串格式化为宽度为 50 的字符串
"<10.2f"	向左对齐格式化项目
">10.2f"	向右对齐格式化项目

检查点

3.17　调用 format 函数，它的返回值是什么？

3.18　如果条目 item 的实际宽度大于格式符里指明的宽度会怎么样？

3.19　显示下面语句的输出：

```
print(format(57.467657, "9.3f"))
print(format(12345678.923, "9.1f"))
print(format(57.4, ".2f"))
print(format(57.4, "10.2f"))
```

3.20　显示下面语句的输出：

```
print(format(57.467657, "9.3e"))
print(format(12345678.923, "9.1e"))
print(format(57.4, ".2e"))
print(format(57.4, "10.2e"))
```

3.21　显示下面语句的输出：

```
print(format(5789.467657, "9.3f"))
print(format(5789.467657, "<9.3f"))
print(format(5789.4, ".2f"))
print(format(5789.4, "<.2f"))
print(format(5789.4, ">9.2f"))
```

3.22　显示下面语句的输出：

```
print(format(0.457467657, "9.3%"))
print(format(0.457467657, "<9.3%"))
```

3.23　显示下面语句的输出：

```
print(format(45, "5d"))
print(format(45, "<5d"))
print(format(45, "5x"))
print(format(45, "<5x"))
```

3.24　显示下面语句的输出：

```
print(format("Programming is fun", "25s"))
print(format("Programming is fun", "<25s"))
print(format("Programming is fun", ">25s"))
```

3.7　绘制各种图形

关键点：Python 的 Turtle 模块里包含移动笔、设置笔的大小、举起和放下笔的方法。

第 1 章介绍如何使用 Turtle 绘画。一个 Turtle 实际上是一个对象，在导入 Turtle 模块时，就创建了对象。然后，可以调用 Turtle 对象的各种方法完成不同的操作。本节将介绍 Turtle 对象更多的方法。

当创建一个 Turtle 对象时，它的位置被设定在（0,0）处——窗口的中心，而且它的方向被设置为向右。Turtle 模块用笔来绘制图形。默认情况下，笔是向下的（就像真实的笔尖触碰着一张纸）。如果笔是向下的，那么当移动 Turtle 的时候，它就会绘制出一条从当前位置到新位置的线。表 3-5 罗列出控制笔的绘制状态的方法，表 3-6 罗列出移动 Turtle 的方法。

表 3-5　Turtle 笔的绘图状态的方法

方法	描述
turtle.pendown()	将笔向下拉——移动的时候绘制
turtle.penup()	将笔向上拉——移动的时候不绘制
turtle.pensize（宽度）	将线的粗细设定为指定宽度

表 3-6　Turtle 运动的方法

方法	描述
turtle.forward(d)	将 Turtle 朝着 Turtle 指向的方向向前移动指定距离
turtle.backward(d)	将 Turtle 朝着 Turtle 指向的反方向向后移动指定距离，Turtle 的方向不改变
turtle.right(angle)	将 Turtle 向右转动指定角度
turtle.left(角度)	将 Turtle 向左转动指定角度
turtle.goto(x,y)	将 Turtle 移动到一个绝对位置

（续）

方法	描述
turtle.setx(x)	将 Turtle 的 x 坐标移动到指定位置
turtle.sety(y)	将 Turtle 的 y 坐标移动到指定位置
turtle.setheading(angle)	将 Turtle 的方向设定为指定角度。0——东、90——北、180——西、270——南
turtle.home()	将 Turtle 移动到起点（0,0）和向东
turtle.circle(r,ext,step)	绘制一个指定半径、范围和阶数的圆
turtle.dot(diameter，color)	绘制一个指定直径和颜色的圆
turtle.undo()	取消（反复）最后一个图操作
turtle.speed(s)	设置 Turtle 的速度为一个在 1 到 10 之间的整数，10 最大

所有的方法都是简单明了的。学习它们的最好方式是写一段测试代码看看每个方法是如何工作的。

circle 方法有三个参数：radius 是必需的，extent 和 step 是可有可无的。extent 是一个角度，它决定绘制圆的哪一部分。step 决定使用的阶数。如果 step 是 3、4、5、6、…，那么 circle 方法将绘制一个里面包含被圆括住的三边、四边、五边、六边或更多边形（即正三角形、正方形、五边形、六边形等）。如果不指定阶数，那么 circle 方法就只画一个圆。

程序清单 3-5 显示了一个绘制三角形、正方形、五边形、六边形以及圆的代码，如图 3-3 所示：

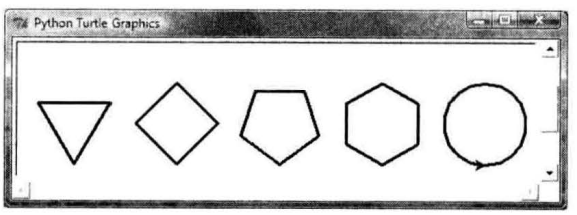


图 3-3　绘制 5 种图形的程序

程序清单 3-5 SimpleShapes.py

```
1  import turtle
2
3  turtle.pensize(3) # Set pen thickness to 3 pixels
4  turtle.penup() # Pull the pen up
5  turtle.goto(-200, -50)
6  turtle.pendown() # Pull the pen down
7  turtle.circle(40, steps = 3) # Draw a triangle
8
9  turtle.penup()
10 turtle.goto(-100, -50)
11 turtle.pendown()
12 turtle.circle(40, steps = 4) # Draw a square
13
14 turtle.penup()
15 turtle.goto(0, -50)
16 turtle.pendown()
17 turtle.circle(40, steps = 5) # Draw a pentagon
18
19 turtle.penup()
20 turtle.goto(100, -50)
21 turtle.pendown()
22 turtle.circle(40, steps = 6) # Draw a hexagon
23
```

```
24  turtle.penup()
25  turtle.goto(200, -50)
26  turtle.pendown()
27  turtle.circle(40) # Draw a circle
28
29  turtle.done()
```

第 1 行导入 Turtle 模块。第 3 行设置笔的粗细为 3 个像素点。第 4 行将笔向上拉，这样就可以在第 5 行将位置改变到（-200，-50）。第 6 行将笔拉下，第 7 行绘制一个三角形。在第 7 行，turtle 对象调用参数 radius 为 40 和阶数为 3 的 circle 方法绘制出一个三角形。类似地，程序的其他部分绘制一个正方形（第 12 行），一个五边形（第 17 行），一个六边形（第 22 行），一个圆（第 27 行）。

检查点

3.25　如何将 turtle 的位置设置在（0,0）？

3.26　如何绘制一个直径为 3 的红点？

3.27　下面的方法将绘制出什么图形？

```
turtle.circle(50, step = 4)
```

3.28　如何使 turtle 快速移动？

3.29　如何取消 turtle 的最后一次操作？

3.8　绘制带颜色和字体的图形

关键点：turtle 对象包含设置颜色和字体的方法。

前一节介绍了如何用 Turtle 模块绘制图形。通过学习掌握如何使用运动方法移动笔以及用笔的方法将笔抬高、降低和控制笔的粗细。本节将介绍更多有关笔的控制方法，如何设置颜色和字体以及编写文本。

表 3-7 罗列出控制绘图、颜色和填充的笔的方法。程序清单 3-6 是一个用不同颜色绘制三角形、正方形、五边形、六边形和圆的简单程序，如图 3-4 所示。程序也为图形添加了文本。

表 3-7　Turtle 笔的颜色、填充和绘制方法

方法	描述
turtle.color(c)	设置笔的颜色
turtle.fillcolor(c)	设置笔填充颜色
turtle.begin_fill()	在填充图形前访问这个方法
turtle.end_fill()	在最后调用 begin_fill 之前填充绘制的图形
turtle.filling()	返回填充状态：True 代表填充，False 代表没有填充
turtle.clear()	清除窗口，turtle 的状态和位置不受影响
turtle.reset()	清除窗口，将状态和位置复位为初始默认值
turtle.screensize(w,h)	设置画布的宽度和高度
turtle.hideturtle()	隐藏 turtle
turtle.showturtle()	显示 turtle
turtle.isvisible()	如果 turtle 可见，返回 True
turtle.write(s,font=("Arial", 8, "normal"))	在 turtle 位置编写字符串 s，字体是由字体名、字体大小和字体类型三部分组成

程序清单 3-6 ColorShapes.py

```
import turtle

turtle.pensize(3) # Set pen thickness to 3 pixels
turtle.penup() # Pull the pen up
turtle.goto(-200, -50)
turtle.pendown() # Pull the pen down
turtle.begin_fill() # Begin to fill color in a shape
turtle.color("red")
turtle.circle(40, steps = 3) # Draw a triangle
turtle.end_fill() # Fill the shape

turtle.penup()
turtle.goto(-100, -50)
turtle.pendown()
turtle.begin_fill() # Begin to fill color in a shape
turtle.color("blue")
turtle.circle(40, steps = 4) # Draw a square
turtle.end_fill() # Fill the shape

turtle.penup()
turtle.goto(0, -50)
turtle.pendown()
turtle.begin_fill() # Begin to fill color in a shape
turtle.color("green")
turtle.circle(40, steps = 5) # Draw a pentagon
turtle.end_fill() # Fill the shape

turtle.penup()
turtle.goto(100, -50)
turtle.pendown()
turtle.begin_fill() # Begin to fill color in a shape
turtle.color("yellow")
turtle.circle(40, steps = 6) # Draw a hexagon
turtle.end_fill() # Fill the shape

turtle.penup()
turtle.goto(200, -50)
turtle.pendown()
turtle.begin_fill() # Begin to fill color in a shape
turtle.color("purple")
turtle.circle(40) # Draw a circle
turtle.end_fill() # Fill the shape
turtle.color("green")
turtle.penup()
turtle.goto(-100, 50)
turtle.pendown()
turtle.write("Cool Colorful Shapes",
    font = ("Times", 18, "bold"))
turtle.hideturtle()

turtle.done()
```

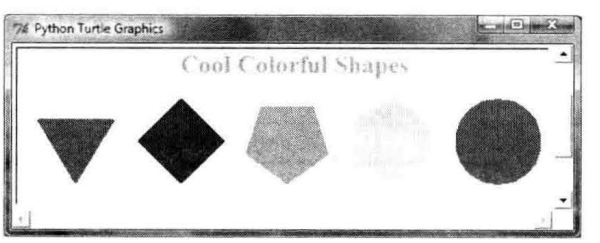


图 3-4　程序用不同颜色绘制 5 个图形

除了给每个图形填充颜色并且写了一个字符串，这个程序和程序清单 3-5 很相似。turtle 对象调用了第 7 行的 begin_fill() 方法来告诉 Python 绘制一个填充颜色的图形。在第 9 行绘制一个三角形。调用 end_fill() 方法（第 10 行）完成了图形颜色的填充。

write 方法是在笔的当前位置绘制一个指定字体的字符串（第 47 ～ 48 行）。注意：如果笔是向下的，只有在笔移动的时候才可以绘制。为了避免绘制，你需要将笔向上拉。调用 hideturtle() 使 turtle 不可见（第 49 行），所以你在窗口里看不到 turtle。

检查点

3.30　如何设置 turtle 的颜色？

3.31　如何给图形填充颜色？

3.32　如何使 turtle 不可见？

关键术语

backslash (\)（反斜杠（\））
character encoding（字符编码）
end-of-line（行尾）
escape sequence（转义序列）
line break（换行符）
methods（方法）
newline（换行符）
object（对象）
string（字符串）
whitespace characters（空白字符）

本章总结

1. Python 提供数学函数：解释器里的 abs、max、min、pow 和 round；math 模块里的 fabs、ceil、floor、exp、log、sqrt、sin、cos、acos、asin、tan、degree 和 radians。
2. 一个字符串是一个字符序列。字符串的值可以用一对单引号或双引号括起来。Python 里并没有字符数据类型；单一字符的字符串代表一个字符。
3. 转义序列是一种特殊的语法，它以“\”开始，再紧跟一个字母或者数字组合，以此来代表一个特殊的字符。例如 \'、\"、\t 和 \n。
4. 字符 ' '、\t、\f、\r 和 \n 被称为空白字符。
5. Python 里所有的数据，包括数字和字符串都是对象。你可以调用方法实现对象上的操作。
6. 你可以使用 format 函数格式化一个数字或字符串，然后返回一个字符串的结果。

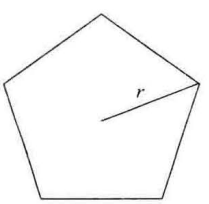
测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 3.2 节

3.1　（几何学：一个五边形的面积）编写一个程序，提示用户输入五边形顶点到中心的距离 r，然后算出五边形的面积，如下图所示。



计算五边形面积的公式是 Area =5 × s × s / (4 × tan(π /5))，这里的 s 是边长。边长的计算公式是 $s = 2r\sin\frac{\pi}{5}$，这里的 r 是顶点到中心的距离。下面是一个示例运行：

```
Enter the length from the center to a vertex: 5.5 Enter
The area of the pentagon is 71.92
```

*3.2 （几何学：大圆距离）大圆距离是球面上两点之间的距离。假设（$x1$，$y1$）和（$x2$，$y2$）是两点的经度和纬度，两点之间的大圆距离可以用下面的公式计算：

$$d = radius \times \arccos(\sin(x_1) \times \sin(x_2) + \cos(x_1) \times \cos(x_2) \times \cos(y_1 - y_2))$$

编写一个程序，提示用户输入地球表面两点经度和纬度的度数然后显示它们的大圆距离。地球的平均半径为 6371.01km。注意：你需要使用 math.radians 函数将度数转换成弧度数，因为 Python 三角函数使用的都是弧度。公式中的经纬度是西经和北纬。用负数表示东经和南纬。下面是一个示例运行。

```
Enter point 1 (latitude and longitude) in degrees:
39.55, -116.25 Enter
Enter point 2 (latitude and longitude) in degrees:
41.5, 87.37 Enter
The distance between the two points is 10691.79183231593 km
```

*3.3 （几何学：估算面积）从网站 www.gps-data-team.com/map/ 上找到佐治亚州亚特兰大、佛罗里达州奥兰多、大草原佐治亚、北卡罗来纳州夏洛特的 GPS 位置，然后计算出这四个城市所围成的区域的大概面积。（提示：可以使用上题 3.2 中的公式计算两个城市之间的距离。将多边形划分成两个三角形，然后用编程题 2.14 中的公式计算三角形的面积。）

3.4 （几何学：五角形的面积）五角形的面积可以使用下面的公式计算（s 是边长）：

$$\text{Area} = \frac{5 \times s^2}{4 \times \tan\left(\frac{\pi}{5}\right)}$$

编写一个程序，提示用户输入五角形的边长，然后显示面积。下面是一个示例运行。

```
Enter the side: 5.5 Enter
The area of the pentagon is 52.04444136781625
```

*3.5 （几何学：一个正多边形的面积）正多边形是边长相等的多边形，而且所有的角相等。计算正多边形面积的公式是：

$$Area = \frac{n \times s^2}{4 \times \tan\left(\frac{\pi}{n}\right)}$$

这里的 s 是边长。编写一个程序，提示用户输入边数以及正多边形的边长，然后显示它的面积。下面是一个示例运行。

```
Enter the number of sides: 5 Enter
Enter the side: 6.5 Enter
The area of the polygon is 72.69017017488385
```

第 3.3 ～ 3.6 节

*3.6 （找出 ASCII 码的字符）编写一个程序，接收一个 ASCII 码值（一个 0 ～ 127 之间的整数），然后显示它对应的字符。例如：如果用户输入 97，程序将显示字符 a。下面一个示例运行：

```
Enter an ASCII code: 69 ↵Enter
The character is E
```

3.7 （随机字符）编写一个程序，使用 time.time() 函数显示一个大写的随机字符。

*3.8 （金融应用程序：货币单元）改写程序清单 3-4，修正将浮点数转换成整数的过程中带来的精度损失。输入一个整数，它的后两位数字代表美分。例如：输入 1156，它代表 11 美元 56 美分。

*3.9 （金融应用程序：工资表）编写一个程序，读取下面的信息，然后打印一个工资报表。

雇员姓名（例如：史密斯）

一周工作时间（例如：10）

每小时报酬（例如：9.75）

联邦预扣税率（例如：20%）

州预扣税率（例如：9%）

一个示例运行如下所示。

```
Enter employee's name: Smith ↵Enter
Enter number of hours worked in a week: 10 ↵Enter
Enter hourly pay rate: 9.75 ↵Enter
Enter federal tax withholding rate: 0.20 ↵Enter
Enter state tax withholding rate: 0.09 ↵Enter

Employee Name: Smith
Hours Worked: 10.0
Pay Rate: $9.75
Gross Pay: $97.5
Deductions:
  Federal Withholding (20.0%): $19.5
  State Withholding (9.0%): $8.77
  Total Deduction: $28.27
Net Pay: $69.22
```

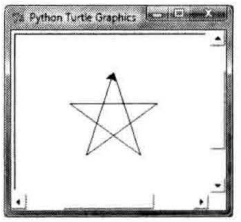
*3.10 （Turtle：显示统一码）编写一个程序，显示希腊字母 $\alpha\beta\gamma\delta\varepsilon\xi\eta\theta$。这些字符的统一码是：\u03b1 \u03b2 \u03b3 \u03b4 \u03b5 \u03b6 \u03b7 \u03b8。

3.11 （反向数字）编写一个程序，提示用户输入一个四位整数，然后显示颠倒各位数字后的数。下面是一个示例运行。

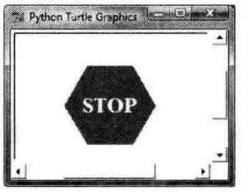
```
Enter an integer: 3125 ↵Enter
The reversed number is 5213
```

第 3.7 ～ 3.8 节

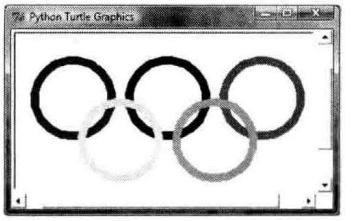
**3.12 （Turtle：绘制一个五角星）编写一个程序，提示用户输入五角星的边长，然后绘制一个五角星，如图 3-5a 所示（提示：五角星每个点的内角是 36 度）。



a）绘制一个五角星



b）显示一个 STOP 牌



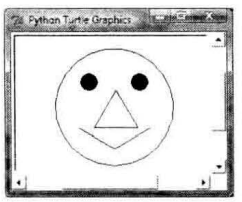
c）绘制一个欧林匹克标志

图　3-5

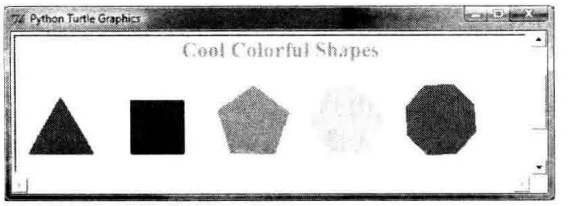
*3.13 （Turtle：显示一个 STOP 牌）编写一个程序，显示一个 STOP 牌，如图 3-5b 所示。六边形是红色的而文字是白色的。

3.14 （Turtle：绘制一个奥运五环标志）编写一个程序，提示用户输入环的半径，然后画出大小相等的五环，颜色依次为：蓝、黑、红、黄、绿，如图 3-5c 所示。

*3.15 （Turtle：绘制一个笑脸）编写一个程序，绘制一个笑脸，如图 3-6a 所示。



a）程序绘制一个笑脸

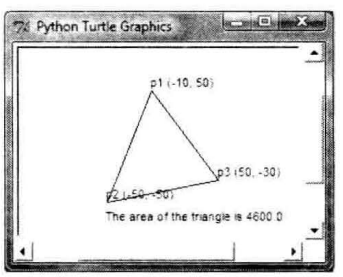


b）程序绘制五个图形，它们的底边是平行于 x 轴的

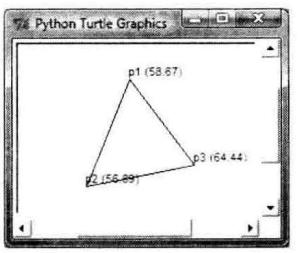
图　3-6

**3.16 （Turtle：绘制图形）编写一个程序，绘制一个三角形、一个正方形、一个五边形、一个六边形和一个八边形，如图 3-6b 所示。注意：这些图形的底边是平行于 x 轴的。（提示：将 turtle 的朝向调整 60 度就可以使三角形的底边平行于 x 轴。）

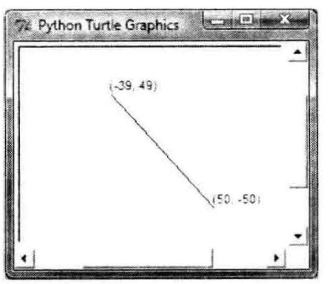
*3.17 （Turtle：三角形面积）编写一个程序，提示用户输入一个三角形的三点：p1、p2、p3，然后在三角形的下面显示三角形的面积，如图 3-7a 所示。计算三角形面积的公式参见编程题 2.14。



a）三角形的面积



b）三角形的角度



c）一条线

图　3-7

*3.18 （Turtle：三角形的角）修改程序清单 3-2，编写一个程序，提示用户输入三角形的三点：p1、p2 和 p3，然后显示它的角度，如图 3-7b 所示。

**3.19 （Turtle：绘制一条线）编写一个程序，提示用户输入两点，然后绘制一条连接两点的线并且显示这些点的坐标，如图 3-7c 所示。

第 4 章
Introduction to Programming Using Python

选　择

学习目标

- 使用比较运算符编写布尔表达式（第 4.2 节）。
- 使用 random.randint(a，b) 或者 random.random( ) 函数来生成随机数（第 4.3 节）。
- 编写布尔表达式（AdditionQuiz）（第 4.3 节）。
- 使用单向 if 语句实现选择控制（第 4.4 节）。
- 使用单向 if 语句编程（第 4.5 节）。
- 使用双向 if-else 语句实现选择控制（第 4.6 节）。
- 使用嵌套 if 和多向 if-elif-else 语句实现选择控制（第 4.7 节）。
- 避免 if 语句里的常见错误（第 4.8 节）。
- 使用选择语句编程（第 4.9 ～ 4.10 节）。
- 使用逻辑运算符（and、or 和 not）组合各种条件（第 4.11 节）。
- 使用带组合条件的选择语句（LeapYear、Lottery）（第 4.12 ～ 4.13 节）。
- 编写使用条件表达式的表达式（第 4.14 节）。
- 了解控制运算符优先权和结合性的规则（第 4.15 节）。
- 检测出一个对象的位置（第 4.16 节）。

4.1　引言

关键点：程序可以根据某个条件决定执行哪条语句。

如果在程序清单 2-2 中输入一个负的 radius 值，程序将产生一个无效结果。如果这个半径是负的，程序将无法计算这个区域。你怎么解决这种情况呢？

就像所有的高级程序设计语言一样，Python 提供选择语句让你可以在两个或多个不同条件下选择不同的动作。你可以使用下面的选择语句来替换程序清单 2-2 中的第 5 行：

```
if radius < 0:
    print("Incorrect input")
else:
    area = radius * radius * math.pi
    print("Area is", area)
```

选择语句使用的条件称为布尔表达式。本章将介绍布尔类型、数值、比较运算符以及表达式。

4.2　布尔类型、数值和表达式

关键点：布尔表达式是能计算出一个布尔值 True 或 False 的表达式。

怎么比较两个数值呢？例如，半径是大于 0、等于 0 还是小于 0？ Python 提供了六种比较运算符（也称为关系运算符），如表 4-1 所示，那么哪个用来比较两个数值呢？（表中假设

使用的是半径 5。）

表 4-1　比较运算符

Python 运算符	算术符号	名称	举例（radius 是 5）	结果
<	<	小于	radius < 0	False
<=	≤	小于等于	radius <= 0	False
>	>	大于	radius > 0	True
>=	≥	大于等于	radius >= 0	True
==	=	等于	radius == 0	False
!=	≠	不等于	radius != 0	True

警告：比较运算符的相等是两个等号（==），而不是单个等号（=），后者是用来赋值的。

比较的结果就是一个布尔逻辑值：True 或 False。例如：下面的语句显示结果为 True。

```
radius = 1
print(radius > 0)
```

存储布尔值的变量被称为布尔变量。布尔数据类型被用来代表布尔值，一个布尔变量可以代表 True 或 False 值中的一个。例如：下面的语句将 True 赋值给变量 lightson。

```
lightsOn = True
```

True 和 False 都是字面量，就像数字 10 是字面量一样。它们都是保留字，不能在程序中被当作标识符。

在计算机内部，Python 使用 1 来表示 True 而使用 0 来表示 False。你可以使用 int 数将布尔值转换为一个整数。

例如：

```
print(int(True))
```

显示 1 而

```
print(int(False))
```

显示 0。

你也可以用 bool 函数将一个数字值转换成一个布尔值。如果值为 0，这个函数返回 False；否则，这个函数总是返回 True。

例如：

```
print(bool(0))
```

显示 False 而

```
print(bool(4))
```

显示 True。

检查点

4.1　列举六种比较运算符。

4.2　下面的转换是允许的吗？如果允许，给出转换后的结果。

```
i = int(True)
j = int(False)

b1 = bool(4)
b2 = bool(0)
```

4.3 产生随机数字

关键点：函数 randint（a，b）可以用来产生一个 a 和 b 之间且包括 a 和 b 的随机整数。

设想你要开发一个帮助一年级学生练习加法的程序。这个程序会随机产生两个一位整数：number1 和 number2，然后显示给学生一个问题：What is 1+7（1+7=？），如程序清单 4-1 所示。在学生输入答案后，程序会显示一条消息表明答案是对还是错。

你可以使用函数 random 模块中的 randint（a，b）函数产生一个随机数字。这个函数返回一个在 a 和 b 之间包括 a 和 b 的随机整数 i。使用 randint（0,9）获取一个在 0 到 9 之间的随机整数。

这个程序会按照如下步骤工作。

第 1 步：产生两个一位整数 number1（例如，4）和 number2（例如，5）。

第 2 步：提示学生回答 What is 4+5。

第 3 步：检测学生的答案是否正确。

程序清单 4-1 AdditionQuiz.py

```
1  import random
2
3  # Generate random numbers
4  number1 = random.randint(0, 9)
5  number2 = random.randint(0, 9)
6
7  # Prompt the user to enter an answer
8  answer = eval(input("What is " + str(number1) + " + "
9      + str(number2) + "? "))
10
11 # Display result
12 print(number1, "+", number2, "=", answer,
13     "is", number1 + number2 == answer)
```

```
What is 1 + 7? 8 ↵Enter
1 + 7 = 8 is True
```

```
What is 4 + 8? 9 ↵Enter
4 + 8 = 9 is False
```

line#	number1	number2	answer	output
4	4			
5		8		
8			9	
12				4 + 8 = 9 is False

程序使用 random 模块中定义的 randint 函数。import 语句会导入这个模块（第 1 行）。

第 4 ～ 5 行产生两个数字：number1 和 number2。第 8 行从用户处获取一个答案，第 13 行使用一个布尔表达式 number1+number2==answer 来判断答案是否正确。

Python 也提供了其他函数：randrange(a, b) 产生一个在 a、b−1 之间的随机整数，它等同于 randint（a,b−1）。例如：randrange(0,10) 和 randint(0,9) 是一样的。因为 randint 更直观，本书例子中更多使用的是 randint。

你也可以使用 random( ) 函数生成一个满足条件 0<=r<=1.0 的随机浮点数 r。例如：

```
1  >>> import random
2  >>> random.random()
3  0.34343
4  >>> random.random()
5  0.20119
6  >>> random.randint(0, 1)
7  0
8  >>> random.randint(0, 1)
9  1
10 >>> random.randrange(0, 1) # This will always be 0
11 0
12 >>>
```

调用 random. random( )（第 2 行和第 4 行）返回一个 0.0 到 1.0 之间（不包括 1.0）的随机浮点数。调用函数 random.randint(0,1)（第 6 行和第 8 行）返回 0 或 1。调用 random. randrange(0,1)（第 10 行）总是返回 0。

检查点

4.3　怎样生成一个满足条件 $0 \leqslant i < 20$ 的随机整数?

4.4　怎样生成一个满足条件 $10 \leqslant i < 20$ 的随机整数?

4.5　怎样生成一个满足条件 $10 \leqslant i \leqslant 50$ 的随机整数?

4.6　怎样生成一个值为 0 或 1 的随机整数?

4.4　if 语句

关键点：如果条件正确就执行一个单向 if 语句。

前面的程序显示一条像“6+2=7 is false”这样的信息。如果你想将信息改成“6+2=7 is incorrect”，你必须使用选择语句来做这种微小改变。

Python 有多种选择语句类型：单向 if 语句、双向 if-else 语句、嵌套 if 语句，多向 if-elif-else 语句以及条件表达式。这节介绍单向 if 语句。

当且仅当条件为 true 时，一条单向 if 语句执行一个动作。单向 if 语句的语法如下：

```
if boolean-expression:
    statement(s)  # Note that the statement(s) must be indented
```

这里 statement（s）必须相对于 if 向右至少缩进一个空白，而每条语句也必须使用同样个数的缩进。为了保持一致性，我们在这本书中缩进四个空白。

图 4-1a 中的流程图解释了 Python 如何执行 if 语句的句法。流程图是描述算法或过程的图表，将步骤显示为不同形状的框图，这些框图之间的顺序是用箭头连接的。框图里面表述过程的操作，而箭头连接它们表示控制流。一个菱形框图用来表示布尔条件，而一个长方形框图用来表示语句。

如果布尔表达式计算的结果为真，那么就会执行 if 块中的语句。if 块里的语句都要在 if 语句之后缩进。例如：

```
if radius >= 0:
    area = radius * radius * math.pi
    print("The area for the circle of radius", radius, "is", area)
```

处理语句的流程图如图 4-1b 所示。如果 radius 的值大于等于 0，那么计算 area，然后显示结果；否则，不执行 if 块中的语句。

在 if 块中的语句必须在 if 行后的一行进行缩进，而且要以相同的空白缩进。例如：下

面的代码就是错误的，因为第 3 行的 print 语句并没有和第 2 行计算面积的语句缩进一样的空白。

```
1  if radius >= 0:
2      area = radius * radius * math.pi  # Compute area
3    print("The area for the circle of radius", radius, "is", area)
```

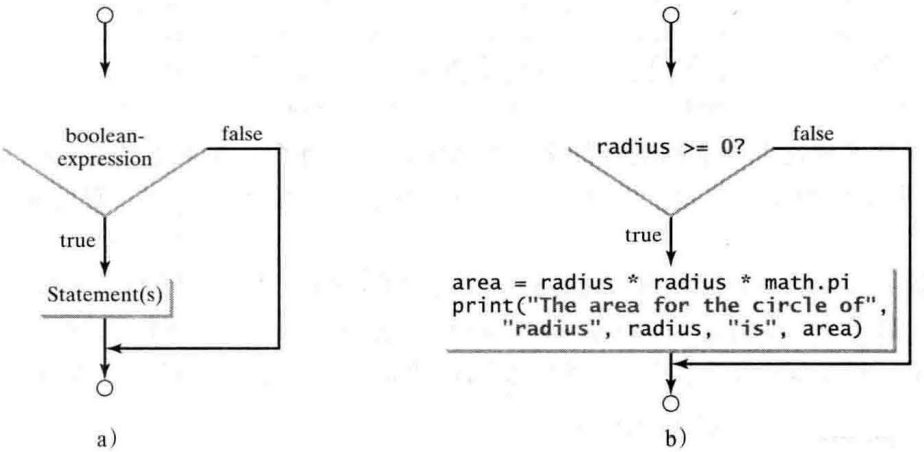


图 4-1 如果 boolean-expression 计算结果是 True，if 语句执行这些语句体

程序清单 4-2 是一个提示用户输入一个整数的程序。如果那个数字是 5 的倍数，程序显示结果 HiFive。如果这个数能被 2 整除，程序显示 HiEven。

程序清单 4-2 SimpleIfDemo.py

```
1  number = eval(input("Enter an integer: "))
2
3  if number % 5 == 0:
4      print("HiFive")
5
6  if number % 2 == 0:
7      print("HiEven")
```

```
Enter an integer: 4 Enter
HiEven
```

```
Enter an integer: 30 Enter
HiFive
HiEven
```

程序提示用户输入一个整数（第 1 行），如果能被 5 整除程序显示 HiFive（第 3 ～ 4 行），如果能被 2 整除程序显式 HiEven（第 6 ～ 7 行）。

检查点

4.7 编写一条如果 y 大于零，将 1 赋值给 x 的 if 语句。

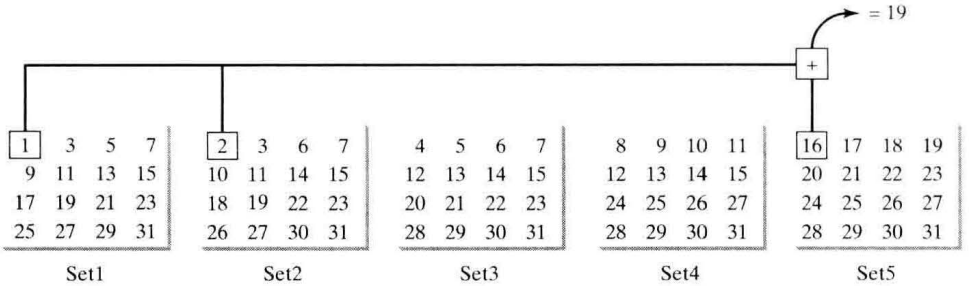
4.8 编写一个如果 score 大于 90，pay 增长 3% 的 if 语句。

4.5 实例研究：猜生日

关键点：猜生日是一个很有趣的用简单程序解决的问题。

你可以通过询问 5 个问题来找出你朋友的生日在一个月中的哪一天。每个问题都在询问

这一天是否在 5 个数字集中。



生日就是出现这个数字的集合的第一个数字的和，例如：如果生日是 19，那它就会在 set1、set2 和 set5 中出现。这三个集合的第一个数字分别是 1、2、16。它们加起来的和就是 19。

程序清单 4-3 是一个提示用户回答这一天是否在 set1（第 4 ～ 13 行）、set2（第 16 ～ 25 行）、set3（第 28 ～ 37 行）、set4（第 40 ～ 49 行）或 set5（第 52 ～ 61 行）中。如果这个数字在集合中，程序就将集合的第一个数加到 day 里面（第 13、25、37、49 和 61 行）。

程序清单 4-3 GuessBirthday.py

```
day = 0 # birth day to be determined

# Prompt the user to answer the first question
question1 = "Is your birthday in Set1?\n" + \
    " 1  3  5  7\n" + \
    " 9 11 13 15\n" + \
    "17 19 21 23\n" + \
    "25 27 29 31" + \
    "\nEnter 0 for No and 1 for Yes: "
answer = eval(input(question1))

if answer == 1:
    day += 1

# Prompt the user to answer the second question
question2 = "Is your birthday in Set2?\n" + \
    " 2  3  6  7\n" + \
    "10 11 14 15\n" + \
    "18 19 22 23\n" + \
    "26 27 30 31" + \
    "\nEnter 0 for No and 1 for Yes: "
answer = eval(input(question2))

if answer == 1:
    day += 2

# Prompt the user to answer the third question
question3 = "Is your birthday in Set3?\n" + \
    " 4  5  6  7\n" + \
    "12 13 14 15\n" + \
    "20 21 22 23\n" + \
    "28 29 30 31" + \
    "\nEnter 0 for No and 1 for Yes: "
answer = eval(input(question3))

if answer == 1:
    day += 4

# Prompt the user to answer the fourth question
```

```
40  question4 = "Is your birthday in Set4?\n" + \
41      " 8  9 10 11\n" + \
42      "12 13 14 15\n" + \
43      "24 25 26 27\n" + \
44      "28 29 30 31" + \
45      "\nEnter 0 for No and 1 for Yes: "
46  answer = eval(input(question4))
47
48  if answer == 1:
49      day += 8
50
51  # Prompt the user to answer the fifth question
52  question5 = "Is your birthday in Set5?\n" + \
53      "16 17 18 19\n"+ \
54      "20 21 22 23\n" + \
55      "24 25 26 27\n" + \
56      "28 29 30 31" + \
57      "\nEnter 0 for No and 1 for Yes: "
58  answer = eval(input(question5))
59
60  if answer == 1:
61      day += 16
62
63  print("\nYour birthday is "+ str(day) + "!")
```

```
Is your birthday in Set1?
 1  3  5  7
 9 11 13 15
17 19 21 23
25 27 29 31
Enter 0 for No and 1 for Yes: 1 Enter

Is your birthday in Set2?
 2  3  6  7
10 11 14 15
18 19 22 23
26 27 30 31
Enter 0 for No and 1 for Yes: 1 Enter

Is your birthday in Set3?
 4  5  6  7
12 13 14 15
20 21 22 23
28 29 30 31
Enter 0 for No and 1 for Yes: 0 Enter

Is your birthday in Set4?
 8  9 10 11
12 13 14 15
24 25 26 27
28 29 30 31
Enter 0 for No and 1 for Yes: 0 Enter

Is your birthday in Set5?
16 17 18 19
20 21 22 23
24 25 26 27
28 29 30 31
Enter 0 for No and 1 for Yes: 1 Enter
Your birthday is 19!
```

line#	day	answer	output
1	0		
10		1	
13	1		
22		1	
25	2		
34		0	
46		0	
58		1	
61	19		
63			Your birthday is 19

在第 4 ～ 8 行末尾出现的字符 \ 是续行符，它告诉解释器语句在下一行继续执行（参见第 2.3 节）。

这个游戏非常容易编写程序，你可能想知道这个游戏是如何产生的。这个游戏背后的数学思想其实是非常简单的。这些数字不是随意地放在一起，它们如何放置在 5 个集合是精心安排的。这 5 个集合的起始数字分别是 1、2、4、8、16，它们分别对应二进制数 1、10、100、1000、10000。1 到 31 之间的十进制数要用二进制数表示则最多需要 5 个数字，如图 4-2a 所示。假设这个数字为 $b_1b_2b_3b_4b_5$，那么 $b_1b_2b_3b_4b_5 = b_5 0000 + b_4 000 + b_3 00 + b_2 0 + b_1$，如图 4-2b 所示。如果某一天的二进制数在 b_k 位置有一个数 1，那么这个数字就应该出现在集合 setk 中。例如：数字 19 的二进制数为 10011，所以它出现在 set1、set2 和 set5 中。它的二进制值是 1 + 10 + 10000 = 10011，而十进制数是 1 + 2 + 16 = 19。数字 31 的二进制数值为 11111，所以它出现在 set1、set2、set3、set4 和 set5 中。它的二进制数是 1 + 10 + 100 + 1000 + 10000 = 11111 而十进制数是 1 + 2 + 4 + 8 + 16 = 31。

Decimal	Binary
1	00001
2	00010
3	00011
...	
19	10011
...	
31	11111

a）1 到 31 之间的数可以用 5 位二进制数表示

b_5 0 0 0 0		10000
b_4 0 0 0		1000
b_3 0 0	10000	100
b_2 0	10	10
+ b_1	+ 1	+ 1
$b_5b_4b_3b_2b_1$	10011	11111
	19	31

b）一个 5 位二进制数可以通过对二进制数 1、10、100、1000 或 10000 相加得到

图　4-2

4.6　双向 if-else 语句

关键点：双向 if-else 语句根据条件是真还是假来决定执行哪些语句。

如果指定条件是 True，那么一条单向 if 语句会完成一个动作。如果条件是 False，那它什么都不做。但是当条件为 False 时，你想要完成一个或多个动作时应该怎么办？你可以使用一个双向 if-else 语句。双向 if-else 语句根据条件是 True 还是 False 指定不同的动作。

下面是一个双向 if-else 语句的句法：

```
if boolean-expression:
    statement(s)-for-the-true-case
else:
    statement(s)-for-the-false-case
```

语句的流程图如图 4-3 所示。

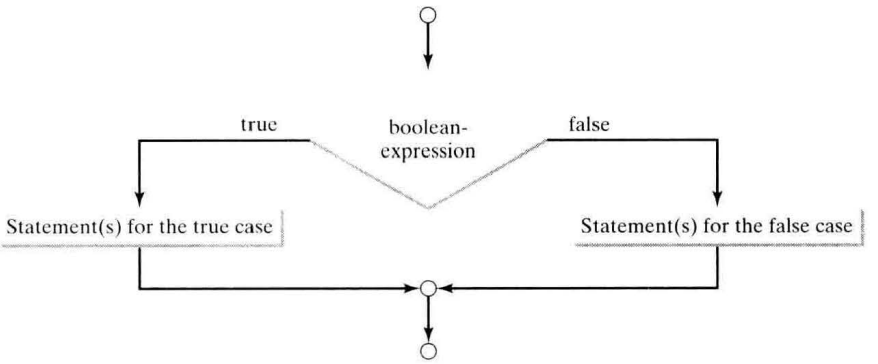


图 4-3

图 4-3 中，如果 boolean-expression 计算结果是 True，那就执行 true 情况下的语句；否则，执行 false 情况下的语句。例如，考虑下面的代码。

```
if radius >= 0:
    area = radius * radius * math.pi
    print("The area for the circle of radius", radius, "is", area)
else:
    print("Negative input")
```

如果 radius>=0 为真，计算 area 并显示它；如果这个条件为 false，就显示消息："negative input"。

这里有一个 if-else 语句的另一个例子。它是用来判断一个数的奇偶性，如下所示：

```
if number % 2 == 0:
    print(number, "is even.")
else:
    print(number, "is odd.")
```

假设你要编写一个训练一年级学生减法的程序。程序随机产生两个一位整数：number1 和 number2，而 number1>=number2，然后向学生提问类似"9-2=？"这样的问题。在回答完问题之后，程序会显示一条信息表明答案是否正确。

程序分为以下几个步骤。

第 1 步：产生两个随机的一位整数 number1 和 number2。

第 2 步：如果 number1<number2，将 number2 的值交换给 number1。

第 3 步：提示学生回答"What is number1-number2?"（number1-number2 的结果是什么）。

第 4 步：检查学生答案是否正确，然后显示输出结果是否正确。

完整的程序如程序清单 4-4 所示。

程序清单 4-4 SubtractionQuiz.py

```
1  import random
2
3  # 1. Generate two random single-digit integers
4  number1 = random.randint(0, 9)
5  number2 = random.randint(0, 9)
```

```
6
7  # 2. If number1 < number2, swap number1 with number2
8  if number1 < number2:
9      number1, number2 = number2, number1  # Simultaneous assignment
10
11 # 3. Prompt the student to answer "What is number1 - number2?"
12 answer = eval(input("What is "+ str(number1) + " - " +
13     str(number2) + "? "))
14
15 # 4. Check the answer and display the result
16 if number1 - number2 == answer:
17     print("You are correct!")
18 else:
19     print("Your answer is wrong.\n", number1, '-',
20         number2, "is", number1 - number2, '.')
```

```
What is 6 - 6? 0 Enter
You are correct!
```

```
What is 9 - 2? 5 Enter
Your answer is wrong.
9 - 2 is 7.
```

line#	number1	number2	answer	output
4	2			
5		9		
9	9	2		
12			5	
19				Your answer is wrong. 9 - 2 is 7.

如果 number1<number2，那么程序将使用同步赋值来交换它们的值。

检查点

4.9　编写一个如果 score 大于 90，pay 上涨 3%，否则上涨 1% 的 if 语句。

4.10　如果 number 分别是 30 和 35，那么 a 中的代码和 b 中的代码的输出结果是什么？

```
if number % 2 == 0:
    print(number, "is even.")

print(number, "is odd.")
```

a)

```
if number % 2 == 0:
    print(number, "is even.")
else
    print(number, "is odd.")
```

b)

4.7　嵌套 if 和多向 if-elif-else 语句

关键点：将一个 if 语句放在另一个 if 语句中就形成了一个嵌套 if 语句。

if 或 if-else 语句中的语句可以是任意一个合法的 Python 语句，甚至可以包括另一个 if 或 if-else 语句。内部 if 语句被称为嵌套在外部 if 语句中。内部 if 语句也可以包含另一个 if 语句；事实上，嵌套的深度是没有限制的。例如，下面的语句是一个嵌套 if 语句：

```
if i > k:
    if j > k:
```

```
        print("i and j are greater than k")
else:
    print("i is less than or equal to k")
```

if j>k 语句是嵌套在 if i>k 语句中的。

嵌套 if 语句可以用来实现多种选择。例如：图 4-4a 通过不同条件，根据分数的大小，给变量 grade 赋值一个字母值。

```
if score >= 90.0:
    grade = 'A'
else:
    if score >= 80.0:
        grade = 'B'
    else:
        if score >= 70.0:
            grade = 'C'
        else:
            if score >= 60.0:
                grade = 'D'
            else:
                grade = 'F'
```

a)

Equivalent

This is better

```
if score >= 90.0:
    grade = 'A'
elif score >= 80.0:
    grade = 'B'
elif score >= 70.0:
    grade = 'C'
elif score >= 60.0:
    grade = 'D'
else:
    grade = 'F'
```

b)

图　4-4

图 4-4 中受推崇使用多向 if-elif-else 语句如图 4-4b 所示。

这个 if 语句如何执行的过程如图 4-5 所示。首先测试第一个条件（score>=90）。如果它为 True，那么成绩为 A。如果条件为 False，就测试第二个条件（score>=80）。如果第二个条件为 True，那么成绩是 B。如果条件为 False，那么测试第三个条件直至所有条件都被测试完，或者所有的条件都被测试过。如果所有的条件都是 False，那么成绩就是 F。注意：一个条件只有这个条件之前的所有条件都变成 False 之后才被测试。

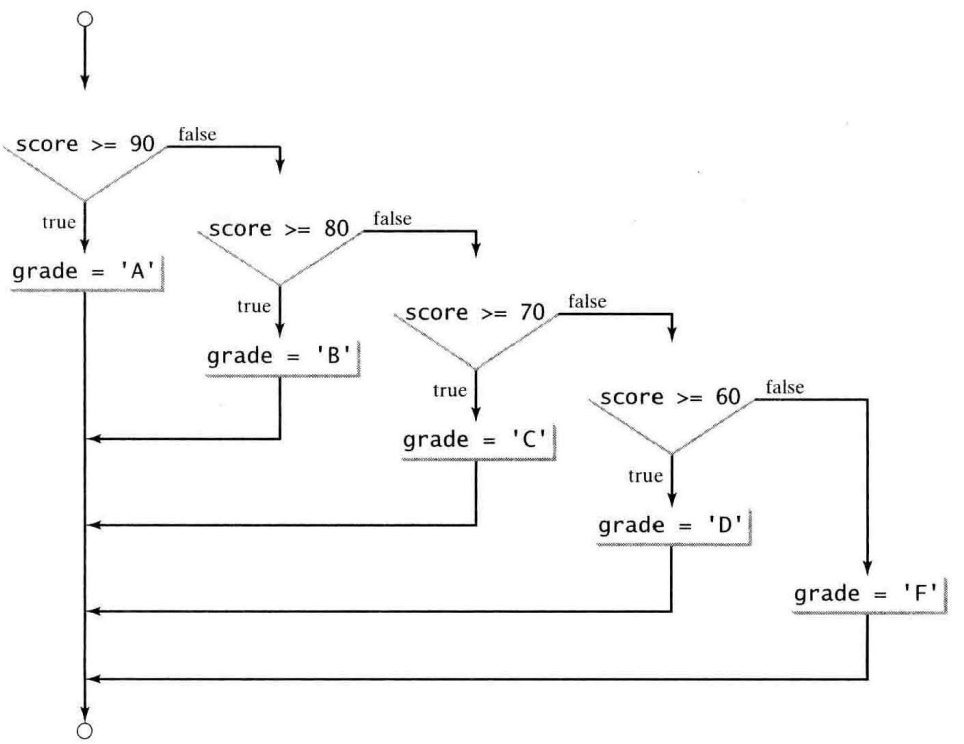


图 4-5　使用一个多向 if-elif-else 语句对成绩赋值

图 4-4a 中的 if 语句等同于图 4-4b 之中的 if 语句。实际上，图 4-4b 是多选 if 语句推崇的编码风格。这种形式的 if 语句被称为*多向 if 语句*，它避免了缩进深度，使程序更容易阅读。这种多向 if 语句使用 if-elif-else 句法，elif（else if 的缩写）也是一个 Python 关键词。

现在，我们编写一个程序来判断一个给定的年份属于哪一个生肖。中国十二生肖以 12 为循环，这个循环的每一年都分别由一个动物表示——猴、鸡、狗、猪、鼠、牛、虎、兔、龙、蛇、马和羊，如图 4-6 所示。

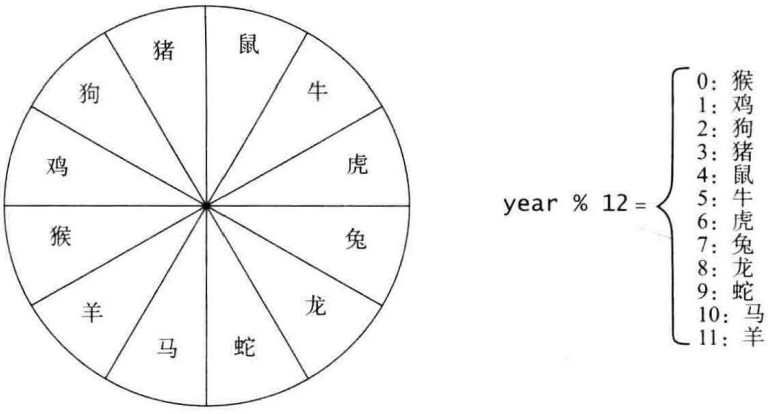


图 4-6　基于 12 年周期的中国十二生肖

year%12 的值决定是什么生肖年。1900 年是兔年，因为 1900%12=4。程序清单 4-5 给出一个程序，提示用户指定一个年份，然后显示表示这一年的动物。

程序清单 4-5　ChineseZodiac.py

```
year = eval(input("Enter a year: "))
zodiacYear = year % 12
if zodiacYear == 0:
    print("monkey")
elif zodiacYear == 1:
    print("rooster")
elif zodiacYear == 2:
    print("dog")
elif zodiacYear == 3:
    print("pig")
elif zodiacYear == 4:
    print("rat")
elif zodiacYear == 5:
    print("ox")
elif zodiacYear == 6:
    print("tiger")
elif zodiacYear == 7:
    print("rabbit")
elif zodiacYear == 8:
    print("dragon")
elif zodiacYear == 9:
    print("snake")
elif zodiacYear == 10:
    print("horse")
else:
    print("sheep")
```

```
Enter a year: 1963 ↵Enter
rabbit
```

```
Enter a year: 1877 ↵Enter
ox
```

检查点

4.11 如果是下面的代码中，假设 x=3 而且 y=2，显示它的结果。如果 x=3 而且 y=4，那么结果是什么？如果 x=2 而且 y=2，那么结果是什么？请为这段代码绘制一个流程图。

```
if x > 2:
    if y > 2:
        z = x + y
        print("z is", z)
    else:
        print("x is", x)
```

4.12 如果是下面的代码，假设 x=2 而且 y=4，显示它的结果。如果 x=3 而且 y=2，那么结果是什么？如果 x=3 而且 y=3，那么结果是什么？（首先正确缩进语句。）

```
if x > 2:
    if y > 2:
        z = x + y
        print("z is", z)
else:
    print("x is", x)
```

4.13 下面的代码错在哪里？

```
if score >= 60.0:
    grade = 'D'
elif score >= 70.0:
    grade = 'C'
elif score >= 80.0:
    grade = 'B'
elif score >= 90.0:
    grade = 'A'
else:
    grade = 'F'
```

4.8 选择语句中的常见错误

关键点：选择语句中的大多数常见错误都是由不正确的缩进问题导致的。

仔细思考 a 和 b 中的代码。

```
radius = -20

if radius >= 0:
    area = radius * radius * math.pi
print("The area is", area)
```

a）错误

```
radius = -20

if radius >= 0:
    area = radius * radius * math.pi
    print("The area is", area)
```

b）正确

在 a 中，print 语句不在 if 语块内。要将它放进 if 块中，你必须像 b 中那样将 print 语句缩进，如图 b 所示。

考虑下面 a 和 b 中代码的另一个例子。a 中的代码有两个 if 子句和一个 else 子句。哪个 if 子句是匹配这个 else 子句的？缩进表明 a 中 else 子句匹配第一个 if 子句而 b 中 else 子句匹配第二个 if 子句。

```
i = 1
j = 2
k = 3

if i > j:
    if i > k:
        print('A')
else:
    print('B')
```

a)

```
i = 1
j = 2
k = 3

if i > j:
    if i > k:
        print('A')
    else:
        print('B')
```

b)

因为（i>j）是 false，所以 a 中的代码显示 B，但是 b 中的语句什么也不显示。

提示：新的程序员经常会这样给一个布尔变量赋值一个测试条件，如 a 中的代码所示：

```
if number % 2 == 0:
    even = True
else:
    even = False
```

a)

Equivalent

This is shorter

```
even = number % 2 == 0
```

b)

代码可以被简化成直接将判断的值赋给变量，如 b 中代码所示。

检查点

4.14　下面代码哪些是等价的？哪些是正确缩进？

```
if i > 0:
    x = 0
    y = 1
else:
    y = 0
    z = 0
```

a)

```
if i > 0:
    x = 0
  y = 1
else:
    y = 0
    z = 0
```

b)

```
if i > 0:
  x = 0
  y = 1
else:
    y = 0
    z = 0
```

c)

```
if i > 0:
  x = 0
  y = 1
else:
    y = 0
  z = 0
```

d)

4.15　使用一个布尔表达式改写下面的语句：

```
if count % 10 == 0:
    newLine = True
else:
    newLine = False
```

4.16　下面的语句正确吗？哪一个更好？

```
if age < 16:
    print("Cannot get a driver's license")
if age >= 16:
    print("Can get a driver's license")
```

a)

```
if age < 16:
    print("Cannot get a driver's license")
else:
    print("Can get a driver's license")
```

b)

4.17　如果 number 分别为 14、15 和 30，那么下面代码的结果是什么？

```
if number % 2 == 0:
    print(number, "is even")
if number % 5 == 0:
    print(number, "is multiple of 5")
```

a)

```
if number % 2 == 0:
    print(number, "is even")
elif number % 5 == 0:
    print(number, "is multiple of 5")
```

b)

4.9 实例研究：计算身体质量指数

关键点：使用嵌套 if 语句编写一个说明身体质量指数的程序。

BMI 是根据体重测量健康的方式。通过以千克为单位的体重除以以米为单位的身高的平方计算出 BMI。下面是 16 岁以上人群的 BMI 图表：

BMI	解释
BMI < 18.5	超轻
18.5 ≤ BMI < 25.0	标准
25.0 ≤ BMI < 30.0	超重
30.0 ≤ BMI	痴肥

编写一个程序，提示用户输入以磅为单位的体重和以英寸为单位的身高，然后显示 BMI 值。注意：1 磅是 0.453 592 37 千克而 1 英寸是 0.0254 米。程序清单 4-6 给出这个程序。

程序清单 4-6 ComputeBMI.py

```
1  # Prompt the user to enter weight in pounds
2  weight = eval(input("Enter weight in pounds: "))
3
4  # Prompt the user to enter height in inches
5  height = eval(input("Enter height in inches: "))
6
7  KILOGRAMS_PER_POUND = 0.45359237  # Constant
8  METERS_PER_INCH = 0.0254          # Constant
9
10 # Compute BMI
11 weightInKilograms = weight * KILOGRAMS_PER_POUND
12 heightInMeters = height * METERS_PER_INCH
13 bmi = weightInKilograms / (heightInMeters * heightInMeters)
14
15 # Display result
16 print("BMI is", format(bmi, ".2f"))
17 if bmi < 18.5:
18     print("Underweight")
19 elif bmi < 25:
20     print("Normal")
21 elif bmi < 30:
22     print("Overweight")
23 else:
24     print("Obese")
```

```
Enter weight in pounds: 146 Enter
Enter height in inches: 70 Enter
BMI is 20.95
Normal
```

line#	weight	height	weightInKilograms	heightInMeters	bmi	output
2	146					
5		70				
11			66.22448602			
12				1.778		
13					20.9486	
16						BMI is 20.95
22						Normal

两个定名常量 KILOGRAMS_PER_POUND 和 METERS_PER_INCH 在第 6 行和第 7 行定义。第 2.7 节中介绍了定名常量。这里使用定名常量可以使程序更容易阅读。然而，在

Python 中没有特殊的句法来定义定名常量。在 Python 中对待定名常量是和变量一样的。本书使用大写所有字母并用下划线 (_) 分隔单词的格式将定名常量和变量加以区分。

4.10　实例研究：计算税款

关键点：使用嵌套的 if 语句来编写一个计算税款的程序。

美国联邦个人收入所得税是根据报税身份和可纳税收入来计算的。报税身份有四种：单身报税人、已婚合并申报人、已婚分开申报人以及户主。税率每年都会改变。表 4-2 显示的是 2009 年的税率，如果你是单身且可纳税收入是 10 000 美元，其中 8350 美元的税率是 10%，而另外的 1650 美元的税率是 15%。所以，你的税款是 1082.5 美元。

表 4-2　2009 年美国联邦个人税率　（单位：美元）

边际税率	单身报税人	已婚合并申报人	已婚分开申报人	户主
10%	0 ~ 8 350	0 ~ 16 700	0 ~ 8 350	0 ~ 11 950
15%	8 351 ~ 33 950	16 701 ~ 67 900	8351 ~ 33 950	11 951 ~ 45 500
25%	33 951 ~ 82 250	67 901 ~ 137 050	33 951 ~ 68 525	45 501 ~ 117 450
28%	82 251 ~ 171 550	137 051 ~ 208 850	68 526 ~ 104 425	117 451 ~ 190 200
33%	171 551 ~ 372 950	208 851 ~ 372 950	104 426 ~ 186 475	190 201 ~ 372 950
35%	372 951+	372 951+	186 476+	372 951+

你要编写一个可以计算个人所得税款的程序。你的程序应该提示用户输入他的报税身份和可纳税收入，然后计算出税款。输入 0 代表单身，1 代表已婚合并申报，2 代表已婚分开申报而 3 代表户主。

程序根据报税身份来计算出可纳税收入的税款。报税身份可以用下面的 if 语句来决定：

```
if status == 0:
    # Compute tax for single filers
elif status == 1:
    # Compute tax for married filing jointly
elif status == 2:
    # Compute tax for married filing separately
elif status == 3:
    # Compute tax for head of household
else:
    # Display wrong status
```

每一种报税身份都有六种税率。每一种税率都可以应用在一定量的可纳税收入上。例如：一个可纳税收入为 400 000 美元的单身报税人，8350 美元之内的税率为 10%，33 950−8350 美元范围的税率为 15%，82 250−33 950 美元范围的税率为 25%，171 550−82 250 美元范围的税率为 28%，372 950−171 550 美元范围的税率为 33%，而 400 000−372 950 美元范围的税率为 35%。

程序清单 4-7 给出了一个计算单身税款的解决方案。更完整的解决方案留在本章最后的编程题 4.13。

程序清单 4-7　ComputeTax.py

```
1  import sys
2
3  # Prompt the user to enter filing status
4  status = eval(input(
5      "(0-single filer, 1-married jointly,\n" +
6      "2-married separately, 3-head of household)\n" +
```

```
 7      "Enter the filing status: "))
 8
 9  # Prompt the user to enter taxable income
10  income = eval(input("Enter the taxable income: "))
11
12  # Compute tax
13  tax = 0
14
15  if status == 0:  # Compute tax for single filers
16      if income <= 8350:
17          tax = income * 0.10
18      elif income <= 33950:
19          tax = 8350 * 0.10 + (income - 8350) * 0.15
20      elif income <= 82250:
21          tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
22              (income - 33950) * 0.25
23      elif income <= 171550:
24          tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
25              (82250 - 33950) * 0.25 + (income - 82250) * 0.28
26      elif income <= 372950:
27          tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
28              (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 + \
29              (income - 171550) * 0.33
30      else:
31          tax = 8350 * 0.10 + (33950 - 8350) * 0.15 + \
32              (82250 - 33950) * 0.25 + (171550 - 82250) * 0.28 + \
33              (372950 - 171550) * 0.33 + (income - 372950) * 0.35;
34  elif status == 1:  # Compute tax for married file jointly
35      print("Left as exercise")
36  elif status == 2:  # Compute tax for married separately
37      print("Left as exercise")
38  elif status == 3:  # Compute tax for head of household
39      print("Left as exercise")
40  else:
41      print("Error: invalid status")
42      sys.exit()
43
44  # Display the result
45  print("Tax is", format(tax, ".2f"))
```

```
(0-single filer, 1-married jointly,
2-married separately, 3-head of household)
Enter the filing status: 0 Enter
Enter the taxable income: 400000 Enter
Tax is 117683.50
```

line#	status	income	tax	output
4	0			
10		400000		
13			0	
17			117683.5	
45				Tax is 117683.50

程序接收到报税身份和可纳税收入。多向选择 if 语句（第 15、34、36、38、40 行）检测报税身份，然后根据报税身份计算税款。

sys.exit()（第 42 行）定义在 sys 模块中。调用这个函数终止程序。

为了测试程序，你需要提供能概括所有可能性的输入值。针对这个程序，你的程序应该

覆盖所有的身份（0、1、2 和 3）。针对每一种身份，测试六个范围的每种情况。所以，一共有 24 种可能。

提示：对于所有的程序，你应该编写一小段代码，在加入更多其他代码之前测试它。这被称为增量开发和测试。这种方法使调试更加简单，因为错误很有可能在你新添加的代码中。

检查点

4.18　下面两条语句相同吗？

```
if income <= 10000:
    tax = income * 0.1
elif income <= 20000:
    tax = 1000 + \
        (income - 10000) * 0.15
```

```
if income <= 10000:
    tax = income * 0.1
elif income > 10000 and
        income <= 20000:
    tax = 1000 + \
        (income - 10000) * 0.15
```

4.19　下面的代码错在哪里？

```
income = 232323

if income <= 10000:
    tax = income * 0.1
elif income > 10000 and income <= 20000:
    tax = 1000 + (income - 10000) * 0.15

print(tax)
```

4.11　逻辑运算符

关键点：逻辑运算符 not、and 和 or 都可以用来创建一个组合条件。

有时候，几个条件组合在一起决定是否执行一条语句。你可以使用逻辑运算符来组合这些条件形成一个组合表达式。逻辑运算符，也被称为布尔运算符，它是在布尔值上的运算并创建出一个新的布尔值。表 4-3 罗列出所有的布尔运算符。表 4-4 定义了 not 运算符。它对 True 取反得 False，对 False 取反得 True。表 4-5 定义了 and 运算符。当且仅当两个操作数都为真时，两个操作数的 and 操作结果是真。表 4-6 定义了 or 运算符。至少有一个操作数为真，两个操作数的 or 操作结果才为真。

表 4-3　布尔运算符

运算符	描述
not	逻辑否
and	逻辑和
or	逻辑或

表 4-4　运算符 not 的真值表

p	not p	示例（假设 age = 24, gender = 'F'）
True	False	not (age > 18) 是 False，因为 (age > 18) 是 True
False	True	not (gender == 'M') 是 True，因为 (gender == 'M') 是 False

表 4-5　运算符 and 的真值表

p_1	p_2	p_1 and p_2	示例（假设 age = 24, gender = 'F'）
False	False	False	(age > 18) and (gender == 'F') 是 True，因为 (age > 18) 和 (gender == 'F') 都是 True
False	True	False	
True	False	False	(age > 18) and (gender != 'F') 是 False，因为 (gender != 'F') 是 False
True	True	True	

表 4-6 运算符 or 的真值表

p_1	p_2	p_1 and p_2	示例 (假设 age = 24, gender = 'F')
False	False	False	(age > 34) or (gender == 'F') 是 True, 因为 (gender == 'F') 是 True
False	True	True	
True	False	True	(age > 34) or (gender == 'M') 是 False, 因为 (age > 34) 和 (gender == 'M') 都是 False
True	True	True	

程序清单 4-8 中的程序检测一个数字是否都能被 2 和 3 整除，能被 2 或 3 整除，以及能被 2 或 3 整除但不能被两者同时都整除。

程序清单 4-8 TestBooleanOperators.py

```
1  # Receive an input
2  number = eval(input("Enter an integer: "))
3
4  if number % 2 == 0 and number % 3 == 0:
5      print(number, "is divisible by 2 and 3")
6
7  if number % 2 == 0 or number % 3 == 0:
8      print(number, "is divisible by 2 or 3")
9
10 if (number % 2 == 0 or number % 3 == 0) and  \
11        not (number % 2 == 0 and number % 3 == 0):
12     print(number, "is divisible by 2 or 3, but not both")
```

```
Enter an integer: 18 ↵Enter
18 is divisible by 2 and 3
18 is divisible by 2 or 3
```

```
Enter an integer: 15 ↵Enter
15 is divisible by 2 or 3
15 is divisible by 2 or 3, but not both
```

在第 4 行中，number % 2 ==0 and number % 3 ==0 检查数字是否能被 2 和 3 整除。第 7 行检测数字是否能被 2 或 3 整除。第 10 ～ 11 行的布尔表达式检测是否能被 2 或 3 整除但不被它们同时整除。

注意：德摩根律是以印度裔英国数学家和逻辑学家奥古斯·德摩根（1806—1871）命名的，可以用来简化布尔表达式。定理陈述的是：

```
not (condition1 and condition2)和not condition1 or not condition2一样
not (condition1 or condition2)和not condition1 and not condition2一样
```

所以，前面例子中的线 ||

```
not (number % 2 == 0 and number % 3 == 0)
```

可以使用等价的表达式简化为

```
(number % 2 != 0 or number % 3 != 0)
```

作为另一个例子

```
not (number == 2 or number == 3)
```

最好写作

```
number != 2 and number != 3
```

如果一个 and 运算符中的一个运算数是 False，则该表达式是 False；如果一个 or 运算符中一个操作数是 True，则该表达式是 True。Python 使用这些特性来提高运算符的性能，当计算 p1 和 p2 时，Python 首先计算 p1，然后如果 p1 为 True，再计算 p2；如果 p1 为 False，就不会再计算 p2。当计算 p1 or p2 时，Python 先计算 p1，然后如果 p1 为 False，再计算 p2；如果 p1 为 True，它就不计算 p2 了。因此，add 又被称为条件或短路 AND 运算符，而 or 被称为条件或短路 OR 运算符。

检查点

4.20　假设 x=1，显示下面布尔表达式的结果。

```
True and (3 > 4)
not (x > 0) and (x > 0)
(x > 0) or (x < 0)
(x != 0) or (x == 0)
(x >= 0) or (x < 0)
(x != 1) == not (x == 1)
```

4.21　编写一个布尔表达式，如果变量 num 的值在 1 到 100 之间，该表达式计算结果为 True。

4.22　编写一个布尔表达式，如果变量 num 的值在 1 到 100 之间或数字为负，该表达式计算结果为 True。

4.23　假设 x=4，y=5，显示下面的布尔表达式的结果。

```
x >= y >= 0
x <= y >= 0
x != y == 5
(x != 0) or (x == 0)
```

4.24　下面的表达式是否等价？

```
(a) (x >= 1) and (x < 10)
(b) (1 <= x < 10)
```

4.25　如果 ch 是 'A'、'p'、'E' 和 '5'，那表达式 ch >= 'A' and ch <= 'Z' 的值是多少？

4.26　假设当你运行下面的程序，从控制台输入 2、3、6，那么输出结果是什么？

```
x, y, z = eval(input("Enter three numbers: "))

print("(x < y and y < z) is", x < y and y < z)
print("(x < y or y < z) is", x < y or y < z)
print("not (x < y) is", not (x < y))
print("(x < y < z) is", x < y < z)
print("not(x < y < z) is", not (x < y < z))
```

4.27　编写一个如果年龄 age 大于 13 小于 18 的布尔表达式。

4.28　编写一个布尔表达式：如果体重 weight 大于 50 或身高 height 小于 160 则结果为真。

4.29　编写一个布尔表达式：如果体重 weight 大于 50 且身高 height 小于 160 则结果为真。

4.30　编写一个布尔表达式：如果体重 weight 大于 50，或身高 height 大于 160 但是这两个条件并不同时满足时结果为真。

4.12　实例研究：判定闰年

关键点：一个年份如果能被 4 整除但不能被 100 整除，或能被 400 整除，那么这个年份就是闰年。

你可以使用下面的布尔表达式来判定某一年是否闰年。

```
# A leap year is divisible by 4
isLeapYear = (year % 4 == 0)
# A leap year is divisible by 4 but not by 100
isLeapYear = isLeapYear and (year % 100 != 0)
# A leap year is divisible by 4 but not by 100 or divisible by 400
isLeapYear = isLeapYear or (year % 400 == 0)
```

你也可以将所有这些表达式组合成一个，如下所示：

```
isLeapYear = (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)
```

程序清单 4-9 就是一个程序，提示用户输入一个年份，然后判断此年份是否是闰年的。

程序清单 4-9 LeapYear.py

```
1  year = eval(input("Enter a year: "))
2
3  # Check if the year is a leap year
4  isLeapYear = (year % 4 == 0 and year % 100 != 0) or \
5      (year % 400 == 0)
6
7  # Display the result
8  print(year, "is a leap year?", isLeapYear)
```

```
Enter a year: 2008 ↵Enter
2008 is a leap year? True
```

```
Enter a year: 1900 ↵Enter
1900 is a leap year? False
```

```
Enter a year: 2002 ↵Enter
2002 is a leap year? False
```

4.13 实例研究：彩票

关键点：这个实例研究中的彩票程序涉及如何产生随机数、对比数字以及使用布尔表达式。

假设你想开发一个玩彩票的程序。程序随机产生一个两位数的数字，然后提示用户输入一个两位数的数字，并根据以下规则判定用户是否赢得奖金。

1）如果用户输入的数字和随机产生的数字完全相同（包括顺序），则奖金为 10 000 美元。

2）如果用户输入的数字和随机产生的数字相同（不包括顺序），则奖金为 3000 美元。

3）如果用户输入的数字和随机产生的数字有一位数相同，则奖金为 1000 美元。

完整的程序如程序清单 4-10 所示。

程序清单 4-10 Lottery.py

```
1  import random
2
3  # Generate a lottery number
4  lottery = random.randint(0, 99)
5
6  # Prompt the user to enter a guess
7  guess = eval(input("Enter your lottery pick (two digits): "))
8
9  # Get digits from lottery
10 lotteryDigit1 = lottery // 10
```

```
11  lotteryDigit2 = lottery % 10
12
13  # Get digits from guess
14  guessDigit1 = guess // 10
15  guessDigit2 = guess % 10
16
17  print("The lottery number is", lottery)
18
19  # Check the guess
20  if guess == lottery:
21      print("Exact match: you win $10,000")
22  elif (guessDigit2 == lotteryDigit1 and \
23      guessDigit1 == lotteryDigit2):
24      print("Match all digits: you win $3,000")
25  elif (guessDigit1 == lotteryDigit1
26          or guessDigit1 == lotteryDigit2
27          or guessDigit2 == lotteryDigit1
28          or guessDigit2 == lotteryDigit2):
29      print("Match one digit: you win $1,000")
30  else:
31      print("Sorry, no match")
```

```
Enter your lottery pick (two digits): 45 ↵Enter
The lottery number is 12
Sorry, no match
```

```
Enter your lottery pick (two digits): 23 ↵Enter
The lottery number is 34
Match one digit: you win $1,000
```

variable \ line#	4	7	10	11	14	15	29
lottery	34						
guess		23					
lotteryDigit1			3				
lotteryDigit2				4			
guessDigit1					2		
guessDigit2						3	
output							Match one digit: you win $1,000

程序使用函数 random.randint（0,99）（第 4 行）产生一个彩票数字，然后提示用户输入一个猜测的数字（第 7 行）。注意：因为数字 guess 是一个两位数，所以使用 guess%10 获得 guess 个位数，使用 guess//10 获得 guess 十位数（第 14 ～ 15 行）。

程序对照彩票数字来检测 guess 数字，顺序如下。

1）首先检测 guess 数字是否和彩票数字完全匹配（第 20 行）。

2）如果不是，检测 guess 数字的逆序是否和彩票数字完全匹配（第 22 ～ 23 行）。

3）如果不是，检测 guess 数字中的一个数是否和 guess 中的一个数字相同（第 25 ～ 28 行）。

4）如果不是，不匹配任何内容，然后显示“Sorry,no match”（第 30 ～ 31 行）。

4.14 条件表达式

关键点：条件表达式是根据某个条件计算一个表达式。

你可能想给一个变量赋值，但又受一些条件的限制。例如：下面的语句在 x 大于 0 时将 1 赋值给 y，在 x 小于等于 0 时将 -1 赋予 y。

```
if x > 0:
    y = 1
else:
    y = -1
```

你还可以像下面的例子一样，使用一个条件表达式来获取同样的结果。

```
y = 1 if x > 0 else -1
```

条件表达式完全是另一种不同风格。句法结构如下所示：

```
expression1 if boolean-expression else expression2
```

如果布尔逻辑表达式（boolean-expression）为真，那么这个条件表达式的结果就是 expression1；否则，这个结果就是 expression2。

假设你想将变量 number1 和 number2 中较大的赋值给 max。你可以使用下面的条件表达式简单地编写一条语句。

```
max = number1 if number1 > number2 else number2
```

对于另一个例子，如果 number 是偶数，下面的语句显示消息“number is even”，否则，显示“number is odd”。

```
print("number is even" if number % 2 == 0 else "number is odd")
```

检查点

4.31 假设你在运行下列程序时，从控制台输入 2、3、6。结果是什么？

```
x, y, z = eval(input("Enter three numbers: "))
print("sorted" if x < y and y < z else "not sorted")
```

4.32 使用一个条件表达式来改写下面的 if 语句：

```
if ages >= 16:
    ticketPrice = 20
else:
    ticketPrice = 10
```

```
if count % 10 == 0:
    print(count)
else:
    print(count, end = " ")
```

4.33 使用 if/else 语句改写下面的条件表达式。

```
(a) score = 3 * scale if x > 10 else 4 * scale
(b) tax = income * 0.2 if income > 10000 else income * 0.17 + 1000
(c) print(i if number % 3 == 0 else j)
```

4.15 运算符的优先级和结合方向

关键点：运算符的优先级和结合方向决定了运算符的计算顺序。

运算符的优先级和结合方向决定 Python 运算符的计算顺序。假设你有如下的表达式：

```
3 + 4 * 4 > 5 * (4 + 3) - 1
```

它的值是多少？这些运算符的执行顺序又是什么？

算术上，最先计算括号内的表达式。（括号也可以嵌套，最先执行的是最里面括号中的表达式。）当计算没有括号的表达式时，可以根据优先规则和组合规则使用运算符。

优先规则定义了运算符的优先性。表 4-7 包含了你至今已经学习过的所有运算符，以从

上到下的顺序罗列出来，其优先级越来越弱。逻辑运算符的优先级低于关系运算符，而关系运算符的优先级小于算术运算符。具有相同的优先级的运算符出现在同一行。

表 4-7　运算符优先级图

优先级	运算符
↓	+, -（一元加 / 减运算符）
	**（指数运算符）
	not
	*, /, //, %（乘、除、整除和余数）
	+, -（二元加 / 减运算符）
	<, <=, >, >=（比较运算符）
	==, !=（相等运算符）
	and
	or
	=, +=, -=, *=, /=, //=, %=（赋值运算符）

如果相同优先级的运算符紧连在一起，那它们的结合方向决定了计算顺序。所有的二元运算符（除赋值运算符外）都是从左到右的结合顺序。例如：因为 + 和 - 都有相同的优先级，所以表达式：

```
a - b + c - d  ═══相等于═══  ((a - b) + c) - d
```

注意：Python 有自己内部计算表达式的方法。Python 计算的结果和它对应的算术计算是一样的。

检查点

4.34　列出布尔运算符的优先级。计算下面的表达式：

```
True or True and False
True and True or False
```

4.35　除了 = 之外的其他所有二元运算符都是从左到右的结合顺序，这种说法是对还是错？

4.36　计算下面的表达式：

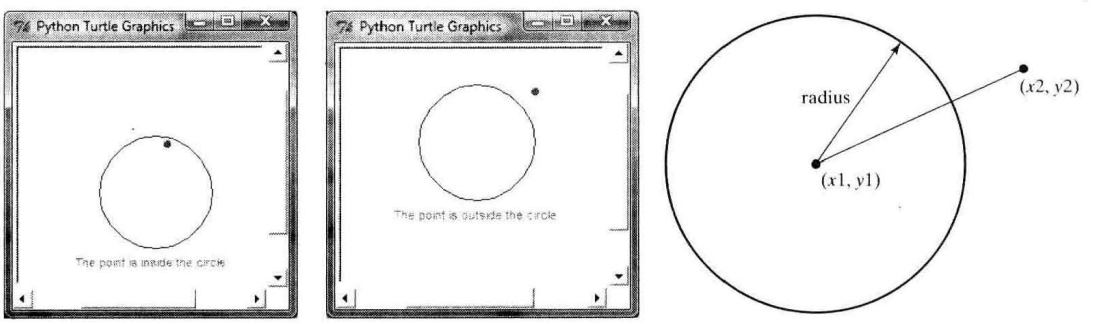
```
2 * 2 - 3 > 2 and 4 - 2 > 5
2 * 2 - 3 > 2 or 4 - 2 > 5
```

4.37　(x > 0 and x < 10) 和 ((x > 0) and (x < 10)) 是否一样？ (x > 0 or x < 10) 和 ((x > 0) or (x < 10)) 是否一样？ (x > 0 or x < 10 and y < 0) 和 (x > 0 or (x < 10 and y < 0)) 是否一样？

4.16　检测一个对象的位置

关键点：在一个游戏编程中，检测一个对象是否在另一个对象中是一个常见任务。

在游戏编程中，你经常需要决定一个对象是否在另一个对象中。这一节给出测试某个点是否在一个圆中的程序。这个程序提示用户输入一个圆心、半径和一个点。然后程序显示这个圆和点以及一条表明这个点是在圆内还是圆外的消息，如图 4-7a、图 4-7b 所示。



a）　b）　c）

图 4-7　程序显示一个圆、一个点以及一条表明这个点实在圆内还是圆外的消息

如果圆内的点到圆心的距离小于或等于圆的半径，如图 4-7c 所示。计算距离的公式为 $\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$。程序清单 4-11 给出这个程序。

程序清单 4-11 PointInCircle.py

```
1  import turtle
2
3  x1, y1 = eval(input("Enter the center of a circle x, y: "))
4  radius = eval(input("Enter the radius of the circle: "))
5  x2, y2 = eval(input("Enter a point x, y: "))
6
7  # Draw the circle
8  turtle.penup()       # Pull the pen up
9  turtle.goto(x1, y1 - radius)
10 turtle.pendown()     # Pull the pen down
11 turtle.circle(radius)
12 # Draw the point
13 turtle.penup()       # Pull the pen up
14 turtle.goto(x2, y2)
15 turtle.pendown()     # Pull the pen down
16 turtle.begin_fill()  # Begin to fill color in a shape
17 turtle.color("red")
18 turtle.circle(3)
19 turtle.end_fill()    # Fill the shape
20
21 # Display the status
22 turtle.penup()       # Pull the pen up
23 turtle.goto(x1 - 70, y1 - radius - 20)
24 turtle.pendown()
25
26 d = ((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1)) ** 0.5
27 if d <= radius:
28     turtle.write("The point is inside the circle")
29 else:
30     turtle.write("The point is outside the circle")
31
32 turtle.hideturtle()
33
34 turtle.done()
```

程序获取圆心的位置和半径（第 3 ～ 4 行）以及某个点的位置。程序显示这个圆（第 8 ～ 11 行）和这个点（第 13 ～ 19 行）。程序计算该点到圆心的距离（第 26 行），然后判断该点是在圆内还是在圆外。

第 16 ～ 19 行的代码绘制一个点，这可以使用 dot 方法简化（如表 3-6 所示）：

```
turtle.dot(6, "red")
```

这个方法画了一个直径为 6 的红点。

关键术语

Boolean expressions（布尔表达式）
Boolean value（布尔值）
operator associativity（运算符结合方向）
operator precedence（运算符优先级）
random module（random 模块）
selection statements（选择语句）
short-circuit evaluation（短路计算）

本章总结

1. 一个布尔类型变量可以储存值 True 或 False。
2. 关系运算符（<、<=、==、！ =、>、>=）是数字和字母一起工作的，产生的结果是一个布尔值。
3. 布尔运算符 and、or 和 not 是在布尔值和变量上的操作。
4. 当计算 p1 and p2 时，Python 首先计算 p1，如果 p1 为 True 才计算 p2；如果 p1 为 False，那它就不

再计算 p2。当计算 p1 or p2 时，Python 首先计算 p1，如果 p1 为 False 才计算 p2；如果 p1 为 True，那它就不再计算 p2。因此，and 称为条件或短路 AND 运算符，而 or 称为条件或短路 OR 运算符。

5. 选择语句是用来解决二选一的程序设计问题。Python 有多种类型的选择语句：单向 if 语句、双向 if-else 语句、嵌套 if-elif-else 语句以及条件表达式。
6. 各种 if 选择语句都是根据布尔表达式作出控制决定的。根据表达式计算的结果为 True 还是 False，这些语句会采用两个可能选项中的一种。
7. 算术表达式中的运算符根据括号、运算符优先性以及运算符结合方向等规则决定运算顺序。
8. 括号可以用来强制改变运算顺序。
9. 首先计算优先级高的运算符。对于优先级一样的运算符，它们的结合方向决定了计算顺序。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

教学建议：对于每道编程题，你应该在编码前仔细分析要解决问题的需求和设计策略。

调试提示：在你请求帮助之前，将程序读给自己并向自己解释，然后通过手动输入多个有代表性的输入或者通过使用一个 IDE 调试器来跟踪程序。你可以通过调试你自己的错误来学习如何编程。

第 4.2 节

*4.1 （代数方面：解一元二次方程）例如：$ax^2 + bx + c = 0$ 的平方根可以使用下面的公式获取。

$$r_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}, \quad r_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

$b^2 - 4ac$ 被称为二次方程的判别式。如果它为正，那么方程有两个实根。如果它为零，那么方程有一个根。如果它为负，那么方程没有实根。

编写程序，提示用户输入 a、b 和 c 的值，然后显示判别式的结果。如果判别式为正，则显示两个根。如果判别式为零，则显示一个根。否则，显示“The equation has no real roots”。下面是一个示例运行。

```
Enter a, b, c: 1.0, 3, 1 Enter
The roots are -0.381966 and -2.61803
```

```
Enter a, b, c: 1, 2.0, 1 Enter
The root is -1
```

```
Enter a, b, c: 1, 2, 3 Enter
The equation has no real roots
```

4.2 （游戏：三个数相加）程序清单 4-1 中的程序产生两个整数，然后提示用户输入这两个整数的和。修改这个程序产生三个一位整数，提示用户输入这三个整数的和。

第 4.3 ～ 4.8 节

*4.3 （代数：解 2 × 2 线性方程）你可以使用克莱姆法则解下面的线性方程 2 × 2 系统：

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \qquad x = \frac{ed - bf}{ad - bc} \qquad y = \frac{af - ec}{ad - bc}$$

编写程序，提示用户输入 a、b、c、d、e 和 f，然后显示结果。如果 $ad-bc$ 为零，呈现“The

equation has no solution”。

```
Enter a, b, c, d, e, f: 9.0, 4.0, 3.0, -5.0, -6.0, -21.0 ↵Enter
x is -2.0 and y is 3.0
```

```
Enter a, b, c, d, e, f: 1.0, 2.0, 2.0, 4.0, 4.0, 5.0 ↵Enter
The equation has no solution
```

**4.4 （游戏：学习加法）编写一个程序产生两个100以下的整数，然后提示用户输入这两个整数的和。如果答案是正确的，程序报告结果为真，否则为假。这个程序类似于程序清单4-1。

*4.5 （找未来数据）编写程序提示用户输入表示今天是一周内哪一天的数字（星期天是0，星期一是1，…，星期六是6）。还要提示用户输入今天之后到未来某天的天数，然后显示未来这天是星期几。下面是一个示例运行。

```
Enter today's day: 1 ↵Enter
Enter the number of days elapsed since today: 3 ↵Enter
Today is Monday and the future day is Thursday
```

```
Enter today's day: 0 ↵Enter
Enter the number of days elapsed since today: 31 ↵Enter
Today is Sunday and the future day is Wednesday
```

*4.6 （健康应用程序：BMI）修改程序清单4-6让用户输入磅表示的用户体重以及英尺英寸表示的用户身高。例如：如果一个人有5英尺10英寸，你就可以输入英尺5和英寸10。下面是一个示例运行。

```
Enter weight in pounds: 140 ↵Enter
Enter feet: 5 ↵Enter
Enter inches: 10 ↵Enter
BMI is 20.087702275404553
You are Normal
```

4.7 （财务应用程序：货币单位）修改程序清单3-4为只显示非零面值的程序，单一的值用单数表示，例如：1dollar（美元）和1penny（美分），而大于1的值用双数表示，例如：2dollars（美元）和3pennies（美分）。

*4.8 （对三个整数排序）编写一个程序提示用户输入三个整数，然后以升序显示它们。

*4.9 （金融方面：比较价钱）假设你购买大米时发现它有两种包装。你会想编写一个程序比较这两种包装的价钱。程序提示用户输入每种包装的重量和价钱，然后显示价钱更好的那种包装。下面是一个示例运行。

```
Enter weight and price for package 1: 50, 24.59 ↵Enter
Enter weight and price for package 2: 25, 11.99 ↵Enter
Package 2 has the better price.
```

4.10 （游戏：乘法测验）程序清单4-4随机产生一个减法问题。修改程序随机产生两个小于100的整数的乘法。

第4.9～4.16节

*4.11 （找出一个月中的天数）编写程序提示用户输入月和年，然后显示这个月的天数。例如：如果用户输入月份2而年份为2000，这个程序应该显示2000年二月份有29天。如果用户输入月份3而年份为2005，这个程序应该显示2005年三月份有31天。

4.12 （检测一个数字）编写一个程序提示用户输入一个整数，然后检测该数字是否能被5和6都整除、能被5或6整除还是只被它们中的一个整除（但又不能被它们同时整除）。下面是一个示例运行。

```
Enter an integer: 10 Enter
Is 10 divisible by 5 and 6? False
Is 10 divisible by 5 or 6? True
Is 10 divisible by 5 or 6, but not both? True
```

*4.13 （财务应用程序：计算税款）程序清单 4-7 给出源代码计算单身报税人的税款。完善程序清单 4-7 给出其他纳税状态的源代码。

4.14 （游戏：头或尾）编写程序让用户猜测一个弹起的硬币显示的是正面还是反面。程序提示用户输入一个猜测值，然后显示这个猜测值是正确的还是错误的。

**4.15 （游戏：彩票）改写程序清单 4-10 产生一个三位彩票数。程序提示用户输入一个三位整数，然后根据下面的规则判断用户是否赢得奖金。

1）如果用户输入的数和彩票数字完全匹配，包括数字的顺序，那么奖金是 10 000 美元。

2）如果用户输入的数匹配彩票数字中的所有数字，那么奖金是 3000 美元。

3）如果用户输入的数中的一个数匹配彩票数字中的一个数，那么奖金是 1000 美元。

4.16 （随机字符）编写程序显示一个随机大写字母。

*4.17 （游戏：剪刀、石头、布）编写程序来玩流行的剪刀 – 石头 – 布的游戏。（剪刀可以剪纸，石头可以磕碰剪刀，而布可以包裹石头。）程序随机产生一个数字 0、1 或 2 来表示剪刀、石头和布。程序提示用户输入数字 0、1 或 2 然后显示一条消息表示用户或计算机是赢、输还是平局。下面是一个示例运行。

```
scissor (0), rock (1), paper (2): 1 Enter
The computer is scissor. You are rock. You won.
```

```
scissor (0), rock (1), paper (2): 2 Enter
The computer is paper. You are paper too. It is a draw.
```

*4.18 （金融问题：货币对换）编写一个程序提示用户输入美元和人民币之间的货币汇率。提示用户输入 0 表示将美元转换为人民币而 1 表示将人民币转换为美元。提示用户输入美元数或人民币数将它分别转换为人民币或美元。下面是一些示例运行。

```
Enter the exchange rate from dollars to RMB: 6.81 Enter
Enter 0 to convert dollars to RMB and 1 vice versa: 0 Enter
Enter the dollar amount: 100 Enter
$100.0 is 681.0 yuan
```

```
Enter the exchange rate from dollars to RMB: 6.81 Enter
Enter 0 to convert dollars to RMB and 1 vice versa: 1 Enter
Enter the RMB amount: 10000 Enter
10000.0 yuan is $1468.43
```

```
Enter the exchange rate from dollars to RMB: 6.81 Enter
Enter 0 to convert dollars to RMB and 1 vice versa: 5 Enter
Incorrect input
```

**4.19 （计算三角形的周长）编写程序读取三角形的三个边，如果输入都是合法的则计算它的周长。否则，显示这个输入是非法的。如果两边之和大于第三边则输入都是合法的。下面是一个示例运行。

```
Enter three edges: 1, 1, 1 Enter
The perimeter is 3
```

```
Enter three edges: 1, 3, 1 ↵Enter
The input is invalid
```

*4.20 （科学方面：风寒温度）编程题2.9给出计算风寒温度的公式。这个公式适用于温度在 -58°F 和41°F之间且风速大于或等于2。编写程序提示用户输入一个温度值和风速。如果输入是合法的，程序就显示风寒温度；否则，它显示一条消息表明这个温度或风速是非法的。

综合题

**4.21 （科学问题：一周的星期几）泽勒的一致性是一个由泽勒开发的算法，用于计算一周的星期几。这个公式是

$$h=\left(q+\left\lfloor\frac{26(m+1)}{10}\right\rfloor+k+\left\lfloor\frac{k}{4}\right\rfloor+\left\lfloor\frac{j}{4}\right\rfloor+5j\right)\%7$$

- 这里的 h 是指一周的星期几（0：星期六；1：星期天；2：星期一；3：星期二；4：星期三；5：星期四；6：星期五）。
- q 是一个月的哪一天。
- m 是月份（3：三月；4：四月；…；12：十二月）。一月和二月都是按照前一年的13月和14月来计数的。
- j 是世纪数（即 $\left\lfloor\frac{year}{100}\right\rfloor$）。
- k 是一个世纪的某一年（即 $year$ % 100）。

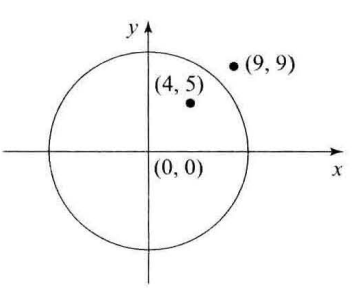
编写程序提示用户输入一个年份、月份以及这个月的某天，然后它会显示它是一周的星期几。下面是一些示例运行。

```
Enter year: (e.g., 2008): 2013 ↵Enter
Enter month: 1-12: 1 ↵Enter
Enter the day of the month: 1-31: 25 ↵Enter
Day of the week is Friday
```

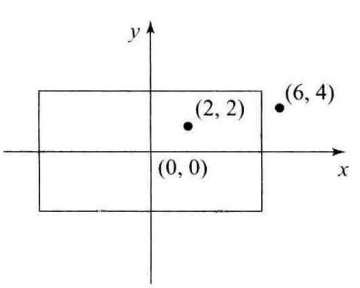
```
Enter year: (e.g., 2008): 2012 ↵Enter
Enter month: 1-12: 5 ↵Enter
Enter the day of the month: 1-31: 12 ↵Enter
Day of the week is Saturday
```

（提示：$\lfloor n\rfloor = n//1$ 其中 n 是一个正数。一月和二月在公式中是以13和14来计算的，所以你需要将用户输入的月份1转换为13和将用户输入的2转换为14，将它们的年份改变为前一年。）

**4.22 （几何问题：点在圆内吗？）编写一个程序提示用户输入一个点 (x,y)，然后检测这个点是否在圆心为 (0,0) 半径为10的圆内。例如：点 (4,5) 在圆内而 (9,9) 在圆外，如图4-8a所示。



a）圆内和圆外的点



b）矩形内和矩形外的点

图 4-8

（提示：如果一个点到 (0,0) 之间的距离小于或等于 10，那它就在圆内。计算距离的公式是 $\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$。测试你的程序考虑所有的情况）。下面是两个示例运行。

```
Enter a point with two coordinates: 4, 5 ↵Enter
Point (4.0, 5.0) is in the circle
```

```
Enter a point with two coordinates: 9, 9 ↵Enter
Point (9.0, 9.0) is not in the circle
```

**4.23 （几何问题：点在矩形内吗？）编写程序提示用户输入点 (x,y)，然后检测这个点是否在以 (0,0) 为中心而宽为 10 高为 5 的矩形内。例如：(2,2) 在矩形内而 (6,4) 在矩形外，如图 4-8b 所示。（提示：如果一个点到 (0,0) 的水平距离小于或等于 10/2 而到 (0,0) 的垂直距离小于或等于 5.0/2。测试你的程序覆盖所有的情况。）下面是两个示例运行。

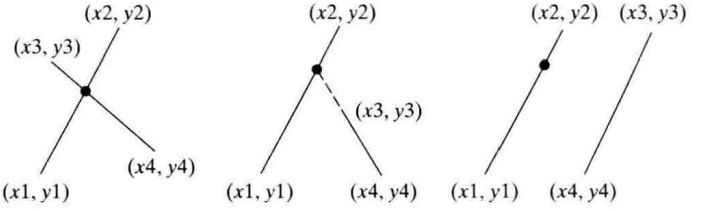
```
Enter a point with two coordinates: 2, 2 ↵Enter
Point (2.0, 2.0) is in the rectangle
```

```
Enter a point with two coordinates: 6, 4 ↵Enter
Point (6.0, 4.0) is not in the rectangle
```

**4.24 （游戏：选出一张牌）编写程序模拟从 52 张牌中选出一张。你的程序应该显示这张牌的大小（Ace、2、3、4、5、6、7、8、9、10、Jack、Queen、King）和花色（梅花、红桃、方块、黑桃）。下面是这个程序的示例运行。

```
The card you picked is the Jack of Hearts
```

*4.25 （几何问题：交点）行 1 上的两个点是 (x1, y1) 和 (x2, y2)，而行 2 上的点是 (x3, y3) 和 (x4, y4)，如图 4-9a、4-9b 所示。



a）～ b）中两线交叉　　c）两线平行

图　4-9

这两条线的交点可以通过解下面的线性等式找出：

$$(y_1-y_2)x-(x_1-x_2)y=(y_1-y_2)x_1-(x_1-x_2)y_1$$

$$(y_3-y_4)x-(x_3-x_4)y=(y_3-y_4)x_3-(x_3-x_4)y_3$$

这个线性等式可以通过使用克莱姆法则解决（参见编程题 4.3）。如果这个等式没有解，那么这两条线平行（图 4-9c）。编写程序提示用户输入四个点，然后显示交点。这里有一些示例运行。

```
Enter x1, y1, x2, y2, x3, y3, x4, y4:
  2, 2, 5, -1, 4, 2, -1, -2 ↵Enter
The intersecting point is at (2.88889, 1.1111)
```

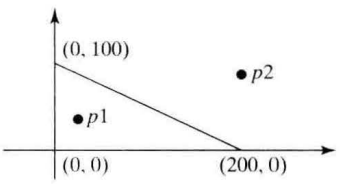
```
Enter x1, y1, x2, y2, x3, y3, x4, y4:
  2, 2, 7, 6, 4, 2, -1, -2 ↵Enter
The two lines are parallel
```

4.26 （回文数）编写程序提示用户输入一个三位整数，然后决定它是否是一个回文数。如果一个数从左向右和从右向左读取时是一样的，那么这个数就是回文数。下面是这个程序的示例运行。

```
Enter a three-digit integer: 121 Enter
121 is a palindrome
```

```
Enter a three-digit integer: 123 Enter
123 is not a palindrome
```

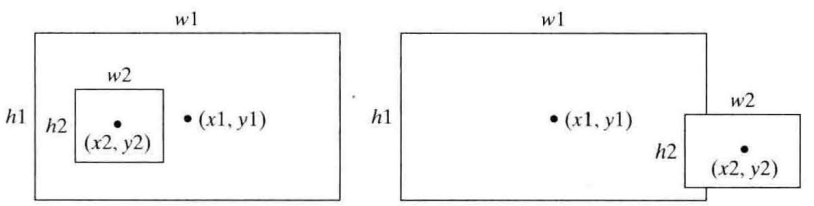
**4.27 （几何问题：点在三角形内吗？）假设一个直角三角形被放在一个水平面上，如下图所示。直角点是在 (0,0) 而另外两个点在 (200,0) 和 (0,100) 处。编写程序提示用户输入一个带 x 坐标和 y 坐标的点，然后决定这个点是否在三角形内。下面是一些示例运行。




```
Enter a point's x- and y-coordinates: 100.5, 25.5 Enter
The point is in the triangle
```

```
Enter a point's x- and y-coordinates: 100.5, 50.5 Enter
The point is not in the triangle
```

**4.28 （几何问题：两个矩形）编写程序提示用户输入两个矩形中心的 x 坐标和 y 坐标以及它们的宽和高，然后决定第二个矩形是否在第一个矩形里还是和第一个矩形有重叠部分，如图 4-10 所示。测试你的程序覆盖所有的情况。



a）一个矩形在另一个矩形内　　b）一个矩形和另一个矩形有重叠

图 4-10

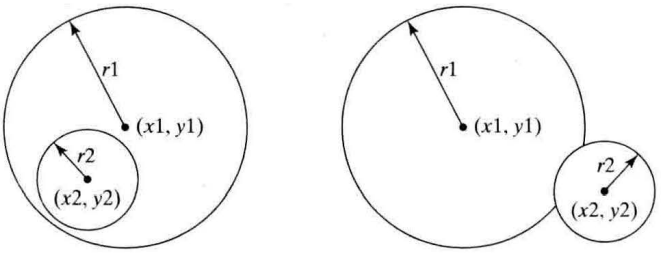
这里是一些示例运行。

```
Enter r1's center x-, y-coordinates, width, and height:
  2.5, 4, 2.5, 43 Enter
Enter r2's center x-, y-coordinates, width, and height:
  1.5, 5, 0.5, 3 Enter
r2 is inside r1
```

```
Enter r1's center x-, y-coordinates, width, and height:
  1, 2, 3, 5.5 Enter
Enter r2's center x-, y-coordinates, width, and height:
  3, 4, 4.5, 5 Enter
r2 overlaps r1
```

```
Enter r1's center x-, y-coordinates, width, and height:
  1, 2, 3, 3 Enter
Enter r2's center x-, y-coordinates, width, and height:
  40, 45, 3, 2 Enter
r2 does not overlap r1
```

**4.29 （几何问题：两个圆）编写程序提示用户输入两个圆的中心的坐标以及它们的半径，然后判断第二个圆是在第一个圆内还是和第一个圆有重叠部分，如图 4-11 所示。（提示：如果两个中心的距离≤ | r1 − r2| 那么 circle2 在 circle1 内，如果两个中心的距离≤ r1 + r2 那么 circle2 是和 circle1 有重叠的。测试你的程序覆盖所有的情况。）



a）一个圆在另一个圆内　　b）一个圆和另一个圆有重叠

图　4-11

下面是一些示例运行。

```
Enter circle1's center x-, y-coordinates, and radius:
  0.5, 5.1, 13 Enter
Enter circle2's center x-, y-coordinates, and radius:
  1, 1.7, 4.5 Enter
circle2 is inside circle1
```

```
Enter circle1's center x-, y-coordinates, and radius:
  4.4, 5.7, 5.5 Enter
Enter circle2's center x-, y-coordinates, and radius:
  6.7, 3.5, 3 Enter
circle2 overlaps circle1
```

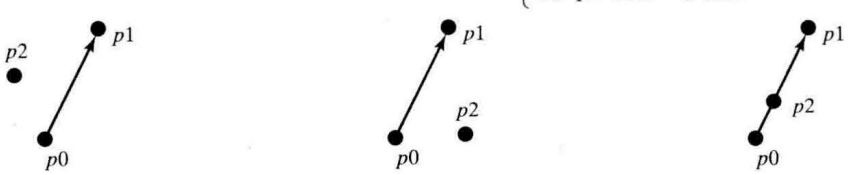
```
Enter circle1's center x-, y-coordinates, and radius:
  4.4, 5.5, 1 Enter
Enter circle2's center x-, y-coordinates, and radius:
  5.5, 7.2, 1 Enter
circle2 does not overlap circle1
```

*4.30 （当前时间）使用 12 小时的时钟修改编程题 2.18 来显示小时数。下面是一个示例运行。

```
Enter the time zone offset to GMT: -5 Enter
The current time is 4:50:34 AM
```

*4.31 （几何问题：点的位置）假设有一个从点 $p0(x0, y0)$ 到 $p1(x1, y1)$ 的有向直线，你可以使用下面的条件决定点 $p2(x2,y2)$ 是在这条线的右边，还是在这条线的左边，或者是在同一条线上（参见图 4-12）：

$$(x1-x0)*(y2-y0)-(x2-x0)*(y1-y0)\begin{cases}>0 & p2\text{ 在线的左边}\\ =0 & p2\text{ 在线的右边}\\ <0 & p2\text{ 在同一条线上}\end{cases}$$



a）$p2$ 在线的左边　　b）$p2$ 在线的右边　　c）$p2$ 在同一条线上

图　4-12

编写程序提示用户输入这三个点 $p0$、$p1$ 和 $p2$ 的 x 坐标和 y 坐标，然后显示 $p2$ 是在从 $p0$ 到 $p1$ 的线的左边、右边还是在同一线上。下面是一些示例运行。

```
Enter coordinates for the three points p0, p1, and p2:
  3.4, 2, 6.5, 9.5, -5, 4 Enter
p2 is on the left side of the line from p0 to p1
```

```
Enter coordinates for the three points p0, p1, and p2:
  1, 1, 5, 5, 2, 2 Enter
p2 is on the same line from p0 to p1
```

```
Enter coordinates for the three points p0, p1, and p2:
  3.4, 2, 6.5, 9.5, 5, 2.5 Enter
p2 is on the right side of the line from p0 to p1
```

*4.32 （几何问题：线段上的点）编程题 4.31 显示如何测试一个点是否在一个无界的行上。修改编程题 4.31 来测试一个点是否在一个线段上。编写程序提示用户输入这三个点 $p0$、$p1$ 和 $p2$ 的 x 坐标和 y 坐标，然后显示 $p2$ 是否在从 $p0$ 到 $p1$ 的线段。下面是一些示例运行。

```
Enter coordinates for the three points p0, p1, and p2:
  1, 1, 2.5, 2.5, 1.5, 1.5 Enter
(1.5, 1.5) is on the line segment from (1.0, 1.0) to (2.5, 2.5)
```

```
Enter coordinates for the three points p0, p1, and p2:
  1, 1, 2, 2, 3.5, 3.5 Enter
(3.5, 3.5) is not on the line segment from (1.0, 1.0) to
  (2.0, 2.0)
```

*4.33 （十进制转十六进制）编写一个程序提示用户输入一个 0 到 15 之间的整数，然后显示它对应的十六进制数。下面是一些示例运行。

```
Enter a decimal value (0 to 15): 11 Enter
The hex value is B
```

```
Enter a decimal value (0 to 15): 5 Enter
The hex value is 5
```

```
Enter a decimal value (0 to 15): 31 Enter
Invalid input
```

*4.34 （十六进制转十进制）编写一个程序提示用户输入一个十六进制的字符，然后显示它对应的十进制整数。下面是一些示例运行。

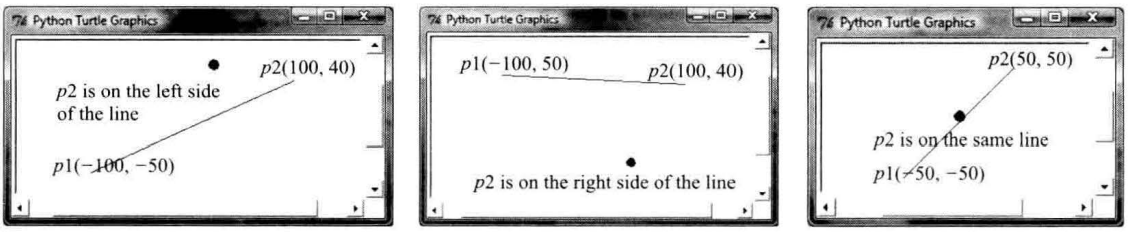
```
Enter a hex character: A Enter
The decimal value is 10
```

```
Enter a hex character: a Enter
The decimal value is 10
```

```
Enter a hex character: 5 Enter
The decimal value is 5
```

```
Enter a hex character: G  Enter
Invalid input
```

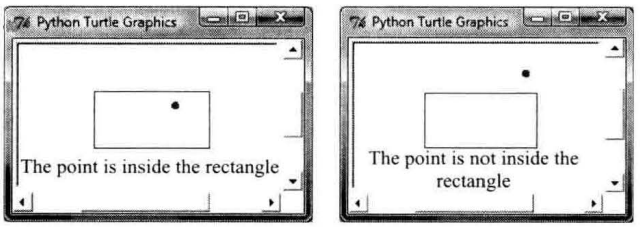
*4.35 （Turtle：点的位置）编写一个程序提示用户输入三个点 $p0$、$p1$ 和 $p2$ 的 x 坐标和 y 坐标，然后显示一条消息表明 $p2$ 是在从 $p0$ 到 $p1$ 的线的右边、左边还是线上，如图 4-13 所示。参见编程题 4.31 确定点的位置。



a）　b）　c）

图 4-13　程序显示点的位置

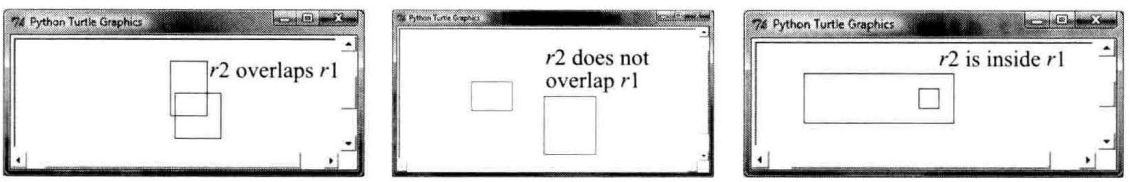
**4.36 （Turtle：点在矩形内吗？）编写一个程序提示用户输入点 (x, y)，然后检测该点是否在以 (0,0) 为中心、宽为 100、高为 50 的矩形内。在屏幕上显示这个点、矩形以及表明这个点是否在矩形内的消息，如图 4-14 所示。



a）　b）

图 4-14　程序显示矩形、点以及表明这个点是否在矩形内的消息

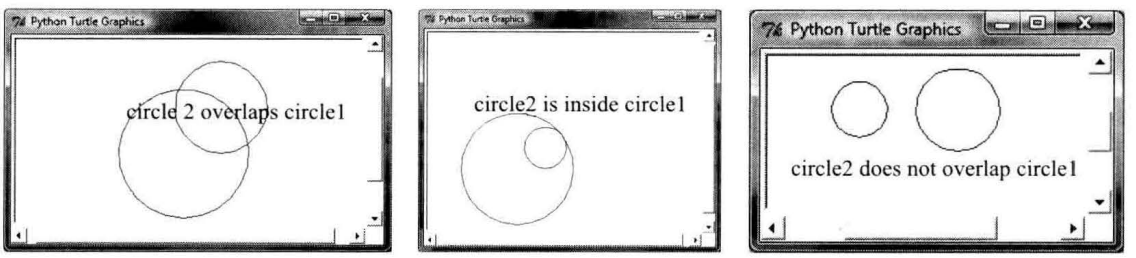
*4.38 （几何问题：两个矩形）编写程序提示用户输入两个矩形的中心的 x 坐标、y 坐标、宽度和高度，然后决定第二个矩形是在第一个矩形内还是和第一个矩形有重叠，如图 4-15 所示。



a）　b）　c）

图 4-15　程序检测一个矩形是在另一个矩形内，和另一个矩形有重叠，还是没有重叠

*4.39 （Turtle：两个圆）编写程序提示用户输入两个圆的圆心的坐标以及两个半径，然后确定第二个圆是在第一个圆内还是和第一个圆有重叠部分，如图 4-16 所示。



a）　b）　c）

图 4-16　程序显示两个圆以及一条表示状态的信息

第 5 章
Introduction to Programming Using Python

循　　环

学习目标

- 通过使用 while 循环编写重复执行的语句（第 5.2 节）。
- 遵从循环的设计策略开发循环（第 5.2.1 ～ 5.2.3 节）。
- 利用用户的确认控制循环（第 5.2.4 节）。
- 用哨兵值控制循环（第 5.2.5 节）。
- 通过使用输入重定向从文件获取大量数据而不是从键盘输入来来获取大量数据，并且使用输出重定向将输出存入文件（第 5.2.6 节）。
- 使用 for 循环来实现计数器控制的循环（第 5.3 节）。
- 编写嵌套循环（第 5.4 节）。
- 学习减少数值错误的技术（第 5.5 节）。
- 从大量的例子里学习循环（GCD、FutureTuition、MonteCarloSimulation、PrimeNumber）（第 5.6、5.8 节）。
- 使用 break 和 continue 控制程序（第 5.7 节）。
- 使用一个循环来模拟随机漫步（第 5.9 节）。

5.1　引言

关键点：可以使用循环来告诉程序重复执行某些语句。

假设你需要显示一个字符串 100 次（例如：Programming is fun!）。直接输入这个语句 100 次将是一个非常乏味的过程：

100 times
```
print("Programming is fun!")
print("Programming is fun!")
...
print("Programming is fun!")
```

那么，你要如何解决这个问题呢？

脚本语言提供了一个非常强大的被称作循环的概念，它能控制系统连续完成一个操作（或一系列操作）的次数。通过使用循环语句，你就不需要编写这条打印语句 100 次；你只要告诉计算机显示这个字符串 100 次。循环语句可以如下编写：

```
count = 0
while count < 100:
    print("Programming is fun!")
    count = count + 1
```

变量 count 初始值是 0。循环检测 count<100 的条件是否为真。如果是真的，它就执行这个循环体，即循环里包含那些被重复的语句部分，显示一条消息“Programming is fun!”同时将 count 增加 1。它将重复执行循环体直到 count<100 变成假（即变量 count 达到 100）。这时，循环将终止然后执行循环体后的下一条语句。

循环是一种控制一个语句块重复执行的结构。循环的概念是程序设计的基础。Python 提供了两种类型的循环语句：while 循环和 for 循环。while 循环是一种条件控制循环，它是根据一个条件的真假来控制的；而 for 循环是一种计数器控制循环，它会重复特定的次数。

5.2 while 循环

关键点：当一个条件保持为真时 while 循环重复执行语句。

while 循环的语法是：

```
while loop-continuation-condition:
    # Loop body
    Statement(s)
```

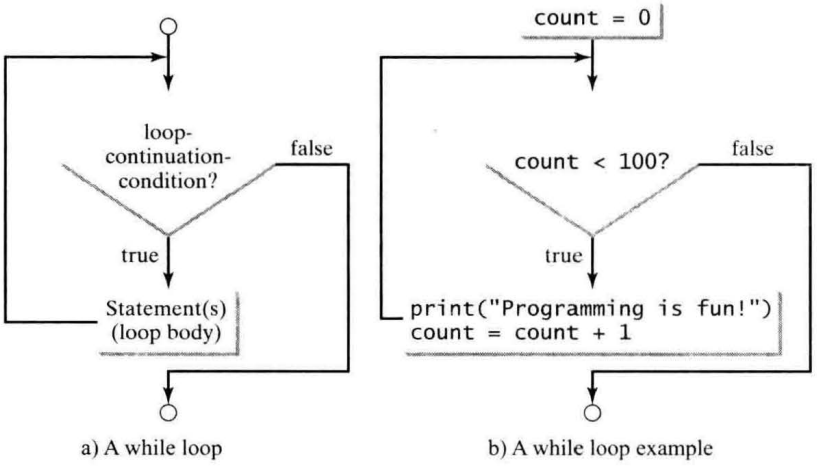
图 5-1a 显示的是 while 循环流程图。一个循环体单次执行被称作循环的一次迭代（或操作）。每个循环都包含一个 loop-continuation-condition（循环继续条件），这是控制循环体执行的布尔表达式。每次都计算它来检测是否应该执行循环体。如果它的计算结果为真，则执行循环体；否则，终止整个循环并且程序控制权转到 while 循环后的语句。

这个显示“programming is fun!”一百次的循环是一个 while 循环的例子。它的流程图如图 5-1b。循环继续条件是 count<100 并且这个循环体包含两条语句：

```
count = 0
while count < 100:
    print("Programming is fun!")
    count = count + 1
```

loop-continuation-condition

loop body



a) A while loop　　b) A while loop example

图 5-1　如果 loop-continuation-condition 计算结果为 true，那么 while 循环重复执行循环体里的语句

下面是另一个图解循环是如何执行的例子：

```
sum = 0
i = 1
while i < 10:
    sum = sum + i
    i = i + 1
print("sum is", sum) # sum is 45
```

如果 i<10 为真，程序把 i 加到 sum 上。变量 i 最初被设置为 1，然后被增加到 2、3 等等直到为 10。当 i 是 10 时，i<10 就为假，然后退出这个循环。所以 sum 就是 1 + 2 + 3 + 4 +

5 + 6 + 7 + 8 + 9 = 45。

假设循环被错误地写成如下所示：

```
sum = 0
i = 1
while i < 10:
    sum = sum + i
i = i + 1
```

注意整个循环体必须被内缩进到循环内部。这里的语句 i=i+1 不在循环体里。这是一个无限循环，因为 i 一直是 1 而 i<10 总是为真。

注意：确保 loop-continuation-condition 最终变成 false 以便结束循环。一个常见的程序设计错误是涉及无限循环（即循环被永远执行）。如果你的程序占用了相当长的一段时间也没有结束，那它可能就是一个无限循环。如果你通过命令行运行这个程序，那么按 Ctrl+c 来停止它。

警告：程序员经常会错误地让循环执行比预期多执行一次或少执行一次。这种错误通常被称作偏离 1 的误差。例如：下面的循环显示 Programming is fun101 次而不是 100 次。错误的原因在于条件，它应该是 count<100 而不是 count<=100.

```
count = 0
while count <= 100:
    print("Programming is fun!")
    count = count + 1
```

回顾程序清单 4-4，它给出一个提示用户输入一个数学题答案的程序。现在通过循环，你可以重写这个程序，让用户一直输入答案直到输入正确答案为止，如程序清单 5-1 所示。

程序清单 5-1 RepeatSubtractionQuiz.py

```
1  import random
2
3  # 1. Generate two random single-digit integers
4  number1 = random.randint(0, 9)
5  number2 = random.randint(0, 9)
6
7  # 2. If number1 < number2, swap number1 with number2
8  if number1 < number2:
9      number1, number2 = number2, number1
10
11 # 3. Prompt the student to answer "What is number1 - number2?"
12 answer = eval(input("What is " + str(number1) + " - "
13     + str(number2) + "? "))
14
15 # 4. Repeatedly ask the question until the answer is correct
16 while number1 - number2 != answer:
17     answer = eval(input("Wrong answer. Try again. What is "
18         + str(number1) + " - " + str(number2) + "? "))
19
20 print("You got it!")
```

```
What is 4 - 3? 4 ↵Enter
Wrong answer. Try again. What is 4 - 3? 5 ↵Enter
Wrong answer. Try again. What is 4 - 3? 1 ↵Enter
You got it!
```

第 16 ～ 18 行的循环会在 number1-number2!=answer 为真时，重复提示用户输入一个答案。一旦语句 number1-number2!=answer 为假时，退出循环。

5.2.1　实例研究：猜数字

这里的问题是猜出电脑里存储的数字是什么。你将要编写一个能够随机生成一个 0 到 100 之间且包括 0 和 100 的数字的程序。这个程序提示用户连续地输入数字直到它与那个随机生成的数字相同。对于每个用户输入的数字，程序会提示它是否过高还是过低，所以，用户可以更明智地选择下一个输入的数字。下面是一个简单的运行：

```
Guess a magic number between 0 and 100
Enter your guess: 50 Enter
Your guess is too high
Enter your guess: 25 Enter
Your guess is too low
Enter your guess: 42 Enter
Your guess is too high
Enter your guess: 39 Enter
Yes, the number is 39
```

这个随机生成的数字在 0 到 100 之间。为了缩小猜测数字的范围，首先输入 50。如果你猜得太高，那么随机生成的数字就在 0 到 49 之间。如果你猜得太低，那么随机生成的数字就在 51 到 100 之间。因此，在猜一次之后，你就可以为接下来的猜测缩小一半的范围。

你怎么编写这个程序呢？你要立刻就开始编写代码吗？不。*在编写代码之前先思考*是非常重要的。想一想在不编写程序的情况下怎样解决这个问题。首先，你需要随机生成一个 0 到 100 之间且包括 0 和 100 的数字，提示用户输入一个数字，然后将这个数字与随机生成的数字进行比较。

*递增地*编码是非常好的一种方式，即每次完成一步。对于涉及循环的程序，如果你不知道如何一下子编写完这个循环，你可以先编写一个执行一次代码的程序，接下来要搞清楚如何在一个循环里重复执行。对于这个程序，你可以创建一个初稿，如程序清单 5-2 所示。

程序清单 5-2　GuessNumberOneTime.py

```
1  import random
2
3  # Generate a random number to be guessed
4  number = random.randint(0, 100)
5
6  print("Guess a magic number between 0 and 100")
7
8  # Prompt the user to guess the number
9  guess = eval(input("Enter your guess: "))
10
11 if guess == number:
12     print("Yes, the number is", number)
13 elif guess > number:
14     print("Your guess is too high")
15 else:
16     print("Your guess is too low")
```

当这个程序运行时，它将提示用户一次输入一个猜测的数字。为了重复输入数字，你可以修改第 11 ～ 16 行的代码将它们改成一个循环，如下所示。

```
1  while True:
2      # Prompt the user to guess the number
3      guess = eval(input("Enter your guess: "))
4
5      if guess == number:
```

```
6           print("Yes, the number is", number)
7       elif guess > number:
8           print("Your guess is too high")
9       else:
10          print("Your guess is too low")
```

这个循环重复提示用户输入猜测的数字。然而，循环还是需要终止的；当输入的数字 guess 与电脑里的数字 number 相匹配时，循环就结束。所以，如下修改这个循环。

```
1  while guess != number:
2      # Prompt the user to guess the number
3      guess = eval(input("Enter your guess: "))
4
5      if guess == number:
6          print("Yes, the number is", number)
7      elif guess > number:
8          print("Your guess is too high")
9      else:
10         print("Your guess is too low")
```

完整的代码在程序清单 5-3 中。

程序清单 5-3 GuessNumber.py

```
1  import random
2
3  # Generate a random number to be guessed
4  number = random.randint(0, 100)
5
6  print("Guess a magic number between 0 and 100")
7
8  guess = -1
9  while guess != number:
10     # Prompt the user to guess the number
11     guess = eval(input("Enter your guess: "))
12
13     if guess == number:
14         print("Yes, the number is", number)
15     elif guess > number:
16         print("Your guess is too high")
17     else:
18         print("Your guess is too low")
```

	line#	number	guess	output
	4	39		
	8		-1	
iteration 1	11		50	
	16			Your guess is too high
iteration 2	11		25	
	18			Your guess is too low
iteration 3	11		42	
	16			Your guess is too high
iteration 4	11		39	
	14			Yes, the number is 39

程序在第 4 行随机生成一个数字，然后提示用户在循环里重复输入一个猜测的数（第 9 ～ 18 行）。针对每次猜测的数，程序判定用户输入的数字是否正确，是太高，还是太低（第 13 ～

18 行）。当猜测的数字正确时，程序就会退出这个循环（第 9 行）。注意：guess 被初始化为 -1。这是为了避免将它初始化为一个 0 到 100 之间的数字，因为那可能就是要猜测的数字。

5.2.2　循环设计策略

对于一个初学编程的人来说，编写一个能够正确工作的循环不是一个容易的任务。编写一个循环时可以考虑以下三步。

第 1 步：确认需要循环的语句。

第 2 步：把这些语句包裹在一个循环，如下所示。

```
while True:
    Statements
```

第 3 步：编写循环继续条件并且添加合适的语句控制循环。

```
while loop-continuation-condition:
    Statements
    Additional statements for controlling the loop
```

5.2.3　实例研究：多道减法题测验

在程序清单 4-4 的减法测试程序中，它只会在每次运行时生成一个问题。你可以使用一个循环来连续生成问题。那么该怎样编写一个能生成五个问题的程序呢？按照循环设计策略。第一步，确定需要被循环的语句，这些语句包括获取两个随机数，提示用户做减法，然后给这个题打分。第二步，将这些语句放置在一个循环里。第三步，添加循环控制变量和循环继续条件来执行五次循环。

程序清单 5-4 是一个能生成五个问题的程序，在一个学生回答完所有的问题后，报告正确答案的个数。这个程序还能显示这个测验所用时间。

程序清单 5-4　SubtractionQuizLoop.py

```
1  import random
2  import time
3
4  correctCount = 0  # Count the number of correct answers
5  count = 0   # Count the number of questions
6  NUMBER_OF_QUESTIONS = 5  # Constant
7
8  startTime = time.time() # Get start time
9
10 while count < NUMBER_OF_QUESTIONS:
11     # Generate two random single-digit integers
12     number1 = random.randint(0, 9)
13     number2 = random.randint(0, 9)
14
15     # If number1 < number2, swap number1 with number2
16     if number1 < number2:
17         number1, number2 = number2, number1
18
19     # Prompt the student to answer "What is number1 - number2?"
20     answer = eval(input("What is " + str(number1) + " - " +
21         str(number2) + "? "))
22
23     # Grade the answer and display the result
24     if number1 - number2 == answer:
25         print("You are correct!")
26         correctCount += 1
```

```
27      else:
28          print("Your answer is wrong.\n", number1, "-",
29              number2, "is", number1 - number2)
30
31      # Increase the count
32      count += 1
33
34  endTime = time.time() # Get end time
35  testTime = int(endTime - startTime) # Get test time
36  print("Correct count is", correctCount, "out of",
37      NUMBER_OF_QUESTIONS, "\nTest time is", testTime, "seconds")
```

```
What is 1 - 1? 0 Enter
You are correct!

What is 7 - 2? 5 Enter
You are correct!

What is 9 - 3? 4 Enter
Your answer is wrong.
9 - 3 is 6

What is 6 - 6? 0 Enter
You are correct!

What is 9 - 6? 2 Enter
Your answer is wrong.
9 - 6 is 3

Correct count is 3 out of 5
Test time is 10 seconds
```

这个程序使用控制变量 count 的数值来控制循环的执行。count 的初始值为 0（第 5 行）而且每次迭代递增 1（第 32 行）。每次迭代显示一个问题并对它进行处理。程序在第 8 行获取这个测验开始的时间而在第 34 行获取测试结束的时间，在第 35 行计算测验所占用的秒数。程序在所有问题都被问完后会给出正确的个数以及测验所占用时间（第 36 ～ 37 行）。

5.2.4 根据用户确认控制循环

上面的例子执行五次循环。如果你想让用户来决定是否还要提问下一个问题，可以为用户提供一个确认语句。程序的模板可以如下编写代码。

```
continueLoop = 'Y'
while continueLoop == 'Y' :
    # Execute the loop body once
    ...

    # Prompt the user for confirmation
    continueLoop = input("Enter Y to continue and N to quit: ")
```

你可以利用用户确认让用户决定是否还要提问下一个问题来改写程序清单 5-4 的程序。

5.2.5 使用哨兵值控制循环

另一个常见的控制循环的技术是指派一个特殊的输入值，这个值被称作哨兵值（sentinel value），它表明输入的结束。使用哨兵值控制循环的这种方式被称作步哨式控制（sentinel-controlled loop）。

程序清单 5-5 里的程序读取并计算一组不确定个数的整数的和。输入 0 表明输入的结束。你不必为每一个输入的值设置一个新变量。而是只需使用一个名为 data 的变量（第 1 行）来存储输入值，使用名为 sum 的变量（第 5 行）来存储这些值的和。只要读入一个值并且它不为 0，那就把它赋给 data 并把 data 添加到 sum 里（第 7 行）。

程序清单 5-5 Sentinelvalue.py

```
1  data = eval(input("Enter an integer (the input ends " +
2      "if it is 0): "))
3
4  # Keep reading data until the input is 0
5  sum = 0
6  while data != 0:
7      sum += data
8
9      data = eval(input("Enter an integer (the input ends " +
10         "if it is 0): "))
11
12 print("The sum is", sum)
```

```
Enter an integer (the input ends if it is 0): 2 Enter
Enter an integer (the input ends if it is 0): 3 Enter
Enter an integer (the input ends if it is 0): 4 Enter
Enter an integer (the input ends if it is 0): 0 Enter
The sum is 9
```

	line#	data	sum	output
	1	2		
	5		0	
iteration 1	7		2	
	9	3		
iteration 2	7		5	
	9	4		
iteration 3	7		9	
	9	0		
	12			The sum is 9

如果 data 不为 0，就把它添加到 sum 里（第 7 行），然后读取下一个输入的条目 data（第 9 ～ 10 行）。如果 data 为 0，就不再执行循环体，并且终止 while 循环。输入值 0 就是这个循环的哨兵值。注意：如果第一个输入的读取值为 0，那么循环体将永远都不会执行，这样得到的和为 0。

警告：在循环控制里不要使用浮点值来比较相等。因为这些值都是近似的，所以它们会导致不精确的计数值。这个例子中 data 使用的是 int 值。考虑下面的代码计算 1+0.9+0.8+…+0.1。

```
item = 1
sum = 0

while item != 0:  # No guarantee item will be 0
```

```
    sum += item
    item -= 0.1

print(sum)
```

变量 item 的值从 1 开始，每执行一次循环体就减少 0.1。循环应该在 item 变为 0 时终止。然而，这个程序并不能保证最终 item 会变为 0，因为浮点值运算是近似的。这个循环表面看起来正确，但是它实际上是一个无限循环。

5.2.6 输入输出重定向

程序清单 5-5 中，如果你要输入很多数据，那么从键盘输入所有的数将是一件非常麻烦的事。你可以把数据存储在一个文本文件（例如：名为 input.txt）里，并使用下面的命令来运行这个程序：

```
python SentinelValue.py < input.txt
```

这个命令被称作输入重定向。用户不再需要在运行时从键盘敲入数据，而是程序从文件 input.txt 中获取输入数据。假设这个文件包含下面的数字，每行一个：

```
2
3
4
0
```

程序运行结果 sum 等于 9。

同样地，输出重定向是把程序运行结果输出到一个文件里而不是输出到屏幕上。输出重定向的命令为：

```
python Script.py > output.txt
```

同一条命令里可以同时使用输入重定向与输出重定向。例如，下面这个命令从 input.txt 中获取输入数据，然后把输出数据发送到文件 output.txt 中。

```
python SentinelValue.py < input.txt > output.txt
```

运行这个程序然后去看看 output.txt 文件中显示的内容是什么。

检查点

5.1　分析下面的代码。在 PointA、PointB 和 PointC 处 count<100 总为 True，总为 False，还是有时 True 有时 False？

```
count = 0
while count < 100:
    # Point A
    print("Programming is fun!")
    count += 1
    # Point B

# Point C
```

5.2　如果把程序清单 5-3 中第 8 行的 guess 初始化为 0，错在哪里？

5.3　下面的循环体被重复了多少次？每次循环的输出结果是什么？

```
i = 1
while i < 10:
    if i % 2 == 0:
        print(i)
```

a)

```
i = 1
while i < 10:
    if i % 2 == 0:
        print(i)
    i += 1
```

b)

```
i = 1
while i < 10:
    if i % 2 == 0:
        print(i)
i += 1
```

c)

5.4　指出下面代码的错误：

```
count = 0
while count < 100:
    print(count)
```

a)

```
count = 0
while count < 100:
    print(count)
    count -= 1
```

b)

```
count = 0
while count < 100:
count += 1
```

c)

5.5　假设输入值为“2 3 4 5 0”(每行一个数)。下面代码的输出结果是什么？

```
number = eval(input("Enter an integer: "))
max = number

while number != 0:
    number = eval(input("Enter an integer: "))
    if number > max:
        max = number

print("max is", max)
print("number", number)
```

5.3　for 循环

关键点：Python 的 for 循环通过一个序列中的每个值来进行迭代。

我们经常是知道循环体需要被执行多少次，所以，使用一个控制变量统计执行的次数。这种类型的循环被称作计数器控制的循环。大体上，这个循环可以编写成如下形式：

```
i = initialValue  # Initialize loop-control variable
while i < endValue:
    # Loop body
    ...
    i += 1  # Adjust loop-control variable
```

for 循环可以用来简化上面的循环：

```
for i in range(initialValue, endValue):
    # Loop body
```

通常，for 循环的语法是：

```
for var in sequence:
    # Loop body
```

sequence 里保存 data 的多个条目，且这些条目按照一个接一个地方式存储。在后面的内容里，本书还将介绍字符串、列表和数组。Python 里它们都是序列类型的对象。变量 var 表示这个序列里每个连续值，针对每个值，循环体内的语句都执行一次循环体。

Range(a,b) 函数返回一系列连续整数 a、a+1、…、b−2 和 b−1。例如：

```
>>> for v in range(4, 8):
...     print(v)
...
4
5
6
7
>>>
```

range 函数有两种或更多形式。你也可以使用 range（a）也可以使用 range（a,b,k）。

range(a) 与 range(0,a) 功能一样。在 range(a,b,k) 中 k 被用作步长值。序列中的第一个数是 a。序列中每一个连续数都会被增加一个步长值 k。b 是界限值。序列中的最后一个数必须小于 b。例如：

```
>>> for v in range(3, 9, 2):
...     print(v)
...
3
5
7
>>>
```

range(3,9,2) 中的步长值为 2，界限值为 9。因此，这个序列就是 3,5,7。

如果函数 range(a,b,k) 中的 k 为负数，则可以反向计数。在这种情况下，序列仍为 a、a+k、a+2k 等等但 k 为负数。最后一个数必须大于 b。例如：

```
>>> for v in range(5, 1, -1):
...     print(v)
...
5
4
3
2
>>>
```

注意：range 函数中的数必须为整数。例如：range(1.5,8.5)、range(8.5) 或 range(1.5,8.5,1) 都是错误的。

检查点

5.6 假设输入是 2 3 4 5 0（每行一个数）。那么下面代码的输出是什么？

```
number = 0
sum = 0

for count in range(5):
    number = eval(input("Enter an integer: "))
    sum += number

print("sum is", sum)
print("count is", count)
```

5.7 你能把任何一个 for 循环转换为 while 循环吗？列出使用 for 循环的优点。

5.8 将下面的 for 循环语句转换成 while 循环。

```
sum = 0
for i in range(1001):
    sum = sum + i
```

5.9 你能将任意 while 循环转换成 for 循环吗？将下面这个 while 循环转换成 for 循环。

```
i = 1
sum = 0

while sum < 10000:
    sum = sum + i
    i += 1
```

5.10 统计下面循环中的迭代次数：

```
count = 0
while count < n:
    count += 1
```

a)

```
for count in range(n):
    print(count)
```

b)

```
count = 5
while count < n:
    count += 1
```

c)

```
count = 5
while count < n:
    count = count + 3
```

d)

5.4　嵌套循环

关键点：一个循环可以嵌套到另一个循环里。

嵌套循环是由一个外层循环和一个或多个内层循环构成。每次重复外层循环时，内层循环都被重新进入并且重新开始。

程序清单 5-6 是一个使用嵌套 for 循环来显示乘法口诀表的程序。

程序清单 5-6　MuliplicationTable.py

```
1  print("          Multiplication Table")
2  # Display the number title
3  print("   ", end = '')
4  for j in range(1, 10):
5      print("  ", j, end = '')
6  print()  # Jump to the new line
7  print("----------------------------------------")
8
9  # Display table body
10 for i in range(1, 10):
11     print(i, "|", end = '')
12     for j in range(1, 10):
13         # Display the product and align properly
14         print(format(i * j, "4d"), end = '')
15     print()  # Jump to the new line
```

```
          Multiplication Table
     1   2   3   4   5   6   7   8   9
----------------------------------------
1 |   1   2   3   4   5   6   7   8   9
2 |   2   4   6   8  10  12  14  16  18
3 |   3   6   9  12  15  18  21  24  27
4 |   4   8  12  16  20  24  28  32  36
5 |   5  10  15  20  25  30  35  40  45
6 |   6  12  18  24  30  36  42  48  54
7 |   7  14  21  28  35  42  49  56  63
8 |   8  16  24  32  40  48  56  64  72
9 |   9  18  27  36  45  54  63  72  81
```

程序在输出的第一行显示标题（第 1 行）。第一个 for 循环（第 4 ～ 5）在第二行显示从数字 1 到 9 这九个数字。第三行显示了一行破折号（-）(第 7 行)。

下一个循环（第 10 ～ 15 行）是一个外层循环使用控制变量 i 而内层循环使用控制变量 j 的嵌套 for 循环。对于每个 i，都有 j 是 1、2、3、…、9，在内层循环中的一行上显示乘积 i*j。

为了正确对齐这些数字，程序使用 format(i*j," 4d") 规范 i*j 的格式（第 14 行）。“4d”指定输出的十进制整数宽度为 4。

通常，print 函数会自动跳转到下一行。但是调用 print(item,end='') 函数（第 3、5、11

和 14 行）输出每一项但不会自动跳转到下一行。注意：第 3.3.5 节详细介绍带 end 参数的 print 函数。

注意：注意嵌套循环可能会花费很长的时间来运行。分三层来考虑下面嵌套的循环：

```
for i in range(1000):
    for j in range(1000):
        for k in range(1000):
            Perform an action
```

动作被执行了 1 000 000 000 次。如果完成这个动作用时 1 毫秒，那么运行这个循环的总时间将会超过 277 小时。

检查点

5.11 显示下面这个程序的输出（提示：绘制一个表格，列出所有的变量来跟踪这个程序）。

```
for i in range(1, 5):
    j = 0
    while j < i:
        print(j, end = " ")
        j += 1
```

a)

```
i = 0
while i < 5:
    for j in range(i, 1, -1):
        print(j, end = " ")
    print("****")
    i += 1
```

b)

```
i = 5
while i >= 1:
    num = 1
    for j in range(1, i + 1):
        print(num, end = "xxx")
        num *= 2
    print()
    i -= 1
```

c)

```
i = 1
while i <= 5:
    num = 1
    for j in range(1, i + 1):
        print(num, end = "G")
        num += 2
    print()
    i += 1
```

d)

5.5 最小化数值错误

关键点：在循环继续条件中使用浮点数可能会导致数值错误。

数值错误涉及浮点数是必然的。这节提供一个如何最小化这种错误的例子。

程序清单 5-7 中的程序是对一个从 0.01 开始到 1.0 的数列中的数求和。这个数列里的数每次递增 0.01，如下所示：0.01+0.02+0.03+…。

程序清单 5-7 TestSum.py

```
1  # Initialize sum
2  sum = 0
3
4  # Add 0.01, 0.02, ..., 0.99, 1 to sum
5  i = 0.01
6  while i <= 1.0:
7      sum += i
8      i = i + 0.01
9
10 # Display result
11 print("The sum is", sum)
```

```
The sum is 49.5
```

最后结果显示 49.5，但是实际上正确的结果应该为 50.5。哪个地方错了？在循环的每次

迭代中变量 i 都递增 0.01。当循环结束时，i 的值稍稍大于 1（而不是真正为 1）。这导致最后一个 i 值并没有被加到 sum 上。最基本的问题是浮点数被近似表示了。

为了改正这个错误，可以使用一个整数计数器来确保所有的数字都被加到了 sum 上。下面是一个新的循环：

```
# Initialize sum
sum = 0

# Add 0.01, 0.02, ..., 0.99, 1 to sum
count = 0
i = 0.01
while count < 100:
    sum += i
    i = i + 0.01
    count += 1  # Increase count

# Display result
print("The sum is", sum)
```

或者，如下所示使用一个 for 循环：

```
# Initialize sum
sum = 0

# Add 0.01, 0.02, ..., 0.99, 1 to sum
i = 0.01
for count in range(100):
    sum += i
    i = i + 0.01

# Display result
print("The sum is", sum)
```

5.6　实例研究

关键点：循环是程序设计的基础。编写循环是学习程序设计的基本能力。

如果你可以使用循环编写程序，那么你就知道如何写程序了。所以，本节给出三个用循环来解决问题的经典例子。

5.6.1　问题：找出最大公约数

两个整数 4 和 2 的最大公约数（GCD）是 2。整数 16 和 24 的最大公约数是 8。怎样找出最大公约数呢？假设输入的两个整数是 $n1$ 和 $n2$。你知道数字 1 是它们的公约数，但它并不是最大公约数。所以，你要检测 k（k=2、3、4、…）是否为 $n1$ 和 $n2$ 的公约数，直到 k 大于 $n1$ 或 $n2$。把公约数存储在一个名为 gcd 的变量中。初始状态时，gcd 的值为 1。每找到一个新的公约数就把它赋给 gcd。当你检测完从 2 到 $n1$ 或从 2 到 $n2$ 的所有可能公约数后，存储在 gcd 中的值就是最大公约数。这个想法可以被翻译成下面的循环：

```
gcd = 1 # Initial gcd is 1
int k = 2 # Possible gcd
while k <= n1 and k <= n2:
    if n1 % k == 0 and n2 % k == 0:
        gcd = k
    k += 1 # Next possible gcd

# After the loop, gcd is the greatest common divisor for n1 and n2
```

程序清单 5-8 给出一个提示用户输入两个正整数，然后求出其最大公约数的程序。

程序清单 5-8 GreatestCommonDivisor.py

```
1  # Prompt the user to enter two integers
2  n1 = eval(input("Enter first integer: "))
3  n2 = eval(input("Enter second integer: "))
4
5  gcd = 1
6  k = 2
7  while k <= n1 and k <= n2:
8      if n1 % k == 0 and n2 % k == 0:
9          gcd = k
10     k += 1
11
12 print("The greatest common divisor for",
13     n1, "and", n2, "is", gcd)
```

```
Enter first integer: 125 Enter
Enter second integer: 2525 Enter
The greatest common divisor for 125 and 2525 is 25
```

你要怎么样编写这个程序呢？你会立即编写代码吗？不，在编代码前先思考是非常重要的。思考可以让你在考虑如何编写代码前产生这个问题的逻辑解决方案。一旦你有一个逻辑上的解决方案，那就把这个解决方案翻译成程序。

一个问题经常有很多种解决方案。这个最大公约数的问题可以用很多方法解决。在本章结尾的编程题 5.16 中建议另一种解决方法。最有效的解决方案就是经典的欧几里得算法。更多信息请参见 www.cut-the-knot.org/blue/euclid.shtml。

检查点

5.12 如果你知道一个数 n1 的公约数不可能大于 n1/2，你就可以试图使用下面的循环来改善你的程序：

```
k = 2
while k <= n1 / 2 and k <= n2 / 2:
    if n1 % k == 0 and n2 % k == 0:
        gcd = k
    k += 1
```

这个程序是错误的。你能找出原因吗？

5.6.2 问题：预测未来学费

假设今年你所在大学的学费为 10 000 美元并且它以每年 7% 的速度增长。那么，多少年之后学费会翻倍？

在你尝试编写程序之前，首先考虑如何手动解决这个问题。第二年的学费是第一年的学费 *1.07。因此，以后每年的学费都是其前一年的学费 *1.07。所以，每年的学费可以被计算为如下所示：

```
year = 0  # Year 0
tuition = 10000

year += 1 # Year 1
tuition = tuition * 1.07

year += 1 # Year 2
tuition = tuition * 1.07
```

```
year += 1 # Year 3
tuition = tuition * 1.07
...
```

持续计算新一年的学费 tuition 直到某一年的学费至少是 20 000 美元为止。这样，你就能知道经过多少年之后学费会翻倍。现在，你可以将这个逻辑翻译成下面的循环：

```
year = 0  # Year 0
tuition = 10000
while tuition < 20000:
    year += 1
    tuition = tuition * 1.07
```

完整的程序如程序清单 5-9 所示。

程序清单 5-9 FutureTuition.py

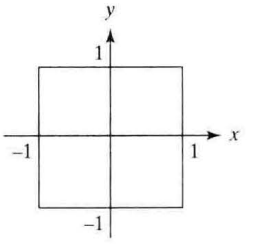
```
1  year = 0  # Year 0
2  tuition = 10000  # Year 1
3
4  while tuition < 20000:
5      year += 1
6      tuition = tuition * 1.07
7
8  print("Tuition will be doubled in", year, "years")
9  print("Tuition will be $" + format(tuition, ".2f"),
10         "in", year, "years")
```

```
Tuition will be doubled in 11 years
Tuition will be $21048.52 in 11 years
```

while 循环（第 4 ～ 6 行）用来计算新一年的学费。当学费 tuition 大于等于 20 000 时循环终止。

5.6.3　问题：蒙特卡罗模拟

蒙特卡罗模拟使用随机数和概率来解决问题。它在计算机数学、物理、化学和经济方面都有非常广泛的应用。现在，我们看一个使用蒙特卡罗模拟来估计 π 的例子。



首先，绘制一个带外接正方形的圆。

假设这个圆的半径为 1。因此，这个圆的面积就是 π，而矩形的面积为 4。在这个正方形内随机产生一个点。这个点落在圆内的概率为 circleArea/squareArea=π/4。

编写一个程序，在正方形内随机产生 1 000 000 个点，使用 numberOfHits 表示落入圆内点的个数。所以，numberOfHits 大约就是 1 000 000*（π/4）。π 就可以被近似表示为 4*numberOfHits/1 000 000。完整的程序如程序清单 5-10 所示。

程序清单 5-10 MoteCarloSimulation.py

```
1  import random
2
3  NUMBER_OF_TRIALS = 1000000  # Constant
4  numberOfHits = 0
5
6  for i in range(NUMBER_OF_TRIALS):
7      x = random.random() * 2 - 1
```

```
 8       y = random.random() * 2 - 1
 9
10       if x * x + y * y <= 1:
11           numberOfHits += 1
12
13   pi = 4 * numberOfHits / NUMBER_OF_TRIALS
14
15   print("PI is", pi)
```

```
PI is 3.14124
```

这个程序的第 7 ～ 8 行在正方形内重复产生随机点（x,y）。

```
x = random.random() * 2 - 1
y = random.random() * 2 - 1
```

函数 random（）返回一个随机的浮点数 r，而且 0<=r<1.0。

如果 $x^2+y^2 \leq 1$，那么这个点就在圆内，并且 numberOfHits 递增 1。在第 13 行 π 被近似为 4*numberOfHits/NUMBER_OF_TRIALS。

5.7 关键字 break 和 continue

关键点：关键字 break 和 continue 提供了另一种控制循环的方式。

教学建议：两个关键字：break 和 continue 都可以为循环语句提供额外的控制。在某些情况下，使用 break 和 continue 可以简化程序设计。然而，如果过度使用或者使用不恰当则会导致程序很难理解和调试。（读者注意：跳过本节也不影响对本书其余知识的理解。）

我们可以在循环中使用关键字 break 来立即终止循环。程序清单 5-11 给出一个程序，它演示在循环中使用 break 的效果。

程序清单 5-11 TestBreak.py

```
 1   sum = 0
 2   number = 0
 3
 4   while number < 20:
 5       number += 1
 6       sum += number
 7       if sum >= 100:
 8           break
 9
10   print("The number is", number)
11   print("The sum is", sum)
```

```
The number is 14
The sum is 105
```

这个程序将从 1 到 20 的整数依次加到 sum 上直到 sum 大于或等于 100。如果没有第 7 到 8 行，这个程序就会计算从 1 到 20 的所有数的和。但是有了第 7 到 8 行，循环会在 sum 大于或等于 100 时终止。没有第 7 到 8 行，输出将会是：

```
The number is 20
The sum is 210
```

我们也可以在循环中使用关键字 continue。当遇到这个关键字时，它会终止当前的迭代并控制程序转到循环体的最后。换句话说，continue 退出一次迭代而 break 退出整个循环。

程序清单 5-12 中的程序显示在循环中使用 continue 的效果。

程序清单 5-12 TestContinue.py

```
1  sum = 0
2  number = 0
3
4  while number < 20:
5      number += 1
6      if number == 10 or number == 11:
7          continue
8      sum += number
9
10 print("The sum is", sum)
```

```
The sum is 189
```

这个程序把除了 10 和 11 的所有从 1 到 20 的数全都加到 sum 里。当 number 变成 10 或 11 时，就会执行 continue 语句。continue 语句终止当次迭代，这样，就不执行循环体中剩余的语句；所以，10 和 11 就不会被加到 sum 上。

没有第 6 行和第 7 行，输出结果将会如下所示。

```
The sum is 210
```

在这种情况下，所有的数字都会被加到 sum 上，即使 number 为 10 或 11 也都被加到 sum 上。所以，结果是 210。

所有的循环都可以不用 break 和 continue（参见编程题 5.15）。一般地，如果要简化程序或使程序更容易读懂，那么使用 break 和 continue 就是合适的。

假设要编写一个程序，它要找出一个整数 n(假设 n>=2) 除了 1 以外最小的因子。可以用 break 语句编写一个简单易懂的程序，如下所示。

```
n = eval(input("Enter an integer >= 2: "))
factor = 2
while factor <= n:
    if n % factor == 0:
        break
    factor += 1
print("The smallest factor other than 1 for", n, "is", factor)
```

可以不使用 break 如下改写代码。

```
n = eval(input("Enter an integer >= 2: "))
found = False
factor = 2
while factor <= n and not found:
    if n % factor == 0:
        found = True
    else:
        factor += 1
print("The smallest factor other than 1 for", n, "is", factor)
```

很明显，在这个例子中，break 语句可以使这个程序变得更简单且更容易理解。但是，你在使用 break 和 continue 时应该小心。过多的 break 和 continue 会使整个程序有太多的退出点从而导致程序很难读懂。

注意：一些程序设计语言有 goto 语句。goto 语句会将控制转到程序中的任意一条语句然后执行它。这会使你的程序很容易出错。Python 里的 break 和 continue 语句与 goto 语

句是不同的。它们只能在循环里起作用。break 语句跳出整个循环，而 continue 语句只是退出循环的当前迭代。

检查点

5.13 关键字 break 的作用是什么？关键字 continue 的作用是什么？下面这个程序会终止吗？如果会，请给出程序运行的结果。

```
balance = 1000
while True:
    if balance < 9:
        break
    balance = balance - 9

print("Balance is", balance)
```

a)

```
balance = 1000
while True:
    if balance < 9:
        continue
    balance = balance - 9

print("Balance is", balance)
```

b)

5.14 左边的 for 循环被转换成右边的 while 循环。哪里出了错误？请改正。

```
for i in range(4):
    if i % 3 == 0:
        continue
    sum += i
```

Converted

Wrong conversion

```
i = 0
while i < 4:
    if i % 3 == 0:
        continue
    sum += i
    i += 1
```

5.15 不使用 break 和 continue 语句改写程序清单 5-11 和 5-12 的程序 TestBreak 和 TestContinue。

5.16 在下面 a 的循环里如果 break 语句执行完后，哪些语句就被终止？显示输出结果。在下面 b 的循环里如果 continue 语句执行完后，哪些语句就被终止？显示输出结果。

```
for i in range(1, 4):
    for j in range(1, 4):
        if i * j > 2:
            break

        print(i * j)

    print(i)
```

a)

```
for i in range(1, 4):
    for j in range(1, 4):
        if i * j > 2:
            continue

        print(i * j)

    print(i)
```

b)

5.8 实例研究：显示素数

关键点：本节涉及的程序是实现在五行里显示前 50 个素数，每行 10 个。

一个大于 1 的整数如果只能被正整数 1 和它本身整除则它就是素数。例如：2、3、5 和 7 都是素数，而 4、6、8、9 则不是。

这个问题可以被分成以下几个任务。

- 决定一个特定的数是否为素数。
- 对于 number=2、3、4、5、6、…，测试这个数字是否为素数。
- 统计所有素数的个数。
- 显示每个素数，每行显示 10 个。

很明显，你要编写一个循环然后重复测试来判断输入的数是否为素数。如果是，count

加 1。count 初值为 1。当它到达 50 时，终止循环。

下面是这个问题的算法。

```
Set the number of prime numbers to be displayed as
    a constant NUMBER_OF_PRIMES
Use count to track the number of prime numbers and
    set an initial count to 0
Set an initial number to 2

while count < NUMBER_OF_PRIMES:
    Test if number is prime

    if number is prime:
        Display the prime number and increase count

    Increment number by 1
```

为了测试这个数是否为素数，那么检测这个数能否被 2、3、4、…，直到 number/2 整除，如果能被整除，那么这个数就不是素数。它的算法可以被描述成如下所示：

```
Use a Boolean variable isPrime to denote whether
    the number is prime; Set isPrime to True initially

for divisor in range(2, number / 2 + 1):
    if number % divisor == 0:
        Set isPrime to False
        Exit the loop
```

完整的程序在程序清单 5-13 中给出。

程序清单 5-13 PrimeNumber.py

```
NUMBER_OF_PRIMES = 50  # Number of primes to display
NUMBER_OF_PRIMES_PER_LINE = 10 # Display 10 per line
count = 0  # Count the number of prime numbers
number = 2  # A number to be tested for primeness

print("The first 50 prime numbers are")

# Repeatedly find prime numbers
while count < NUMBER_OF_PRIMES:
    # Assume the number is prime
    isPrime = True  # Is the current number prime?

    # Test if number is prime
    divisor = 2
    while divisor <= number / 2:
        if number % divisor == 0:
            # If true, the number is not prime
            isPrime = False  # Set isPrime to false
            break # Exit the for loop
        divisor += 1

    # Display the prime number and increase the count
    if isPrime:
        count += 1  # Increase the count

        print(format(number, "5d"), end = '')
        if count % NUMBER_OF_PRIMES_PER_LINE == 0:
            # Display the number and advance to the new line
            print()  # Jump to the new line

    # Check if the next number is prime
    number += 1
```

```
The first 50 prime numbers are

   2    3    5    7   11   13   17   19   23   29
  31   37   41   43   47   53   59   61   67   71
  73   79   83   89   97  101  103  107  109  113
 127  131  137  139  149  151  157  163  167  173
 179  181  191  193  197  199  211  223  227  229
```

这对于初学者来说是一个复杂的例子。开发一个程序来解决某个问题（或很多其他问题）的关键是把它分成几个子问题，然后依次开发每个子问题的解决方案。不要试图一开始就开发一个完整的解决方案。而是，先编写代码判断给定的那个数是否为素数，然后再用一个循环来扩展它判断其他数字是否为素数。

为了确定一个数是否为素数，检查它能否被从 2 到 number/2 的数（包括 2 和 number/2）整除，如果能，那它就不是一个素数；否则它就是一个素数。如果是素数，就显示它。如果 count 可以被 10 整除，就进入下一行。当 count 达到 50 时，程序终止。

当发现输入的数字不是素数时，程序在第 19 行使用 break 语句退出 for 循环。你也可以重新编写一个没有使用 break 语句的循环（第 15 ～ 20 行），如下所示。

```
while divisor <= number / 2 and isPrime:
    if number % divisor == 0:
        # If True, the number is not prime
        isPrime = False # Set isPrime to False
    divisor += 1
```

但是，在这个例子中使用 break 语句能够使整个程序变得更简单和易读懂。

5.9 实例研究：随意行走

关键点：你可以使用 Turtle 图形来模拟随意行走。

本节中，我们要编写一个 Turtle 程序来模仿在格子里随意行走（例如：就像在花园里散步转过去看一朵花），从中心位置开始，然后在边缘处的某点停下来，如图 5-2 所示。程序清单 5.14 给出这个程序。

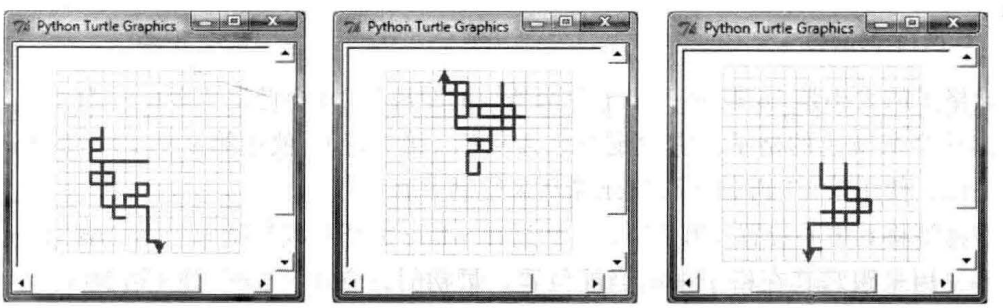


图 5-2 程序模拟格子里的随意行走

程序清单 5-14 RandomWalk.py

```
1  import turtle
2  from random import randint
3
4  turtle.speed(1)  # Set turtle speed to slowest
5
6  # Draw 16-by-16 lattice
7  turtle.color("gray")  # Color for lattice
```

```
x = -80
for y in range(-80, 80 + 1, 10):
    turtle.penup()
    turtle.goto(x, y)  # Draw a horizontal line
    turtle.pendown()
    turtle.forward(160)

y = 80
turtle.right(90)
for x in range(-80, 80 + 1, 10):
    turtle.penup()
    turtle.goto(x, y)  # Draw a vertical line
    turtle.pendown()
    turtle.forward(160)

turtle.pensize(3)
turtle.color("red")

turtle.penup()
turtle.goto(0, 0)  # Go to the center
turtle.pendown()

x = y = 0 # Current pen location at the center of lattice
while abs(x) < 80 and abs(y) < 80:
    r = randint(0, 3)
    if r == 0:
        x += 10 # Walk right
        turtle.setheading(0)
        turtle.forward(10)
    elif r == 1:
        y -= 10 # Walk down
        turtle.setheading(270)
        turtle.forward(10)
    elif r == 2:
        x -= 10 # Walk left
        turtle.setheading(180)
        turtle.forward(10)
    elif r == 3:
        y += 10 # Walk up
        turtle.setheading(90)
        turtle.forward(10)

turtle.done()
```

假设格子的大小是 16 乘 16，格子中两条线的距离为 10 个像素（第 6 ～ 21 行）。程序首先绘制出这个灰颜色的格子。它设定颜色为灰色（第 7 行），使用第 9 到 13 行的 for 循环来绘制水平线，使用第 17 到 21 行的 for 循环来绘制垂直线。

程序将笔移到中心位置（第 27 行），然后在 while 循环中模拟随意行走（第 31 ～ 48 行）。变量 x 和 y 用来跟踪其在格子中的当前位置。最初时，它在（0,0）处（第 30 行）。在第 32 行生成一个 0 到 3 之间的随机数。每个数字代表了一个方向：东、南、西和北。考虑下面四种情况：

- 如果向东走一步，x 增加 10（第 34 行）然后笔向右移动（第 35 ～ 36 行）。
- 如果向南走一步，y 减少 10（第 38 行）然后笔向下移动（第 39 ～ 40 行）。
- 如果向西走一步，x 减少 10（第 42 行）然后笔向左移动（第 43 ～ 44 行）。
- 如果向北走一步，y 增加 10（第 46 行）然后笔向上移动（第 47 ～ 48 行）。

当 abs(x) 或 abs(y) 为 80 的时候停止行走。

一种更有趣的行走被称作自我回避行走。它是指在格子里随意行走时不会两次移向同一个点。在本书的后面将会学习如何编写一个程序来模拟自我回避行走。

关键术语

break keyword（break 关键字）
condition-controlled loop（条件控制循环）
continue keyword（continue 关键字）
count-controlled loop（计数器控制的循环）
infinite loop（无限循环）
input redirection（输入重定向）
iteration（迭代）
loop（循环）
loop body（循环体）
loop-continuationcondition（循环继续条件）
nested loop（嵌套循环）
off-by-one error（偏离 1 的误差）
output redirection（输出重定向）
sentinel value（哨兵值）

本章总结

1. 两种类型的循环语句：while 循环和 for 循环。
2. 循环中需要被重复执行的语句被称为循环体。
3. 循环体的一次执行被叫做循环的一次迭代。
4. 一个无限循环是指循环体的语句无限次被执行。
5. 在设计一个循环时，你不仅仅要考虑循环控制结构还要考虑循环体。
6. while 循环首先检查循环继续条件。如果条件为真，则执行循环体；否则，循环终止。
7. 哨兵值是一个特殊的值，它表明输入值的结束。
8. for 循环是计数器控制的循环，循环体执行可预见次数遍。
9. break 和 continue 两个关键字都可以被用在循环中。
10. break 关键字立即结束包含这个 break 的最内层循环。
11. continue 只终止当前迭代。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

教学建议：对每一个问题，都要多读几遍直到你读懂了这个问题。在编写代码之前先想想怎么解决这个问题。将你的逻辑思路转换成代码。

一个问题可以有多种解法。你应该设法想出多种解决方案。

第 5.2 ～ 5.7 节

*5.1 （统计正数和负数的个数然后计算这些数的平均值）编写一个程序来读入不指定个数的整数，然后决定已经读取的整数中有多少个正数和多少个负数并计算这些输入值（不统计 0）的总和，最终得出它们的平均值。这个程序以输入值 0 来结束。使用浮点数显示这个平均值。下面是一个简单的示例运行。

```
Enter an integer, the input ends if it is 0: 1 ↵Enter
Enter an integer, the input ends if it is 0: 2 ↵Enter
Enter an integer, the input ends if it is 0: -1 ↵Enter
Enter an integer, the input ends if it is 0: 3 ↵Enter
```

```
Enter an integer, the input ends if it is 0: 0 Enter
The number of positives is 3
The number of negatives is 1
The total is 5
The average is 1.25


Enter an integer, the input ends if it is 0: 0 Enter
You didn't enter any number
```

5.2 （累加）程序清单 5-4 中产生五个随机减法的问题。改写这个程序，产生两个在 1 到 15 之间的随机数的加法问题，显示回答正确的次数和测试所用时间。

5.3 （公斤转换成磅）编写一个程序能显示下面的表格（1 公斤是 2.2 磅）。

公斤	磅
1	2.2
3	6.6
...	
197	433.4
199	437.8

5.4 （英里转换成公里）编写一个程序能显示下面的表格（注意：1 英里是 1.609 公里）。

英里	公里
1	1.609
2	3.218
...	
9	14.481
10	16.090

*5.5 （公斤转换成磅，磅转换成公斤）编写一个程序能显示下面两个相邻的表格（注意：1 公斤等于 2.2 磅）。

公斤	磅	公斤	磅
1	2.2	9.09	20
3	6.6	11.36	25
...		...	
197	433.4	231.82	510
199	437.8	234.09	515

*5.6 （将英里转换成公里，公里转换成英里）编写一个程序能显示下面两个相邻的表格（注意：1 英里等于 1.609 公里）。

英里	公斤	英里	公斤
1	1.609	12.430	20
2	3.218	15.538	25
...		...	
9	14.481	37.290	60
10	16.090	40.398	65

5.7 （使用三角函数）打印下面的表格显示从 0 度到 360 度每隔 10 度的角度的 sin 值和 cos 值。四舍五入这些值，保持小数点后四位。

度	Sin	Cos
0	0.0000	1.0000
10	0.1736	0.9848
...		
350	−0.1736	0.9848
360	0.0000	1.0000

5.8 （使用 math.sqrt 函数）使用 math 模块中的 sqrt 函数来编写程序输出下面的表格。

Number	Square Root
0	0.0000
2	1.4142
...	
18	4.2426
20	4.4721

**5.9 （财务应用程序：计算未来学费）假设大学今年的学费是 10 000 美元，且以每年 5% 增长。编写程序计算十年之后的学费以及从现在开始到十年后大学四年的总学费。

5.10 （找出最高分）编写程序提示用户输入学生个数以及每个学生的分数，然后显示最高分。假设输入是存储在一个名为 score.txt 的文件，程序从这个文件获取输入。

*5.11 （找出两个最高分）编写程序提示用户输入学生个数以及每个学生的分数，然后显示最高分和次高分的分数。

5.12 （找出可被 5 和 6 同时整除的数）编写程序找出在 100 和 1000 之间所有被 5 和 6 同时整除的数，每行显示 10 个数。这些数被一个空格隔开。

5.13 （找出可被 5 或 6 整除但又不能被它俩同时整除的数）编写程序找出在 100 和 200 之间所有被 5 或 6 整除但又不能被它俩同时整除的数，每行显示 10 个数。这些数被一个空格隔开。

5.14 （找出最小的 n 满足 n^2>12 000）使用 while 循环找出最小的整数 n 满足大于 12 000。

5.15 （找出最大的 n 满足 n^3<12 000）使用 while 循环找出最大的整数 n 满足小于 12 000。

*5.16 （计算最大公约数）对于程序清单 5-8，另外一种找出两个整数 n1 和 n2 的最大公约数的解决方案如下所示：首先找出 n1 和 n2 的最小数 d，然后以 d、d−1、d−2、…、2、1 的顺序依次检测它们是否是 n1 和 n2 的公因子。第一个这样的公约数就是 n1 和 n2 的最大公约数。

第 5.8 节

*5.17 （显示 ASCII 字符表）编写程序显示 ASCII 字符表中从！到～的字符。每行显示十个字符，字符被一个空格隔开。

**5.18 （找出一个整数的所有因子）编写程序读取一个整数，然后显示它所有的最小因子，也称之为素因子。例如：如果输入整数为 120，那么输出应该如下所示。

```
2, 2, 2, 3, 5
```

**5.19 （显示一个金字塔）编写程序提示用户输入一个在 1 到 15 之间的整数，然后显示一个金字塔，示例运行如下所示。

```
Enter the number of lines: 7 Enter
                    1
                  2 1 2
                3 2 1 2 3
              4 3 2 1 2 3 4
            5 4 3 2 1 2 3 4 5
          6 5 4 3 2 1 2 3 4 5 6
        7 6 5 4 3 2 1 2 3 4 5 6 7
```

*5.20 （使用循环显示四种模式）使用嵌套循环在四个独立的程序中显示下面四种模式。

```
模式 A          模式 B          模式 C          模式 D

1               1 2 3 4 5 6         1       1 2 3 4 5 6
1 2             1 2 3 4 5         2 1         1 2 3 4 5
1 2 3           1 2 3 4         3 2 1           1 2 3 4
1 2 3 4         1 2 3         4 3 2 1             1 2 3
1 2 3 4 5       1 2         5 4 3 2 1               1 2
1 2 3 4 5 6     1         6 5 4 3 2 1                 1
```

**5.21 （在金字塔模式中显示数字）编写一个嵌套 for 循环来显示下面的输出。

```
                    1
                  1   2   1
                1   2   4   2   1
              1   2   4   8   4   2   1
            1   2   4   8  16   8   4   2   1
          1   2   4   8  16  32  16   8   4   2   1
        1   2   4   8  16  32  64  32  16   8   4   2   1
1   2   4   8  16  32  64 128  64  32  16   8   4   2   1
```

*5.22 （显示在 2 和 1000 之间的素数）修改程序清单 5-13，显示在 2 和 1000 之间且包括 2 和 1000 的素数，每行显示 8 个素数。

综合题

**5.23 （财务应用程序：比较不同利率的贷款）编写程序让用户输入贷款额以及以年为单位的贷款周期，然后显示利率从 5% 开始，每次增加 1/8，直到 8% 的每月还贷额和总的还款额。下面是一个示例运行。

```
Loan Amount: 10000 Enter
Number of Years: 5 Enter
Interest Rate    Monthly Payment    Total Payment

5.000%           188.71             11322.74
5.125%           189.28             11357.13
5.250%           189.85             11391.59
...
7.875%           202.17             12129.97
8.000%           202.76             12165.83
```

计算每月还款额的公式参见程序清单 2-8。

**5.24 （财务应用程序：贷款摊销时间表）一笔给定的贷款每月支付额包括本金和利息。月利息可以通过计算月利率乘以结余（余下的本金）得到。每个月支付的本金就是每月支付额加上每个月利息。编写程序让用户输入贷款额、年数以及利率，然后显示贷款摊销时间表。下面是一个示例运行。

```
Loan Amount: 10000 Enter
Number of Years: 1 Enter
Annual Interest Rate: 7 Enter

Monthly Payment: 865.26
Total Payment: 10383.21

Payment#      Interest      Principal      Balance
1             58.33         806.93         9193.07
2             53.62         811.64         8381.43
...
11            10.00         855.26          860.27
12             5.01         860.25            0.01
```

注意：最后一次支付之后的结余可能不会为0。如果是这样，那么最后一次支付额应该是正常每月支付额加最后的结余。

提示：编写循环显示一个表格。因为每月支付额对每个月而言都是一样的，所以应该在循环之前计算它。结余的初始值是贷款总数。对循环中的每次迭代，计算利息和本金然后更新结余。这个循环看起来如下所示。

```
for i in range(1, numberOfYears * 12 + 1):
    interest = monthlyInterestRate * balance
    principal = monthlyPayment - interest
    balance = balance - principal
    print(i, "\t\t", interest, "\t\t", principal, "\t\t",
      balance)
```

*5.25 （演示消除错误）当你操作一个非常大的数和一个非常小的数时，就会出现消除错误。大数可能会抵消比较小的数。例如：100000000.0+0.000000001的结果是100000000.0。为了避免消除错误并获取更精确的结果，应该仔细选择计算的顺序。例如：在计算下面数列的过程中，你可以从右向左而不是从左向右计算，这样将会获取更精确的结果。

$$1+\frac{1}{2}+\frac{1}{3}+\ldots+\frac{1}{n}$$

编写程序比较从左到右和从右向左计算上面数列和的结果，其中n=50 000。

*5.26 （数列求和）编写程序对下面的数列求和。

$$\frac{1}{3}+\frac{3}{5}+\frac{5}{7}+\frac{7}{9}+\frac{9}{11}+\frac{11}{13}+\ldots+\frac{95}{97}+\frac{97}{99}$$

**5.27 （计算π）你可以使用下面的数列近似计算π。

$$\pi=4\left(1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\frac{1}{9}-\frac{1}{11}+\ldots+\frac{(-1)^{i+1}}{2i-1}\right)$$

编写程序显示当i=10 000、20 000、…、100 000时π的值。

**5.28 （计算e）你可以使用下面的数列近似计算e。

$$e=1+\frac{1}{1!}+\frac{1}{2!}+\frac{1}{3!}+\frac{1}{4!}+\ldots+\frac{1}{i!}$$

编写程序显示当i=10 000、20 000、…、100 000时e的值（提示：因为$i!=i\times(i-1)\times\cdots\times2\times1$，而$\frac{1}{i!}$是$\frac{\quad}{i(i\quad1)}$。初始化e和item为1，然后不停将一个新的item加到e中。新item是前一个item除以i，其中i=2、3、4、…）。

5.29 （显示闰年）编写程序显示21世纪（从2001年到2100年）里所有的闰年，每行显示10个闰年。这些年被一个空格隔开。

**5.30 （显示每个月的第一天）编写程序提示用户输入年份以及该年的第一天是星期几，然后在控制台上显示该年每个月的第一天是星期几。例如，如果用户为2013年1月1日输入的年份是2013并输入2表示星期二，那你的程序应该显示下面的输出。

```
January 1, 2013 is Tuesday
...
December 1, 2013 is Sunday
```

**5.31 （显示日历）编写程序提示用户输入年份以及该年的第一天是星期几，然后在控制台上显示该年的日历表。例如，如果用户为2005年1月1日输入的年份2005并输入6表示星期六，则程序应该显示该年每个月的日历，如下所示。

January 2005

Sun	Mon	Tue	Wed	Thu	Fri	Sat
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

...

December 2005

Sun	Mon	Tue	Wed	Thu	Fri	Sat
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

*5.32 （财务应用程序：复合值）假设你每月给储蓄账户存储 100 美元，且年利率为 5%。所以，每月利率是 0.05/12=0.004 17。第一个月之后，账户的值变成

```
100 * (1 + 0.00417) = 100.417
```

第二个月之后，账户的值变成

```
(100 + 100.417) * (1 + 0.00417) = 201.252
```

第三个月后，账户的值变成

```
(100 + 201.252) * (1 + 0.00417) = 302.507
```

依此类推。

编写程序提示用户输入一个数额（例如：100），年利率（例如：5）和月份数（例如：6），然后显示给定的月份之后储蓄账户上的数额。

*5.33 （财务应用程序：计算 CD 值）假设你将 10 000 投资买 CD，CD 的年收益率为 5.75%。一个月之后，CD 价值

```
10000 + 10000 * 5.75 / 1200 = 10047.91
```

在两个月之后，CD 价值

```
10047.91 + 10047.91 * 5.75 / 1200 = 10096.06
```

在三个月之后，CD 价值

```
10096.06 + 10096.06 * 5.75 / 1200 = 10144.44
```

依此类推。

编写程序提示用户输入一个数额（例如：10 000）、年收益率（例如：5.75）以及月份数（例如：18），然后显示如下所示的示例运行结果。

```
Enter the initial deposit amount: 10000 ↵Enter
Enter annual percentage yield: 5.75 ↵Enter
Enter maturity period (number of months): 18 ↵Enter

Month  CD Value
1      10047.91
2      10096.06
...
17     10846.56
18     10898.54
```

**5.34 （游戏：彩票）改写程序清单 4-10 来随机产生一个两位数的抽奖数。数字中的两位是不同的（提示：随机产生第一位，然后使用循环继续产生第二位直到它和第一位不同为止）。

**5.35 （完全数）如果一个正整数等于除了它本身之外所有正因子的和，那么这个数被称为完全数。例如，6是第一个完全数，因为6=3+2+1。下一个完全数是28=14+7+4+2+1。小于10 000的完全数有四个。编写程序找出这四个数。

***5.36 （游戏：石头、剪刀、布）编程题4.17给出玩石头、剪刀、布游戏的程序。改写程序让用户不断玩直到用户或计算机中的某一方能够赢得游戏超过两次。

*5.37 （求和）编写程序计算下面的和。

$$\frac{1}{1+\sqrt{2}}+\frac{1}{\sqrt{2}+\sqrt{3}}+\frac{1}{\sqrt{3}+\sqrt{4}}+\ldots+\frac{1}{\sqrt{624}+\sqrt{625}}$$

*5.38 （模拟：时钟倒计时）你可以使用time模块中的time.sleep(seconds)函数让程序暂停指定的秒数。编写程序提示用户输入秒数，每秒显示一条消息，然后当时间到期时终止。下面是一个示例运行。

```
Enter the number of seconds: 3 Enter
2 seconds remaining
1 second remaining
Stopped
```

*5.39 （财务应用程序：找出销售额）你刚刚在百货商店开始销售工作。你的报酬包括基本工资和佣金。基本工资是5000美元。下面的方案给出如何确定佣金率。

销售额	佣金率
$0.01-$5 000	8 percent
$5 000.01-$10 000	10 percent
$10 000.01 and above	12 percent

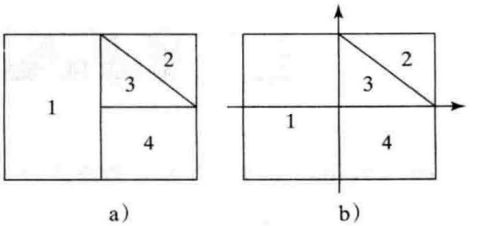
你的目标是每年挣30 000美元。编写程序找出为了挣30 000美元，你的最小销售额。

5.40 （模拟：硬币正反面）编写程序模拟将硬币翻一百万次，然后显示硬币出现正面和反面的次数。

**5.41 （最大数的出现）编写程序读取整数，找出它们中的最大值，然后计算它的出现次数。假设输入以数字0结束。假设你输入的是“3 5 2 5 5 5 0”；程序找出的最大数是5，而5的出现次数是4。（提示：维护两个变量max和count。变量max存储的是当前最大数，而count存储的是它的出现次数。初始状态下，将第一个值赋值给max，将1赋值给count。将max和每个随后的数字进行比较。如果这个数字大于max，就将它赋值给max且将count重置为1。如果这个数等于max，给count自增1。）

```
Enter a number (0: for end of input): 3 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 2 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 5 Enter
Enter a number (0: for end of input): 0 Enter
The largest number is 5
The occurrence count of the largest number is 4
```

**5.42 （蒙特卡罗模拟）一个正方形被分为四个更小的区域，如图a所示。如果你投掷一个飞镖到这个正方形一百万次，这个飞镖落在一个奇数区域里的概率是多少？编写程序模拟这个过程然后显式结果。（提示：将这个正方形的中心放在坐标系统的中心位置，如图b所示。在正方形中随机产生一个点，然后统计这个点落入奇数区域的次数。）



a) b)

*5.43 （数学问题：组合）编写程序显示从 1 到 7 的整数中选取两个数的所有可能组合，同时显示组合的总个数。

```
1 2
1 3
...
...
The total number of all combinations is 21
```

**5.44 （十进制到二进制）编写程序提示用户输入一个十进制整数，然后显示它对应的二进制数。

**5.45 （十进制到十六进制）编写程序提示用户输入一个十进制数，然后显示它对应的十六进制数。

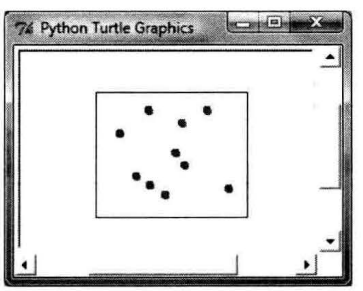
**5.46 （统计方面：计算均值和标准方差）在商业应用程序中，你经常会被问到计算数据的均值和标准方差。均值只是这些数的平均值。标准方差是一个统计值，它告诉你所有各种数据是多么紧密地靠近数据集的均值。例如：一个班学生的平均年龄是多少？他们的年龄的接近程度？如果所有的同学都是相同的年纪，那么方差就是 0。编写一个程序提示用户输入 10 个数字，然后使用下面的公式显示均值和标准方差。

$$mean = \frac{\sum_{i=1}^{n} x_i}{n} = \frac{x_1 + x_2 + \ldots + x_n}{n} \qquad deviation = \sqrt{\frac{\sum_{i=1}^{n} x_i^2 - (\sum_{i=1}^{n} x_i)^2}{n-1}}$$

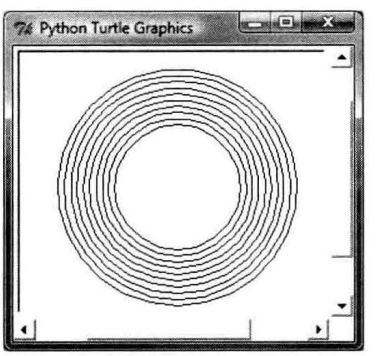
下面是一个示例运行。

```
Enter ten numbers: 1 Enter
2 Enter
3 Enter
5.5 Enter
5.6 Enter
6 Enter
7 Enter
8 Enter
9 Enter
10 Enter
The mean is 5.61
The standard deviation is 2.99794
```

**5.47 （Turtle：绘制随机球）编写程序显示 10 个随机球，它们在一个宽 120 高 100 的矩形里，这个矩形的中心点在 (0,0)，如图 5-3a 所示。



a）绘制 10 个随机球



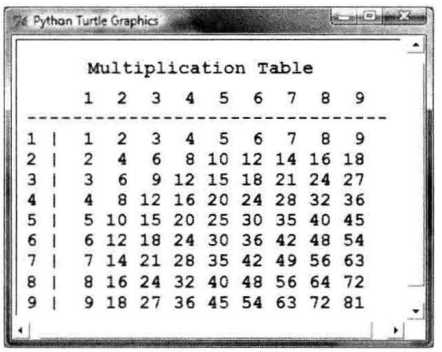
b）绘制 10 个圆

图　5-3

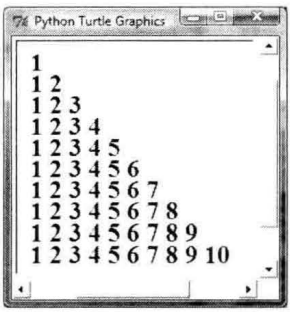
**5.48 （Turtle：绘制圆）编写程序绘制 10 个圆，中心选在 (0,0)，如图 5-3b 所示。

**5.49 （Turtle：显示乘法口诀表）编写程序显示一个乘法口诀表，如图 5-4a 所示。

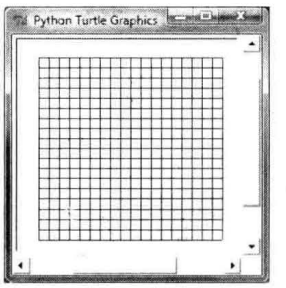
**5.50 （Turtle：显示三角形图案的数字）编写程序显示三角形图案的数字，如图 5-4b 所示。



a）程序显示一个乘法口诀表



b）程序显示三角形图案的数字



c）程序显示 18×18 的格子

图 5-4

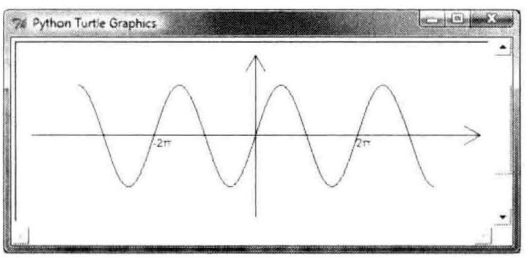
**5.51 （Turtle：显示一个格子）编写程序显示 18×18 的格子，如图 5-4c 所示。

**5.52 （Turtle：绘制 sin 函数）编写程序绘制 sin 函数，如图 5-5a 所示。

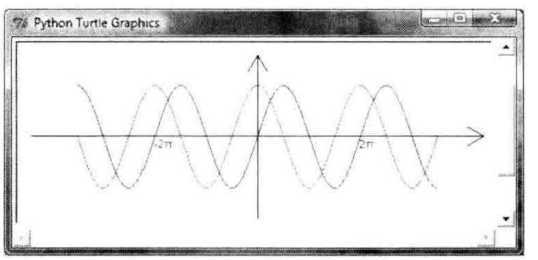
提示：π 的统一码是 \u03c0。为了显示 -2π，使用 turtle.write("-2\u03c0")。对于像 $\sin(x)$ 这样的三角函数，x 是弧度值。使用下面的循环绘制 sin 函数。

```
for x in range(-175, 176):
    turtle.goto(x, 50 * math.sin((x / 100) * 2 * math.pi))
```

在点 (−100,−15) 处显示 -2π，轴的中心位置在 (0,0) 处，而点 (100,−15) 处显示 2π。



a）程序绘制 sin 函数

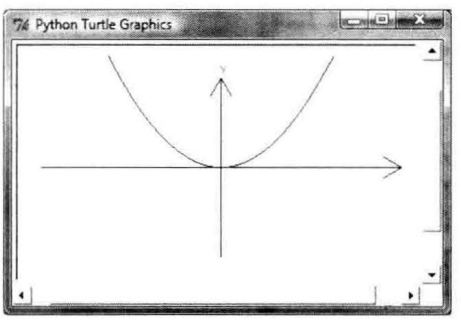


b）程序绘制蓝色的 sin 函数以及红色的 cos 函数

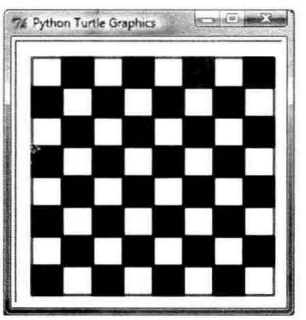
图 5-5

**5.53 （Turtle：绘制 sin 和 cos 函数）编写程序绘制蓝色的 sin 函数和红色的 cos 函数，如图 5-5b 所示。

**5.54 （Turtle：绘制平方函数）编写程序绘制函数 $f(x)=x^2$（参见图 5-6a）。



a）程序绘制函数 $f(x)=x^2$



b）程序绘制棋盘

图 5-6

**5.55 （Turtle：棋盘）编写程序绘制一个棋盘，如图 5-6b 所示。

第 6 章

Introduction to Programming Using Python

函　　数

学习目标

- 定义带形参的函数（第 6.2 节）。
- 用实参来调用函数（第 6.3 节）。
- 区分带返回值和不带返回值的函数（第 6.4 节）。
- 使用位置参数和关键字参数调用函数（第 6.5 节）。
- 通过传参数的引用值来传递参数（第 6.6 节）。
- 开发可重用代码来模块化程序，使程序易读、易调试和易维护（第 6.7 节）。
- 为可重用函数创建模块（第 6.7 ～ 6.8 节）。
- 决定变量的作用域（第 6.9 节）。
- 定义带默认参数的函数（第 6.10 节）。
- 定义一个返回多个值的函数（第 6.11 节）。
- 在软件开发中使用函数抽象的概念（第 6.12 节）。
- 用逐步求精的方法设计和实现函数（第 6.13 节）。
- 使用可重用代码简化程序（第 6.14 节）

6.1　引言

关键点：函数可以用来定义可重用代码、组织和简化代码。

假设你需要对 1 到 10、20 到 37 以及 35 到 49 分别求和。如果你创建一个程序来对这三个集合求和，你的代码可能会像下面这样：

```
sum = 0
for i in range(1, 11):
    sum += i
print("Sum from 1 to 10 is", sum)

sum = 0
for i in range(20, 38):
    sum += i
print("Sum from 20 to 37 is", sum)

sum = 0
for i in range(35, 50):
    sum += i
print("Sum from 35 to 49 is", sum)
```

你可能已经发现这些计算和的代码除了开始和结束的两个数字不同其他都非常相似。一次编写一个通用的代码然后重复使用会不会更好？你可以定义一个函数，这样你就可以创建可重用代码。例如，上面的代码使用函数后可简化成下面的代码：

```
1  def sum(i1, i2):
2      result = 0
```

```
3       for i in range(i1, i2 + 1):
4           result += i
5
6       return result
7
8   def main():
9       print("Sum from 1 to 10 is", sum(1, 10))
10      print("Sum from 20 to 37 is", sum(20, 37))
11      print("Sum from 35 to 49 is", sum(35, 49))
12
13  main() # Call the main function
```

在第 1 到 6 行定义了一个带两个参数 i1 和 i2 的 sum 函数。第 8 到 11 行定义了 main 函数，它通过调用 sum(1,10)、sum(20,37) 和 sum(35,49) 分别计算 1 到 10、20 到 37 以及 35 到 49 的和。

函数是为实现一个操作而集合在一起的语句集。在前面的章节中，我们已经学习了像 eval("numricString") 和 random.randint(a,b) 这样的函数。例如：当调用 random.randint(a,b) 函数时，系统会执行函数里的这些语句，并返回结果。在本章里，我们将学习如何定义和使用函数以及如何应用函数抽象去解决复杂的问题。

6.2　定义一个函数

关键点：函数定义包括函数名称、形参以及函数体。

定义函数的语法如下所示：

```
def functionName(list of parameters)
    # Function body
```

我们来看一个用来找出两个数中哪个比较大的函数。这个函数被命名为 max，它有两个参数：num1 和 num2，函数返回这两个数中较大的那个。图 6-1 解释了这个函数的组件。

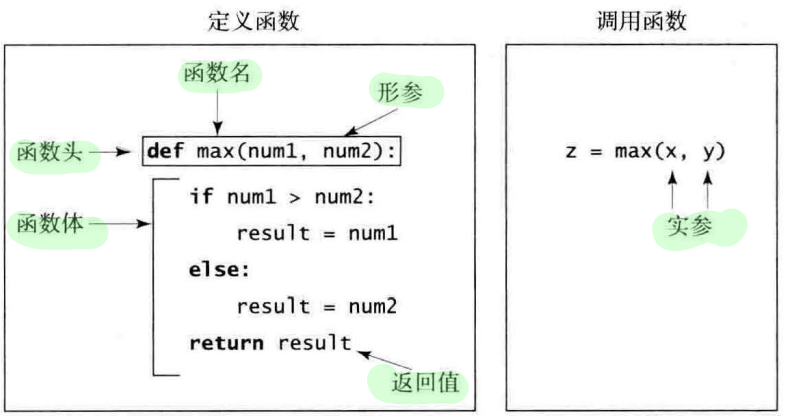


图 6-1　你可以通过参数来定义和调用函数

函数包括函数头和函数体。函数头以一个 def 关键字开始，后面紧接着函数名以及形参并以冒号结束。

函数头中的参数被称为形式参数或简称为形参。参数就像一个占位符：当调用函数时，就将一个值传递给参数。这个值被称为实际参数或实参。参数是可选的；也就是说，函数可以不包含参数。例如：函数 random.random() 就不包含参数。

某些函数有返回值，而一些其他的函数会完成要求的操作而不返回值。如果函数有返回值，则被称为带返回值的函数。

函数体包含一个定义函数做什么的语句集。例如：函数 max 的函数体使用 if 语句来判断哪个数更大然后返回这个数的值。一个带返回值的函数需要一个使用关键字 return 的返回语句来返回一个值。执行 return 语句意味着函数的终止。

6.3　调用一个函数

关键点：调用一个函数来执行函数中的代码。

在函数的定义中，定义函数要做什么。为了使用函数，必须调用它。调用函数的程序被称为调用者。根据函数是否有返回值，调用函数有两种方式。

如果函数带有返回值，对这种函数的调用通常当作一个值处理。例如：

```
larger = max(3, 4)
```

调用 max(3,4) 并将函数的结果赋值给变量 larger。

另外一个把它当作值处理的调用例子是

```
print(max(3, 4))
```

这条语句输出调用函数 max(3,4) 后的返回值。

如果函数没有返回值，那么对函数的调用必须是一条语句。例如：函数 print 没有返回值。下面的调用就是一条语句：

```
print("Programming is fun!")
```

注意：带返回值的函数也可以当作语句被调用。在这种情况下，函数返回值就会被忽略掉。这是很少见的，但如果函数调用者对返回值不感兴趣，这样也是允许的。

当程序调用一个函数时，程序控制权就会转移到被调用的函数上。当执行完函数的返回语句或执行到函数结束时，被调用函数就会将程序控制权交还给调用者。

程序清单 6-1 给出用于测试函数 max 的完整程序。

程序清单 6-1　TestMax.py

```
1  # Return the max of two numbers
2  def max(num1, num2):
3      if num1 > num2:
4          result = num1
5      else:
6          result = num2
7
8      return result
9
10 def main():
11     i = 5
12     j = 2
13     k = max(i, j) # Call the max function
14     print("The larger number of", i, "and", j, "is", k)
15
16 main() # Call the main function
```

```
The larger number of 5 and 2 is 5
```

	Line#	i	j	k	num1	num2	result
	11	5					
	12		2				
Invoke max	2				5	2	
	4						5
	13			5			

这个程序包含 max 函数和 main 函数。程序脚本在第 16 行调用 main 函数。习惯上，程序里通常定义一个包含程序主要功能的名为 main 的函数。

这个程序是怎么执行的？翻译器从文件的第 1 行开始一行一行地读取脚本语言。因为第 1 行是注释，所以它就被忽略掉了。当它读取第 2 行的函数头时，将函数以及函数体（第 2 到 8 行）存储在内存中。尽管函数的定义对函数进行了定义，但它不会让函数执行。然后，翻译器将 main 函数的定义（第 10 ～ 14 行）读取到内存。最后，解释器读取第 16 行时，它会调用 main 函数，即 main 函数被执行。程序的控制转移到 main 函数，如图 6-2 所示。

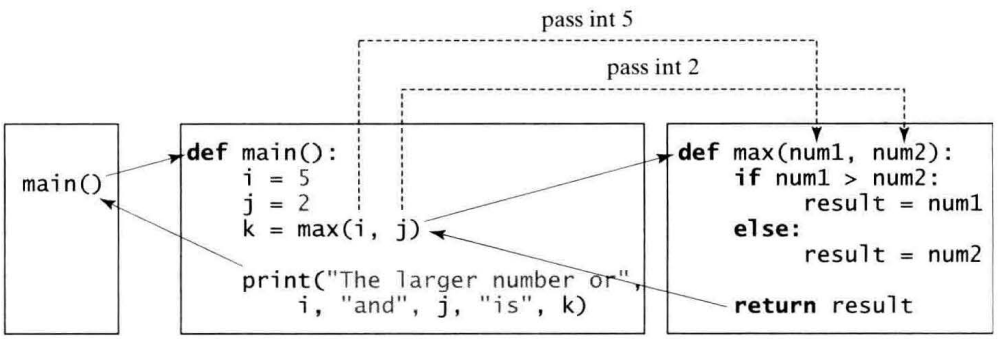


图 6-2 当函数被调用时，程序控制权就转移到这个函数，
当函数结束时程序控制权又转移到函数被调用的地方

main 函数的执行从第 11 行开始。它把 5 赋值给 i，2 赋值给 j（第 11 ～ 12 行），然后调用函数 max(i,j)（第 13 行）。

当 max 函数被调用时（第 13 行），变量 i 的值被传递给 num1，变量 j 的值被传递给 num2。程序控制权转移到 max 函数，然后就开始执行 max 函数。当 max 函数的 return 语句被执行后，max 函数就将程序控制权转移给调用者（在这种情况下，调用者就是 main 函数）。

max 函数结束之后，max 函数的返回值就会赋值给 k（第 13 行）。main 函数打印结果（第 14 行）。main 函数结束，它就将程序控制权返回给调用者（第 16 行）。现在，程序就结束了。

注意：这里的 main 函数是定义在 max 函数之后。但在 Python 中，因为函数在内存中被调用，函数可以定义在脚本文件的任意位置。你也可以在 max 函数之前定义 main 函数。

调用栈

每次调用一个函数时，系统就会创建一个为函数存储它的参数和变量的激活记录，然后将这个激活记录放在一个被称为堆栈的内存区域。调用栈又被称为执行堆栈、运行堆栈或机器堆栈，经常被简称为堆栈。当一个函数调用另一个函数时，调用者的激活记录保持不变，然后为新函数的调用创建一个新的激活记录。当一个函数结束了它的工作之后就将程序控制权转移给它的调用者，同时从堆栈中删除它的激活记录。

堆栈采用后进先出的方式存储激活记录。最后被调用的函数的激活记录将最先从栈里删除。假设函数 m1 调用 m2，然后调用 m3。运行时系统将 m1 的激活记录压入栈中，然后是将 m2 的激活记录压入栈中，最后是将 m3 的激活记录压入栈中。在完成 m3 的工作之后，它的激活记录被弹出栈。在完成 m2 的工作之后，它的激活记录被弹出栈，在完成 m1 的工作之后，它的激活记录被弹出栈。

理解调用堆栈有助于我们理解函数是如何被调用的。当调用 main 函数时，就会创建一个激活记录来存储变量 i 和 j，如图 6-3a 所示。切记：在 Python 中所有的数据都是对象。Python 在一个被称为堆的内存空间里创建和存储对象。变量 i 和 j 实际包含的是 int 对象 5 和 2 的引用值，如图 6-3a 所示。

调用函数 max(i,j) 时，将 i 和 j 的值传递给函数 max 的参数 num1 和 num2。现在 num1 和 num2 指向 int 对象 5 和 2，如图 6-3b 所示。函数 max 找出最大数并将它赋值给 result。所以现在 result 指向 int 对象 5，如图 6-3c 所示。result 返回给 main 函数并赋值给变量 k。现在 k 指向 int 对象 5，如图 6-3d 所示。在 main 函数结束后，栈被清空，如图 6-3e 所示。当堆中的对象不再需要时，Python 会自动将这些对象清空。

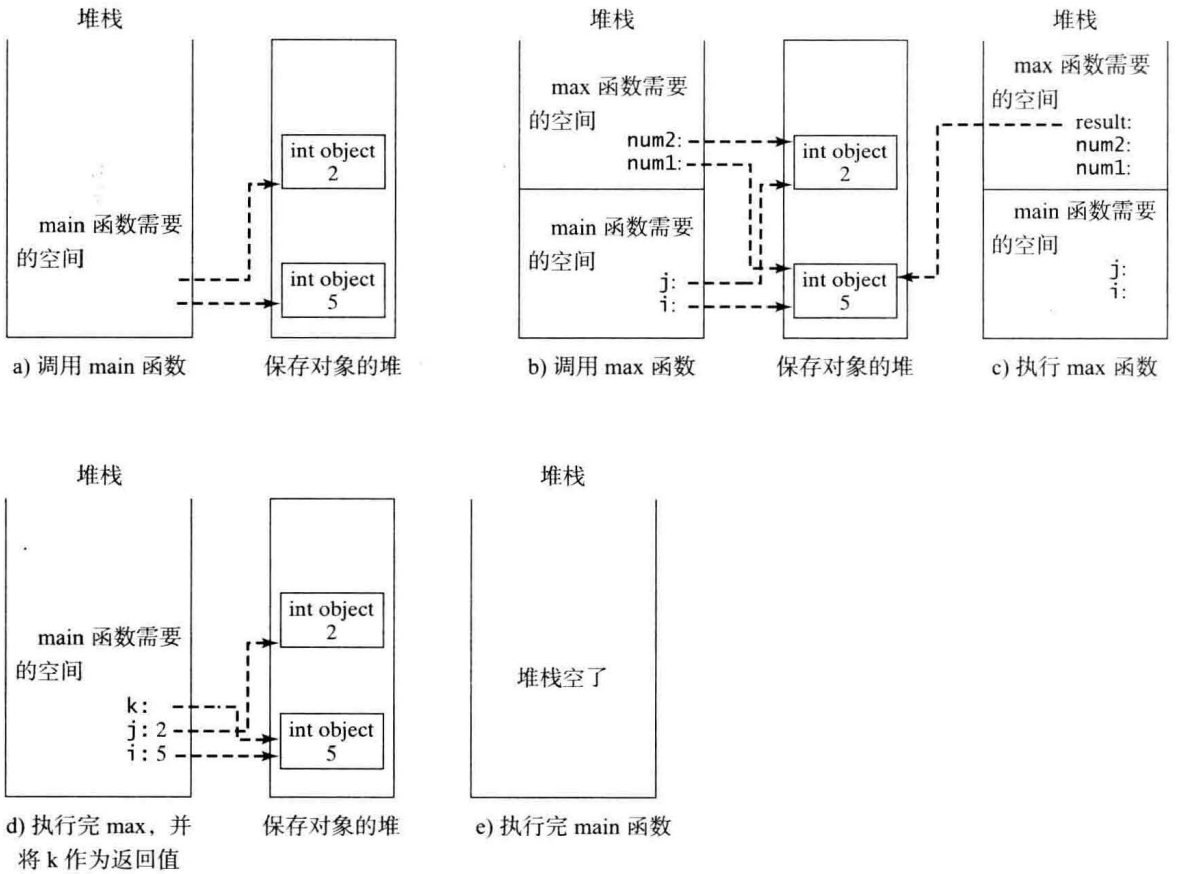


图 6-3　当调用函数时，创建一个激活记录来存储函数中的变量，函数完成后释放激活记录

6.4　带返回值或不带返回值的函数

关键点：函数不是一定有返回值。

第 6.3 节给出一个带返回值函数的例子。本节将介绍如何定义和调用不带返回值的函

数。通常，这样的函数的程序设计术语是*无返回值函数*。

程序清单 6-2 定义了一个叫 printGrade 的函数，然后调用它打印给定分数的等级。

程序清单 6-2 PrintGradeFunction.py

```
1  # Print grade for the score
2  def printGrade(score):
3      if score >= 90.0:
4          print('A')
5      elif score >= 80.0:
6          print('B')
7      elif score >= 70.0:
8          print('C')
9      elif score >= 60.0:
10         print('D')
11     else:
12         print('F')
13
14 def main():
15     score = eval(input("Enter a score: "))
16     print("The grade is ", end = " ")
17     printGrade(score)
18
19 main() # Call the main function
```

```
Enter a score: 78.5 Enter
The grade is C
```

printGrade 函数不返回任何值。所以，在 main 函数的第 17 行，它被当作一个语句调用。

为了体现带返回值或无返回值的函数的区别，让我们重新设计 printGrade 函数返回一个值。我们调用一个新函数 getGrade 返回一个等级，如程序清单 6-3 所示。

程序清单 6-3 ReturnGradeFunction.py

```
1  # Return the grade for the score
2  def getGrade(score):
3      if score >= 90.0:
4          return 'A'
5      elif score >= 80.0:
6          return 'B'
7      elif score >= 70.0:
8          return 'C'
9      elif score >= 60.0:
10         return 'D'
11     else:
12         return 'F'
13
14 def main():
15     score = eval(input("Enter a score: "))
16     print("The grade is", getGrade(score))
17
18 main() # Call the main function
```

```
Enter a score: 78.5 Enter
The grade is C
```

第 2 ～ 12 行定义的函数 getgrade 返回一个基于数字分值的字符等级。调用者在第 16 行调用这个函数。

函数 getGrade 返回一个字符，它可以像调用一个字符一样使用。函数 printGrade 无返

回值，因而只能被当作一条语句被调用。

注意：实际上，不管你是否使用 return，所有 Python 的函数都将返回一个值。如果某个函数没有返回值，默认情况下，它返回一个特殊值 None。因此，无返回值函数不会返回值，它也被称作 None 函数。None 函数可以赋值给一个变量来表明这个变量不指向任何对象。例如，如果你运行下面程序：

```
def sum(number1, number2):
    total = number1 + number2

print(sum(1, 2))
```

你会发现输出为 None，因为 sum 函数没有 return 语句。默认情况下，它返回 None。

注意：None 函数不是一定需要 return 语句，但它能用于终止函数并将控制权返回函数调用者。它的语法是

```
return
```

或

```
return None
```

这种用法很少使用，但对于改变函数的正常流程是很有用的。例如，当分数值是无效值时，下面的代码就用 return 语句结束这个函数。

```
# Print grade for the score
def printGrade(score):
    if score < 0 or score > 100:
        print("Invalid score")
        return # Same as return None

    if score >= 90.0:
        print('A')
    elif score >= 80.0:
        print('B')
    elif score >= 70.0:
        print('C')
    elif score >= 60.0:
        print('D')
    else:
        print('F')
```

检查点

6.1　使用函数的好处是什么？

6.2　如何定义一个函数？如何调用一个函数？

6.3　你能使用传统的表达式简化程序清单 6-1 中的 max 函数吗？

6.4　对 None 函数的调用总是它自己语句本身，但是对带返回值函数的调用总是表达式的一部分。这种说法是对还是错？

6.5　None 函数能不能有 return 语句吗？下面语句中的 return 函数是否会造成语法错误？

```
def xFunction(x, y):
    print(x + y)
    return
```

6.6　给出术语函数头、形参、实参的定义。

6.7　编写下面函数的函数头（并指出函数是否有返回值）：

- 给定销售额和提成率，然后计算销售提成。

- 给定年份和月份，然后打印该月的日历。
- 计算平方根。
- 判断一个数是不是偶数，如果是则返回 true。
- 按指定次数打印一条消息。
- 给定贷款额、还款年数和年利率，然后计算月支付额。
- 对于给定的小写字母，给出相应的大写字母。

6.8 确定并改正下面程序中的错误：

```
def function1(n, m):
function2(3.4)

def function2(n):
    if n > 0:
     return 1
  elif n == 0:
    return 0
  elif n < 0:
    return -1

function1(2, 3)
```

6.9 显示下面代码的输出：

```
def main():
    print(min(5, 6))

def min(n1, n2):
    smallest = n1
    if n2 < smallest:
        smallest = n2

main() # Call the main function
```

6.10 运行下面程序时会出现什么错误？

```
def main():
    print(min(min(5, 6), (51, 6)))

def min(n1, n2):
    smallest = n1
    if n2 < smallest:
        smallest = n2

main() # Call the main function
```

6.5 位置参数和关键字参数

关键点：函数实参是作为位置参数和关键字参数被传递。

函数的作用就在于它处理参数的能力。当调用函数时，需要将实参传递给形参。实参有两种类型：位置参数和关键字参数。使用位置参数要求参数按它们在函数头的顺序进行传递。例如，下面的函数输出消息 n 次：

```
def nPrintln(message, n):
    for i in range(n):
        print(message)
```

你可以使用 nPrintln('a',3) 输出 a 三次。nPrintln('a',3) 语句将 a 传递给 message，将 3 传递给 n，然后输出 a 三次。但是，语句 nPrintln(3，'a') 则有不同的含义。它将 3 传递给

message，将 a 传递给 n。当我们调用这样的函数时，就被称为使用位置参数。实参必须和函数头中定义的形参在顺序、个数和类型上匹配。

你也可以使用关键字参数调用函数，通过 name=value 的形式传递每个参数。例如：nPlintln(n=5,message="good") 将 5 传递给 n，将 "good" 传递给 message。使用关键字参数时，参数可以以任何顺序出现。

位置参数和关键字参数很可能被混在一起，但位置参数不能出现在任何关键字参数之后。假设函数头是：

```
def f(p1, p2, p3):
```

你可以通过使用

```
f(30, p2 = 4, p3 = 10)
```

调用它。

但是，你通过使用

```
f(30, p2 = 4, 10)
```

调用它则会出错。因为位置参数 10 出现在关键字参数 p2=4 之后。

检查点

6.11　比较位置参数和关键字参数。

6.12　假设函数头如下所示：

```
def f(p1, p2, p3, p4):
```

下面哪些调用是正确的？

```
f(1, p2 = 3, p3 = 4, p4 = 4)
f(1, p2 = 3, 4, p4 = 4)
f(p1 = 1, p2 = 3, 4, p4 = 4)
f(p1 = 1, p2 = 3, p3 = 4, p4 = 4)
f(p4 = 1, p2 = 3, p3 = 4, p1 = 4)
```

6.6　通过传引用来传递参数

关键点：当你调用带参数的函数时，每个实参的引用值被传递给函数的形参。

因为 Python 中的所有数据都是对象，所以对象的变量通常都是指向对象的引用。当你调用一个带参数的函数时，每个实参的引用值就被传递给形参。这在程序设计术语中被称为通过值传递。简单地说，调用函数时，实参的值就被传递给形参。这个值通常就是对象的引用值。

如果实参是一个数字或者是一个字符串，那么不管函数中的形参有没有变化，这个实参是不受影响的。程序清单 6-4 给出一个例子。

程序清单 6-4　Increment.py

```
1  def main():
2      x = 1
3      print("Before the call, x is", x)
4      increment(x)
5      print("After the call, x is", x)
6
```

```
 7  def increment(n):
 8      n += 1
 9      print("\tn inside the function is", n)
10
11  main() # Call the main function
```

```
Before the call, x is 1
        n inside the function is 2
After the call, x is 1
```

如程序清单 6-4 的输出所示，x(1) 的值被传递给形参 n 来调用 increment 函数（第 4 行）。函数中参数 n 递增 1，但是不论函数做什么 x 都不会改变。

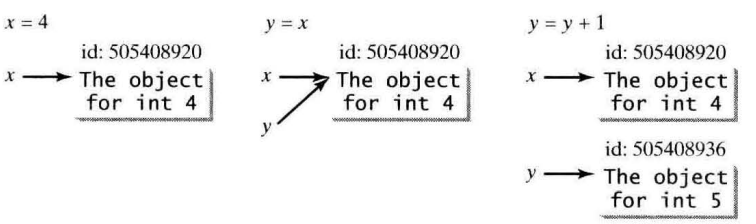
这样的原因是因为数字和字符串被称为*不可变对象*。不可变对象的内容是不能被改变的。当你将一个新数字赋值给变量时，Python 就会为这个新数字创建新对象，然后将这个新对象的引用赋值给这个变量。

考虑下面的代码：

```
>>> x = 4
>>> y = x
>>> id(x)  # The reference of x
505408920
>>> id(y) # The reference of y is the same as the reference of x
505408920
>>>
```

你把 x 赋值给 y，现在，x 和 y 都指向同一个对象（整数 4），如图 6-4a ～ b 所示。如果你将 1 加到 y，那就会创建一个新对象然后它被赋给 y，如图 6-4c 所示。现在，y 指向一个新对象，如下面代码所示：

```
>>> y = y + 1 # y now points to a new int object with value 5
>>> id(y)
505408936
>>>
```



a）4 被赋值给 x　　b）x 被赋值给 y　　c）y+1 被赋值给 y

图　6-4

检查点

6.13　什么是值传递？

6.14　形参和实参能同名吗？

6.15　显示下面函数的结果。

```
def main():
    max = 0
    getMax(1, 2, max)
    print(max)

def getMax(value1, value2, max):
    if value1 > value2:
        max = value1
    else:
        max = value2

main()
```

a)

```
def main():
    i = 1
    while i <= 6:
        print(function1(i, 2))
        i += 1

def function1(i, num):
    line = ""
    for j in range(1, i):
        line += str(num) + " "
        num *= 2
    return line

main()
```

b)

```
def main():
    # Initialize times
    times = 3
    print("Before the call, variable",
        "times is", times)

    # Invoke nPrintln and display times
    nPrint("Welcome to CS!", times)
    print("After the call, variable",
        "times is", times)

# Print the message n times
def nPrint(message, n):
    while n > 0:
        print("n = ", n)
        print(message)
        n -= 1

main()
```

c)

```
def main():
    i = 0
    while i <= 4:
        function1(i)
        i += 1

        print("i is", i)

def function1(i):
    line = " "
    while i >= 1:
        if i % 3 != 0:
            line += str(i) + " "
            i -= 1

    print(line)

main()
```

d)

6.16　在前面问题的 a 中，给出调用函数 max 之前也就是刚刚进入 max 时栈的内容，以及 max 被返回后栈的内容。

6.7　模块化代码

关键点：模块化可以使代码易于维护和调试，并且提高代码的重用性。

函数可以用来减少冗余的代码并提高代码的可重用性。函数也可以用来模块化代码并提高程序的质量。在 Python 中，你可以将函数的定义放在一个被称为模块的文件中，这种文件的后缀名是 .py。之后这些模块可以被导入到程序中以便重复使用。这些模块文件应该和其他程序一起放在同一个地方。一个模块可以包含不止一个函数。一个模块的每个函数都有不同的名字。注意：turtle、random 和 math 是定义在 Python 库里的模块，这样，它们可以被导入到任何一个 Python 程序中。

程序清单 5-8 是一个提示用户输入两个整数然后显示它们最大公约数的程序。你可重写一个使用函数的程序，然后将它放在一个称作 GCDFuntion.py 的模块，如程序清单 6-5 所示。

程序清单 6-5 GCDFunction.py

```
1  # Return the gcd of two integers
2  def gcd(n1, n2):
3      gcd = 1 # Initial gcd is 1
4      k = 2   # Possible gcd
5
6      while k <= n1 and k <= n2:
7          if n1 % k == 0 and n2 % k == 0:
8              gcd = k # Update gcd
9          k += 1
10
11     return gcd # Return gcd
```

现在，我们编写一个独立的程序使用 gcd 函数，如程序清单 6-6 所示。

程序清单 6-6 TestGCDFunction.py

```
1  from GCDFunction import gcd # Import the gcd function
2
3  # Prompt the user to enter two integers
4  n1 = eval(input("Enter the first integer: "))
5  n2 = eval(input("Enter the second integer: "))
6
7  print("The greatest common divisor for", n1,
8      "and", n2, "is", gcd(n1, n2))
```

```
Enter the first integer: 45 Enter
Enter the second integer: 75 Enter
The greatest common divisor for 45 and 75 is 15
```

第 1 行从模块 GCDFuntion 中导入 gcd 函数，这样，你就可以在程序中调用 gcd 函数（第 8 行）。你也可以使用下面的语句导入它：

```
import GCDFunction
```

如果使用这个语句，你必须使用 GCDFuntion.gcd 才能调用函数 gcd。

通过将求最大公约数的代码封装在函数 gcd 中，这个程序具备了以下几个优点。

1）它将计算最大公约数的代码和其他代码分隔开，这样使程序的逻辑更加清晰而且程序的可读性更强。

2）计算最大公约数的任何错误就被限定在函数 gcd 中，这样就缩小了调试的范围。

3）现在，其他程序就可以重用函数 gcd 了。

如果你在一个模块里定义了两个同名函数，那会出现什么情况呢？在这种情况下不会出现语法错误，但后者的优先级高。

程序清单 6-7 应用模块的概念来改进程序清单 5-13。程序定义了两个新函数：isPrime 和 printPrimeNumbers。函数 isPrime 判断一个数是不是素数，函数 printPrimeNumbers 打印素数。

程序清单 6-7 PrimeNumberFunction.py

```
1  # Check whether number is prime
2  def isPrime(number):
3      divisor = 2
4      while divisor <= number / 2:
5          if number % divisor == 0:
6              # If true, number is not prime
```

```
            return False # number is not a prime
        divisor += 1

    return True # number is prime

def printPrimeNumbers(numberOfPrimes):
    NUMBER_OF_PRIMES = 50  # Number of primes to display
    NUMBER_OF_PRIMES_PER_LINE = 10  # Display 10 per line
    count = 0  # Count the number of prime numbers
    number = 2  # A number to be tested for primeness

    # Repeatedly find prime numbers
    while count < numberOfPrimes:
        # Print the prime number and increase the count
        if isPrime(number):
            count += 1 # Increase the count

            print(number, end = " ")
            if count % NUMBER_OF_PRIMES_PER_LINE == 0:
                # Print the number and advance to the new line
                print()

        # Check if the next number is prime
        number += 1

def main():
    print("The first 50 prime numbers are")
    printPrimeNumbers(50)

main()  # Call the main function
```

```
The first 50 prime numbers are

2   3   5   7   11  13  17  19  23  29
31  37  41  43  47  53  59  61  67  71
73  79  83  89  97  101 103 107 109 113
127 131 137 139 149 151 157 163 167 173
179 181 191 193 197 199 211 223 227 229
```

这个程序将一个大问题分成两个小问题。这样，新程序更易读且更易于调试。同时，其他程序也可以重复使用 printPrimeNumbers 和 isPrime 函数。

6.8　实例研究：将十进制数转换为十六进制数

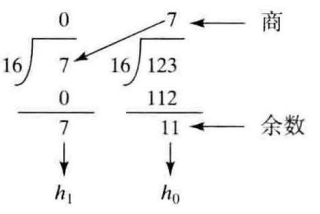
关键点：本节给出一个将十进制数转换为十六进制数的程序。

计算机程序设计中会经常用到十六进制数（第 3 章曾介绍过）（参见附录 C 中对数系的介绍）。将十进制数 d 转换为一个十六进制数就是找到满足下面条件的十六进制数：

$$d=h_n\times 16^n+h_{n-1}\times 16^{n-1}+h_{n-2}\times 16^{n-2}+\cdots+h_2\times 16^2+h_1\times 16^1+h_0\times 16^0$$

这些十六进制数可以通过不断用 d 除以 16 直到商为 0 而得到。依次得到的余数是 h_0，h_1，…，h_{n-1} 和 h_n。十六进制数包括 0、1、2、3、4、5、6、7、8、9，而 A 是十进制数 10，B 是十进制数 11，C 是十进制数 12，D 是十进制数 13，E 是十进制数 14，F 是十进制数 15。

例如：十进制数 123 转化成十六进制数就是 7B。这个转化过程如右所示：



123 除以 16 的余数是 11，它是十六进制数 B。这次除法的商是 7。16 除以 7 余数是 7，商是 0。所以，7B 就代表十进制数 123。

程序清单 6-8 的程序，提示用户输进一个十进制数，然后将它转换成字符串形式的十六进制数。

程序清单 6-8 Decimal2HexConversion.py

```
1  # Convert a decimal to a hex as a string
2  def decimalToHex(decimalValue):
3      hex = ""
4
5      while decimalValue != 0:
6          hexValue = decimalValue % 16
7          hex = toHexChar(hexValue) + hex
8          decimalValue = decimalValue // 16
9
10     return hex
11
12 # Convert an integer to a single hex digit as a character
13 def toHexChar(hexValue):
14     if 0 <= hexValue <= 9:
15         return chr(hexValue + ord('0'))
16     else:  # 10 <= hexValue <= 15
17         return chr(hexValue - 10 + ord('A'))
18
19 def main():
20     # Prompt the user to enter a decimal integer
21     decimalValue = eval(input("Enter a decimal number: "))
22
23     print("The hex number for decimal",
24         decimalValue, "is", decimalToHex(decimalValue))
25
26 main() # Call the main function
```

```
Enter a decimal number: 1234 ↵Enter
The hex number for decimal 1234 is 4D2
```

	line#	decimalValue	hex	hexValue	toHexChar(hexValue)
	21	1234			
	3		""		
iteration 1	6			2	
	7		"2"		"2"
	8	77			
iteration 2	6			13	
	7		"D2"		"D"
	8	4			
iteration 3	6			4	
	7		"4D2"		"4"
	8	0			

十六进制的字符串 hex 初始时为空（第 3 行）。程序使用函数 decimalToHex 将十进制数转换成字符串形式的十六进制数（第 2 ～ 10 行）。该函数得到这个十进制数除以 16 之后的余数 (第 6 行）。调用函数 toHexChar 将得到的余数转化为一个字符然后转为一个十六进制字符串（第 7 行）。将这个十进制数除以 16，就从该数中去掉一个十六进制数（第 8 行）。函数在一个循环中重复执行这些操作，直到商变成 0（第 5 ～ 8 行）。

函数 toHexChar 将 0 到 15 之间的数 hexValue 转换为一个十六进制字符。如果 hexValue

在 0 到 9 之间，那它就被转换为 chr(hexValue +ord('0'))（第 15 行）。例如，如果 hexValue 是 5，那么 chr(hexValue +ord('0')) 返回 5。类似地，如果 hexValue 在 10 到 15 之间，它就被转换为 chr(hexValue−10 +ord('A'))（第 17 行）。例如，如果 hexValue 是 11，那么 chr(hexValue−10 +ord('A')) 返回 B。

6.9 变量的作用域

关键点：变量的作用域是指该变量可以在程序中被引用的范围。

第 2 章介绍了变量的作用域。这节讨论在函数范围中变量的作用域。在函数内部定义的变量被称为*局部变量*。局部变量只能在函数内部被访问。局部变量的作用域从创建变量的地方开始，直到包含该变量的函数结束为止。

在 Python 中，你也可以使用*全局变量*。它们在所有的函数之外创建，可以被所有的函数访问。考虑下面的例子。

1. 示例 1

```
1  globalVar = 1
2  def f1():
3      localVar = 2
4      print(globalVar)
5      print(localVar)
6
7  f1()
8  print(globalVar)
9  print(localVar) # Out of scope, so this gives an error
```

在第 1 行创建一个全局变量。它可以在函数里访问（第 4 行），也可以在函数外访问（第 8 行）。第 3 行创建了一个局部变量。它可以在第 5 行的函数里访问。第 9 行试图在函数外访问局部变量就会造成错误。

2. 示例 2

```
1  x = 1
2  def f1():
3      x = 2
4      print(x) # Displays 2
5
6  f1()
7  print(x) # Displays 1
```

第 1 行定义一个全局变量 x，而第 3 行创建另一个同名的局部变量（x）。从这里开始，全局变量 x 就不可以在函数中被访问。但是在函数之外，全局变量 x 仍旧可访问。所以，第 7 行输出 1。

3. 示例 3

```
1  x = eval(input("Enter a number: "))
2  if x > 0:
3      y = 4
4
5  print(y) # This gives an error if y is not created
```

这里，如果 x>0 则变量 y 才会被创建。如果输入一个大于 0 的正数，程序正常运行。如果你输入一个非正数，那第 5 行程序将出错，因为 y 没有被创建。

4. 示例 4

```
1  sum = 0
2  for i in range(5):
3      sum += i
4
5  print(i)
```

这里，在循环中创建了变量 i。循环结束后，i 的值是 4，所以，第 5 行显示 4。

你可以将一个局部变量的作用域绑定为全局的。你也可以在函数中创建一个变量然后在函数外使用它。为了实现上述两个功能，可以使用 global 语句，如下面的例子所示。

5. 示例 5

```
1  x = 1
2  def increase():
3      global x
4      x =  x + 1
5      print(x) # Displays 2
6
7  increase()
8  print(x) # Displays 2
```

这里，第 1 行创建了一个全局变量 x，然后在第 3 行将 x 限定在这个函数中，这意味着函数里的 x 和函数外的 x 是一样的，所以，程序在第 5 行和第 8 行输出 2。

注意：尽管允许使用全局变量，你也可以看到全局变量在其他程序中使用。但在一个函数中允许修改全局变量并不是一个好习惯。因为这样做可能使程序更易出错。但是，可以定义全局变量以便模块中的所有函数都能使用它。

检查点

6.17　下面代码的打印结果什么？

```
def function(x):
    print(x)
    x = 4.5
    y = 3.4
    print(y)

x = 2
y = 4
function(x)
print(x)
print(y)
```

a)

```
def f(x, y = 1, z = 2):
    return x + y + z

print(f(1, 1, 1))
print(f(y = 1, x = 2, z = 3))
print(f(1, z = 3))
```

b)

6.18　下面的代码有什么错误？

```
1  def function():
2      x = 4.5
3      y = 3.4
4      print(x)
5      print(y)
6
7  function()
8  print(x)
9  print(y)
```

6.19　下面代码能运行吗？如果能，打印结果是什么？

```
x = 10
if x < 0:
```

```
    y = -1
else:
    y = 1

print("y is", y)
```

6.10 默认参数

关键点：Python 允许定义带默认参数值的函数。当函数被调用时无参数，那么这些默认值就会被传递给实参。

程序清单 6-9 演示如何定义带默认参数值的函数以及如何调用这样的函数。

程序清单 6-9 DefaultArgumentDemo.py

```
1  def printArea(width = 1, height = 2):
2      area = width * height
3      print("width:", width, "\theight:", height, "\tarea:", area)
4
5  printArea() # Default arguments width = 1 and height = 2
6  printArea(4, 2.5) # Positional arguments width = 4 and height = 2.5
7  printArea(height = 5, width = 3) # Keyword arguments width
8  printArea(width = 1.2) # Default height = 2
9  printArea(height = 6.2) # Default width = 1
```

```
width: 1        height: 2       area: 2
width: 4        height: 2.5     area: 10.0
width: 3        height: 5       area: 15
width: 1.2      height: 2       area: 2.4
width: 1        height: 6.2     area: 6.2
```

第 1 行用参数 width 和 height 定义函数 printArea。width 的默认值是 1，而 height 的默认值是 2。第 5 行是在没有传递实参的情况下调用这个函数。所以，程序就将默认值 1 赋给 width，将默认值 2 赋给 height。第 6 行通过将 4 和 6 分别赋给 width 和 height 来调用这个函数。第 7 行通过将 3 和 5 分别赋值给 width 和 height 来调用这个函数。注意：我们也可以通过指定参数名来传递实参的值，如第 8 行和第 9 行所示。

注意：函数可以混用默认值参数和非默认值参数。这种情况下，非默认值参数必须定义在默认值参数之前。

注意：尽管许多语言支持在同一个模块里定义两个同名的函数，但是 Python 并不支持这个特点。通过默认参数你只可以定义函数一次，但可以通过许多不同的方式调用函数。这和在其他语言中定义同名的多个函数的效果一样。如果你在 Python 中定义了多个函数，那么后面的函数就会取代先前的函数。

检查点

6.20　显示下面代码的打印结果：

```
def f(w = 1, h = 2):
    print(w, h)

f()
f(w = 5)
f(h = 24)
f(4, 5)
```

6.21　确定下面程序的错误并改正：

```
def main():
    nPrintln(5)

def nPrintln(message = "Welcome to Python!", n):
    for i in range(n):
        print(message)

main() # Call the main function
```

6.22　如果在同一模块里定义两个同名的函数会发生什么？

6.11　返回多个值

关键点：Python 的 return 语句可以返回多个值。

Python 允许函数返回多个值。程序清单 6-10 定义了一个输入两个数并以升序返回这两个数的函数。

程序清单 6-10　MultipleReturnValueDemo.py

```
def sort(number1, number2):
    if number1 < number2:
        return number1, number2
    else:
        return number2, number1

n1, n2 = sort(3, 2)
print("n1 is", n1)
print("n2 is", n2)
```

```
n1 is 2
n2 is 3
```

sort 函数返回两个值。当它被调用时，你需要同时赋值传递这些返回值。

检查点

6.23　函数能否返回多个值？显示下面程序的打印结果。

```
def f(x, y):
    return x + y, x - y, x * y, x / y

t1, t2, t3, t4 = f(9, 5)
print(t1, t2, t3, t4)
```

6.12　实例研究：生成随机 ASCII 码字符

关键点：字符是用整型数字来编码的。生成随机字符就是生成一个整型数。

计算机处理的是数值数据和字符。你已经看到了许多涉及数值数据的例子。了解字符以及如何处理字符都很重要。本节给出一个生成随机 ASCII 码字符的例子。

正如第 3.3 节中介绍的那样，每个字符都有一个在 0 到 127 之间的唯一 ASCII 码。要生成一个随机字符首先生成一个在 0 到 127 之间的 ASCII 码，然后使用下面的代码从函数 chr 中从整数中获取字符：

```
chr(randint(0, 127))
```

让我们考虑一下如何生成一个随机的小写字母。小写字母的 ASCII 码是一连串连续的整数，从小写字母 "a" 的编码开始，然后是 "b"、"c"、…、"z"。"a" 的编码是：

```
ord('a')
```

所以，ord('a') 和 ord('z') 之间的随机数是

```
randint(ord('a'), ord('z'))
```

因此，一个随机小写字母是

```
chr(randint(ord('a'), ord('z')))
```

这样，任意两个字符 ch1 和 ch2 之间且满足 ch1<ch2 的随机字符可以使用如下所示的代码生成：

```
chr(randint(ord(ch1), ord(ch2)))
```

这是一个简单且有用的发现。在程序清单 6-11 中，我们创建了一个名为 RandomCharacter.py 的包含 5 个能随机生成特定类型字符的函数。你可以在以后的程序里使用这些函数。

程序清单 6-11 RandomCharacter.py

```
1  from random import randint # import randint
2
3  # Generate a random character between ch1 and ch2
4  def getRandomCharacter(ch1, ch2):
5      return chr(randint(ord(ch1), ord(ch2)))
6
7  # Generate a random lowercase letter
8  def getRandomLowerCaseLetter():
9      return getRandomCharacter('a', 'z')
10
11 # Generate a random uppercase letter
12 def getRandomUpperCaseLetter():
13     return getRandomCharacter('A', 'Z')
14
15 # Generate a random digit character
16 def getRandomDigitCharacter():
17     return getRandomCharacter('0', '9')
18
19 # Generate a random character
20 def getRandomASCIICharacter():
21     return chr(randint(0, 127))
```

程序清单 6-12 是一个测试程序，它显示 175 个随机小写字母。

程序清单 6-12 TestRandomCharacter.py

```
1  import RandomCharacter
2
3  NUMBER_OF_CHARS = 175 # Number of characters to generate
4  CHARS_PER_LINE = 25 # Number of characters to display per line
5
6  # Print random characters between 'a' and 'z', 25 chars per line
7  for i in range(NUMBER_OF_CHARS):
8      print(RandomCharacter.getRandomLowerCaseLetter(), end = " ")
9      if (i + 1) % CHARS_PER_LINE == 0:
10         print()  # Jump to the new line
```

```
gmjsohezfkgtazqgmswfclrao
pnrunulnwmaztlfjedmpchcif
lalqdgivxkxpbzulrmqmbhikr
lbnrjlsopfxahssqhwuuljvbe
```

```
xbhdotzhpehbqmuwsfktwsoli
cbuwkzgxpmtzihgatdslvbwbz
bfesoklwbhnooygiigzdxuqni
```

第 1 行导入 RandomCharacter 模块，因为程序调用了这个模块里定义的函数。

调用函数 getRandomLowerCaseLetter() 返回一个小写字母（第 8 行）。

注意：函数 getRandomLowerCaseLetter() 没有任何形参，但当你定义和调用它时，必须使用括号。

检查点

6.24　编写一个返回 34 到 55 之间，包括 33 和 55 的随机整数的表达式。

6.25　编写一个返回 B 到 M 之间，包括 B 和 M 的随机字符的表达式。

6.26　编写一个返回 6.5 到 56.5（不包括 56.5）之间的随机数的表达式。

6.27　编写一个返回随机小写字母的表达式。

6.13　函数抽象和逐步求精

关键点：函数抽象就是将函数的使用和函数的实现分开来实现的。

软件开发的关键就是应用抽象的概念。本书中介绍了许多不同层次的抽象。函数抽象将函数的使用从函数的实现分离出来。一个用户程序或简称为用户，可以在不知道函数是如何实现的情况下使用函数。函数的实现细节被封装在函数内，并对调用该函数的用户隐藏。这被称为信息隐藏或封装。如果决定改变函数的实现，只要不改变函数名，用户程序就不会受影响。函数的实现对用户而言是隐藏在一个黑匣子中，如图 6-5 所示。

我们已经在用户程序中使用过许多 Python 内置函数，并且知道如何在程序中编写代码来调用这些函数，但是，作为这些函数的使用者，我们并不需要知道它们是如何实现的。

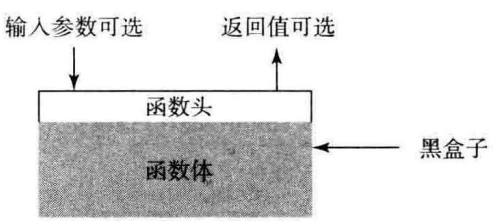


图 6-5　函数体可以被认为是一个黑盒子，它包含这个函数的详细实现

函数抽象的概念可以被应用到开发程序的过程中。当编写一个大程序时，你可以使用分治策略，也称之为逐步求精，将大问题分解为子问题。这些子问题又被分为更小更容易管理的问题。

假设要编写一个程序，它显示给定年月的日历。程序提示用户输入年月，然后显示该月的整个日历，如下示例运行所示：

```
Enter full year (e.g., 2001): 2011 ↵Enter
Enter month as number between 1 and 12: 9 ↵Enter
         September 2011
---------------------------------
 Sun  Mon  Tue  Wed  Thu  Fri  Sat
                     1    2    3
   4    5    6    7    8    9   10
  11   12   13   14   15   16   17
  18   19   20   21   22   23   24
  25   26   27   28   29   30
```

让我们使用这个例子演示分治法。

6.13.1　自顶向下设计

如何开始编写这样一个函数呢？你会立即开始编写代码吗？程序员新手常常一开始就想制订解决方案的每一个细节。尽管细节对最终程序很重要，但前期过多关注细节会阻碍问题的解决进程。为使解决问题流程尽可能地流畅，本例先用函数抽象把细节和设计分离，只在最后才实现这些细节。

对本例来说，先把问题分成两个子问题：①获取用户的输入，②打印该月的日历。在这一步应该考虑还能分成什么子问题，而不是如何获取输入和打印一个月的日历。你可以绘制一个结构图，这有助于看清楚问题的分解过程（参见图 6-6a）。

你可以使用函数 input 读取输入的年份和月份。如何打印给定月份的日历问题可以分成两个子问题：①打印日历标题，②打印日历主体，如图 6-6b 所示。日历标题由三行组成：年月、虚线、每周七天的星期名称。你需要通过表示月份的数字（例如：1）来获取该月的全称（例如：January）。这可以由 getMonthName 完成（如图 6-7a 所示）。

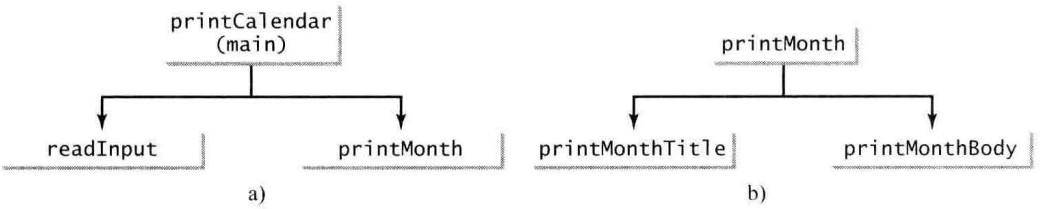


图 6-6　结构图显示 printCalendar 问题被分成两个子问题，readInput 和 printMonth，而 printMonth 又被分成两个更小的子问题，printMonthTitle 和 printMonthBody

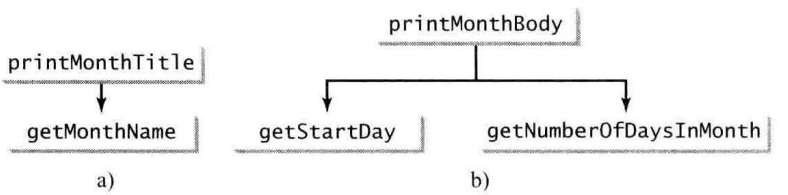


图 6-7　a）为实现 printMonthTitle，你需要 getMonthName
b）printMonthBody 问题被分成几个更小的问题

为了打印日历的主体，你需要知道这个月的第 1 天是星期几（getStartDay），以及这个月共有多少天（getNumberOfDaysInMonth），如图 6-7b 所示。例如：2005 年 12 月有 31 天，2005 年 12 月 1 日是星期四。

你如何知道一个月的第 1 天是星期几？下面有几个方法。假设你知道 1800 年 1 月 1 日是星期三（START_DAY_FOR_JAN_1_1800 = 3）。你可以计算这个月的第 1 天和 1800 年 1 月 1 日之间有多少天（totalNumberOfDays）。那么，这个月的第一天就是 (totalNumberOfDays+startDay1800)%7，因为每个星期有七天。因此，getStartDay 问题就被进一步提炼成 getTotalNumberOfDays，如图 6-8a 所示。

为获取总天数，你需要知道该年是不是闰年以及每个月的天数。因此，getTotalNumberOfDays 需要被进一步提炼成两个小问题：isLeapYear 和 getNumberOfDaysInMonth，如图 6-8b 所示。完整的结构图如图 6-9 所示。

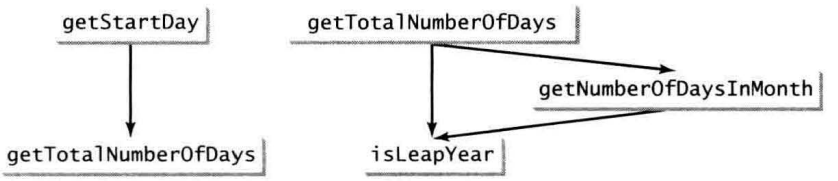


图 6-8　a）为实现 getStartDay，你需要 getTotalNumberOfDays
b) 问题 getTotalNumberOfDays 被提炼成两个更小的问题

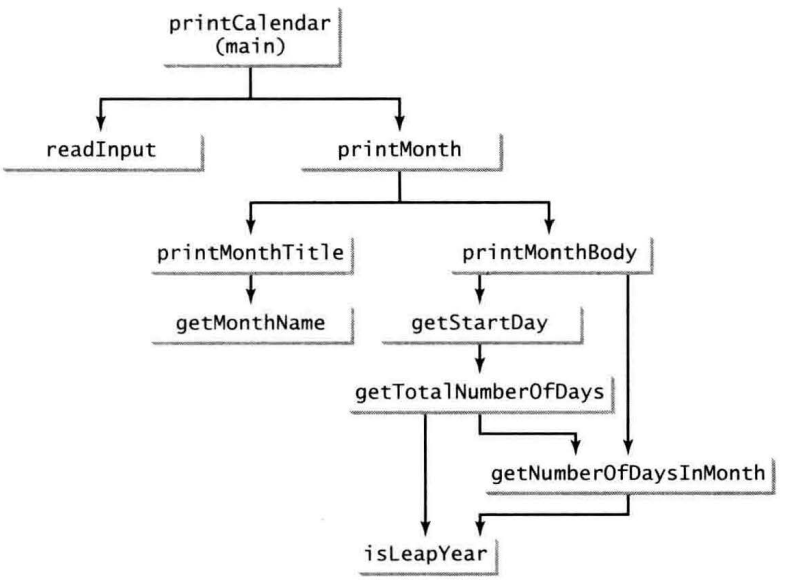


图 6-9　结构图显示程序里子问题之间的层次关系

6.13.2　自顶向下和自底向上的实现

现在，我们把注意力转移到实现上。通常，一个子问题在实现过程中对应一个函数，尽管某些子问题太简单，以至于都不需要函数来实现。你需要决定哪些模块需要函数实现，而哪些模块需要结合其他函数。这种决策应该基于整个程序是否更易读。在本例中，子问题 readInput 只在 main 函数中实现。

你既可以采用自顶向下的方法，也可以采用自底向上的方法。“自顶向下”的方法是自上而下，每次实现结构图中的一个函数。*待完善部分*（stub）是函数的一个简单但不完整的版本，可以用它表示等待实现的函数。使用待完善方式可以快速地构建程序的框架。首先，实现 main 方法，然后使用 printMonth 函数的待完善部分。例如，让 printMonth 的待完善部分显示年份和月份。那么，程序就可以像这样开始：

```
# A stub for printMonth may look like this
def printMonth(year, month):
    print(year, month)

# A stub for printMonthTitle may look like this
def printMonthTitle(year, month):
    print("printMonthTitle")
```

```
# A stub for getMonthBody may look like this
def getMonthBody(year, month):
    print("getMonthBody")

# A stub for getMonthName may look like this
def getMonthName(month):
    print("getMonthName")

# A stub for getStartDay may look like this
def getStartDay(year, month):
    print("getStartDay")

# A stub for getTotalNumberOfDays may look like this
def getTotalNumberOfDays(year, month):
    print("getTotalNumberOfDays")

# A stub for getNumberOfDaysInMonth may look like this
def getNumberOfDaysInMonth(year, month):
    print("getNumberOfDaysInMonth")

# A stub for isLeapYear may look like this
def isLeapYear(year):
    print("isLeapYear")

def main():
    # Prompt the user to enter year and month
    year = eval(input("Enter full year (e.g., 2001): "))
    month = eval(input((
       "Enter month as number between 1 and 12: ")))

    # Print calendar for the month of the year
    printMonth(year, month)

main() # Call the main function
```

运行并测试这个程序，然后修改所有的错误。现在，你可以实现 printMonth 函数。对于从 printMonth 函数中调用的所有函数，你可以再次使用待完善方式。

自底向上方法是从下向上每次实现结构图中的一个函数。对每一个实现的函数都编写一个被称为驱动程序的测试程序进行测试。

自顶向下和自底向上都是不错的方法。它们都是逐步地实现函数，这有助于分离程序设计错误，使调试变得更加容易。这两种方法可以一起使用。

6.13.3　实现细节

函数 isLeapYear(year) 可以使用下面的代码实现（参见第 4.12 节）：

```
return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)
```

使用下面的事实实现函数 getTotalNumberOfDays(year,month)：

- 一月、三月、五月、七月、八月、十月、十二月都是 31 天。
- 四月、六月、九月、十一月都是 30 天。
- 二月有 28 天，但在闰年有 29 天。因此，一年有 365 天，闰年有 366 天。

为了实现函数 getTotalNumberOfDays(year,month)，你需要计算从 1800 年 1 月 1 日到该月的第一天的总天数（totalNumberOfDays）以及这个月的第一天。你可以求出 1800 年到该年的总天数以及在这个月之前该年的天数。这两个天数之和就是 totalNumberOfDays。

要打印日历体，首先要在第一天之前填充一些空格，然后为每个星期打印一行。

完整的程序在程序清单 6-13 中给出。

程序清单 6-13 PrintCalendar.py

```
# Print the calendar for a month in a year
def printMonth(year, month):
    # Print the headings of the calendar
    printMonthTitle(year, month)

    # Print the body of the calendar
    printMonthBody(year, month)

# Print the month title, e.g., May 1999
def printMonthTitle(year, month):
    print("         ", getMonthName(month), " ", year)
    print("-----------------------------------------")
    print(" Sun Mon Tue Wed Thu Fri Sat")

# Print month body
def printMonthBody(year, month):
    # Get start day of the week for the first date in the month
    startDay = getStartDay(year, month)

    # Get number of days in the month
    numberOfDaysInMonth = getNumberOfDaysInMonth(year, month)

    # Pad space before the first day of the month
    i = 0
    for i in range(0, startDay):
       print("    ", end = " ")

    for i in range(1, numberOfDaysInMonth + 1):
        print(format(i, "4d"), end = " ")

        if (i + startDay) % 7 == 0:
            print() # Jump to the new line

# Get the English name for the month
def getMonthName(month):
    if month == 1:
        monthName = "January"
    elif month == 2:
        monthName = "February"
    elif month == 3:
        monthName = "March"
    elif month == 4:
        monthName = "April"
    elif month == 5:
        monthName = "May"
    elif month == 6:
        monthName = "June"
    elif month == 7:
        monthName = "July"
    elif month == 8:
        monthName = "August"
    elif month == 9:
        monthName = "September"
    elif month == 10:
        monthName = "October"
    elif month == 11:
        monthName = "November"
    else:
        monthName = "December"

```

```
    return monthName

# Get the start day of month/1/year
def getStartDay(year, month):
    START_DAY_FOR_JAN_1_1800 = 3

    # Get total number of days from 1/1/1800 to month/1/year
    totalNumberOfDays = getTotalNumberOfDays(year, month)

    # Return the start day for month/1/year
    return (totalNumberOfDays + START_DAY_FOR_JAN_1_1800) % 7

# Get the total number of days since January 1, 1800
def getTotalNumberOfDays(year, month):
    total = 0

    # Get the total days from 1800 to 1/1/year
    for i in range(1800, year):
        if isLeapYear(i):
            total = total + 366
        else:
            total = total + 365

    # Add days from Jan to the month prior to the calendar month
    for i in range(1, month):
        total = total + getNumberOfDaysInMonth(year, i)

    return total

# Get the number of days in a month
def getNumberOfDaysInMonth(year, month):
    if (month == 1 or month == 3 or month == 5 or month == 7 or
        month == 8 or month == 10 or month == 12):
        return 31

    if month == 4 or month == 6 or month == 9 or month == 11:
        return 30

    if month == 2:
        return 29 if isLeapYear(year) else 28

    return 0 # If month is incorrect

# Determine if it is a leap year
def isLeapYear(year):
    return year % 400 == 0 or (year % 4 == 0 and year % 100 != 0)

def main():
    # Prompt the user to enter year and month
    year = eval(input("Enter full year (e.g., 2001): "))
    month = eval(input(("Enter month as number between 1 and 12: ")))

    # Print calendar for the month of the year
    printMonth(year, month)

main() # Call the main function
```

这个程序没有检测用户输入的有效性。例如：用户输入的月份如果不在 1 到 12 之间，或者年份在 1800 以前，那么程序就会显示错误的日历。为了避免这样的错误，可以添加 if 语句在打印之前检查输入。

该程序可以打印一个月的日历，也可以很容易地修改为打印整年的日历。尽管它现在只能打印 1800 年 1 月以后的月份，但稍加修改便能打印 1800 年之前的月份。

6.13.4 逐步求精的优势

逐步求精将一个大问题分解成更小的易于解决的子问题。每个子问题都可以用函数实现。这种方法使程序易于编写、重用、调试、测试、修改和维护。

1. 简化程序

打印日历的程序很长。逐步求精将它分解成许多小函数，而不是在一个函数中编写一段很长的语句。这可以简化程序，使整个程序更易读和理解。

2. 重用函数

逐步求精促使函数在程序中重复使用。函数 isLeapYear 只被定义了一次，但却可以被函数 getTotalNumberOfDays 和 getTotalNumberOfDaysInMonth 调用。这减少了冗余的代码。

3. 易于开发、调试和测试

因为每个子问题都是由一个函数来解决，所以一个函数可以被单独开发、编译和测试。它将错误分离出来，使程序更易于开发、调试和测试。

当实现一个大程序时，使用自顶向下或自底向上的方法。不要一次性写完整个程序。使用这些方法似乎要占用更多的开发时间（因为要反复地运行程序），但它实际上更省时间和更易于调试。

4. 更好地实现团队合作

因为一个大程序被分成了许多子程序，这些子程序可以被分给不同的程序员。这使程序员更易于团队合作。

6.14 实例研究：可重用图形函数

关键点：在 turtle 模块中，你可以开发可重用函数来简化代码。

你经常需要在两点之间绘制一条线，在一个指定的位置显示文本或一个小点，描绘一个指定圆心和半径的圆，或者创建一个指定中心、宽和高的矩形。如果这些函数可重用，那么将大大简化程序设计。程序清单 6-14 在一个名为 UseFullTurtleFuntions 的模块里定义了这些函数。

程序清单 6-14 UsefulTurtleFunctions.py

```
1  import turtle
2
3  # Draw a line from (x1, y1) to (x2, y2)
4  def drawLine(x1, y1, x2, y2):
5      turtle.penup()
6      turtle.goto(x1, y1)
7      turtle.pendown()
8      turtle.goto(x2, y2)
9
10 # Write a string s at the specified location (x, y)
11 def writeText(s, x, y):
12     turtle.penup() # Pull the pen up
13     turtle.goto(x, y)
14     turtle.pendown() # Pull the pen down
15     turtle.write(s) # Write a string
16
17 # Draw a point at the specified location (x, y)
18 def drawPoint(x, y):
19     turtle.penup() # Pull the pen up
20     turtle.goto(x, y)
```

```
    turtle.pendown() # Pull the pen down
    turtle.begin_fill() # Begin to fill color in a shape
    turtle.circle(3)
    turtle.end_fill() # Fill the shape

# Draw a circle centered at (x, y) with the specified radius
def drawCircle(x = 0, y = 0, radius = 10):
    turtle.penup() # Pull the pen up
    turtle.goto(x, y - radius)
    turtle.pendown() # Pull the pen down
    turtle.circle(radius)

# Draw a rectangle at (x, y) with the specified width and height
def drawRectangle(x = 0, y = 0, width = 10, height = 10):
    turtle.penup() # Pull the pen up
    turtle.goto(x + width / 2, y + height / 2)
    turtle.pendown() # Pull the pen down
    turtle.right(90)
    turtle.forward(height)
    turtle.right(90)
    turtle.forward(width)
    turtle.right(90)
    turtle.forward(height)
    turtle.right(90)
    turtle.forward(width)
```

现在，你已经写好了这个代码，你可以使用这些函数绘制图形。程序清单 6-15 给出一个测试程序来使用 UseFullTurtleFuntions 模块中的函数来绘制一条线、编写一些文本，以及创建一个点、一个圆和一个矩形，如图 6-10 所示。

程序清单 6-15 UseCustomTurtleFunctions.py

```
1  import turtle
2  from UsefulTurtleFunctions import *
3
4  # Draw a line from (-50, -50) to (50, 50)
5  drawLine(-50, -50, 50, 50)
6
7  # Write text at (-50, -60)
8  writeText("Testing useful Turtle functions", -50, -60)
9
10 # Draw a point at (0, 0)
11 drawPoint(0, 0)
12
13 # Draw a circle at (0, 0) with radius 80
14 drawCircle(0, 0, 80)
15
16 # Draw a rectangle at (0, 0) with width 60 and height 40
17 drawRectangle(0, 0, 60, 40)
18
19 turtle.hideturtle()
20 turtle.done()
```

第 2 行的星号（*）导入模块 UseFullTurtleFuntions 中的所有函数。第 5 行调用函数 drawLine 来绘制一条线，而第 8 行调用函数 writeText 来绘制一个文本字符串。函数 drawPoint（第 11 行）绘制一个点，而函数 drawCircle（第 14 行）绘制一个圆。第 17 行调用函数 drawRetangle 来绘制一个矩形。

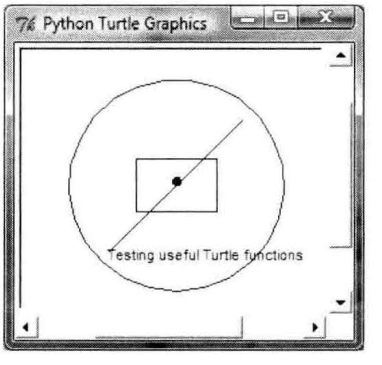


图 6-10　程序使用自定制函数绘制图形

关键术语

actual parameter（实际参数）
argument（实参）
caller（调用者）
default argument（默认参数）
divide and conquer（分治）
formal parameters (i.e., parameter)（形式参数（即形参））
functions（函数）
function abstraction（函数抽象）
function header（函数头）
global variable（全局变量）
immutable objects（不可变对象）
information hiding（信息隐藏）
keyword arguments（关键字参数）
local variable（局部变量）
None function（None 函数）
parameters（参数）
pass-by-value（传值）
positional arguments（位置参数）
return value（返回值）
scope of a variable（变量的作用域）
stepwise refinement（逐步求精）
stub（待完善方式）

本章总结

1. 程序模块化和可重用性是软件工程的中心目标之一。函数可以实现这个目标。
2. 函数头由关键字 def 开始，接下来是函数名和形式参数，最后以冒号结束。
3. 形式参数是可选的；也就是说，函数可以不包含任何形式参数。
4. 无返回值的函数被称为 void 或 None 函数。
5. 一个 return 语句可以在 void 函数中用来终止函数并将程序控制权返回给函数的调用者。有时，这对保证函数控制流正常是非常有用的。
6. 传给函数的参数必须和定义在函数头里的形参在数目、类型和顺序上保持一致。
7. 当程序调用一个函数时，程序的控制权就转移到被调用的函数。当执行到函数的 return 语句或执行到函数的最后一条语句时，被调用的函数就将控制权转给调用者。
8. 带返回值函数也可以当作 Python 语句被调用。在这种情况下，函数的返回值被忽略。
9. 函数参数可以当作位置参数或关键字参数传递。
10. 当调用一个带形式参数的函数时，实参的值就被传给形参。这用程序设计术语讲就是值传递。
11. 函数中创建的变量被称为局部变量。局部变量的作用域从它被创建的位置开始，直到函数返回为止都存在。变量必须在使用前被创建。
12. 全局变量被定义在所有函数之外，而且它们可以被所有函数访问。
13. Python 允许用默认参数值定义函数。当无参数调用函数时，默认值就被传给形参。
14. Python 的 return 语句可以返回多个值。
15. 函数抽象是通过将函数的使用和实现分开实现。一个用户可以在不知道函数是如何实现的情况下使用函数。函数的实现细节被封装在函数内，并对调用该函数的用户来说是隐藏的。这被称为信息隐藏或封装。
16. 函数抽象将程序模块化为整齐、分层的形式。程序被写成简洁函数的集合，这样使程序更易于编写、调试、维护和修改。这种编写风格会提高函数的可重用性。
17. 当实现一个大程序时，使用自顶向下或自底向上的编码方法。不要一次性编写整个程序。这个方法似乎要占用更多的编码时间（因为要反复地运行这个程序），但它实际上更省时间和更易于调试。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 6.2 ～ 6.9 节

6.1 （数学方面：五角数）一个五角数被定义为 $n(3n-1)/2$，其中 $n=1、2、\cdots$。所以，开始的几个数是 1、5、12、22、…，编写一个带下面函数头的函数返回五角数。

```
def getPentagonalNumber(n):
```

编写一个测试程序来使用这个函数显示前 100 个五角数，每行显示 10 个。

*6.2 （求一个整数各个数字的和）编写一个函数，计算一个整数各个数字的和。使用下面的函数头：

```
def sumDigits(n):
```

例如：sumDigits(234) 返回 9（2+3+4）。（提示：使用求余运算符 % 提取数字，而使用除号 // 去掉提取出来的数字。例如：使用 234%10（=4）抽取 4，然后使用 234//10（=23）从 234 中去掉 4。使用一个循环来反复提取和去掉每个数字，直到所有数字被提取完为止。）编写程序提示用户输入一个整数，然后显示这个整数所有数字的和。

**6.3 （回文整数）编写带下面函数头的函数。

```
# Return the reversal of an integer, e.g. reverse(456) returns
# 654
def reverse(number):

# Return true if number is a palindrome
def isPalindrome(number):
```

使用函数 reverse 实现 isPalindrome。如果一个数的反向数和它的顺向数一样，那么这个数就被称为回文数。编写一个测试程序，提示用户输入一个整数，然后输出这个整数是不是回文数。

*6.4 （反向显示一个整数）编写下面的函数，反向显示一个整数。

```
def reverse(number):
```

例如：reserse(3456) 显示 6543。编写一个测试程序，提示用户输入一个整数，然后显示它的反向数。

*6.5 （对三个数排序）编写下面的函数，以升序显示三个数。

```
def displaySortedNumbers(num1, num2, num3):
```

编写一个测试程序，提示用户输入三个整数，然后调用函数按升序显示三个数。下面是一些示例运行。

```
Enter three numbers: 3, 2.4, 5 Enter
The sorted numbers are 2.4 3 5
Enter three numbers: 31, 12.4, 15 Enter
The sorted numbers are 12.4 15 31
```

*6.6 （显示模式）编写函数显示如下模式。

```
        1
      2 1
    3 2 1
...
n n-1 ... 3 2 1
```

这个函数头是

```
def displayPattern(n):
```

编写一个测试程序，提示用户输入一个数 n 然后调用 displayPattern(n) 来显示这种模式。

*6.7 （财务应用程序：计算未来投资值）编写一个函数计算指定年数以给定的利率来计算未来投资值。未来投资就是用编程题 2.19 中的计算公式计算得到的。

使用下面的函数头：

```
def futureInvestmentValue(
    investmentAmount, monthlyInterestRate, years):
```

例如：futureInvestmentValue(10000,0.05/12,50) 返回 12833.59。

编写一个程序提示用户输入投资额和百分比格式的年利率，然后输出一份表格显示年份从 1 到 30 年的未来值。下面是一个示例运行。

```
The amount invested: 1000 Enter
Annual interest rate: 9 Enter

Years    Future Value
1           1093.80
2           1196.41
...
29         13467.25
30         14730.57
```

6.8 （摄氏度和华氏度之间的转换）编写一个包含下面两个函数的模块。

```
# Converts from Celsius to Fahrenheit
def celsiusToFahrenheit(celsius):

# Converts from Fahrenheit to Celsius
def fahrenheitToCelsius(fahrenheit):
```

转换公式是：

```
celsius = (5 / 9) * (fahrenheit - 32)
fahrenheit = (9 / 5) * celsius + 32
```

编写一个测试程序，调用这两个函数来显示下面的表格。

Celsius	Fahrenheit	Fahrenheit	Celsius
40.0	104.0	120.0	48.89
39.0	102.2	110.0	43.33
...			
32.0	89.6	40.0	4.44
31.0	87.8	30.0	−1.11

6.9 （英尺和米之间的转换）编写一个包含下面两个函数的模块：

```
# Converts from feet to meters
def footToMeter(foot):

# Converts from meters to feet
def meterToFoot(meter):
```

转换公式如下。

```
foot = meter / 0.305
meter = 0.305 * foot
```

编写一个测试程序，调用这两个函数来显示如下表格。

Feet	Meters	Meters	Feet
1.0	0.305	20.0	66.574
2.0	0.610	25.0	81.967
...			
9.0	2.745	60.0	196.721
10.0	3.050	65.0	213.115

6.10 （使用 isPrime 函数）程序清单 6-7 提供了 isPrime(number) 函数测试某个数字是不是素数。使用这个函数找出小于 10 000 的素数的个数。

6.11 （财务应用程序：计算佣金）编写一个函数，利用编程题 5.39 的方案计算佣金。这个函数的函数头是：

```
def computeCommission(salesAmount):
```

编写一个测试程序，显示下面的表格。

Sales Amount	Commission
10000	900.0
15000	1500.0
...	
95000	11100.0
100000	11700.0

6.12 （显示字符）使用下面的函数头，编写一个打印字符的函数。

```
def printChars(ch1, ch2, numberPerLine):
```

这个函数打印 ch1 到 ch2 之间的字符，按每行指定某个数来打印。编写一个测试程序，打印 "1" 到 "Z" 的字符，每行打印 10 个。

*6.13 （数列求和）编写一个函数计算下面的数列。

$$m(i)=\frac{1}{2}+\frac{2}{3}+\cdots+\frac{i}{i+1}$$

编写一个测试程序显示下面的表格。

i	m(i)
1	0.5000
2	1.1667
...	
19	16.4023
20	17.3546

*6.14 （估算 π 值）可以用下面的函数近似计算 π。

$$m(i)=4\left(1-\frac{1}{3}+\frac{1}{5}-\frac{1}{7}+\frac{1}{9}-\frac{1}{11}+\cdots+\frac{(-1)^{i+1}}{2i-1}\right)$$

编写一个函数，给定 i 返回 m(i)，编写一个测试程序显示下面的表格。

i	m(i)
1	4.0000
101	3.1515
201	3.1466

（续）

i	m(i)
301	3.1449
401	3.1441
501	3.1436
601	3.1433
701	3.1430
801	3.1428
901	3.1427

*6.15 （财务应用程序：打印税款表）程序清单 4-7，给出计算税款的程序。使用下面的函数头编写一个计算税款的函数。

```
def computeTax(status, taxableIncome):
```

使用这个函数编写程序，打印可征税收入从 50 000 美元到 60 000 美元、收入间隔为 50 美元的四种纳税人的纳税表，如下所示。

Taxable Income	Single	Married Joint	Married Separate	Head of a House
50000	8688	6665	8688	7352
50050	8700	6673	8700	7365
...				
59950	11175	8158	11175	9840
60000	11188	8165	11188	9852

*6.16 （一年的天数）使用下面的函数头编写一个函数，返回一年的天数。

```
def numberOfDaysInAYear(year):
```

编写一个测试程序，显示从 2010 年到 2020 年每年的天数。

第 6.10 ～ 6.11 节

*6.17 （MyTriangle 模块）创建一个名叫 MyTriangle 的模块，它包含下面两个函数。

```
# Returns true if the sum of any two sides is
#  greater than the third side.
def isValid(side1, side2, side3):

# Returns the area of the triangle.
def area(side1, side2, side3):
```

编写一个测试程序，读入三角形三边的值，若输入有效则计算面积。否则，显示输入无效。计算三角形面积的公式在编程题 2.14 中给出。下面是一些示例运行。

```
Enter three sides in double: 1, 3, 1 Enter
Input is invalid
```

```
Enter three sides in double: 1, 1, 1 Enter
The area of the triangle is 0.4330127018922193
```

*6.18 （显示 0 和 1 构成的矩阵）编写一个函数，使用下面的函数头显示 $n \times n$ 矩阵。

```
def printMatrix(n):
```

每个元素都是随机产生的 0 或 1。编写一个测试程序提示用户输入整数 n，然后显示 $n \times n$

的矩阵。下面是一个示例运行。

```
Enter n: 3 ↵Enter
0 1 0
0 0 0
1 1 1
```

*6.19 （几何问题：点的位置）编程题 4.31 显示如何测试一个点是在一条有向线的左边、右边或刚好在线上。编写下面的函数。

```
# Return true if point (x2, y2) is on the left side of the
#  directed line from (x0, y0) to (x1, y1)
def leftOfTheLine(x0, y0, x1, y1, x2, y2):

# Return true if point (x2, y2) is on the same
#  line from (x0, y0) to (x1, y1)
def onTheSameLine(x0, y0, x1, y1, x2, y2):

# Return true if point (x2, y2) is on the
# line segment from (x0, y0) to (x1, y1)
def onTheLineSegment(x0, y0, x1, y1, x2, y2):
```

编写一个程序提示用户输入三个点 p0、p1 和 p2，然后显示点 p2 是在从 p0 到 p1 的线的左边、右边还是在线上。这个程序的示例运行和编程题 4.31 是一样的。

*6.20 （几何问题：显示角）重写程序清单 2-9 使用下面的函数计算两点之间的距离。

```
def distance(x1, y1, x2, y2):
```

*6.21 （数学问题：平方根的近似求法）math 模块里有几种实现 sqrt 函数的方法。其中一种方法就是巴比伦函数。它通过重复地使用下面的公式计算求出 n 的平方根的近似值。

```
nextGuess = (lastGuess + (n / lastGuess)) / 2
```

当 nextGuess 和 lastGuess 很接近时，nextGuess 就是平方根的近似值。初始的猜测值可以是任意的正数（例如：1）。这个值将是 lastGuess 的开始值。如果 nestGuess 和 lastGuess 的差别非常小时，例如：0.0001，你可以说 nestGuess 就是 n 的平方根近似值。否则，nextGuess 就变成 lastGuess，这个近似过程继续。实现下面的函数返回 n 的平方根。

```
def sqrt(n):
```

第 6.12 ～ 6.13 节

**6.22 （显示当前日期和时间）程序清单 2-7 显示当前时间。改进这个例子，显示当前日期和时间。（提示：可参考程序清单 6-13 中的日历例子获得一些如何求年、月和日的想法。）

**6.23 （将毫秒换成小时数、分钟数和秒数）使用下面的函数头，编写一个将毫秒换成小时数、分钟数和秒数的函数。

```
def convertMillis(millis):
```

该函数返回一个形如“小时：分钟：秒”的字符串。例如：convertMillis(5500) 返回字符串 0：0：5，convertMillis(100000) 返回字符串 0：1：40，而 convertMillis(555550000) 返回字符串 154：19：10。

编写一个程序提示用户输入一个毫秒数，然后显示形如“小时：分钟：秒”的字符串。

**6.24 （回文素数）*回文素数*是指一个数既是素数又是回文数。例如，131 既是素数也是回文数。数字 313 和 717 都是如此。编写程序显示前 100 个回文素数。每行显示 10 个数字，并且准确对齐，如下所示。

```
  2    3    5    7   11  101  131  151  181  191
313  353  373  383  727  757  787  797  919  929
```

**6.25 （反素数）反素数（逆向拼写的素数）是指一个将其逆向拼写后也是一个素数的非回文数。例如：17 和 71 都是素数，所以，17 和 71 都是反素数。编写程序显示前 100 个反素数。每行显示 10 个数字，并且准确对齐，如下所示。

```
 13   17   31   37   71   73   79   97  107  113
149  157  167  179  199  311  337  347  359  389
...
```

**6.26 （梅森素数）如果一个素数可以写成 2^{p-1} 的形式，其中 p 是某个正整数，那么这个数就被称作梅森素数。编写程序找出所有 $p \leq 31$ 的梅森素数。然后显示如下结果。

```
      p         2^p - 1
      2           3
      3           7
      5          31
...
```

**6.27 （双素数）双素数是指一对差值为 2 的素数。例如：3 和 5 就是一对双素数，5 和 7 就是一对双素数，11 和 13 也是一对双素数。编写程序，找出所有小于 1000 的双素数。显示结果如下所示。

```
(3, 5)
(5, 7)...
```

**6.28 （游戏：掷骰子）掷色子是赌场里一种非常流行的游戏。编写程序玩这个游戏的变种，如下所示。

掷两个骰子。每个骰子有六个面，分别表示值 1、2、…、6。检查两个骰子的和。如果和为 2、3 和 12，你就输了；如果和为 7 或 11，你就赢了；如果和是其他数字（4、5、6、8、9 或 10），就确定了一个点。继续掷骰子，直到掷出一个 7 或者掷出和刚才相同的点数。如果掷出的是 7，你就输了，如果掷出的点数和你前一次掷出的相同，你就赢了。程序扮演一个单独的玩家。下面是一些示例运行。

```
You rolled 5 + 6 = 11
You win
```

```
You rolled 1 + 2 = 3
You lose
```

```
You rolled 4 + 4 = 8
point is 8
You rolled 6 + 2 = 8
You win
```

```
You rolled 3 + 2 = 5
point is 5
You rolled 2 + 5 = 7
You lose
```

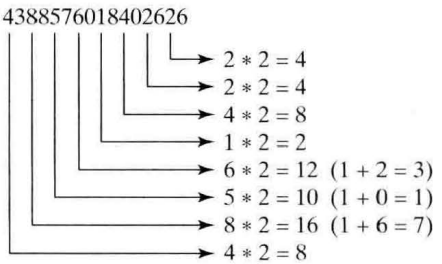
**6.29 （财务应用程序：信用卡号的合法性）信用卡号遵循下面的模式：一个信用卡号必须是 13 位到 16 位的整数，它的开头必须是：

- 4 是指 Visa 卡。
- 5 是指 Master 卡。
- 37 是指 American Express 卡。
- 6 是指 Discover 卡。

在 1954 年，IBM 的 Hans Luhn 提出一种算法，该算法可以验证信用卡号的有效性。这个算

法在验证信用卡号是否有效或者信用卡是否被扫描仪正确扫描方面是非常有用的。遵循这种合法性检测，可以生成所有的信用卡卡号，通常称为 Luhn 检测或 Mod 10 检测，它可以如下描述（为了方便解释，假设卡号为 4388576018402626）：

1）从左到右对每个数字翻倍。如果对某个数字翻倍后的结果是两位数，那么就将这两位加在一起得到一个一位数。



2）现在，将第一步得到的所有一位数相加。

4+4+8+2+3+1+7+8=37

3）将卡号里从左到右在奇数位上的所有数字相加。

6+6+0+8+0+7+8+3=38

4）将第 2 步和第 3 步得到的结果相加。

37+38=75

5）如果第 4 步得到的结果能被 10 整除，那么卡号就是合法的；否则，卡号是不合法的。例如：卡号 4388576018402626 是不合法的，但是卡号 438857601840707 是合法的。

编写程序，提示用户输入一个整数的信用卡卡号。显示这个数字是合法的还是非法的。设计你的程序使用下面的函数。

```
# Return true if the card number is valid
def isValid(number):

# Get the result from Step 2
def sumOfDoubleEvenPlace(number):


# Return this number if it is a single digit, otherwise, return
# the sum of the two digits
def getDigit(number):

# Return sum of odd place digits in number
def sumOfOddPlace(number):

# Return true if the digit d is a prefix for number
def prefixMatched(number, d):

# Return the number of digits in d
def getSize(d):

# Return the first k number of digits from number. If the
# number of digits in number is less than k, return number.
def getPrefix(number, k):
```

**6.30 （游戏：赢取骰子游戏的机会）修改编程题 6.28 使该程序运行 10 000 次，然后显示赢得游戏的次数。

***6.31 （当前时间和日期）调用 time.time() 返回从 1970 年 1 月 1 日 0 点开始的毫秒数。编写程序显示日期和时间。下面是一个示例运行。

```
Current date and time is May 16, 2012 10:34:23
```

**6.32 （打印日历）编程题 4.21 使用 Zeller 一致性原理来计算某天是星期几。使用 Zeller 的算法来简化程序清单 6-13 以获得每月开始的第一天是星期几。

**6.33 （几何问题：五边形的面积）使用下面的函数重写编程题 3.4 来返回五边形的面积。

```
def area(s):
```

*6.34 （几何问题：正多边形的面积）使用下面的函数重写编程题 3.5 返回正多边形的面积。

```
def area(n, side):
```

*6.35 （计算概率）使用程序清单 6-11 中的函数 RandomCharacter 生成 10 000 个大写字母，然后计算 A 的出现次数。

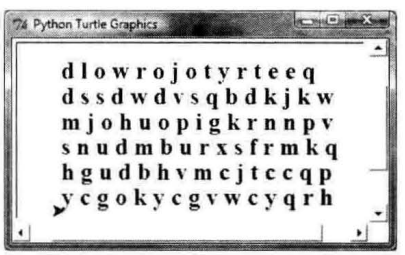
*6.36 （随机生成字符）使用程序清单 6-11 中的函数 RandomCharacter 生成 100 个大写字母，每行打印 10 个。

第 6.14 节

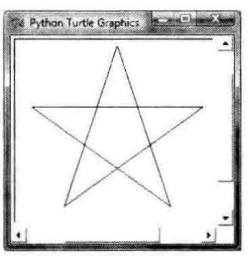
*6.37 （Turtle 模块：随机生成字符）使用程序清单 6-11 中的函数 RandomCharacter 生成 100 个小写字母，每行 15 个，如图 6-11a 所示。

**6.38 （绘制一条线）编写下面的函数绘制一条从点（$x1$,$y1$）到（$x2$,$y2$）带指定颜色（默认为黑色）和指定线宽（默认为 1）的线。

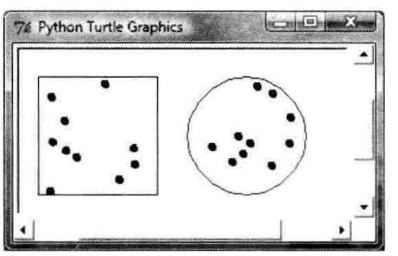
```
def drawLine(x1, y1, x2, y2, color = "black", size = 1):
```



a）程序显示随机小写字母



b）程序绘制一颗星星



c）程序在一个矩形和一个圆中绘制随机点

图　6-11

**6.39 （Turtle：绘制一颗星）使用定义在编程题 6.38 中的函数编写一个程序绘制一颗星星，如图 6-11b 所示。

**6.40 （Turtle：填充矩形和圆）编写下面的函数使其填充一个指定颜色、中心、宽度和高度的矩形以及一个指定颜色、圆心和半径的圆。

```
# Fill a rectangle
def drawRectangle(color = "black",
    x = 0, y = 0, width = 30, height = 30):

# Fill a circle
def drawCircle(color = "black", x = 0, y = 0, radius = 50):
```

**6.41 （Turtle：绘制点、矩形和圆）使用定义在程序清单 6-14 中的函数编写一个程序来显示一个中心在（−75，0）、高和宽为 100 的矩形以及圆心为 (50,0) 而半径为 50 的圆。在矩形和圆中填充 10 个任意点，如图 6-11c 所示。

**6.42 （Turtle：绘制 sin 函数）使用程序清单 6-14 中的函数简化编程题 5.52 的代码。

**6.43 （Turtle：绘制 sin 和 cos 函数）使用程序清单 6-14 中的函数简化编程题 5.53 的代码。

**6.44 （Turtle：绘制平方函数）使用程序清单 6-14 中的函数简化编程题 5.54 的代码。

**6.45 （Turtle：绘制一个正多边形）编写下面的函数绘制一个正多边形。

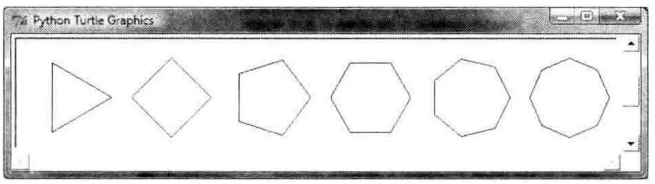
```
def drawPolygon(x = 0, y = 0, radius = 50, numberOfSides = 3):
```

多边形的中心在（x,y），指定该多边形的外接圆半径以及多边形的边数。编写一个程序显示三角形、四边形、五边形、六边形、七边形和八边形，如图 6-12a 所示。

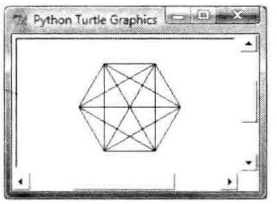
*6.46 （Turtle：连接六边形的所有点）编写一个程序显示六边形，所有点都互相连接，如图 6-12b 所示。

*6.47 （Turtle：两个棋盘）编写一个程序显示两个棋盘，如图 6-13 所示。你的程序必须至少定义成下面的函数。

```
# Draw one chessboard whose upper-left corner is at
# (startx, starty) and bottom-right corner is at (endx, endy)
def drawChessboard(startx, endx, starty, endy):
```



a）程序显示几个 *n* 边形



b）程序显示一个所有点都互连的六边形

图　6-12

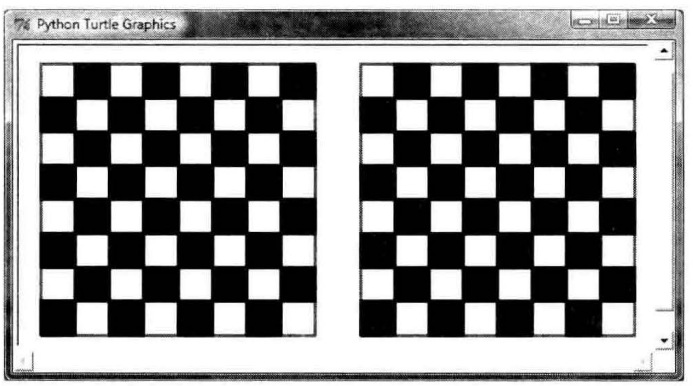


图 6-13　程序绘制两个棋盘

*6.48 （格式化一个整型数）使用下面的函数头编写一个函数格式化整数为指定宽度。

```
def format(number, width):
```

这个函数返回一个前缀为一个或多个 0 的数字。字符串的大小就是宽度。例如：format(34,5) 返回“00034”。如果数字比指定宽度要长，那么，函数就返回表示这个数的字符串。例如：format(34,1) 返回“34”。编写一个测试程序，提示用户输入一个数以及它的宽度，并显示从调用函数 format(number,width) 返回的字符串。下面是一个示例运行。

```
Enter an integer: 453 Enter
Enter the width: 6 Enter
The formatted number is 000453
```

第二部分

Introduction to Programming Using Python

面向对象程序设计

第 7 章

Introduction to Programming Using Python

对象和类

学习目标

- 描述对象和类，以及使用类来建模对象（第 7.2 节）。
- 定义带数据域和方法的类（第 7.2.1 节）。
- 使用构造方法调用初始化程序来创建和初始化数据域以构建一个对象（第 7.2.2 节）。
- 使用圆点运算符（.）访问对象成员（第 7.2.3 节）。
- 使用 self 参数引用对象本身（第 7.2.4 节）。
- 使用 UML 图符号来描述类和对象（第 7.3 节）。
- 区分不可变对象和可变对象（第 7.4 节）。
- 隐藏数据域以避免数据域损坏并使类更易于维护（第 7.5 节）。
- 在软件开发过程中应用类的抽象和封装（第 7.6 节）。
- 探究面向过程范式和面向对象范式的差异（第 7.7 节）。

7.1 引言

关键点：面向对象程序设计可以让你高效地开发大型软件和图形用户界面。

学习前几章的内容之后，现在，我们可以使用选择、循环和函数来解决许多程序设计问题。但是，这些特征对开发图形用户界面（GUI，读作 goo-ee）或大型软件系统来讲并不够用。假设你想开发如图 7-1 所示的图形用户界面。你该如何编写这个程序呢？

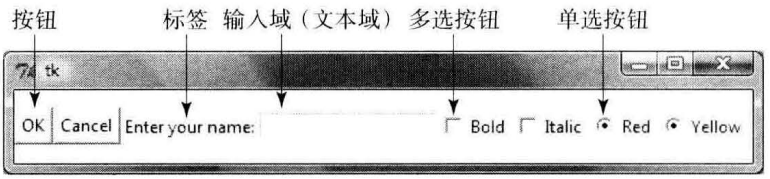


图 7-1 你可以使用面向对象程序设计创建类似这样的 GUI 对象

本章介绍面向对象程序设计，它会构建一个基础，让你在接下来的章节中开发图形用户界面和大型软件系统。

7.2 为对象定义类

关键点：类定义对象的特征和行为。

第 3.5 节介绍了对象和方法，并展示如何使用对象。对象由类创建。本节将详细介绍如何定义自定制的类。

面向对象程序设计（OOP）是关于如何使用对象创建程序。对象代表现实世界中可以被明确辨识的实体。例如：一个学生、一张桌子、一个圆、一个按钮甚至一笔贷款都可以认为是一个对象。一个对象有独特的特性、状态和行为。

- 一个对象的特性就像人的身份证号码。Python 会在运行时自动对每个对象赋予一个独特的 id 来辨识这个对象。
- 一个对象的状态（也被称为它的特征或属性）是用变量表示的，称之为数据域。例如：一个圆对象具有数据域 radius，它表示圆的一个属性。一个矩形对象有数据域 width 和 height，它们表示矩形的属性。
- Python 使用方法来定义一个对象的行为（也称为它的动作）。回顾一下，方法也被称为函数。通过调用对象上的方法，你可以让对象完成某个动作。例如：你可以为圆对象定义名为 getArea() 和 getPerimeter() 的方法。这样，圆对象就可以调用 getArea() 方法返回它的面积，调用 getPerimeter() 方法来返回它的周长。

使用通用类来定义同一种类型的对象。类和对象的关系就像苹果派食谱和苹果派之间的关系。你可以根据一张苹果派食谱（类）制作出任意多个苹果派（对象）。

一个 Python 类使用变量存储数据域，定义方法来完成动作。类就是一份契约（有时也称之为模板或蓝本），它定义对象的数据域和方法。

对象是类的一个实例，你可以创建一个类的多个对象。创建类的一个实例的过程被称为实例化。术语对象和实例经常是可互换的。对象就是实例，而实例就是对象。

图 7-2 显示一个名为 Circle 的类以及它的三个对象。

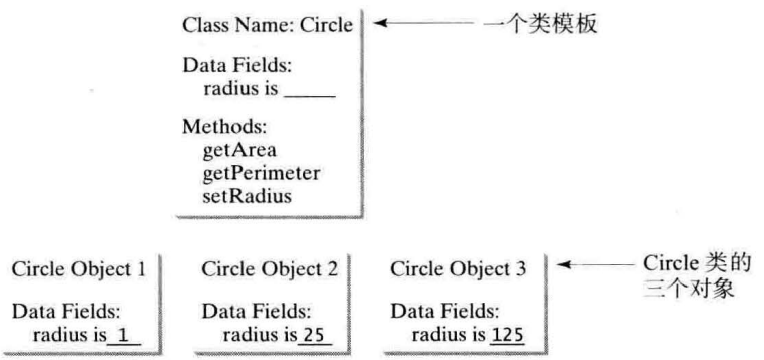


图 7-2　类是一个创建对象的模板或合约

7.2.1　定义类

除了使用变量存储数据域和定义方法，一个类还提供了一种特殊的方法：__init__。这个方法被称为初始化程序，它是在创建和初始化这个新对象时被调用的。初始化程序能完成任何动作，但初始化程序被设计为完成初始化动作，例如：使用初始值创建对象的数据域。

Python 使用下面的语法定义一个类：

```
class ClassName:
    initializer
    methods
```

程序清单 7-1 定义了 Circle 类。类名通常是在关键词 class 之后，其后紧随一个冒号（：）。初始化程序总是被命名为 __init__（第 5 行），这是一个特殊的方法。注意：init 需要前后加两个下划线。数据域 radius 在初始化程序中创建（第 6 行）。定义方法 getPerimeter 和 getArea 返回一个圆的周长和面积（第 8 ～ 12 行）。接下来的几节会介绍更多关于初始化程序、数据域和方法的细节。

程序清单 7-1 Circle.py

```
1  import math
2
3  class Circle:
4      # Construct a circle object
5      def __init__(self, radius = 1):
6          self.radius = radius
7
8      def getPerimeter(self):
9          return 2 * self.radius * math.pi
10
11     def getArea(self):
12         return self.radius * self.radius * math.pi
13
14     def setRadius(self, radius):
15         self.radius = radius
```

注意：类名的命名风格在 Python 库中不是始终如一的。在本书中，我们会采用类名中每个单词的首字母大写的方式。例如：Circle、Linear Equation 和 LinkedList 都是遵循我们习惯的正确类名。

7.2.2　构造对象

一旦定义了一个类，你就可以使用构造方法由类来创建对象。构造方法完成两个任务：

- 在内存中为类创建一个对象。
- 调用类的 __init__ 方法来初始化对象。

包括初始化程序的所有方法，都有第一个参数 self。这个参数指向调用方法的对象。__init__ 方法中的 self 参数被自动地设置为引用刚被创建的对象。你可以为这个参数指定任何一个名字，但是按照惯例，经常使用的是 self。我们将在第 7.2.4 节探讨 self 的作用。

构造方法的语法规则是：

类名（参数）

图 7-3 显示对象是如何被创建并初始化的。在对象被建立之后，self 可以被用来指向对象。

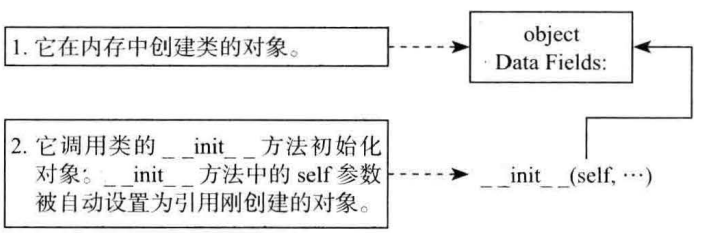


图 7-3　构建一个对象是在内存中创建对象并调用它的初始化程序

构造方法的参数和无 self 的 __init__ 方法中的参数匹配。例如：因为程序清单 7-1 中的第 5 行 __init__ 方法被定义为 __init__ (self,radius=1)，所以，为了构建一个半径 radius 为 5 的 Circle 对象，那你就应该使用 Circle(5)。图 7-4 显示使用 Circle(5) 构建 Circle 对象的效果。首先，Circle 对象在内存中被创建，然后调用初始化程序将半径 radius 设置为 5。

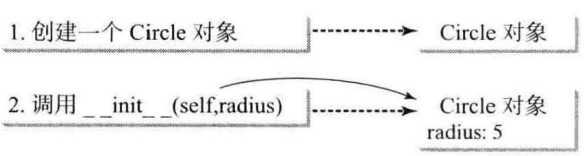


图 7-4　使用 Circle(5) 构建一个圆对象

Circle 类中的初始化程序有默认的 radius 值 1。接下来，构造方法创建了默认半径为 1 的 Circle 对象：

```
Circle()
```

7.2.3　访问对象成员

对象成员是指它的数据域和方法。数据域也被称为实例变量，因为每个对象（实例）的数据域中都有一个特定值。方法也被称为实例方法，因为方法被一个对象（实例）调用来完成对象上的动作，例如，改变对象数据域中的值。为了访问一个对象的数据域以及调用对象的方法，你需要使用下面的语法将对象赋给一个变量：

```
objectRefVar = ClassName(arguments)
```

例如：

```
c1 = Circle(5)
c2 = Circle()
```

你可以使用圆点运算符（.）访问对象的数据域并调用它的方法，它也被称为对象成员访问运算符。使用圆点运算符的语法是：

```
objectRefVar.datafield
objectRefVar.method(args)
```

例如：下面的代码访问 radius 数据域（第 3 行），然后调用 getPerimeter 方法（第 5 行）以及 getArea 方法（第 7 行）。注意：第 1 行导入程序清单 7-1 里 Circle 模块中定义的 Circle 类。

```
1  >>> from Circle import Circle
2  >>> c = Circle(5)
3  >>> c.radius
4  5
5  >>> c.getPerimeter()
6  31.41592653589793
7  >>> c.getArea()
8  78.53981633974483
9  >>>
```

注意：通常，你创建一个对象并将它赋给一个变量。随后，你可以使用变量指代这个对象。偶尔，对象也不需要随后被引用。在这种情况下，你可以创建一个对象而不需要明确将它赋值给变量，如下所示：

```
print("Area is", Circle(5).getArea())
```

这个语句创建 Circle 对象并调用它的 getArea 方法来返回它的面积。以这种方式创建的对象被称为匿名对象。

7.2.4　self 参数

如之前提到的，定义的每个方法的第一个参数就是 self。这个参数被用在方法的实现中，但不是用在方法被调用的时候。那么，这个参数 self 是干什么的？为什么 Python 需要它？

self 是指向对象本身的参数。你可以使用 self 访问在类定义中的对象成员。例如：你可以使用语法 self.x 访问实例变量 x，而使用语法 self.m1() 来调用类的对象 self 的实例方法 m1，如图 7-5 所示。

一旦一个实例变量被创建，那么它的作用域就是整个类。在图 7-5 中，self.x 是一个在 __init__ 方法中创建的实例变量。它可以在方法 m2 中被访问。实例变量 self.y 在方法 m1

中被设置为 2，在方法 m2 中被设置为 3。注意：你也可以在方法中创建局部变量。局部变量的作用域是在该方法内。局部变量 z 在方法 m1 中被创建，而它的作用域就是从它创建时起到方法 m1 结束。

图 7-5　实例变量的作用域是整个类

7.2.5　举例：使用类

前面几节演示了类和对象的概念。我们已经学习了如何使用初始化程序、数据域和方法定义一个类，以及如何使用构造方法创建一个对象。本节给出一个测试程序构建半径分别为 1、25、125 的三个圆对象，程序清单 7-2 中显示每个圆的半径和面积。然后，程序将第二个对象的半径改为 100，并显示它的新半径和面积。

程序清单 7-2　TestCircle.py

```
1  from Circle import Circle
2
3  def main():
4      # Create a circle with radius 1
5      circle1 = Circle()
6      print("The area of the circle of radius",
7          circle1.radius , "is", circle1.getArea())
8
9      # Create a circle with radius 25
10     circle2 = Circle(25)
11     print("The area of the circle of radius",
12         circle2.radius , "is", circle2.getArea() )
13
14     # Create a circle with radius 125
15     circle3 = Circle(125)
16     print("The area of the circle of radius",
17         circle3.radius , "is", circle3.getArea() )
18
19     # Modify circle radius
20     circle2.radius = 100 # or circle2.setRadius(100)
21     print("The area of the circle of radius",
22         circle2.radius , "is", circle2.getArea())
23
24 main() # Call the main function
```

```
The area of the circle of radius 1.0 is 3.141592653589793
The area of the circle of radius 25.0 is 1963.4954084936207
The area of the circle of radius 125.0 is 49087.385212340516
The area of the circle of radius 100.0 is 31415.926535897932
```

程序使用 Circle 类创建 Circle 对象。这种使用类（例如：Circle）的程序被称作类的客户端。

Circle 类被定义在程序清单 7-1 中，这个程序在第 1 行使用语法 from Circle import Circle 将其导入。程序创建了一个默认半径为 1 的 Circle 对象（第 5 行）并创建两个指定半径的 Circle 对象（第 10、15 行），然后获取 radius 属性，并调用对象上的 getArea() 方法获取面积（第 7、12、17 行）。程序给 Circle2 设置一个新的 radius 属性（第 20 行）。这个也可以通过使用 circle2.setRadius（100）完成。

注意：看似保存一个对象的变量实际上包含的是指向这个对象的引用。严格地讲，变量和对象是不同的，但大多数情况下，两者的区别是可以忽略的。所以，为了简单起见，最好说“circle1 是一个 Circle 对象”，而不会长冗地描述为“circle1 是一个变量，它包含一个指向 Circle 对象的引用”。

检查点

7.1　描述对象和它的类定义之间的关系。

7.2　如何定义一个类？

7.3　如何创建一个对象？

7.4　初始化方法的名字是什么？

7.5　习惯上，初始化方法的第一个参数被命名为 self。self 的作用是什么？

7.6　构建一个对象的语法是什么？ Python 在创建一个对象时做了些什么？

7.7　一个初始化程序和一个方法的区别是什么？

7.8　对象成员访问运算符是干什么的？

7.9　运行下面的程序会出现什么问题？如何修正它？

```
class A:
    def __init__(self, i):
        self.i = i

def main():
    a = A()
    print(a.i)

main() # Call the main function
```

7.10　下面的程序有什么错误？

```
1  class A:
2      # Construct an object of the class
3      def A(self):
4          radius = 3
```

a)

```
1  class A:
2      # Construct an object of the class
3      def __init__(self):
4          radius = 3
5
6      def setRadius(radius):
7          self.radius = radius
```

b)

7.3　UML 类图

关键点：UML 类图表示用图形符号描述类。

对图 7-2 中类模板和对象的阐释可以使用 UML（统一建模语言）符号来规范。这种符号，如图 7-6 所示，被称作 UML 类图或简称为类图，是独立于语言的；也就是说，其他程序设计语言也使用同样的模型和注释。在 UML 类图中，数据域被表示为：

```
dataFieldName: dataFieldType
```

构造方法如下所示：

```
ClassName(parameterName: parameterType)
```

方法被表示为：

```
methodName(parameterName: parameterType): returnType
```

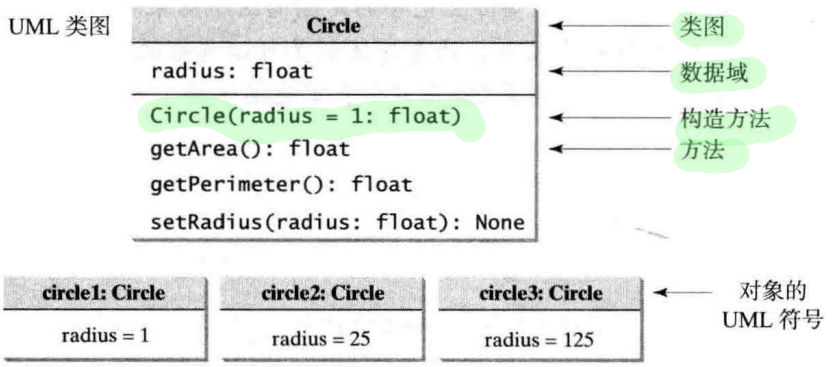


图 7-6　类和对象都可以使用 UML 符号表示

类中方法定义总有特殊的 self 参数，但在 UML 图中并不包括它，因为客户端不需要知道这个参数而且不会使用参数调用方法。

方法 _ _init_ _ 同样不需要罗列在 UML 图中，因为它被构造方法调用，它的参数与构造方法的参数是一样的。

UML 图就像客户端的合约（模板），这样，客户端就知道怎么使用这个类。它为用户端描述如何创建对象以及如何调用对象上的方法。

考虑将电视机作为一个例子。每个电视机都是一个带有多个状态（即，当前频道、当前音量、电源开或关，这些都是数据域所代表的电视机的属性）和行为（换频道、调音量、打开 / 关闭，这些都是每个电视机对象用方法执行的动作）的对象。你可以使用类定义电视机。TV 类的 UML 图如图 7-7 所示。

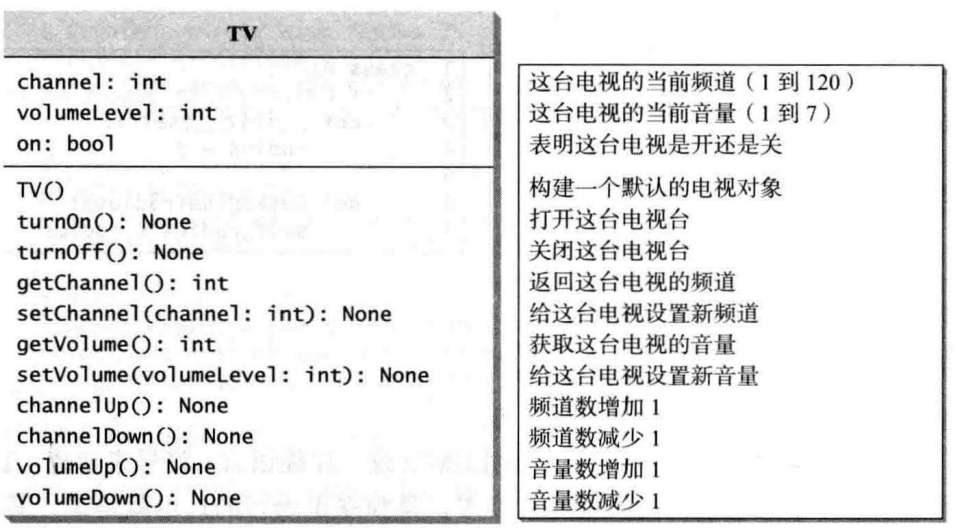


图 7-7　TV 类定义电视机

程序清单 7-3 给出定义 TV 类的 Python 代码。

程序清单 7-3 TV.py

```
1  class TV:
2      def __init__(self):
3          self.channel = 1   # Default channel is 1
4          self.volumeLevel = 1   # Default volume level is 1
5          self.on = False    # Initially, TV is off
6
7      def turnOn(self):
8          self.on = True
9
10     def turnOff(self) :
11         self.on = False
12
13     def getChannel(self):
14         return self.channel
15
16     def setChannel(self, channel):
17         if self.on and 1 <= self.channel <= 120:
18             self.channel = channel
19
20     def getVolumeLevel(self):
21         return self.volumeLevel
22
23     def setVolume(self, volumeLevel):
24         if self.on and \
25               1 <= self.volumeLevel <= 7:
26             self.volumeLevel = volumeLevel
27
28     def channelUp(self):
29         if self.on and self.channel < 120:
30             self.channel += 1
31
32     def channelDown(self):
33         if self.on and self.channel > 1:
34             self.channel -= 1
35
36     def volumeUp(self):
37         if self.on and self.volumeLevel < 7:
38             self.volumeLevel += 1
39
40     def volumeDown(self):
41         if self.on and self.volumeLevel > 1:
42             self.volumeLevel -= 1
```

初始化程序为 TV 对象中的数据域创建实例变量 channel、volumeLevel 和 on（第 2 ～ 5 行）。注意：这个初始化程序除了 self 就没有其他的参数。

如果这个电视不是打开状态（第 16 ～ 18 和 23 ～ 26 行），那么频道和音量就不会被改变。在它们中的任何一个被改变之前，它的当前值都会被检查以确保是在正确范围内。

程序清单 7-4 是使用 TV 类创建两个对象的程序。

程序清单 7-4 TestTV.py

```
1  from TV import TV
2
3  def main():
4      tv1 = TV()
5      tv1.turnOn()
6      tv1.setChannel(30)
7      tv1.setVolume(3)
```

```
 8
 9      tv2 = TV()
10      tv2.turnOn()
11      tv2.channelUp()
12      tv2.channelUp()
13      tv2.volumeUp()
14
15      print("tv1's channel is", tv1.getChannel() ,
16          "and volume level is", tv1.getVolumeLevel() )
17      print("tv2's channel is", tv2.getChannel(),
18          "and volume level is", tv2.getVolumeLevel())
19
20  main() # Call the main function
```

```
tv1's channel is 30 and volume level is 3
tv2's channel is 3 and volume level is 2
```

程序创建了两个 TV 对象：tv1 和 tv2（第 4 和 9 行），调用对象上的方法来完成设置频道和音量及增加频道和音量等动作。tv1 通过调用第 5 行的 tv1.turnOn() 被打开，通过调用第 6 行的 tv1.setChannel(30) 将频道设置为 30，而在第 7 行将音量设置为 3 。tv2 在第 10 行被打开，它的频道通过调用第 11 行的 tv2.channelUp() 增加 1 ，然后在第 12 行又增加 1 。因为初始频道被设置为 1 （TV.py 中的第 3 行)），tv2 的频道现在是 3。tv2 的音量通过调用第 13 行的 tv2.volumeUp() 增加 1 。因为初始音量被设置为 1 （TV.py 中的第 4 行），tv2 的音量现在是 2。

程序在第 15 ～ 18 行显示对象的状态。使用方法 getChannel() 和 getVolumnLevel() 读取数据域。

7.4　不变对象和可变对象

关键点：当将一个可变对象传给函数时，函数可能会改变这个对象的内容。

回顾一下，Python 中的数字和字符串都是不可变对象。它们的内容不能被改变。当将一个不可变对象传给函数时，对象不会被改变。但是，如果你给函数传递一个可变对象，那么对象的内容就可能有变化。程序清单 7-5 中的例子演示不可变对象和可变对象参数在函数中的不同。

程序清单 7-5　TestPassMutableObject.py

```
 1  from Circle import Circle
 2
 3  def main():
 4      # Create a Circle object with radius 1
 5      myCircle = Circle()
 6
 7      # Print areas for radius 1, 2, 3, 4, and 5
 8      n = 5
 9      printAreas(myCircle, n)
10
11      # Display myCircle.radius and times
12      print("\nRadius is", myCircle.radius)
13      print("n is", n)
14
15  # Print a table of areas for radius
16  def printAreas(c, times):
17      print("Radius \t\tArea")
18      while times >= 1:
```